

完全零基础 · 手把手

★ Star 支持持续更新

Claude Code 零基础入门指南

不会编程、没用过终端，也能从零读到精通

27 章 + 9 附录

17 万字

43 张思维导图

Mac / Windows 双平台



作者：马力 · github.com/mali9527/claude-code-full-guide-for-beginners

Claude Code 零基础入门指南

针对 Claude Opus 4.7 全面更新

马力 (Ma Li)

2026 · 首版

github.com/mali9527/claude-code-full-guide-for-beginners

Claude Code 零基础入门指南 · 首版 2026-04

作者：马力 (Ma Li) · @mali9527 · li.ma.aria@gmail.com

GitHub 仓库：github.com/mali9527/claude-code-full-guide-for-beginners

本书以 CC BY-NC-SA 4.0 协议开源。允许自由传播 / 修改 / 翻译，禁止商用，修改后须保持同样协议开源。详见 LICENSE 文件。

本书内容针对 Claude Opus 4.7 全面更新 (Anthropic 2026-04-16 发布)。后续版本迭代会同步修订，最新版本请访问 GitHub 仓库。

目录

第 0 章：本书怎么读	1
0.1 这本书适合谁	2
0.2 读完你能做什么	2
0.3 本书用到的记号	2
0.4 章节地图	3
0.5 别搞错——还有两种“会干活的 Claude”	4
0.6 每章都有的三件套	4
本章小结	4
动手任务	5
如果你卡住了	5
🌿 支线 0.A：读这本书的节奏建议	6
🌿 支线 0.B：这本书和其他 Claude Code 教程有啥不一样	6
第 1 章：先把电脑准备好	9
开场三问	9
1.1 认识终端	10
1.2 打开你的终端	11
1.3 第一次使用：看懂提示符，输入 <code>pwd</code>	12
1.4 在文件夹间穿梭： <code>ls</code> 和 <code>cd</code>	13
1.5 理解“路径”	14
1.6 收尾：退出终端 + 三个常见报错	15
本章小结	16
动手任务	17
如果你卡住了	17
🌿 支线 1.A：打开终端的另一种方式，以及把它固定到 Dock / 任务栏	18
🌿 支线 1.B：命令用得更灵活	19
🌿 支线 1.C：路径的两个细节坑	20
第 2 章：安装 Claude Code	22
开场三问	22

2.1 装之前：两件事要先备好	23
2.2 Mac 上安装 Claude Code	24
2.3 Windows 上安装 Claude Code	25
2.4 登录并验证	26
2.5 装不上？三个最常见的失败	27
本章小结	28
动手任务	28
如果你卡住了	29
🌿 支线 2.A：计费 and 隐私——几个最关心的问题（简答版）	30
🌿 支线 2.B：除了 npm 还有别的装法	31
第 3 章：你的第一次对话	32
开场三问	32
3.1 先准备一个“练习文件夹”	33
3.2 进入工作文件夹再启动	34
3.3 认识 Claude Code 的界面	35
3.4 发第一条消息，让它读一个文件	36
3.5 退出，并在下次继续	37
本章小结	38
动手任务	38
如果你卡住了	39
🌿 支线 3.A：用 <code>--resume</code> 从历史对话里挑一个继续	40
🌿 支线 3.B：状态栏到底在告诉你什么	41
第 4 章：Claude Code 到底是什么	43
开场三问	43
4.1 它不是聊天机器人——它是你电脑里的“实习生”	44
4.2 它能做什么、不能做什么	45
4.3 和 ChatGPT / Claude 网页版的本质区别	46
4.4 三种产品形态：你该用哪一个	47
4.5 调整你的预期：5 个心智转变	48
本章小结	49
动手任务	50
如果你卡住了	50
🌿 支线 4.A：Claude Code 底层是怎么运作的（通俗版）	51

🌿 支线 4.B: 什么是“agentic coding”（智能体编码）	52
第 5 章：让它读文件	55
开场三问	55
5.1 让 Claude “看到”内容的三种方式	56
5.2 @ 引用的具体用法	57
5.3 提问的“WHAT / WHERE / HOW / VERIFY”模板	58
5.4 为什么“具体”永远比“模糊”好	58
5.5 让它看图片：截图是你的好朋友	59
本章小结	60
动手任务	60
如果你卡住了	61
🌿 支线 5.A: 让 Claude 浏览整个文件夹（不读内容只看结构）	62
🌿 支线 5.B: 让 Claude 读网页 / URL	63
第 6 章：让它改文件	64
开场三问	64
6.1 什么是 diff（修改对比图）	65
6.2 三档审核习惯——别每次都细读	66
6.3 三个选项：Accept / Reject / Edit	67
6.4 不只是改——让它建、删、重命名	68
6.5 改错了怎么挽回——四种路	69
本章小结	70
动手任务	70
如果你卡住了	71
🌿 支线 6.A: 批量改多个文件	72
🌿 支线 6.B: 让它改得更准的三个技巧	73
第 7 章：让它跑命令 + 权限机制	74
开场三问	74
7.1 “让它跑命令”是什么意思	75
7.2 权限机制：5 档简化为 3 档	75
7.3 怎么读权限弹窗	77
7.4 四类高风险命令——必须识别	78
7.5 权限设置在哪改	80
本章小结	81

动手任务	81
如果你卡住了	82
🌿 支线 7.A: 什么时候值得用自动接受模式 (acceptEdits)	83
🌿 支线 7.B: 建一个私人白名单	84
第 8 章: 交互循环七步法	86
开场三问	86
8.1 为什么要讲“交互循环”	87
8.2 七步走一遍完整流程	87
8.3 每一步你的职责	88
8.4 三种常见跑偏 + 怎么拉回来	90
8.5 把循环刻进肌肉——一个完整演示	90
本章小结	91
动手任务	91
如果你卡住了	92
🌿 支线 8.A: 为什么叫“循环”，不是“流程”	92
🌿 支线 8.B: 多轮迭代的小技巧	93
第 9 章: 计划模式与撤销——Claude Code 的安全网三件套	95
开场三问	95
9.1 安全网三件套——先认一下	96
9.2 /plan 计划模式——让 Claude 只看不动	97
9.3 Esc Esc——快速撤销最近一步	98
9.4 /rewind——精确回到历史检查点	99
9.5 三件套和 git 的关系	100
本章小结	101
动手任务	102
如果你卡住了	103
🌿 支线 9.A: 把 plan → execute 当默认 workflow	103
🌿 支线 9.B: /rewind 的边界——它能和不能救什么	104
第 10 章: 信任校准——什么时候信，什么时候核对	106
开场三问	106
10.1 先承认一个事实: AI 确实会犯错	107
10.2 验证频谱——按改动风险选检查深度	108
10.3 Vibe review——中间策略	108

10.4 必须细审的几类场景（警戒清单）	109
10.5 建立你自己的“信任档案”	110
本章小结	111
动手任务	111
如果你卡住了	112
🌿 支线 10.A: AI 最容易出错的几个典型场景	112
🌿 支线 10.B: 团队里怎么统一审核标准	113
第 11 章：你的数据去哪了——隐私与数据流	117
开场三问	117
11.1 先看一张图：一次对话的数据去了哪	118
11.2 “不训练模型”到底是什么意思	119
11.3 红线清单：这些东西绝对别发给 Claude	120
11.4 黄线清单：要小心的内容（可以发但先处理）	121
11.5 公司 / 团队用户的决策清单	121
本章小结	123
动手任务	123
如果你卡住了	124
🌿 支线 11.A: 数据保留的几个细节	125
🌿 支线 11.B: 真的不敢上传，有没有本地替代	125
第 12 章：Claude 为什么会变糊涂——上下文原理	127
开场三问	127
12.1 比喻先行：一个熬夜的人	128
12.2 用技术名词说：上下文窗口（context window）	129
12.3 Claude Code 的状态栏：看懂 Ctx%	129
12.4 三种清理策略：/compact、/clear、新会话	130
12.5 预防比清理更重要：省 Ctx 的 5 个习惯	132
本章小结	133
动手任务	133
如果你卡住了	134
🌿 支线 12.A: Token 是什么——再深一层	135
🌿 支线 12.B: 长上下文模型（1M 版本）什么时候值得用	136
🌿 支线 12.C: /compact 的工作原理	137

第 13 章：三层记忆——会话 / 自动记忆 / CLAUDE.md	139
开场三问	139
13.1 三层记忆是什么——先看比喻	140
13.2 第一层：会话记忆	141
13.3 第二层：自动记忆 (MEMORY.md)	142
13.4 第三层：项目记忆 (CLAUDE.md)	143
13.5 第四层（特殊）：Skills——按需加载	144
13.6 快速决策表：这条信息放哪层	145
本章小结	146
动手任务	146
如果你卡住了	148
🌿 支线 13.A：高效管理你的 MEMORY.md	148
🌿 支线 13.B：全局 CLAUDE.md vs 项目 CLAUDE.md	149
第 14 章：写好你的 CLAUDE.md——全书最高杠杆的一章	152
开场三问	152
14.1 第一反直觉警告：不要用 <code>/init</code> 一键生成	153
14.2 CLAUDE.md 的三个核心作用	154
14.3 容量铁律：越短越好	155
14.4 反模式清单：这些东西不要写进 CLAUDE.md	155
14.5 正面模板：一个好的 CLAUDE.md 长什么样	158
14.6 渐进披露原则：慢慢加、不要一次堆	160
本章小结	161
动手任务	161
如果你卡住了	162
🌿 支线 14.A：团队场景下的 CLAUDE.md	163
🌿 支线 14.B：CLAUDE.md 的版本管理	164
第 15 章：模型与成本——怎么选、怎么算、怎么省	166
开场三问	166
15.1 三款模型——一句话对比	167
15.2 怎么选——按任务类型	168
15.3 怎么切换—— <code>/model</code> 命令	169
15.4 怎么查用了多少—— <code>/status</code> 和账单页	170
15.5 一个月大概要花多少——量级感	171

15.6 五个省钱习惯	172
本章小结	173
动手任务	173
如果你卡住了	174
🌿 支线 15.A: 按量付费 vs 订阅——怎么选	175
🌿 支线 15.B: 团队 / 公司场景的一点建议	176
第 16 章: 新手 10 大错误——看完能避开的坑	179
开场三问	179
16.1 错误 1: 一来就跑 <code>/init</code>	180
16.2 错误 2: 把 <code>.env</code> 或密钥文件 <code>@</code> 进来	181
16.3 错误 3: 不看 diff 就无脑 Yes	181
16.4 错误 4: 在一个会话里做 10 件不相关的事	181
16.5 错误 5: 文件太大 / 目录太大直接 <code>@</code>	182
16.6 错误 6: 让 Claude 干完全不懂的活然后全盘信	182
16.7 错误 7: 用模糊语言给任务	182
16.8 错误 8: 忽视状态栏	183
16.9 错误 9: 对 Claude 的错误愤怒升级	183
16.10 错误 10: 把 CLAUDE.md 当“通灵板”	184
本章小结	184
动手任务	185
如果你卡住了	186
🌿 支线 16.A: 两个进阶错误	187
🌿 支线 16.B: 把错误清单当项目迭代	188
第 17 章: 什么时候该停下来——不要和 AI 对抗	189
开场三问	189
17.1 核心原则: 5 分钟原则	190
17.2 “和 AI 对抗”的四个信号	191
17.3 卡住时的三档应对	192
17.4 人和 AI 各自擅长什么	193
17.5 建立你自己的“止损规则”	194
本章小结	195
动手任务	195
如果你卡住了	196

🌿 支线 17.A: 什么时候坚持——避免过度止损	197
🌿 支线 17.B: 团队 / pair programming 场景	197
第 18 章: 常用快捷命令全览——斜杠命令速查	199
开场三问	199
18.1 斜杠命令是什么	200
18.2 核心 12 个命令——按场景分组	200
18.3 一图看懂: 什么情境用什么命令	202
18.4 命令组合使用——“套路”	203
18.5 ! 和 @ —— 严格来说不是斜杠命令, 但属于本章	204
18.6 自定义命令的一个预告	205
本章小结	205
动手任务	206
如果你卡住了	206
🌿 支线 18.A: 自定义“常用套路”为一键命令	207
第 19 章: 输入技巧——把效率榨干	209
开场三问	209
19.1 ! 快跑 shell 命令 (已在 Ch 18 提过, 这里说透)	210
19.2 @ 精准引用——进阶用法	210
19.3 图片 / 截图输入	211
19.4 多行输入——长提示词的正确姿势	212
19.5 历史回溯	213
19.6 粘贴大段文字的正确姿势	214
本章小结	215
动手任务	215
如果你卡住了	216
🌿 支线 19.A: 键盘快捷键收藏单	217
🌿 支线 19.B: 输入模板库的搭建	218
第 20 章: Skills 入门——可复用的操作手册	221
开场三问	221
20.1 Skill 是什么	222
20.2 Skills 存在哪里	223
20.3 第一个 Skill: 装一个现成的 (15 分钟)	223
原则	225

常见坑	225
原则	228
常见坑	228
🌿 支线 20.B: Skill 共享 / 社区	232
第 21 章: 自定义 Slash 命令——给你自己的快捷键	234
开场三问	234
21.1 自定义 Slash 命令是什么	235
21.2 建你的第一个命令 (10 分钟)	236
21.4 什么时候值得做成命令	239
21.5 命令、Skills、CLAUDE.md 三者统一对比	239
本章小结	240
动手任务	241
如果你卡住了	241
🌿 支线 21.A: 团队共享命令的最佳实践	242
vX.Y.Z	245
新增	245
修复	245
破坏性变更	245
第 22 章: Subagents 入门——派一个专业实习生去干活	246
开场三问	246
22.1 Subagent 是什么	247
22.2 和 Skill / Command 的本质区别	248
22.3 最简例子: 一个“代码审查员” Subagent (20 分钟)	248
原则	250
22.6 三者 (Skill / Command / Subagent) 联合 workflows	253
本章小结	254
动手任务	254
如果你卡住了	255
🌿 支线 22.A: 并行派多个 subagent	256
🌿 支线 22.B: Subagent 和“Agent 协作”生态	257
第 23 章: Hooks 入门——事件触发的自动化	258
开场三问	258
23.1 Hook 是什么	259

23.2 一个 Hook 长什么样	260
23.3 建你的第一个 Hook (15 分钟)	261
23.4 什么时候值得用 Hook	263
23.5 Hook 的三大坑	264
23.6 Hook 的三个经典模板	265
本章小结	266
动手任务	267
如果你卡住了	267
🌿 支线 23.A: 阻塞型 Hook 和条件判断	268
第 24 章: MCP 入门——给 Claude 接外部工具	270
开场三问	270
24.1 MCP 是什么	271
24.2 MCP 的工作原理 (概念)	272
24.3 装一个最简单的 MCP: 文件系统 (15 分钟)	273
24.4 新手最有用的三个 MCP	274
24.5 装 MCP 的通用步骤 (适用所有)	276
24.6 MCP 的安全红线	276
24.7 和其他扩展的关系	277
本章小结	277
动手任务	278
如果你卡住了	278
🌿 支线 24.A: 自己写一个 MCP server 的概念	279
🌿 支线 24.B: MCP 生态的现状和未来	280
第 25 章: 团队协作基础——和同事一起用 Claude Code	283
开场三问	283
25.1 共享什么、不共享什么——一张表	284
25.2 团队 CLAUDE.md 的维护节奏	285
25.3 三个团队场景容易踩的坑	286
25.4 和 git 工作流的配合	287
25.5 团队第一次用的“30 天接入路线”	289
本章小结	289
动手任务	290
如果你卡住了	291

🌿 支线 25.A: Claude Code 和 CI/CD 的边界	292
第 26 章: 进阶能力速览——指路牌	293
开场三问	293
26.1 本章不教你“怎么做”——只告诉你“有这些”	294
26.2 进阶方向 1: 多 Agent 编排	294
26.3 进阶方向 2: CI/CD 集成	295
26.4 进阶方向 3: Claude Code 桌面应用 (2026-04 大改版)	295
26.5 进阶方向 4: 非交互模式 / 脚本化	296
26.6 进阶方向 5: Anthropic API 直接调用	296
26.7 进阶方向 6: prompt engineering 深度	297
26.8 进阶方向 7: 模型对比 + 模型切换	298
26.9 进阶方向 8: Observability / 日志追踪	298
26.10 你的进阶路线图	299
本章小结	299
动手任务	299
如果你卡住了	300
🌿 支线 26.A: AI 工具未来 1-2 年的几个趋势	301
第 27 章: 下一步路线——读完书只是起点	303
开场三问	303
27.1 先认识自己现在的水平	304
27.2 30 天挑战清单	305
27.3 推荐资源	306
27.4 学习节奏建议	307
27.5 关于 AI 时代的工作心态	308
本章小结	309
动手任务 (最后一个!)	309
如果你卡住了	310
写在最后	310
🌿 支线 27.A: 书没写的那些	311
附录 A: Mac 终端速查	314
最常用 15 个命令	315
最常用快捷键	316
提示符符号速读	316

常见错误翻译	316
一些“我希望早点知道”	317
附录 B: Windows PowerShell 速查	318
最常用 15 个命令	319
最常用快捷键	319
提示符符号速读	320
常见错误翻译	320
Windows 特别要注意的路径差异	320
PowerShell 的“别名”	320
一些“我希望早点知道”	321
附录 C: Claude Code Slash 命令全表	322
上下文管理	322
安全网	323
模型与配置	323
记忆与项目	323
会话管理	323
特殊前缀（非 slash 但属于“特殊输入”）	323
常用组合套路	324
自定义命令	324
注意	324
附录 D: 决策流程图	325
图 1: 权限决策流程（遇到弹窗怎么选）	325
图 2: 模型选择流程（任务来了用哪个）	326
图 3: Ctx% 使用率应对流程	326
图 4: 卡住了怎么办（5 分钟原则）	327
图 5: 信息分层决策（该放在哪一层）	327
图 6: 扩展机制选型	328
附录 E: FAQ（常见问题 10 问）	329
1. Claude 会偷看我的文件吗？	329
2. 用 Claude Code 大概多少钱一个月？	329
3. 它改坏我的文件了怎么办？	330
4. 我在公司用会不会违反数据政策？	330
5. 有哪些模型？怎么选？	330
6. Claude 会记住我上次说的话吗？	330

7. 一句话: CLAUDE.md 怎么写?	331
8. Skill / Command / Subagent / Hook / MCP 怎么分?	331
9. 我可以和同事一起用吗?	331
10. 读完书了, 接下来干嘛?	331
附录 F: 术语表	333
A-Z 英文术语	334
本书内置概念	336
按键与符号	337
缩写速查	338
附录 G: 常见报错与自救手册	339
安装阶段	339
登录 / 认证	340
使用阶段	340
权限 / 改文件	341
Hooks / 扩展	342
Git 相关	342
通用排查顺序	343
还是解决不了?	343
附录 H: Claude Code 桌面应用简明导览	344
什么时候切桌面应用值	344
什么时候终端版更好	345
桌面应用的五大界面元素	345
概念对照表 (终端 ↔ 桌面)	346
本书所有概念在桌面应用里适用吗?	346
哪里下载	347
一个建议的迁移节奏	347
附录 I: 延伸阅读	348
一级: 官方权威 (最该看)	348
二级: 高质量长文 (核心思想)	349
三级: 社区内容 (获取手感)	350
四级: 进阶话题 (有需求再看)	350
五级: 基础知识补课 (零基础补充)	351
六级: AI 全景视野 (大方向思考)	351
不推荐 (避雷)	352

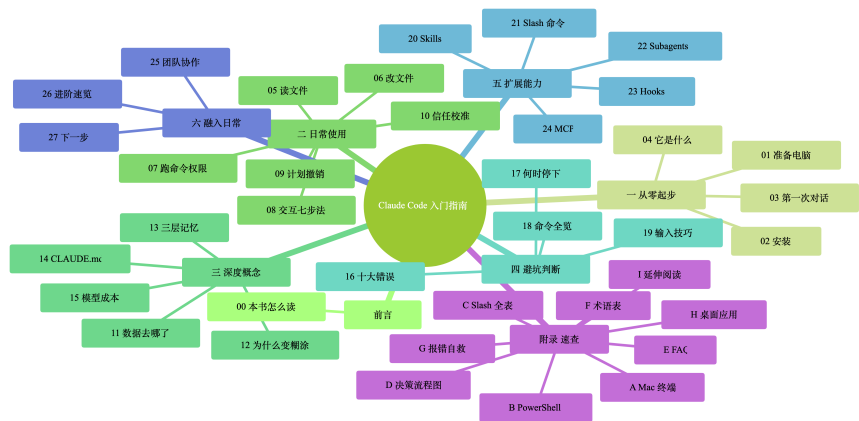
一份推荐的 90 天学习节奏	352
附录 J: Claude Opus 4.7 新手指南	353
J.1 一句话概括 Opus 4.7	353
J.2 你会直接感受到的差异	354
J.3 新概念: effort 档位	355
J.4 在 Claude Code 里用 Opus 4.7	356
J.5 prompt 写法的几个微调	356
J.6 新手最容易踩的 5 个坑	357
J.7 什么时候该用 Opus 4.7?	358
J.8 官方权威资源	359
本节小结	359

第 0 章：本书怎么读

我是马力，这本书的作者。欢迎。正式开始前，先花 10 分钟搞清楚——这本书是给谁写的、怎么读最省力、读完你能得到什么。

全书地图（一张图看清整本书脉络）

下图是整本书的结构。先用 30 秒扫一眼，你会对“我要读的是什么”有个整体感——后面每章开头也会有一张更细的“章首导览”，但这张图只画一次，它是整本书的坐标系。



图较宽——移动端可左右滑动查看完整结构。

🎯【主线】——本章必读核心

下面 6 节是全书导读的骨架。读完即可进入第 1 章。预估 10 分钟。

0.1 这本书适合谁

你同时满足下面几乎全部条件时，这本书最适合你：

- 不会编程——从没写过一行代码，也不想专门去学
- 没用过命令行——打开“终端”或“PowerShell”会发懵，甚至不知道它们在哪、长什么样
- 没用过 AI 编程工具——Cursor、Copilot、Windsurf 这些名字对你陌生的
- 有真实的工作场景想解决——比如整理一堆下载的文件、把杂乱的表格清洗一下、把一堆资料总结成一篇文档、给自己做点小自动化
- 用 Mac 或 Windows 电脑（Linux 用户本书不专门覆盖，可自行类推）

如果你已经会编程或熟悉终端，本书前半部分对你会略显啰嗦——但第 10、12、14 章（信任校准、上下文、CLAUDE.md）对所有层次的读者都有价值。

0.2 读完你能做什么

认真读完并完成所有动手任务后，你能：

1. 独立安装并启动 Claude Code
2. 在终端里发起有效对话——让它读文件、改文件、跑命令
3. 读懂权限提示，知道什么时候该允许、什么时候该拒绝
4. 在 Claude 变糊涂时知道该怎么办
5. 写一份自己的 CLAUDE.md，让 Claude 长期记住你的偏好
6. 识别 10 个新手常踩的坑
7. 至少装一个扩展能力（Skill、自定义命令 或 MCP）
8. 明白什么时候该停下来、改手工做

简言之：从“不敢打开终端”到“能独立用 Claude Code 解决日常工作任务”。

0.3 本书用到的记号

全书使用这几种固定记号：

记号	含义
	危险操作或容易出错的地方，停下来看清楚
	小贴士，可选但有理
	重点，必须记住
	常见疑问
反引号里的字	命令、代码、文件名、按键
你：…	示例对话中你说的话（粗体）
Claude：…	Claude 的回复

当你看到类似 `ls ~/Desktop` 这种代码方框里的文字，那是需要你一字一字打进终端的命令，不是普通的中文阅读内容。

还有两个贯穿全书的重要标志——告诉你一段内容是“必读”还是“可跳过”：

-  **【主线】** = 本章必读核心。读完就合格，可以进下一章
 -  **【支线】** = 可选深入。第一遍读可以直接跳过，用熟了再回头看
- 比如本章——你正读到的就是“主线”。本章末尾有两条“支线”，第一遍不读完全没问题。

0.4 章节地图

全书 27 章 + 10 个附录，分 6 个部分：

部分	章节	讲什么
一、从零起步	Ch 0-4	电脑怎么准备、怎么装、第一次对话、Claude Code 是什么
二、日常使用	Ch 5-10	读文件、改文件、跑命令、权限机制、审核、信任校准
三、深度概念	Ch 11-15	隐私与数据、上下文原理、三层记忆、CLAUDE.md、模型与成本
四、避坑与判断力	Ch 16-19	10 大错误、何时停下、常用命令、输入技巧
五、扩展能力	Ch 20-24	Skills、自定义命令、Subagents、Hooks、MCP
六、融入日常	Ch 25-27	团队协作、进阶速览、下一步

 强烈建议按顺序读——后面的章节都假设你已经掌握前面的内容。每章的“主线”读完就行，支线可以最后再回来翻。

0.5 别搞错——还有两种“会干活的 Claude”

Anthropic（开发 Claude 的公司）目前有三种不同形态的产品都能帮你“读文件、改文件、跑任务”，一定要对号入座：

产品	界面	适合谁
① Claude Code（终端版）	命令行窗口，只能打字	本书主讲——愿意学一点基础终端的人
② Claude Code 桌面应用	图形界面	不爱切终端、想多会话可视化编排的人
③ Claude Cowork	全图形，不写代码	完全不想学终端、任务是整理文件/做表格/处理文档的知识工作者

 一句话分流：

- 愿意学点基础终端，换一套威力大的长期工具 → 继续读本书
- 只想让 AI 帮整文件、做表、写文档，对终端没兴趣 → 去 claude.com/cowork 用 Cowork，和本书教的类似但更直观

我们不强留你——你节省时间，对大家都好。

0.6 每章都有的三件套

从第1章起，每章主线末尾都有这三件套，顺序固定：

1. 本章小结——合上书也能复述的几条要点
2. 动手任务——带预估时间、步骤、成功标志
3. 如果你卡住了——列最可能的 2-3 个卡点 + 解决办法

动手任务一定要做。光看不练，真打开终端时会发现手是陌生的——动手任务本身就是最好的自测，能做出来就是真懂了。

本章小结

- 这本书给完全零基础的普通人，用 Mac 或 Windows
- 读完能从“不敢打开终端”到“独立解决日常工作任务”
- Claude Code 有三种形态：终端版（本书主讲）、桌面应用、Cowork——看清楚自己该用哪个

- 每章分 🌟 主线 / 🌿 支线——第一遍读只读主线就够
 - 每章主线末尾有三件套：小结 / 动手任务 / 卡住了
 - 按顺序读最省力
-

动手任务

任务 1：想清楚你为什么读这本书（2 分钟）

合上书，对自己说一遍：“我为什么要读这本书？我期望学完能具体解决什么问题？”

成功标志：你能具体说出 1 件你最近想让 AI 帮你处理、但不知道怎么做的事。比如：“把我下载文件夹里 300 个 PDF 按主题分类”、“把 10 个 Excel 合并成一个”、“把一年的会议笔记整理成摘要”。

为什么做这个任务：带着具体目标读书，比泛泛地“学习一下”效率高 5 倍。

如果你卡住了

症状：你读完本章觉得“术语太多、好复杂”

- 原因：Ch 0 会提到终端、PowerShell、Cowork 等好几个名字，你可能一个都没听过。这正常。
- 解决：别担心，后面都会讲。特别是第 1 章会从“什么是终端”从零讲起。直接往下读就行。

症状：不确定自己是不是“真零基础”

- 原因：你可能用过命令行一次两次、或者跟着教程跑过一段代码。
 - 解决：这仍然算“零基础”。只要你没把终端当日常工具用，本书就适合你。顶多前两章读得快一点。
-

主线到这里结束。如果你想进第 1 章，往下翻，直接跳到下一章的文件。

下面是两条支线——对本书怎么用得更好感兴趣的人可以看看，第一遍读完全可以跳过。

🌿 【支线】—— 可选深入（学有余力再看）

下面两小节是关于“怎么读这本书”的补充建议和本书背景。第一遍读时完全可以直接跳到第 1 章；等你读完前三部分、对本书节奏有感觉了，再回来看看。

支线 0.A：读这本书的节奏建议

这段讲什么：怎么分配阅读时间、哪些章可以跳、遇到不懂怎么办。什么时候回头读：读完 Ch 1-2 觉得信息量大想规划一下，就回来看这段。

- 每次读 1-2 小时，别一口气灌完——认知负担过载会让你记不住
- 第一部分（Ch 0-4）尽量连着读完——心理门槛都在这几章，一鼓作气过
- 每章先读 🎯 主线、完成动手任务，再决定要不要读当章支线
- 进阶章节（第五部分 Ch 20-24）可以先跳——用熟了基础再回来
- 附录随时查，不用死记硬背
- 遇到看不懂的术语：先往下读一两段，通常下一段会解释；还不懂，翻附录 F 术语表
- 第二遍阅读：建议在用 Claude Code 1 个月左右时重读——那时你的问题和第一次不一样，支线会突然变得有用

支线 0.B：这本书和其他 Claude Code 教程有啥不一样

这段讲什么：为什么你读这本书，而不是其他同类教程。什么时候回头读：好奇作者视角 / 想对比其他资源的时候。

市面上已有不少 Claude Code 教程（英文版 Ultimate Guide、中文 awesome-claudecode-tutorial 等），但它们几乎都有同一个问题：默认读者已经会编程或熟悉终端。

本书的差异化取舍：

维度	大多数同类教程	本书
前置知识	假设会 git、熟悉 API 等	从“什么是终端”开始讲
前几章密度	快节奏，新手容易卡	前 5 章颗粒度最细，每步都手把手
示例类型	React 组件、后端 API	日常任务（整文件、做表格、写文档）
结构	线性长文	主线 / 支线双轨
CLAUDE.md 这类高杠杆内容	常常一句“跑 /init”带过	整整一章讲透，含反模式

简单讲：如果你已经是程序员，本书显得啰嗦；如果你是零基础，其他教程读起来像天书。这本书就是填“零基础到入门”这段路——这也是我（马力）写它的原因。

好了。准备好就进入第 1 章，我们从最基础的“什么是终端”讲起。
——马力，2026 年春

第一部分 · 从零起步

第 1 章：先把电脑准备好

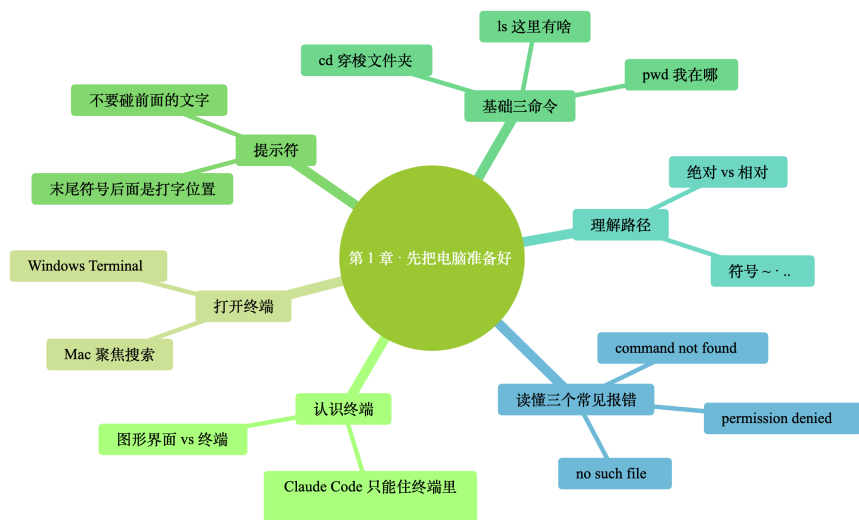
本章在全书的位置：第一部分 · 第 1 章 / 预估阅读+动手时长：主线 40-50 分钟 + 支线 15-20 分钟

学完能做什么（主线）：独立打开终端、看懂提示符、用 `pwd` / `ls` / `cd` 在文件夹间穿梭、理解“路径”、读懂三个常见报错。这些是后面所有章节的前提。

开场三问

- 你会遇到什么问题：下一章就要用终端装 Claude Code——连终端都没打开过的话，那一章会变噩梦。
- 不读这章会踩什么坑：看到 `$` `%` `>` 这些符号不知道干什么；看到“请 `cd` 到你的工作文件夹”手足无措；看到 `command not found` 慌。
- 读完你会多会什么事：像切换微信和浏览器那样自然地在终端里进出文件夹、看看里面有啥。

⚠ 本章适用“颗粒度最细”原则——主线每一步都会写出来，包括“按回车”这种级别。真零基础请逐字跟着做。



🎯 【主线】—— 本章必读核心

下面 6 节 + 动手任务，让你从“没见过终端”走到“能在终端里自由行走”。预估 40-50 分钟。

1.1 认识终端

平时你用鼠标点图标、打开微信、用 Word——这叫图形界面。图形界面很直观，但有个缺点：只能做软件作者做好的功能。

终端（terminal）是另一种和电脑打交道的方式——一个只能打字的窗口。你在里面一行一行打命令，电脑一条一条回复你。这样你能让电脑干很多图形界面里做不到的事，比如“把这个文件夹里所有 PDF 的文件名前加上今天的日期”、“统计这 30 个表格里‘已付款’的条数”——这些事图形界面要么做不了，要么得点几百下。

💡 生活化比喻：图形界面像外卖 App（菜单有什么你点什么），终端像直接跟后厨说话（你说啥他给你做啥）。Claude Code 就是后厨里那位能听懂你

自然语言的大厨——所以它只能住在终端里。不学会打开终端，就没法用 Claude Code。

好消息：本书只需要你学 5-6 个最基础的命令，就能走完所有章节。你不需要成为命令行专家。

两个平台的终端叫什么名字：

平台	名字	里面跑什么
Mac	终端 (Terminal)，系统自带	新系统用 zsh，老系统用 bash (对用户没区别)
Windows	Windows Terminal (新版系统)，或直接用 PowerShell (老版系统)	都跑 PowerShell 命令语言

两个平台的终端外观不同、少数命令有差异，但概念完全一样。本书涉及平台差异时会分段讲。

1.2 打开你的终端

Mac 用户

步骤 1：按住 **Command** 键 (⌘，空格键左边)，再按空格键。

你会看到：屏幕中央弹出一个搜索框。这是 聚焦搜索 (Spotlight) ——Mac 自带的快速搜索工具。

步骤 2：搜索框里打“终端”或英文 **Terminal**。

步骤 3：按回车键。

你会看到：弹出一个窗口——通常白色底黑色字（也可能反过来，看系统主题）。第一行类似：

```
Last login: Sat Apr 19 14:32:10 on ttys000
你的名字@你的电脑名字 ~ %
```

👉 这就是终端。末尾那个 **%** 符号（或 **\$**，看系统版本）后面有一个闪烁的光标——它在等你打字。

💡 还有别的打开方式 / 想把图标固定到 Dock？见本章末尾 🌱 支线 1.A。

Windows 用户

Windows 要先分清自己的版本。按 **Windows 键 + R**，弹出“运行”小窗口，打 **winver** 回车——会弹窗告诉你 Windows 版本。

步骤 1：按键盘左下角的 Windows 键（有 Windows 图标那个键），或点屏幕左下角的开始按钮。

步骤 2：

- Win11 / 新版 Win10：打 **terminal** 或 **终端**
- 老版 Win10（搜不到 terminal）：打 **powershell**

步骤 3：点击搜索结果里的 Windows Terminal（或 Windows PowerShell）。

你会看到：弹出一个深蓝或黑色窗口，内容类似：

```
PS C:\Users\你的名字>
```

 前面 **PS** 是 PowerShell 的缩写。末尾的 **>** 是提示符。

 本书后面所有 Windows 操作都在 PowerShell 里进行（不管是 Windows Terminal 里的 PowerShell 还是直接打开的 PowerShell）。

 **千万别用“命令提示符”（cmd.exe）**：Windows 还有一个更老的终端叫“命令提示符”，标题栏写“命令提示符”或“cmd”。它的命令和 PowerShell 不一样，本书不覆盖。如果你不小心开了它，关掉重新开 PowerShell 或 Windows Terminal。

1.3 第一次使用：看懂提示符，输入 **pwd**

认识提示符

打开终端后，每一行末尾都有一段类似这样的文字——提示符（prompt）：

- Mac（新系统）：**你的名字@你的电脑 ~ %**
- Mac（老系统）：**你的名字@你的电脑 ~ \$**
- Windows PowerShell：**PS C:\Users\你的名字>**

提示符告诉你“轮到你打字了”。

 **关键就一条**：末尾那个符号（**%** 或 **\$** 或 **>**）后面的光标位置，就是你该打字的地方。不要打到前面的文字里去，也不要动前面已经显示的内容。

 想知道提示符每一段分别是什么意思？见  支线 1.C。

输入你的第一个命令：pwd

步骤 1：确认光标在提示符后面闪。步骤 2：一个字一个字打 `pwd`（全小写）。

步骤 3：按回车键。

你会看到：

- Mac 显示： `/Users/你的名字`
- Windows 显示： `C:\Users\你的名字`

这是什么意思

`pwd` 是“print working directory”的缩写，意思是“打印当前所在文件夹”。

终端刚打开时默认把你放在 家目录（home directory）里——也就是 `/Users/你的名字`（Mac）或 `C:\Users\你的名字`（Windows）。你的桌面、下载、文档都在家目录下面。

💡 为什么一上来就要知道“我在哪”：你后面所有命令都是相对于当前位置起作用的。“列出这里的文件”的“这里”就是 `pwd` 返回的那个地方。每次做事前打一下 `pwd` 确认位置，是个好习惯。

1.4 在文件夹间穿梭：ls 和 cd

这两个命令一起用——`ls` 看当前文件夹里有什么，`cd` 进入/离开文件夹。

ls ——列出当前文件夹里的东西

打 `ls`，按回车。

Mac 你会看到（一行或几行简洁列表）：

Desktop	Documents	Downloads	Music	Pictures	...
---------	-----------	-----------	-------	----------	-----

Windows PowerShell 你会看到（带列的表格）：

Mode	LastWriteTime	Length	Name
d-----	2026/4/10 14:00		Desktop
d-----	2026/4/10 14:00		Documents

左边 `Mode` 列里有 `d` 就是文件夹，没 `d` 就是文件。

`ls` 是“list”（列出）的缩写。Windows 也可以用 `dir`（效果相同），但本书统一用 `ls`，两平台都认。

💡 想看隐藏文件？想看文件大小？见 🌿 支线 1.B。

`cd` ——进入 / 离开文件夹

`cd` 是“change directory”（切换文件夹）的缩写。三种用法：

① 进入某个子文件夹

```
cd Desktop
```

（`cd` 和 `Desktop` 之间必须有空格；Mac 大小写敏感，`desktop` 进不去桌面）
按回车后，提示符会变——末尾出现你刚进的文件夹名。再打 `ls` 就是桌面里的东西了。

② 退回上一层

```
cd ..
```

（`cd` 和 `..` 之间有空格）

`..` 是“上一层文件夹”的符号，两平台都一样。

③ 一步回家

```
cd ~
```

或直接 `cd` 什么都不带。无论你在哪，都能一步跳回家目录。

💡 一次跳好几层 / 路径里有空格怎么办？见 🌿 支线 1.B。

1.5 理解“路径”

`pwd` 返回的 `/Users/你的名字` 或 `C:\Users\你的名字` ——这种东西叫 路径（path）。

路径就是文件或文件夹在电脑里的地址写法。就像“北京市海淀区中关村大街1号”是一个地址，`/Users/小明/Desktop/笔记.txt` 也是一个地址，表示“小明的家目录里、桌面里、叫笔记.txt 的那个文件”。

绝对路径 vs 相对路径

路径有两种写法：

类型	特征	例子	什么时候用
绝对路径	从最顶层开始写（Mac 以 <code>/</code> 开头，Windows 以 <code>C:\</code> ）；不管当前在哪都能找到	<code>/Users/小明/Desktop/笔记.txt</code> 、 <code>C:\Users\小明\Desktop\笔记.txt</code>	告诉 Claude Code 在哪操作、写自动化脚本时——更保险
相对路径	从当前位置开始写	<code>Desktop/笔记.txt</code> （相对于家目录）、 <code>../其他</code>	手打命令时——更省力

三个必须认识的路径符号

这三个符号在路径里经常出现，一定要认识：

符号	含义	例子
<code>~</code>	家目录	<code>~/Desktop</code> = 你的桌面（不管当前在哪）
<code>.</code>	当前文件夹	<code>./笔记.txt</code> = 当前位置里的笔记.txt
<code>..</code>	上一层文件夹	<code>../其他文件夹</code>

💡 Mac 和 Windows 路径的所有细节差异 / `~/` 和 `./` 别混淆——见 🌱 支线 1.C。

1.6 收尾：退出终端 + 三个常见报错

退出终端

两种方式都行：

1. 打 `exit` 回车 —— 干净退出
2. 直接关窗口（Mac `Cmd + Q` / Windows 点右上角 `×`）

终端里没有“未保存的修改”，关窗口不会丢东西。

三个最常见的报错——读懂就不慌

报错 1： `command not found`（Mac） / 不是内部或外部命令（Windows）

```
zsh: command not found: lss
```

意思：你打的这个词终端不认识。99% 是打错字。

怎么办：对比原命令——少个字母、大小写错、多/少空格，都会这样。

报错 2: no such file or directory (Mac)/ Cannot find path (Windows)

```
cd: no such file or directory: Desktopp
```

意思：你想进的文件/文件夹不存在。

怎么办：

1. 先 `pwd` 确认当前位置
2. 再 `ls` 看这里实际有什么
3. 对照拼写——Mac 用户特别检查大小写（`desktop` $\neq$ `Desktop`）

报错 3: permission denied

```
permission denied: /System/somefile
```

意思：这地方你没权限动。

怎么办：

- 不要急着用 `sudo` 或其他提权命令——很多时候是你本来就不该去那个地方
- 确认你想动的是不是自己的文件（家目录 `~` 下通常都有权限）
- 真遇到需要权限的情况，第 7 章会讲 Claude Code 的权限机制

本章小结

- 终端是只能打字的窗口，Claude Code 只能住在里面
- Mac 叫“终端”、Windows 推荐“Windows Terminal + PowerShell”；不要用 `cmd.exe`
- 提示符末尾符号（`%` / `$` / `>`）后面就是你打字的地方
- 三个基础命令：`pwd`（我在哪）/ `ls`（这里有啥）/ `cd`（去/回）
- 路径是文件的地址；绝对路径从根开始、相对路径从当前开始
- 三个符号：`~`（家）/ `.`（当前）/ `..`（上一层）
- 三个报错：`command not found`（打错字）/ `no such file`（不存在）/ `permission denied`（没权限）

动手任务

任务 1：打开终端并找到自己（3 分钟）

步骤：用你平台的方法打开终端 → `pwd` 回车 → 对照输出。成功标志：终端显示你的家目录路径（`/Users/你的名字` 或 `C:\Users\你的名字`）。失败了怎么办：报 `command not found` 检查是不是打成大写 `PWD` 或多打了空格。

任务 2：进桌面看看有啥（5 分钟）

步骤：

1. `cd ~/Desktop` 回车
2. 确认提示符变了（末尾出现 `Desktop`）
3. `ls` 回车
4. 在输出里找一个你认识的文件或文件夹名

成功标志：在 `ls` 输出里认出至少一个桌面上的东西。失败了怎么办：`cd ~/Desktop` 报 `no such file`，你的桌面可能有别的名子。`cd ~` 回家后 `ls` 看看实际叫什么。

任务 3：往下钻两层再回来（5 分钟）

步骤：

1. 右键桌面 → 新建一个文件夹叫 `测试`（图形操作没问题）
2. 终端里 `cd ~/Desktop/测试`
3. `pwd` 确认你在 `~/Desktop/测试`
4. `cd ..` 回到桌面
5. `cd ..` 再回到家目录

成功标志：提示符一步步变化，最后回到家。

如果你卡住了

症状 A：终端打开后是乱码或看不到提示符

- 原因：主题或字体不对。
- 解决：Mac 点菜单栏“终端 → 偏好设置”选默认主题；Windows Terminal 设置里选“Campbell”或“One Half Dark”。

症状 B：Windows 打 `ls` 报 `command not found`

- 原因：极个别老版 PowerShell 不认 `ls`。
- 解决：改用 `dir`，或升级到 PowerShell 7+。

症状 C：中文文件夹名打不进去

- 原因：输入法问题。
- 解决：切英文输入法再打；或先 `cd` 进父文件夹 `ls` 看真实名字后复制粘贴。

还是不行？→ 见附录 G（常见报错自救），或直接 Google 那条英文报错，通常第一条结果就有答案。

主线到这里结束。你已经有了进入第 2 章的所有基础。下面是 3 条支线，挑你感兴趣的看；第一遍读完全可以跳过，直接去第 2 章。

【支线】—— 可选深入（学有余力再看）

本章三条支线分别讲：打开终端的另一种方式、命令用得更灵活、路径的两个细节坑。

不读不影响后续章节。建议用熟了 Claude Code（比如第 2-5 章做完）再回头。

支线 1.A：打开终端的另一种方式，以及把它固定到 Dock / 任务栏

这段讲什么：除了聚焦搜索和开始菜单，还能从访达/开始菜单直接找到终端；以及把它钉在屏幕下方，下次一键打开。什么时候回头读：你发现自己经常要打开终端、想省点事的时候。

Mac：从访达找终端

1. 点屏幕顶部左上角苹果图标  → 选“访达”
2. 菜单栏点“前往” → 选“应用程序”
3. 在应用程序里向下滚，找到“实用工具”文件夹双击进入
4. 找到“终端”，双击打开

和聚焦搜索打开的是同一个应用，没有区别。

Mac：把终端图标固定到 Dock

终端打开后，图标会出现在屏幕底部的 Dock（应用栏）上。右键这个图标 → 选“选项” → 勾选“在程序坞中保留”。以后直接点 Dock 上的图标就能开，不用每次搜索。

Windows：把终端固定到任务栏

Windows Terminal 打开后，图标会出现在屏幕底部的任务栏上。右键这个图标 → 选“固定到任务栏”。以后直接点就能开。

支线 1.B：命令用得更灵活

这段讲什么：`ls` 看隐藏文件、`cd` 一次跳多层、路径里带空格怎么办。什么时候回头读：装 Claude Code 时看到 `.claude/` 这种以 `.` 开头的文件夹；或遇到空格路径报错时。

看到隐藏文件：`ls -a` / `ls -Force`

默认 `ls` 不显示以 `.` 开头的“隐藏文件”——它们通常是系统或工具的配置文件，平时不显示避免干扰。

- Mac: `ls -a`（`-a` 是“all”——全部）
- Windows PowerShell: `ls -Force`

会看到一些原来看不见的东西：`.zshrc`（Mac 终端配置）、`.claude/`（Claude Code 的项目配置目录，本书后面会用到）、`.git/`（git 版本控制）等。

一次跳好几层：`cd` 多层/路径

不用一层一层 `cd`，可以写一个多层路径一步跳到：

```
cd Documents/工作/2026
```

等于先进 `Documents`、再进 `工作`、再进 `2026`，三步合一。

绝对路径也行：


```
cd /Users/小明/Desktop/练习      # Mac
cd C:/Users/小明/Desktop/练习    # Windows (PowerShell 也认
`/`)
```

路径里有空格：加双引号

文件夹名有空格时，必须用双引号包起来，不然空格会被当成命令分隔符报错：

```
cd "My Documents/Work Stuff"
```

错误示范：`cd My Documents`——终端会以为你想进一个叫 `My` 的文件夹，然后报错“no such file”。

支线 1.C：路径的两个细节坑

这段讲什么：Mac vs Windows 路径系统的完整差异，以及一个最常被混淆的写法——`~/Desktop` 和 `./Desktop` 是完全不同的东西。什么时候回头读：同时用两种电脑时；或混用 `~/` 和 `./` 导致报错时。

Mac 和 Windows 路径系统的完整对比

维度	Mac	Windows
分隔符	<code>/</code> （正斜杠）	<code>\</code> （反斜杠）或 <code>/</code> （PowerShell 两个都认）
家目录位置	<code>/Users/你的名字</code>	<code>C:\Users\你的名字</code>
家目录缩写	<code>~</code>	<code>~</code> 或 <code>\$HOME</code>
路径大小写	区分（ <code>Desktop</code> $\neq$ <code>desktop</code> ）	不区分
盘符	没有——所有东西在 <code>/</code> 下	有（ <code>C:\</code> 、 <code>D:\</code> 、 <code>E:\</code> ）
根目录	<code>/</code> （唯一一个）	每个盘符都有自己的根

⚠ Mac 的大小写敏感最坑人：`cd desktop` 和 `cd Desktop` 在 Mac 上是两个完全不同的操作（一个报错、一个成功）；Windows 上两个等价。跨平台复制别人的命令时特别注意。

📌 本书写命令时统一用 `/` 做分隔符——Mac 必须用、Windows PowerShell 也认，这样一套命令两平台都能跑。

~/Desktop 和 ./Desktop 别混淆
两个路径写法看起来像，含义差远了：

写法	含义	找得到
<code>~/Desktop</code>	家目录下的 Desktop	无论当前在哪都能找到
<code>./Desktop</code>	当前文件夹下的 Desktop	看当前文件夹里是否真有

举例：

```
# 你现在在 ~/Documents
pwd                               # 显示：/Users/你/Documents
cd ~/Desktop                     # ✅ 成功—跳到 /Users/你/Desktop
cd ./Desktop                     # ❌ 报错—Documents 里没有叫 Desktop 的子
文件夹
```

📌 记法：`~` 是“家”，`.` 是“这里”。“回家”和“去这里”明显不一样。

好了，第 1 章到这里。进入第 2 章——装 Claude Code。

第 2 章：安装 Claude Code

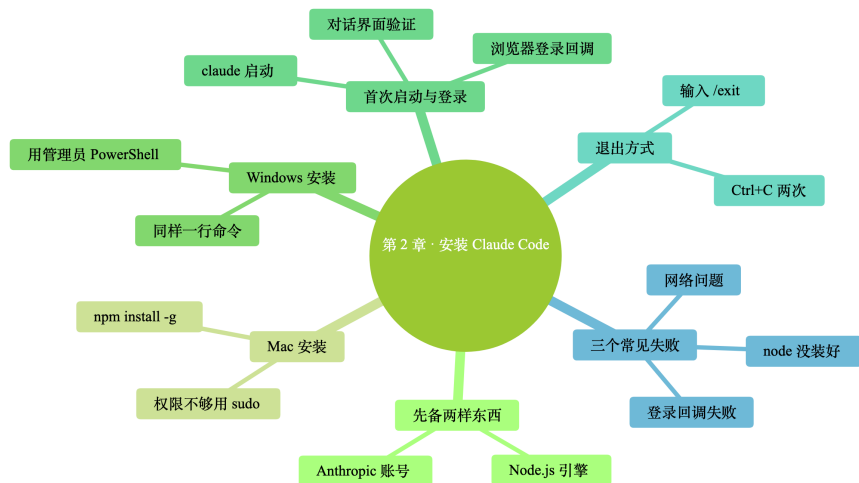
本章在全书的位置：第一部分 · 第 2 章 / 预估阅读+动手时长：主线 30-40 分钟 + 支线 10-15 分钟

前置章节：必须先完成第 1 章——下面每一步都要在终端里操作。

学完能做什么（主线）：你的电脑上已经装好 Claude Code，能用 `claude` 命令启动它，能登录自己的账号并看到它回话。

开场三问

- 你会遇到什么问题：装不上、装上了但启动失败、账号登录一直转圈——这三类坑是新手装 Claude Code 最常见的。
- 不读这章会踩什么坑：直接按网上教程敲命令，遇到报错不知道什么原因；装了一半放弃。
- 读完你会多会什么事：能独立走完安装和登录的完整流程，遇到失败能判断是哪一步出问题。



🎯 【主线】—— 本章必读核心

下面 5 节 + 动手任务，走完你就能启动 Claude Code 了。预估 30-40 分钟。

2.1 装之前：两件事要先备好

装 Claude Code 之前，你需要两样东西准备好。我们先解释这两样是什么、为什么需要。

① 一个 Anthropic 账号

Anthropic 是做 Claude 的公司。Claude Code 需要登录 Anthropic 账号才能用——因为它每次对话都要把你说的话发到 Anthropic 的服务器，服务器处理完再返回，这就需要知道“是你在用”。

注册方式：浏览器打开 <https://claude.ai> → 点“Sign up” → 用邮箱注册。如果你之前用过 Claude 的网页版、订阅过 Claude Pro/Max，那个账号就能直接用。

付费问题的简短版：Claude Code 有免费额度（对轻度使用够用），想用更多就订阅或者按 API 计费。新手先用免费额度试，不会产生扣费。深入的成本问题第 15 章专讲。

② Node.js（一个叫“node”的东西）

Claude Code 是一个用 JavaScript 写的命令程序——JavaScript 这种程序在电脑上跑起来，需要一个叫 Node.js（或简称 node）的“引擎”。就像 Word 文档要 Word 打开、PDF 要 Acrobat 打开一样，JavaScript 程序要 Node.js 打开。

💡 不用搞懂 Node.js 是啥，只要知道它是“Claude Code 运行所需的引擎”。装一次就能一直用。

怎么知道自己电脑有没有装 Node.js？

打开终端（见第 1 章 1.2 节），输入：

```
node -v
```

按回车。

- 已经装了：会显示类似 `v20.11.0` 的版本号——版本号 18 或更高都可以，跳到 2.2 节
- 没装：会报 `command not found`（Mac）或 不是内部或外部命令（Windows）——需要先装 Node.js

怎么装 Node.js：浏览器打开 nodejs.org，点左边绿色的“LTS”按钮（LTS = “长期支持版”，最稳定）下载安装包。下载完双击安装，一路点“下一步/继续”即可。

装完关掉终端重新打开（让新装的 node 生效），再打一次 `node -v`，看到版本号就说明成功了。

2.2 Mac 上安装 Claude Code

准备好 Node.js 和账号后，装 Claude Code 本体就很简单——一行命令。

安装

打开终端，输入：

```
npm install -g @anthropic-ai/claude-code
```

按回车。

这行命令在做什么：

- `npm` 是 Node.js 自带的“应用商店”（Node Package Manager，节点包管理器）
- `install` = 安装
- `-g` = global（全局），意思是装到全电脑都能用的位置，而不是只装到当前文件夹
- `@anthropic-ai/claude-code` = 要装的东西的名字

你会看到：终端里刷一串文字，可能会有一些黄色或绿色的进度条。这正常，说明它在下载。等它跑完——最后一行通常是 `added XXX packages in XXs` 之类。时间约 30 秒到 2 分钟，看网速。

可能遇到的权限问题

如果跑完后看到 `permission denied`（权限被拒绝），说明 `-g` 全局安装需要管理员权限。改用：

```
sudo npm install -g @anthropic-ai/claude-code
```

`sudo` = super user do（以管理员身份做），会弹出让你输 Mac 登录密码。输入时屏幕上不会显示任何字符（连 `*` 都不显示），这是故意的，正常打完密码回车就行。

2.3 Windows 上安装 Claude Code

同样一行命令，在 PowerShell 里：

```
npm install -g @anthropic-ai/claude-code
```

按回车。

你会看到：和 Mac 一样，终端里会刷一串文字显示下载进度。等跑完。

可能遇到的权限问题

Windows 上如果报权限错误，不要用 `sudo`（Windows 没有这个命令）。改为：

1. 关掉当前 PowerShell 窗口
2. 找到 Windows Terminal 或 PowerShell 的图标

3. 右键 → 选“以管理员身份运行”

4. 在新打开的管理员窗口里重新跑

```
npm install -g @anthropic-ai/claude-code
```

管理员 PowerShell 的标题栏会写“管理员”字样，辨认清楚。

2.4 登录并验证

装完后，Claude Code 已经在你电脑里了。启动它。

启动

在终端（Mac）或 PowerShell（Windows）里随便哪个位置，输入：

```
claude
```

按回车。

第一次启动时：Claude Code 会引导你登录 Anthropic 账号。会发生下面的事情：

1. 终端里显示一个欢迎信息和一个登录链接（URL）
2. 你的默认浏览器自动打开那个链接（或者终端提示你手动复制到浏览器）
3. 浏览器里，登录你的 Anthropic 账号
4. 登录成功后浏览器显示“You can close this window”（可以关闭此窗口）
5. 切回终端——Claude Code 应该已经显示登录成功，进入对话界面

Claude Code 界面长什么样

登录后，终端会变成 Claude Code 的对话界面，大致是这样：

```
* Welcome to Claude Code
```

```
/help for help, /status for status
```

```
>
```

最下面那个 > 是 Claude Code 的输入框（和终端的提示符不是一回事）。光标在它后面闪，等你打话。

验证它能工作

打一句话进去：

你好，告诉我你能做什么

按回车。

你会看到：Claude 开始在你的终端里一行一行吐出回复，讲它自己。这就说明装成功了、登录成功了、能正常用了。

退出

按两次 `Ctrl + C`（同时按 `Ctrl` 和 `C`，按两次），或者输入 `/exit` 回车。

💡 之后每次用 Claude Code，直接在终端里 `cd` 到你的工作文件夹、`claude` 启动就行，不用重新登录（登录信息会保存一段时间）。

2.5 装不上？三个最常见的失败

如果上面某一步挂了，大概率是下面三种情况之一。

失败 1： `command not found: npm` 或 `command not found: node`
意思：Node.js 没装好，或装了但终端没找到。

怎么办：

- 确认你从 nodejs.org 装的是 LTS 版本
- 装完后彻底关掉终端窗口（不是新开标签页，是关窗口），重新开终端
- 再试 `node -v` 和 `npm -v`，都应该显示版本号

失败 2： `npm install` 卡住不动或报 `network error`

意思：网络问题，连不上 npm 的服务器。

怎么办：

- 检查网络（能不能打开别的网站）
- 如果在中国大陆，可能需要切换 npm 源到国内镜像。PowerShell 或 Mac 终端里跑：

```
npm config set registry https://registry.npmmirror.com
```

再重新 `npm install -g @anthropic-ai/claude-code`

- 公司网有防火墙的话，问 IT 同事

失败 3：登录浏览器页面跳转失败 / 登录完终端没反应

意思：登录回调没成功关联回终端。

怎么办：

1. 终端里按 `Ctrl + C` 退出 Claude Code
2. 重新 `claude` 启动
3. 登录页面出现时，注意看终端有没有显示“可以手动粘贴”的选项——把浏览器登录成功后 URL 里的一段代码复制到终端

如果反复不行，登录官网 claude.ai 确认网页版自己能登进去（排除账号问题）。

本章小结

- 装 Claude Code 前要备好两样：Anthropic 账号 + Node.js 引擎
- 装命令一行：`npm install -g @anthropic-ai/claude-code` (Mac 可能需要前面加 `sudo`)
- 启动命令：`claude`
- 第一次启动会自动引导浏览器登录，登完切回终端
- 退出按 `Ctrl + C` 两次或 `/exit`
- 三个最常见失败：Node.js 没装好、网络问题、登录回调失败

动手任务

任务 1：检查或安装 Node.js (5–10 分钟)

步骤：

1. 打开终端，输入 `node -v`
 2. 看到版本号 (18+) → 跳过本任务
 3. 报错 → 去 nodejs.org 下 LTS 版本安装
 4. 装完关掉终端重开，再 `node -v` 确认
- 成功标志：`node -v` 显示 `v18.x.x` 或更高。

任务 2：安装 Claude Code（2–5 分钟，看网速）

步骤：

1. 终端里跑 `npm install -g @anthropic-ai/claude-code`（报权限错误就 Mac 加 `sudo`、Windows 用管理员窗口重跑）
2. 等它跑完，看到 `added ... packages`

成功标志：无报错、最后一行显示成功安装。

任务 3：启动并发第一条消息（3–5 分钟）

步骤：

1. 终端里 `claude` 回车
2. 按提示完成浏览器登录
3. 回到终端，在 `>` 后面打 `你好` 回车
4. 看 Claude 回你话

成功标志：Claude 给出一段中文回复。做到这一步，你已经成功用上 Claude Code 了 🎉

任务 4：干净退出（1 分钟）

步骤：按两次 `Ctrl + C`，或打 `/exit` 回车。

成功标志：回到你原来的终端提示符（`%` 或 `>`）。

如果你卡住了

症状 A：`npm install` 跑到一半红字报一堆东西

- 最可能原因：权限不够（Mac）或网络（两平台都可能）
- 解决：参考 2.5 节对应的失败 1 / 2

症状 B：浏览器打开登录页后点同意但终端一直转圈

- 最可能原因：登录回调没收到
- 解决：`Ctrl + C` 退出，重新 `claude`，看有没有手动粘贴代码的选项

症状 C：打 `claude` 报 `command not found`，但 `npm install` 明明跑成功了

- 最可能原因：`npm` 的全局安装路径没加进终端的“搜索路径”
- 解决：关掉终端重开；如果还不行，Google 搜“`npm global path not in PATH`”+ 你的系统名字

还是不行? → 见附录 G (常见报错自救), 或查官方安装文档 code.claude.com/docs。

主线到这里结束。你的电脑上已经装好 Claude Code 了。下面两条支线讲计费/隐私的快问快答, 以及除了 npm 之外的其他安装方式, 想了解可以看看。

🌿 【支线】—— 可选深入 (学有余力再看)

🌿 支线 2.A: 计费和隐私——几个最关心的问题 (简答版)

这段讲什么: 你现在最容易担心的几个问题的简短回答。什么时候回头读: 装完开始用时脑子里冒出“这玩意儿会乱扣费吗 / 我的文件会不会被偷看”就回来看这段。深入的版本分别见第 11 章 (隐私) 和第 15 章 (成本)。

Q: 用 Claude Code 会不会乱扣我钱? A: 不会。免费额度用完它会提示你, 不会偷偷扣。订阅了 Claude Pro/Max 的按套餐扣, 用 API key 的按 token 扣——两种模式你都能看到明细。

Q: 我打进去的话、让它看的文件, 会被 Anthropic 偷看/用来训练模型吗? A: 不会用来训练模型 (Anthropic 官方承诺)。会短暂存在服务器上用来响应你, 30 天后删除。传输全程加密。

Q: 那 Anthropic 员工理论上能看到吗? A: 技术上有访问控制, 但不是绝对不可能 (极少数 abuse 检测场景)。如果你的文件特别敏感 (含银行卡、密码、未公开的机密), 不要用 Claude Code 处理。

Q: 公司电脑上能用吗? A: 得看你公司政策。有些公司禁止用云端 AI 工具处理公司代码/数据。用之前问一下 IT。

更深入的版本见第 11 章 (数据流与隐私) 和第 15 章 (模型与成本)。

🌿 支线 2.B：除了 npm 还有别的装法

这段讲什么：Mac 的 Homebrew、Windows 的 winget、或者用 pnpm 等替代方式。什么时候回头读：你已经在用 Homebrew / winget / pnpm 管理其他工具，想统一。

Mac：用 Homebrew

如果你已经装了 Homebrew（Mac 上的另一个“应用商店”），可以：

```
brew install anthropic/tap/claude-code
```

好处：Homebrew 会自动处理路径、升级更方便（`brew upgrade`）。

Windows：用 winget

Windows 11 自带 winget（Windows 包管理器）：

```
winget install Anthropic.ClaudeCode
```

（命令名以官方最新为准，查 code.claude.com/docs）

其他 Node.js 包管理器

如果你用 `pnpm` 或 `yarn` 而不是 `npm`，对应命令：

```
pnpm install -g @anthropic-ai/claude-code  
yarn global add @anthropic-ai/claude-code
```

效果一样。

⚠ 不要同时用多种方式装——会混乱。选一种坚持用。

装好了。下一章开始用——你的第一次对话。

第 3 章：你的第一次对话

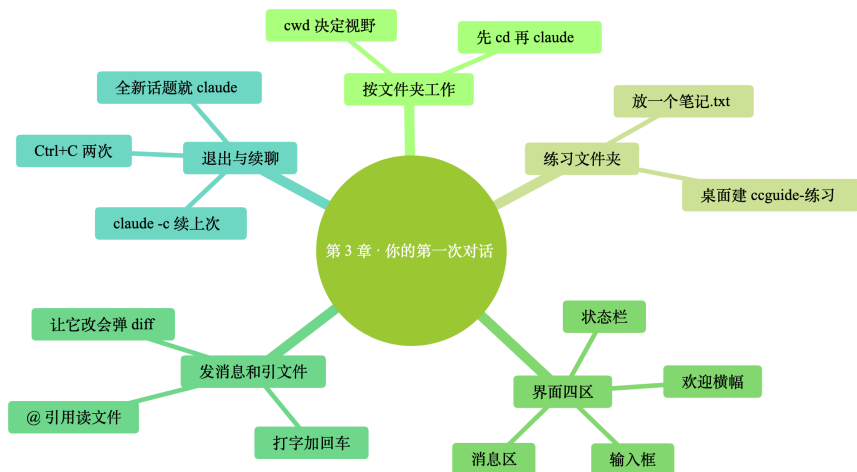
本章在本书的位置：第一部分 · 第 3 章 / 预估阅读+动手时长：主线 30-40 分钟 + 支线 10-15 分钟

前置章节：第 1 章（用终端）+ 第 2 章（装 Claude Code）

学完能做什么（主线）：能进入一个工作文件夹启动 Claude Code，看懂它的界面，发消息、让它读文件、干净退出，并在下次进同一个文件夹时继续之前的对话。

开场三问

- 你会遇到什么问题：装好了 Claude Code，打开它一脸懵——界面上每个东西是什么、该怎么开始说话。
- 不读这章会踩什么坑：不知道要先 `cd` 到对应文件夹再启动（结果 Claude 看不到你的文件）；不知道怎么让它读文件；不知道怎么退出或怎么继续之前的对话。
- 读完你会多会什么事：能在任意文件夹启动 Claude Code，顺畅地和它对话、给它看文件、干净退出；下次还能接着上次聊。



🎯 【主线】—— 本章必读核心

5 节 + 动手任务，让你从“启动成功”走到“会用 Claude Code 处理真实任务”。
预估 30-40 分钟。

3.1 先准备一个“练习文件夹”

Claude Code 是按文件夹来工作的——你在哪个文件夹启动它，它的“视野”就限制在这个文件夹里。它只能看见、只能改这个文件夹里的东西（除非你明确告诉它去别处）。

💡 为什么这样设计：这是一种安全保护。避免你在“系统文件夹”里一启动 Claude Code，它就可能动到操作系统重要文件。你先 `cd` 到你想要它干活的地方，它就只在那儿活动。

动手建一个练习文件夹

我们先建一个专门用来跟着本书练习的文件夹——叫 `ccguide-练习` 放在桌面。

步骤 1：在桌面右键 → 新建文件夹 → 命名 `ccguide-练习`（用图形界面就行）。

步骤 2：在这个文件夹里建一个简单的测试文件。方法：

- **Mac**：双击进入文件夹 → 右键空白处 → 新建文件（如果没这项，可以用 Finder 的“新建文稿”或者下面用终端建）
- **Windows**：双击进入文件夹 → 右键空白处 → 新建 → 文本文档 → 起名 笔记.txt

文件里随便写几句话，比如：

```
今天的待办：
- 买菜
- 给妈打电话
- 写周报
```

保存。这个文件等会儿让 Claude 读。

3.2 进入工作文件夹再启动

打开终端，先 `cd` 到你刚建的文件夹，再启动 Claude Code。

```
cd ~/Desktop/ccguide-练习
claude
```

第一行把你“搬”进练习文件夹，第二行启动 Claude Code。

你会看到：

```
* Welcome to Claude Code

/help for help, /status for status

cwd: /Users/你的名字/Desktop/ccguide-练习

>
```

📌 重点看那行 `cwd:`（current working directory = 当前工作目录）——它必须指向你刚建的 `ccguide-练习`。如果不是，说明你忘了 `cd`，按 `Ctrl+C` 退出，重新 `cd` 后再 `claude`。

⚠ 永远先 `cd` 再 `claude`。这是本书反复强调的第一个好习惯。

3.3 认识 Claude Code 的界面

Claude Code 启动后，你的终端变成了下面几个区域——熟悉它们能让你少走弯路。



① 欢迎横幅

第一次启动时出现，显示版本号、工作目录、可用的快捷提示。之后对话起来就会被滚上去看不到了。

② 消息区

所有的对话（你说的话、Claude 的回复、权限提示、工具调用提示）都在这里按时间顺序往下堆。

③ 状态栏（在一些版本里位于输入框上方）

通常显示：

- 当前用的是哪个模型（Sonnet / Opus / Haiku）
- 当前对话用了多少上下文（Context）——形如 `Ctx: 12%`，这个以后超级重要（第12章专讲）
- 有时还有预估成本

💡 状态栏的更多细节——见 🌱 支线 3.B。

④ 输入框

最下面那个 `>`，后面闪光标的地方——这是你打字的地方。

- 打字 + 回车 = 发送
- `Ctrl+C` 按两次 = 退出 Claude Code
- `Shift+回车` = 换行（消息里想打多行）

3.4 发第一条消息，让它读一个文件

发一条打招呼

在输入框里打：

你好，简单说一下你是什么、能做什么。

按回车。

你会看到：Claude 开始一行一行输出回复——一般是一段简短介绍。你不需要打断它，等它自己说完。说完后光标回到输入框，等你下一句。

🔴 这种一来一回就叫“对话”。你说一句、它回一次、然后等你下一句。中间它不会主动说话。

让它读你之前建的笔记文件

在输入框里打：

@笔记.txt 这个文件里写了什么？用 3 句话概括。

按回车。

关键符号：@——这是 Claude Code 的“引用文件”符号。打 @ 后跟文件名，Claude 就会把这个文件读进对话。你不用自己复制粘贴文件内容。

💡 生活化比喻：@笔记.txt 就像给秘书递过一页纸说“看这张”。和你说“帮我看一下桌上那份文件”不同——@ 是精确指向。

你会看到：Claude 会先告诉你它读了 笔记.txt （可能会简短复述你写的内容），然后给出三句话的概括。

让它改这个文件（初尝权限机制）

再试一句：

```
@笔记.txt 把这份待办清单的每一条前面加上编号 1、2、3。
```

按回车。

你会看到：Claude 不会直接动文件。它会显示一个 diff（修改对比）——左边旧内容、右边它想改成的样子，然后弹出一个选项让你选：

```
Do you want to apply this change?
) Yes
  Yes, and allow similar changes for the rest of this session
  No, tell Claude what to do differently
```

（选项用**↑ ↓ 方向键**选，回车确认）

🔴 这就是 Claude Code 最重要的安全机制：改文件、跑命令之前都会问你。你选 Yes 它才会真的改。第6章和第7章会把这套机制彻底讲透；本章先体验一下。

选 Yes 回车。Claude 会改完文件，并告诉你改完了。

去图形界面打开 `笔记.txt` 看一眼——内容应该真的被编了号。这就是 Claude Code 在干活的样子。

3.5 退出，并在下次继续

干净退出

两种方式：

- 按 `Ctrl + C` 两次（按一下、松开、再按一下）——Claude Code 会确认并退出
- 或者在输入框里打 `/exit` 按回车

你会回到你原来的终端提示符（`%` 或 `>`）。

下次怎么继续

下次你想接着用，只需要：

```
cd ~/Desktop/ccguide-练习
claude
```

注意：直接 `claude` 启动的是全新对话——之前的对话内容 Claude 不记得了。如果你想接着上次聊（保留之前的对话上下文），启动时加一个参数：

```
claude -c
```

或者：

```
claude --continue
```

`-c / --continue` 意思是“continue the previous session”（继续上次会话）。

💡 选哪个？

- 新任务、新话题 → 直接 `claude`（开新对话，也方便让 Claude 轻装上阵）
- 接着上次没完的事 → `claude -c`

第 12 章会解释为什么新任务最好新开对话（涉及“上下文”管理）。现在先记住这两种启动方式。

💡 还有 `claude --resume` 选特定历史对话——见 🌿 支线 3.A。

本章小结

- Claude Code 按文件夹工作——永远先 `cd` 到工作文件夹再 `claude`
- 启动后界面有 4 块：欢迎横幅 / 消息区 / 状态栏 / 输入框
- `@文件名` = 让 Claude 读这个文件
- 改文件前 Claude 会显示 diff 并问你，选 Yes 才真改——这是核心安全机制
- 退出：`Ctrl+C` 两次，或 `/exit`
- 下次继续上次对话：`claude -c`（新话题就直接 `claude`）

动手任务

任务 1：建练习文件夹并启动（3 分钟）

步骤：

1. 桌面建文件夹 `ccguide-练习`（若已建跳过）

2. 里面建文件 `笔记.txt`，写 3 条任意内容
3. 终端 `cd ~/Desktop/ccguide-练习`
4. `claude` 启动

成功标志：启动画面里 `cwd:` 指向 `ccguide-练习`。

任务 2：让它读并概括（3 分钟）

步骤：在输入框打 `@笔记.txt` 用一句话概括这份内容，回车。成功标志：Claude 返回一句符合内容的概括。

任务 3：让它改并审核（5 分钟）

步骤：

1. 输入 `@笔记.txt` 帮我在每行前加一个 `□` 符号（作为待办框）
2. Claude 显示 diff 和确认选项 → 选 Yes 回车
3. 切到图形界面打开 `笔记.txt` 看结果

成功标志：文件里每行前面真的多了 `□`。

任务 4：退出后再用 `-c` 继续（3 分钟）

步骤：

1. `Ctrl+C` 两次退出
2. 终端里 `claude -c`（或 `claude --continue`）
3. 问它：“刚才我们做了什么？”

成功标志：Claude 能回忆起刚才读过 `笔记.txt` 并改过内容。

如果你卡住了

症状 A：启动后 `cwd:` 不是你建的练习文件夹

- 原因：忘了 `cd`，或 `cd` 路径写错
- 解决：`Ctrl+C` 两次退出 → `pwd` 确认在哪 → `cd ~/Desktop/ccguide-练习` 后重新 `claude`

症状 B：打 `@笔记.txt` 后 Claude 说找不到这个文件

- 原因：文件名对不上（大小写、扩展名、是否真在这个文件夹）
- 解决：退出 Claude Code，`ls` 看这个文件夹里实际有什么；对照文件名精确重新试

症状 C：Claude 改文件后选了 Yes，但文件内容没变

- 原因：极少见的文件权限问题，或图形界面没刷新
- 解决：先在图形界面关闭 笔记.txt 再重新打开；还不行就 `ls -la` (Mac) / `ls -Force` (Windows) 看文件有没有被锁

症状 D：`claude -c` 说没有可继续的对话

- 原因：你从上次对话后换了文件夹——会话是按文件夹记录的
- 解决：确认当前在上次同一个文件夹里；或用 `claude --resume` 从列表里选（见支线 3.A）

还是不行？→ 见附录 G。

主线结束。下面两条支线讲如何从历史对话列表里精确挑一个继续，以及状态栏到底在告诉你什么。

 【支线】—— 可选深入（学有余力再看）

支线 3.A：用 `--resume` 从历史对话里挑一个继续

这段讲什么：`claude -c` 是“接上最近那次”，但如果你上周、上上周有几次对话都想回去看，需要另一个命令。什么时候回头读：你用了一两周发现自己在同一个文件夹有多次对话想翻出来的时候。

启动时加 `--resume`（或简写 `-r`）：

```
claude --resume
```

Claude Code 会弹出一个历史对话列表，按时间倒序列出你在当前文件夹的所有会话。用**↑↓方向键**选，回车进入。

每条列表项通常显示：

- 开始时间
- 消息数
- 第一条消息的前几个字（帮你认出是哪次聊的什么）

`-c` vs `--resume` 一句话：

- `-c` = 接最近那次
- `--resume` = 挑一次

⚠ 跨文件夹不通：每个文件夹有自己的对话历史。在 练习A 文件夹里聊的，到 练习B 里 `--resume` 是看不到的。

支线 3.B：状态栏到底在告诉你什么

这段讲什么：Claude Code 屏幕上那些数字（Ctx: 12%、Sonnet、成本）具体含义。什么时候回头读：你发现状态栏在变数字、好奇那是啥。第12章会专门讲上下文（context），15章讲模型和成本。

典型状态栏（随版本略有差异）：

● Sonnet 4.6 | Ctx: 23% used | \$0.12

分三段：

① 当前用的模型

- Sonnet：默认，性价比最高，适合 80% 场景
- Opus：更聪明、更贵，适合复杂任务
- Haiku：最快最便宜，适合简单任务

切换用 `/model` 命令（第15章详解）。

② Ctx: XX% used（上下文占用百分比）

这是整本书最重要的一个数字。它告诉你 Claude 这次对话的“短期记忆”用掉了多少。

- 50% 以下：健康
 - 70% 左右：考虑用 `/compact` 压一下（让 Claude 把对话浓缩）
 - 90%+：必须 `/compact` 或 `/clear`，不然回复质量会明显下降
- 详细机制第12章专讲。现在只需知道这个数字存在，平时扫一眼。

③ 成本预估（\$ 数字）

当前这次对话累计花了多少钱（订阅用户显示相对占用、API 用户显示美元）。

- 免费额度用户：通常看不到 \$，会看到额度余量
- API 用户：看 \$ 实际金额
- Pro/Max 订阅用户：看占你本月配额的比例

💡 新手先不用操心这个数字——默认 Sonnet 跑日常任务，一次对话通常几分到一毛几人民币的量级。详见第 15 章。

你已经完成了“和 Claude Code 正常对话”的全流程。下一章讲一个更重要的东西——Claude Code 到底是什么、你该用什么心态对待它。

第 4 章：Claude Code 到底是什么

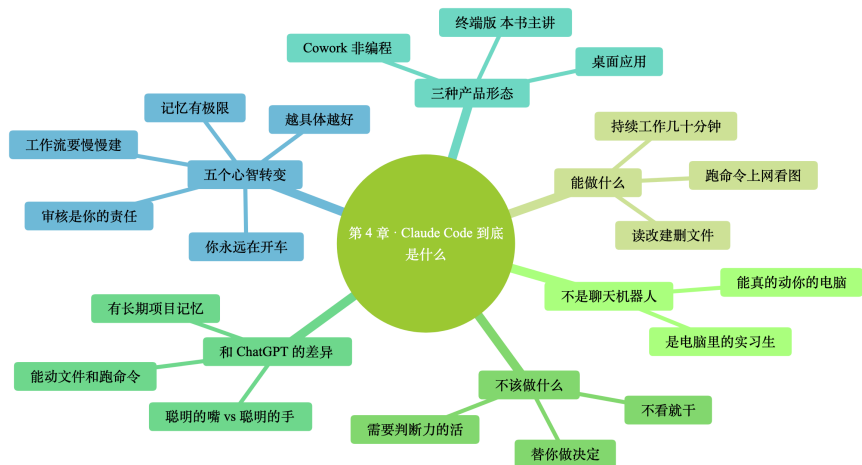
本章在全书的位置：第一部分 · 第 4 章 / 预估阅读+动手时长：主线 20-30 分钟 + 支线 10-15 分钟

前置章节：最好做完第 3 章（有过实际对话体验）再读这章——空谈心智模型容易飘。

学完能做什么（主线）：建立起用 Claude Code 时的正确心智模型——它能做什么、不能做什么、和其他 AI 聊天工具的本质差异、你该对它抱什么预期。没有这个，后面学再多技巧都会跑偏。

开场三问

- 你会遇到什么问题：第一次用 Claude Code，很容易把它当成“更高级的 ChatGPT”——结果要么预期过高（以为它能自动搞定一切），要么预期过低（只当聊天用）。
- 不读这章会踩什么坑：把它当聊天机器人每天闲聊；或者相反，把它当“AI 自动化机器”放任不管结果改坏文件。这一章帮你建立正确预期。
- 读完你会多会什么事：清楚它的能力边界，什么场景用它，什么场景别用它，以及你自己在整个流程里该干什么。



🎯 【主线】—— 本章必读核心

5 节，全章无需动手——这是一章心智模型章节。读完你会对 Claude Code 有更清楚的直觉。预估 20-30 分钟。

4.1 它不是聊天机器人——它是你电脑里的“实习生”

这是理解 Claude Code 最重要的一句话：

📌 Claude Code 不是聊天机器人，是一个住在你电脑里的实习生。

我们展开说。

聊天机器人（ChatGPT、Claude 网页版）的模式

你打字 → AI 打字回 → 你打字 → AI 打字回 → ...

它只会输出文字——除此之外什么都做不了。它能帮你写文案、解释概念、想主意，但它碰不到你电脑上的任何文件、装不了任何软件、跑不了任何命令。

Claude Code 的模式

你打电话 → Claude 决定要做什么 →

- 读你的文件? → 读
- 改你的文件? → 问你要不要改, 你说 Yes 才改
- 跑个命令 (比如统计、压缩、重命名)? → 问你要不要跑, 你说 Yes 才跑

→ 看结果, 可能再决定下一步 →

→ 最后告诉你成果

它真的在动你的电脑。这是本质差异。

为什么“实习生”这个比喻好

想象你招了个实习生:

- 他能做很多事 (读文件、写文档、跑程序)
- 但他不知道你具体想要什么, 需要你讲清楚
- 他每做一件可能影响重要东西的事前会问你: “这样改可以吗?”
- 他不会完美——会犯错、会理解偏差, 需要你审核
- 他没有你的经验——同一件事你做了 10 年, 他第一天来

用 Claude Code 时, 你不是在用一个 App, 你是在带一个实习生。这个视角一建立, 后面很多东西都通了: 为什么要审核它的改动、为什么要写 CLAUDE.md (岗前培训)、为什么它偶尔会犯傻 (经验不够)。

4.2 它能做什么、不能做什么

✅ 它能做的事

- 读任何它能访问的文件 (当前工作文件夹内的, 默认都能读)
- 改文件 (会先给你看 diff)
- 建新文件、删文件、重命名、移动 (都会先问)
- 跑命令: 整理文件、压缩图片、统计数据、装软件、调 API…… (会先问)
- 上网 (如果你允许): 搜资料、查文档、下载东西
- 看图片: 你贴截图、设计稿、流程图进去它能理解
- 持续工作几十分钟: 给它一个复杂目标, 它会分解、执行、回报

💡 生活化例子:

- “把下载文件夹里所有 PDF 按文件名里的日期分类到子文件夹”✅ 能做
- “读这份会议录音转录文字，帮我整理成会议纪要”✅ 能做（有合适工具的话）
- “把这 50 个 Excel 的‘已付款’列加起来”✅ 能做
- “帮我把我公众号草稿改成知乎风格”✅ 能做
- “重构这个 React 项目的登录模块”✅ 能做（如果你懂代码更好）

❌ 它不能做（或不该让它做）的事

- 替你做决定——它能给选项、讲优劣，但**“选哪个”是你的事**
- 不看就干——它永远先问你权限，你盲目选 Yes 出事它不负责
- 需要你本人创意和判断的活——比如“写一篇打动我客户的开场白”，它能起草，但最后的味道需要你改
- 跨越它视野的事——你在 A 文件夹启动它，它不能直接改 B 文件夹（除非你给绝对路径）
- 实时的事——它不知道今天什么天气、股价多少（除非你让它上网查）

⚠️ 它能做但要小心的事

- 写代码（如果你不懂代码，它写的代码你看不出对错——风险高）
- 删文件 / 覆盖现有文件（一旦 Yes，恢复不容易）
- 跑从网上下的脚本（可能有安全风险）
- 操作系统级设置（能毁电脑）

这几类第 7 章（权限机制）会专门讲怎么识别和应对。

4.3 和 ChatGPT / Claude 网页版的本质区别

很多人第一次接触 Claude Code 会问：“我已经在用 ChatGPT 了，这个有什么不一样？”

一张表说清楚：

维度	ChatGPT / Claude 网页版	Claude Code
能不能动你的文件	不能（只能输出文字让你复制）	能（在你允许下）
能不能跑命令	不能	能（在你允许下）
持续工作	主要是一问一答	可以分解任务持续干
长期记忆	部分（按账号保存）	按项目保存（ <code>CLAUDE.md</code> 、对话历史）
隐私	传云端	传云端（但可通过 API 或 Bedrock 等提升隔离）
适合场景	想主意、写草稿、解释概念	真正动手的事：整文件、做自动化、改项目
学习曲线	几乎没有	要先学点终端基础

📌 一句话：ChatGPT 是聪明的嘴，Claude Code 是聪明的手。两个互补，不替代。你问“这段怎么优化”用 ChatGPT 都行；你说“帮我把这个文件夹里 100 份 Excel 合并成一个总表”——必须用 Claude Code（或 Cowork）。

4.4 三种产品形态：你该用哪一个

第 0 章简单提过，这里系统讲一遍——Anthropic 目前有三种能“干活”的产品，你得知道自己该用哪个：

产品	界面	典型用户	典型任务
① Claude Code (终端版)	命令行	程序员 + 愿意学点终端的零基础人	改代码、重构项目、做自动化脚本、深度文件操作
② Claude Code 桌面应用 (2026-04 大改版)	图形界面：多会话侧栏、拖拽面板、内嵌终端	上面一样的人，但想要可视化多任务编排	同 ①，但一次搞 3-4 个并行任务
③ Claude Cowork	全图形，不需要命令行	完全不写代码的知识工作者：研究员、分析师、运营、法务、财务	整理文件、拆分下载、做结构化文档、数据清洗

对号入座

如果你的典型需求是：

- “改代码 / 做项目”类 → 用 ① 终端版（本书）
- “想要 ① 的能力但不爱切终端” → 用 ② 桌面应用（本书所有概念都通用，见附录 H）
- “整文件、做表格、写文档，完全不碰代码” → 用 ③ Cowork（claude.com/cowork），不用看本书

⚠ 不是越复杂越好——如果 Cowork 能满足你，用 Cowork 反而更快上手。别为了“酷”而硬学终端。

4.5 调整你的预期：5 个心智转变

很多零基础用户用了一两天 Claude Code 就放弃，通常是预期没调整好。下面 5 个心智转变必须做——这是本章最值得画线的一节。

① 它会犯错——审核是你的责任

AI 不是“绝对正确”，它生成的东西可能有错。日常任务错误率不高，但一旦发生，错误责任是你的（你点的 Yes）。

做法：养成 diff 审核习惯（第 6 章专讲）——改文件前那个对比图，扫一眼有没有意外改动。

② 你永远在开车——它只是助手

它可以建议、可以执行，但方向盘在你手上。

反模式：“Claude 你看着办”→ 它会瞎做；“Claude 这个怎么样”→ 你让它替你拿主意。

正确姿势：你想清楚要什么，告诉它 WHAT / WHERE / HOW / VERIFY（第 5 章讲这个提问模板），它执行。

③ 描述越具体，结果越好

模糊的指令：帮我整理一下笔记——它会猜你要什么，很可能猜错。具体的指令：

把 ~/Documents/2026会议笔记 里所有 .md 文件按主题分到三个子文件夹：产品 / 运营 / 技术，分类按每个文件头部的标题判断

——一次就能搞定。

④ 它有“记忆极限”——聊太久会变糊涂

Claude Code 每次对话的记忆是有限的（这是上下文，第 12 章专讲）。一次对话做太多不相关的事、或聊得太长，它会开始犯蠢。

做法：一个任务做完就 `/clear` 开新对话。不要把 10 个不相关的活塞一个对话里。

⑤ 前两周别指望“起飞”——先建 workflow

新手前两周通常是“先学会安全地用”——审核、权限、退出。两周后你开始写 CLAUDE.md（第 14 章）、装第一个 Skill（第 20 章），workflow 才真正成型。

💡 别着急。把实习生培养成骨干员工，本来就需要时间。

本章小结

- Claude Code 是电脑里的实习生，不是聊天机器人、不是自动化机器
- 它能动文件、跑命令（在你允许下），这是它和 ChatGPT 的本质区别
- 三种产品形态：终端版（本书）、桌面应用（同引擎，图形界面）、Cowork（非编程）
- 5 个心智转变：
 1. 它会犯错，审核是你的责任
 2. 你永远在开车，它是助手
 3. 描述越具体，结果越好

4. 它有记忆极限，聊久会糊涂
5. 前两周别指望起飞，先建工作流

动手任务

任务 1：心智自测——三道判断题（2 分钟）

对下面三句话做判断（心里答就行）：

1. 我让 Claude Code 改一份文件，它改错了，这是谁的责任？
2. 我应该把每天所有杂活放到同一个 Claude Code 对话里，这样省事？
3. Claude Code 就是“能操作我电脑的 ChatGPT”，没本质区别？

正确答案（建议自己先想一遍再对）：

1. 我的责任——我点了 Yes
2. 不应该——会导致上下文混乱、Claude 变糊涂；一事一对话
3. 有本质区别——ChatGPT 只能输出文字，Claude Code 能真正动电脑

任务 2：对号入座——你该用哪个产品（3 分钟）

场景：假设你接下来有以下 3 个任务，各自适合用哪个？（① 终端 Claude Code / ② 桌面应用 / ③ Cowork）

A. “整理我去年的 500 张照片，按拍摄日期分到月份文件夹” B. “重构我手里那个半成品 Python 脚本，让它支持批量处理” C. “同时跑 3 个并行任务，每个任务 Claude 都要做 20 分钟”

参考答案：

- A → ③ Cowork（非编程、文件整理）；也可以 ① 或 ②，但 Cowork 最直接
- B → ① 终端 或 ② 桌面应用
- C → ② 桌面应用（并行编排是它的强项）

如果你卡住了

症状：读完这章仍然觉得“它和 ChatGPT 有啥区别”不是很明白

- 原因：大概率是你还没真正体验到“它改文件/跑命令”的场景。
- 解决：回到第 3 章任务 3（让它改 `笔记.txt` 并审核 diff）——亲身做一次胜过读十遍。

症状：不确定该用终端版还是 Cowork

- 原因：你的使用场景还没定。
- 解决：用一句话描述你最想解决的问题——如果里面出现“代码、项目、脚本”这类词，用终端版；出现“整理、分类、合并、摘要”这类日常词，用 Cowork。

主线结束。下面两条支线讲一些更深入的背景——好奇底层怎么运作、想知道“agentic coding”到底是什么的人可以看看。

【支线】—— 可选深入（学有余力再看）

支线 4.A：Claude Code 底层是怎么运作的（通俗版）

这段讲什么：你在终端里打字，到 Claude 改了文件，中间到底发生了什么。什么时候回头读：用了一段时间，好奇“为什么会是这样”。

一个简化流程：

你在终端打字

↓

Claude Code 把你的话 + 最近对话 + 当前文件夹结构 一起加密发到 Anthropic 服务器

↓

服务器上的 Claude 大模型“思考”，决定要做什么：

- “我需要先读 X 文件吗？”
- “我要改什么文件？怎么改？”
- “需要跑什么命令？”

↓

它把行动计划发回来

↓

Claude Code（在你电脑上）按计划执行：

- 要读文件 → 直接读

- 要改文件 → 弹 diff 问你 → 你同意才改
- 要跑命令 → 弹权限提示 → 你同意才跑

↓

执行结果发回服务器，Claude 看结果再决定下一步

↓

循环直到完成，最后回复你

关键点：

- “大脑”在云端（Anthropic 服务器）
- “手脚”在你电脑（Claude Code 本地程序）
- 两边通过加密通道通信
- 所有改动你都会看到、都需要你同意（默认情况下）

这个架构叫 **agentic coding**（智能体编码）。下一条支线讲。

支线 4.B：什么是“agentic coding”（智能体编码）

这段讲什么：Claude Code 的这种工作方式有个专门名字，叫 **agentic coding**。

什么时候回头读：你在别人博客/视频里看到这个词，不知道啥意思。

传统 AI 聊天 vs agentic（智能体）模式

传统聊天：你问一句，AI 答一段，结束。AI 不记得你做了什么、也不主动做事。

Agentic 模式：AI 是一个能自主决定行动、反复迭代直到达成目标的实体——也就是“智能体”（agent）。

一个 agentic 系统包括：

1. 目标（你告诉它要什么）
2. 感知（读环境：文件、命令输出、之前的对话）
3. 决策（思考下一步）
4. 行动（真的执行：改文件、跑命令）
5. 观察结果（看自己行动之后发生了什么）
6. 回到 2，继续迭代直到任务完成

Claude Code 就是一个典型的 **agentic** 工具。

Agentic 的好处与坑

好处：

- 复杂任务一条命令搞定（你不用拆分）
- 能自我纠错（发现错了重来）
- 能长时间工作

坑：

- 可能“自作主张”做你没想到的事（所以要审核！）
- 可能误入歧途浪费时间（需要你看着点）
- 复杂任务用的 token 多（成本高）

💡 与 agentic 相关的其他工具：Cursor、Aider、Devin 也都是 agentic 编码工具，各有特色。Claude Code 的长处是终端原生 + Anthropic 大模型 + 权限控制细致。

你已经完成第一部分（从零起步）的全部 5 章。你现在的状态：能装、能启动、知道怎么对话、知道它是什么东西、建立了正确预期。

下一部分（日常使用）开始教你真正“用它干活”——从读文件讲起。

💡 本书以 Claude Opus 4.7（2026-04-16 发布）为基线写作。如果你想一次性了解 4.7 相对 4.6 的所有新变化和新手坑，可以直接读 [附录 J · Opus 4.7 新手指南](#)——不过那是附录，现在不读不影响主线学习。

第二部分 · 日常使用

第 5 章：让它读文件

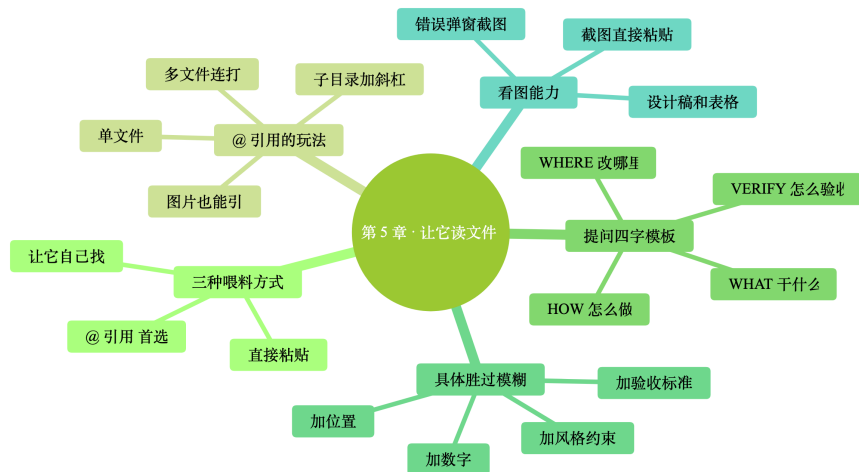
本章在全书的位置：第二部分 · 第 5 章 / 预估阅读+动手时长：主线 30-40 分钟 + 支线 10-15 分钟

前置章节：第 3 章（会启动和对话）

学完能做什么（主线）：能用 @ 引用让 Claude 精准读到你想让它看的文件、会用“WHAT/WHERE/HOW/VERIFY”模板提出清晰指令、让它看懂截图和多文件。

开场三问

- 你会遇到什么问题：你让 Claude 做事，它做出来的不是你要的——十有八九是你没让它“看到”你真正想让它处理的东西。
- 不读这章会踩什么坑：每次都要把文件内容复制粘贴到对话里（上下文爆炸）；问得模糊，Claude 瞎猜、结果不对。
- 读完你会多会什么事：能精准让它读指定文件、用结构化模板问清楚需求、让它看图片和截图。



【主线】—— 本章必读核心

5.1 让 Claude “看到”内容的三种方式

要 Claude 处理什么，它得先“看到”。有三种方式：

方式	怎么用	适合什么场景
① @ 引用	输入框打 @文件名	处理特定文件——首选
② 直接粘贴	Ctrl+V / Cmd+V 把内容贴进对话	短文本片段、从浏览器复制的一段话
③ 让它自己找	“看看当前文件夹里有啥”	你不确定要处理哪个文件，让它先探查

✚ 最重要的是掌握方式 ①——它是 Claude Code 和 ChatGPT 的最大差异之一。

5.2 @ 引用的具体用法

@ 是 Claude Code 的“精确指路”符号。在输入框里打 @，会弹出一个自动补全菜单列出当前文件夹里的文件，用方向键选或者继续打字过滤。

单文件

```
@笔记.txt 总结一下这份内容
```

Claude 会先读 笔记.txt，再按你要求总结。

子文件夹里的文件

```
@Documents/周报.md 帮我改一下这份周报的语气
```

路径可以带子目录。

一个文件夹（让它看这里所有文件）

```
@会议纪要/ 里面每份笔记分别抓一个关键点
```

@文件夹名/ 末尾带斜杠，Claude 会把这个文件夹里的文件都扫一遍。

⚠ 大文件夹慎用：如果里面有几十上百个文件，一次扫完会消耗大量上下文（下一章概念“上下文”详讲）。精准到具体文件比“扫一个文件夹”更好。

多个文件一起引

```
@第一章.md @第二章.md 把这两章合并成一份摘要
```

连续打几个 @ 没问题。

图片（Mac 和 Windows 都行）

```
@设计稿.png 看这张图，帮我把布局描述成文字
```

Claude 能“看”截图、设计稿、流程图。这是个很被低估的能力。

💡 让它看当前文件夹结构而不读具体内容 → 见 🌱 支线 5.A。

5.3 提问的“WHAT / WHERE / HOW / VERIFY”模板

新手最常见的毛病：问得太笼统。看两个例子：

模糊版 ❌

你：@周报.md 帮我改一下这份周报

Claude 会猜：你想改语气？改错别字？加内容？删内容？——猜中算你走运，猜偏你得反复纠正。

清晰版 ✅

你：@周报.md

- WHAT：把这份周报的语气从“严肃正式”改成“轻松简洁”
- WHERE：只改正文部分，不动标题和日期
- HOW：长句拆短；去掉“兹此”、“特此”之类的官话
- VERIFY：改完后字数不超过 500 字，看起来像朋友之间汇报进度

Claude 一次就能做对。

模板解释

字段	含义	作用
WHAT	要干什么	定义目标
WHERE	改哪里 / 限制范围	防止它乱动
HOW	具体做法、风格	统一审美
VERIFY	怎么验证做对了	给出可检查的成果标准

💡 不一定每次都要写这四项——简单任务一句话就行（“把文件名里的空格换成下划线”）。但任务越复杂，越要用这个模板。

5.4 为什么“具体”永远比“模糊”好

再看两组对比：

整理文件

模糊：帮我整理一下下载文件夹 具体：
@Downloads/ 把里面的 PDF 按文件名里的年份分到 2024/、2025/、2026/ 三个子文件夹；没有年份的放到“待分类/”

写东西

模糊：帮我写个文案 具体：
写一段 80 字朋友圈文案，宣传明天下午 3 点的免费书法公开课。风格亲切、口语化，带一个 call-to-action（比如“扫码预约”），不要用表情符号

调代码

模糊：帮我看看这代码哪里不对 具体：
@app.py 运行到第 42 行 toggle_dark_mode 函数时报 AttributeError。帮我定位原因，给修改建议，但先不改——我要先看方案

规律：

- 加数字（80 字、3 点）
- 加位置（第 42 行、哪个文件夹）
- 加风格约束（亲切、不要表情）
- 加验收标准（带 CTA、不要改动，先看方案）

每多一个具体限制，Claude 输出的“中签率”就高一截。

5.5 让它看图片：截图是你的好朋友

Claude 能看懂图片——截图、设计稿、手绘流程图、表格截图、错误界面……这个能力特别适合：

- 贴个错误弹窗截图：@error.png 这个报错是什么意思？怎么解决？
- 贴个设计稿让它生成文字描述：@mockup.png 这个登录页要什么元素？给我列出来
- 贴张手绘思维导图：@脑图.jpg 帮我把这张图整理成结构化大纲
- 贴 Excel 截图让它分析：@sales.png 这张销售表里哪个季度最好？

怎么把截图放进去

方式 A（推荐，最快）：在对话里直接粘贴——截图（Mac `Cmd+Shift+4` / Windows `Win+Shift+S`）截好后直接 `Cmd+V` / `Ctrl+V`，Claude Code 会把图片贴进对话。

方式 B：把图片文件保存到工作文件夹，然后 @图片名.png 引用。

方式 C：拖拽图片文件到 Claude Code 的终端窗口（某些终端支持）。

💡 对零基础用户特别有用：很多场景“用文字描述好累”——截个图最快。比如遇到奇怪的报错、想让它帮看看 Excel 某一列、想参考某个设计稿……

本章小结

- @ 引用是让 Claude 精准看到内容的最重要方式（单文件、文件夹、图片都能引）
- WHAT / WHERE / HOW / VERIFY 四字提问模板——任务越复杂越要用
- 具体永远比“模糊”好：加数字、加位置、加风格、加验收
- 图片截图直接粘贴进对话，Claude 能看懂——很多场景比文字描述快 10 倍

动手任务

任务 1：用 @ 引用读并总结（5 分钟）

步骤：

1. 进 `ccguide-练习` 文件夹，`claude` 启动
2. 输入 `@笔记.txt 用 3 条要点概括这份内容`，回车
3. 看结果

成功标志：Claude 输出的 3 条要点和文件内容对得上。

任务 2：用 WHAT/WHERE/HOW/VERIFY 改一份文件（10 分钟）

步骤：

1. 桌面建一个新文件 `草稿.md`，粘贴任意一段你写过的文字（周报、邮件、总结，200 字以上）
2. 把它复制到 `~/Desktop/ccguide-练习/` 里
3. Claude Code 里输入：

@草稿.md

- WHAT：改进这段文字的可读性
- WHERE：只改正文，标题和签名不动
- HOW：长句拆短、去重复、关键词加粗
- VERIFY：改完看起来像适合手机阅读的简洁版

4. 审核 diff，选 Yes 或 No

成功标志：Claude 改出来的版本确实更好读，你基本认可。

任务 3：让它看截图（5 分钟）

步骤：

1. 截一张你电脑桌面的图（Mac `Cmd+Shift+4` / Windows `Win+Shift+S`）
2. 切到 Claude Code 输入框
3. 粘贴（`Cmd+V` / `Ctrl+V`）
4. 问：“这张图里有什么内容？”

成功标志：Claude 能说出截图里有的东西（应用、文件夹、大概几个图标等）。

如果你卡住了

症状 A：打 @ 没有弹出文件列表

- 原因：你不在一个有文件的文件夹，或那个文件夹里文件太多被限制了。
- 解决：`Ctrl+C` 两次退出 → `cd` 到目标文件夹 → 重新 `claude`

症状 B：Claude 说找不到你引用的文件

- 原因：文件名打错、扩展名少了（`.txt` 和 `.md` 区分）、或文件不在当前文件夹里
- 解决：退出 → `ls` 看文件夹里的准确名字 → 重新引

症状 C：粘贴截图没反应

- 原因：极少数老终端不支持。
- 解决：把截图保存成文件（`Cmd+Shift+4` / `Win+Shift+S` 后选保存到桌面），再 `@截图名.png` 引用

主线结束。下面支线讲让 Claude 浏览整个文件夹、以及让它读网页 URL 内容。

🌿【支线】—— 可选深入（学有余力再看）

🌿 支线 5.A：让 Claude 浏览整个文件夹（不读内容 只看结构）

这段讲什么：你不确定要处理哪个文件，想让 Claude 先“扫一眼”整个文件夹的结构（不读内容）。什么时候回头读：你要让 Claude 处理一堆文件，但不想让它一次把内容全读进来（上下文爆炸）。

场景

你有个文件夹 `~/Documents/工作` 下面 30 多个 Word 和 PDF，想让 Claude 先告诉你每份文件大概是啥主题，再决定让它处理哪几个。

做法

帮我列出 `~/Documents/工作/` 里所有文件的文件名和大小，不要读内容。

Claude 会跑一个命令（比如 `ls -la`）列出来，不读文件内容，节省上下文。然后你根据列表挑：

@季度报告Q1.pdf @季度报告Q2.pdf 把这两份的要点放到一张对比表里

💡 原则：先浏览，再精读——尤其文件夹里东西多时，这样能省大量上下文。

支线 5.B：让 Claude 读网页 / URL

这段讲什么：Claude Code 能读网址里的内容（在你允许的情况下）。什么时候回头读：你想让 Claude 参考一篇博客、查一个文档、看一个产品页。

方式 A：直接贴 URL

读一下 `https://example.com/xxx` 这篇博客，帮我总结 5 个要点

Claude 会问你是否允许它上网，你选 Yes 它就会用内置工具去取内容。

方式 B：你自己用浏览器复制粘贴

简单粗暴但可靠——浏览器里全选复制，贴到对话里。适合：

- 页面需要登录才能看
- 页面有很多 JS 渲染（Claude 可能取不全）
- 你只想要某一段

注意

- URL 取内容会消耗较多 token（算在成本/上下文里）
- 敏感信息不要用 URL 方式（会把 URL 传到云端）
- 局域网或需认证的网页通常 Claude 取不到——用方式 B

下一章讲你已经初步体验过的——让它改文件，但会把 diff 审核、三档习惯、挽回机制讲透。

第 6 章：让它改文件

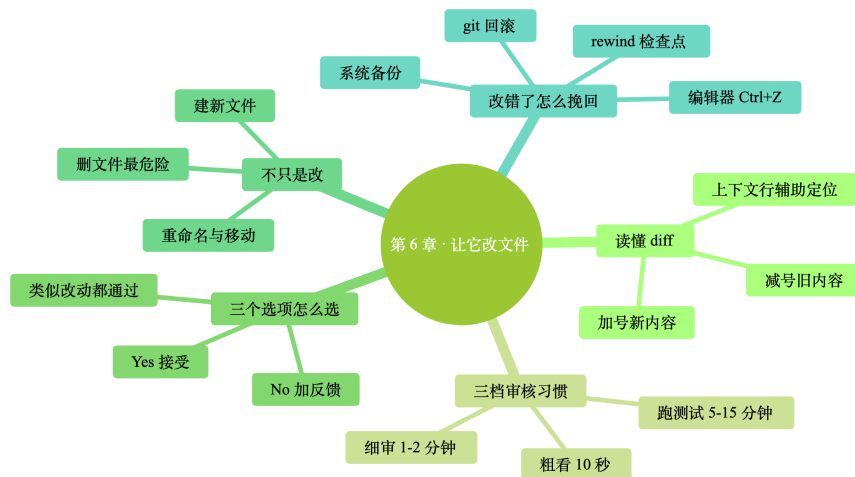
本章在全书的位置：第二部分 · 第 6 章 / 预估阅读+动手时长：主线 30-40 分钟 + 支线 10 分钟

前置章节：Ch 3（体验过一次改文件）+ Ch 5（会用 @ 引用）

学完能做什么（主线）：看懂 diff（修改对比图）、养成三档审核习惯、用熟 Accept/Reject/Edit 三个选项、让它建/删/重命名文件、改错了知道怎么挽回。

开场三问

- 你会遇到什么问题：Claude 弹出一堆红绿对比文字，你不知道看哪里、哪些是关键、哪些该警惕。
- 不读这章会踩什么坑：不看 diff 直接 Yes——十次里有一次你会发现它顺便改了你不想改的地方。
- 读完你会多会什么事：30 秒扫一个 diff 做出准确判断；知道什么时候必须细审、什么时候可以粗看；改错了不慌。



🧵【主线】—— 本章必读核心

6.1 什么是 diff（修改对比图）

diff = difference 的缩写，意思是“修改前后的对比”。每次 Claude 要改文件，它都会先给你看 diff，让你确认后再真改。

一个简单 diff 长什么样

假设原文件 `笔记.txt` 内容是：

```
买菜
写周报
给妈打电话
```

你让 Claude “每行前加编号”。它生成的 diff 是：

```
Edit: 笔记.txt

- 买菜
```

- 写周报
- 给妈打电话
- + 1. 买菜
- + 2. 写周报
- + 3. 给妈打电话

怎么读

- 红色 - 开头的行：这行被删除或替换（旧内容）
- 绿色 + 开头的行：这行是新增或替换后（新内容）
- 没 -/+ 的行：没变，不用看

💡 一句话记法：减号是旧的、加号是新的。你看到的是“文件即将变成的样子”——绿色留下、红色消失。

复杂 diff 多一个上下文

文件很大时，Claude 只会显示改动附近的几行（叫“上下文行”），省屏幕：

```
Edit: config.txt
...
port: 8080
- debug: false
+ debug: true
timeout: 30
...
```

中间的 `port` 和 `timeout` 行没动（没 -/+ ），显示是为了让你看到改动在文件的什么位置。

6.2 三档审核习惯——别每次都细读

不是所有 diff 都要逐字看。新手最常见的错误是两极化：要么不看、要么纠结到看不完。正确做法是分三档：

档	什么时候用	看什么	时间
粗看（扫 10 秒）	小改动、低风险文件（笔记、草稿、自己写的说明）	只看改动范围合不合理、有没有“意外加/删”的文件	10 秒
细审（读 1-2 分钟）	中等改动、涉及关键内容（简历、合同、代码）	逐行读 <code>-/+</code> ，确认每一处改动都是你要的	1-2 分钟
跑测试（5-15 分钟）	涉及正式数据、会被别人看到的內容、代码	接受后，实际运行一遍看效果；代码类的跑测试	5-15 分钟

🔑 关键判断：改坏了恢复成本多高？恢复难的必须细审、要跑测试；恢复易的粗看就行。

三个典型例子

- 改草稿笔记：粗看 → Yes
- 改你明天发给客户的邮件：细审每一句
- 改生产环境的配置文件：细审 + 跑测试 + 留备份

6.3 三个选项：Accept / Reject / Edit

每个 diff 弹出后，你有三个选项（方向键选、回车确认）：

```
Do you want to apply this change?
) Yes
  Yes, and allow similar changes for the rest of this session
  No, tell Claude what to do differently
```

① Yes（接受）

选了就改。文件立刻被写成新版本。适合你审核通过、没异议。

② Yes, and allow similar changes（这次对话里类似改动都通过）

本次对话结束前，类似的改动 Claude 直接改不用再问。

⚠ 新手谨慎用这个——如果任务有几十次类似改动（比如批量改所有文件里的一个词），用这个省事；但如果你不确定“类似”会扩散到哪，选普通 Yes 更稳。

③ No（拒绝 + 告诉它怎么改更好）

选了后 Claude 不会改，而是等你说清楚哪里不对。

常见用法：

- “这个改动范围太大了，只改 A 部分”
- “这种语气不对，太正式了，口语化一点”
- “保留原来的结构，只替换关键词”

Claude 会根据你的反馈重新生成 diff 给你看。不满意再 No，直到满意。

💡 反复 No 并不‘烦 Claude’——它不会生气、不会累。反而比你将就接受然后再改回来更省时间。

有些版本还有 Edit 选项

一些 Claude Code 版本提供 `e`（Edit）——让你手动编辑这个 diff，改成你想要的样子，然后 Claude 按你的版本应用。适合你想局部微调而不是整体重做。

6.4 不只是改——让它建、删、重命名

Claude 能做的文件操作远不止改：

建新文件

在当前文件夹建一个 `todo.md`，里面写今天的三件事：买菜、写周报、打电话。

Claude 会显示“即将创建”的内容，你 Yes 它就真建。

删文件

删掉 `~/Downloads/` 里所有超过一年没动过的 `.zip` 文件。

⚠ 删文件是最危险的操作——一旦 Yes，恢复可能很麻烦（图形界面的“废纸篓”有时能救、终端删的可能直接永久消失）。

做法：

- 删之前让它先列出要删的清单（“先给我看要删的文件名列表，别删”）
- 确认清单没问题再让它删
- 涉及重要文件前先备份一份到其他文件夹

重命名 / 移动

把这个文件夹里所有 "2024-" 开头的文件，改名把前缀去掉

改名也是先给 diff（新旧名对照），你 Yes 再执行。

6.5 改错了怎么挽回——四种路

改下去发现不对，有四种挽回方式，按优先级从简单到麻烦：

路 1: Ctrl + Z（如果文件还在编辑器里打开）

如果这个文件在 VS Code、Word、任何编辑器里打开着，切到编辑器按撤销——很多时候直接就回去了。

路 2: Claude Code 的 /rewind

在 Claude Code 里打：

```
/rewind
```

会列出对话中最近的几个检查点（checkpoints），你选一个回退到那个时刻——对话历史 + 文件状态都会恢复。

还有快捷键：按两次 Esc 键 (Esc Esc)，等于快速 `/rewind` 回到上一个检查点。

💡 这是 Claude Code 一个超好用的救命功能。第 9 章会专门讲 `/rewind` 的细节。

路 3: 用 git（如果你在用 git）

如果你的文件夹是一个 git 仓库，改坏了：

```
# 看看改了啥
git diff

# 回滚到上次 commit
git checkout 文件名

# 或者回滚全部
git reset --hard
```

⚠ 这些 git 命令会永久丢弃未 commit 的改动——确定要回滚再用。

💡 不懂 git? 第 25 章会简单讲。最基础的做法：每次重要改动前 `git add . && git commit -m "备份"` 一下，改坏了就 `git checkout .` 回滚。

路 4：从备份恢复

Mac 的 Time Machine、Windows 的文件历史记录、iCloud/Dropbox 的版本历史——这些都能恢复单个文件的旧版本。适合改动发生了一段时间才发现问题的。

本章小结

- diff = 修改前后对比；红色是旧（消失）、绿色是新（留下）
- 三档审核：粗看（10 秒）/ 细审（1-2 分钟）/ 跑测试（5-15 分钟）——按恢复难度选档
- 三个选项：Yes（接受）/ Yes 类似改动都通过（谨慎用）/ No 告诉它哪里不对
- 删文件最危险——先列清单确认再真删
- 改错了四种挽回：Ctrl+Z（编辑器里）→ `/rewind` → git → 备份

动手任务

任务 1：体验完整改文件流程（8 分钟）

步骤：

1. 在 `ccguide-练习` 里新建文件 `日程.md`，写 3 条安排（比如“9 点 meeting、11 点写稿、下午 review”）
2. 启动 Claude Code，输入：`@日程.md` 把时间从 24 小时制改成 12 小时制并加 AM/PM
3. 审核 diff，选 Yes
4. 切图形界面打开 `日程.md`，确认变化

成功标志：文件内容按要求改了，且图形界面能看到。

任务 2：主动用 No 纠正（8 分钟）

步骤：

1. Claude Code 里输入：`@日程.md` 大幅精简一下，只保留核心关键词，20 字以内
2. Claude 会生成 diff——看完后故意选 No
3. 在反馈里输入：“保留原来的时间格式，但把描述换成更简练的动词开头的短语”
4. 看 Claude 重新给的 diff，这次 Yes

成功标志：体验到 No + 反馈是让 Claude 调整的正常路径。

任务 3：用 `/rewind` 回退（5 分钟）

步骤：

1. Claude Code 里输入 `/rewind`
2. 看弹出的检查点列表
3. 选“任务 2 开始之前”的那个检查点，确认
4. 切图形界面看 `日程.md` 是否回到任务 2 之前的状态

成功标志：文件回到之前的样子。

如果你卡住了

症状 A：diff 很长看着眼花

- 原因：改动范围大。
- 解决：先看有没有你不想动的文件出现在 diff 里（比如你让改 A，它顺便改了 B）；有就 No 让它只改 A。改动内容本身按三档原则办。

症状 B：选 Yes 后文件没变

- 原因：极少数文件被其他程序锁住（比如 Word 打开着正在编辑）。
- 解决：关掉那个程序再试。

症状 C：想 `/rewind` 但说“没有可回退的检查点”

- 原因：当前对话还没有“写文件”这种动作产生检查点，或上个检查点已经过期。
- 解决：改用 git、编辑器 `Ctrl+Z`、或从备份恢复。

主线结束。下面支线讲批量改多文件、和让它“改得更准”的技巧。

 【支线】—— 可选深入（学有余力再看）

支线 6.A：批量改多个文件

这段讲什么：一条命令改几个、几十个文件。什么时候回头读：你碰到“所有这些文件都要做类似改动”的场景。

让它同时改多文件

```
@第一章.md @第二章.md @第三章.md  
把这三章开头的"2024 年"都改成"2026 年"
```

Claude 会依次对每个文件生成 diff，你一个个审核。

三个 diff 都类似，简化审核

如果你已经看完第一个 diff 觉得很好、其余两个类似，选：

```
Yes, and allow similar changes for the rest of this session
```

就不会每个都弹。但前提：你真看清了模式，确信“类似改动都是安全的”。

更复杂：跑个脚本批量改

```
把 ~/Documents/会议/ 下所有 .md 文件的首行日期格式统一成 2026-04-19  
这种
```

Claude 可能不生成 50 个 diff，而是跑一个脚本遍历改。这种情况下它会展示脚本让你审（变成“权限-跑命令”提示，见第 7 章）。

支线 6.B：让它改得更准的三个技巧

这段讲什么：提高 Claude 改对率的三个习惯。什么时候回头读：你发现 Claude 经常改偏、需要反复纠正。

技巧 1：给一两个“期望的例子”

@章节列表.md 把每行末尾的字数统计格式从"(1200字)"改成"· 1.2k 字"
例子：原来 "第一章：开头 (1200字)" 改成 "第一章：开头 · 1.2k 字"

一个具体例子胜过十句解释。

技巧 2：明确“不要做什么”

帮我把这段英文翻译成中文。

▲ 不要改变段落结构，不要合并句子，不要意译——保持直译的节奏。

告诉 Claude 别做什么和告诉它做什么一样重要。

技巧 3：要求它“先说计划再动手”

@大文件.md 我想把它拆成三个章节。

先告诉我你打算怎么拆（在哪里断开、各章标题是什么），我 OK 你再开始改。

Claude 会先说方案、等你同意后才生成 diff——省得方案错了改一半发现问题。（这种做法和 /plan 计划模式本质一样，第 9 章会详讲）

下一章讲 Claude Code 最重要的一个安全机制——让它跑命令 + 权限系统。

第 7 章：让它跑命令 + 权限机制

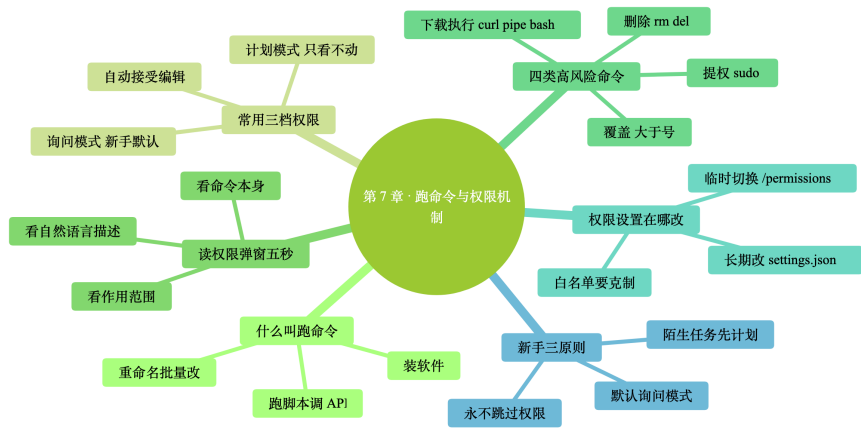
本章在全书的位置：第二部分 · 第 7 章 / 预估阅读+动手时长：主线 40-50 分钟 + 支线 15 分钟

前置章节：Ch 6（改文件的权限/审核概念已建立）

学完能做什么（主线）：理解 Claude Code 的三档权限模式、看懂权限弹窗、识别危险命令、知道什么时候该选允许、什么时候该停下来。这是 Claude Code 整个工具最重要的安全机制。

开场三问

- 你会遇到什么问题：Claude 弹一个权限框问你允不允许跑一条命令——命令里一堆英文单词和符号，你一脸懵：“这是什么？我该点 Yes 吗？”
- 不读这章会踩什么坑：闭眼 Yes 跑到危险命令（删文件、装奇怪的软件、修改系统设置）；或反过来——啥都 No 导致 Claude 根本干不了活。
- 读完你会多会什么事：一眼识别命令类别和风险等级，该 Yes 果断 Yes、该 No 果断 No，还知道该用哪档权限模式最适合你当前场景。



🎯 【主线】—— 本章必读核心

7.1 “让它跑命令”是什么意思

前面几章 Claude 在干的都是读文件、改文件。但它还有一个更强的能力：在你电脑上跑命令。

“跑命令”就是让终端执行一条指令——你在第 1 章学过的 `ls` / `cd` 是命令；装软件的 `npm install` 是命令；重命名文件的 `mv`、删东西的 `rm`、下载的 `curl`、`git` 操作、跑 Python 脚本……全都是命令。

典型场景

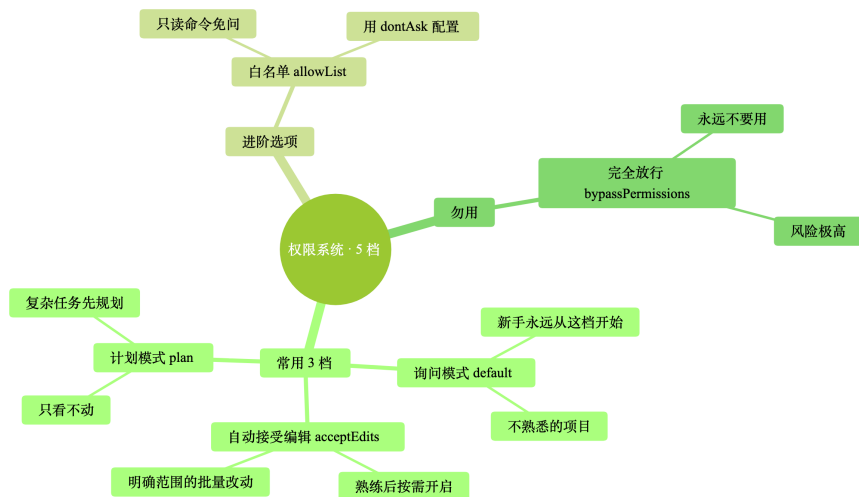
- “把 ~/Downloads 里所有 PDF 重命名，前面加今天的日期” → Claude 会跑 `mv` 循环
- “统计这 30 个 Excel 里‘已付款’多少条” → 跑 Python 脚本
- “压缩这张大图到 500KB 以下” → 跑 `sips` 或 `imagemagick`
- “把这个项目里的依赖都更新一下” → 跑 `npm update`
- “看看这个进程在占资源吗” → 跑 `ps` / `top`

这个能力就是 Claude Code 和 ChatGPT 最大的区别之一——但因为命令能真改动你电脑，必须有权限机制。

7.2 权限机制：5 档简化为 3 档

Claude Code 提供五档权限模式，但对零基础用户，只需理解并用三档：

档	官方名字	什么感觉	什么时候用
1. 询问模式	default（默认）	每次改文件 / 跑命令都弹窗问你	新手永远从这档开始；不熟悉的项目；重要文件
2. 自动接受编辑模式	acceptEdits	改文件不问（自动 Yes），跑命令仍问	你对这次任务很有把握，只是不想一直点 Yes
3. 计划模式	plan	只读不写——Claude 只能看和分析，不能改文件、不能跑命令	复杂任务想先让它规划，还没准备好让它动手



另外两档（简单了解）

- dontAsk / allowList（白名单）：把特定命令（比如 `ls`、`git status`）标记为“以后不用问”
- bypassPermissions（完全放行）：⚠ 永远不要用——所有权限都跳过，改文件删文件跑任何命令都不问

怎么切换权限模式

三种方式：

方式 A：启动时指定

```

claude                                # 默认询问模式
claude --permission-mode plan         # 计划模式启动（最安全的探索方式）

```

方式 B：会话中切换

在对话输入框里打：

```
/permissions
```

会弹出界面，让你选当前会话用哪档。

方式 C：某次操作时临时选

每个权限弹窗的第二个选项“**Yes, and allow similar changes...**”（见第 6 章）就是**把这类操作的权限升到“本次会话内自动允许”**。

新手的三原则

1. 默认用询问模式——别图快一上来就自动接受
2. 探索新项目 / 复杂任务用计划模式——先让 Claude 看和分析、规划好方案再改
3. 永远不要用 `--dangerously-skip-permissions` ——这是 `bypassPermissions` 的命令行开关，仅限机器之间自动化场景用

7.3 怎么读权限弹窗

看到 Claude 弹出这样的窗口时，你需要 3 秒做决定：

```

Claude wants to run:

rm -rf ~/Downloads/*.zip

This will remove all .zip files in ~/Downloads.

) Allow once
  Allow always for this session
  Deny and tell Claude what to do differently

```

扫视顺序（5 秒做决定）

Step 1: 看命令本身——最关键那一行是什么？

- `rm` = 删
- `mv` = 移动 / 重命名
- `cp` = 复制
- `curl` / `wget` = 下载
- `sudo ...` = 用管理员权限（高度警惕）
- `npm install` / `pip install` = 装 Node.js / Python 包
- `git commit` / `git push` = 提交代码 / 推送代码

Step 2: 看命令作用范围——动的是你预期的文件/位置吗?

- `~/Downloads/*.zip` = 只动下载文件夹里的 zip ☒ 预期
- `~/*` 或 `/*` = 整个家目录/整个磁盘 ☐ 危险
- `.`（当前文件夹）= 当前位置 ☒ 通常安全

Step 3: 看描述——Claude Code 通常会在命令下一行用自然语言解释这条命令在做什么，没看懂命令就读描述。

三个选项怎么选

选项	含义	什么时候
Allow once	允许一次，下次类似命令还会问	命令你看懂了、没问题
Allow always for this session	本次对话里这类命令以后不问	你确信这种命令接下来会反复出现且都安全
Deny	不允许 + 让 Claude 换个方法	看不懂、或这不是你想要的命令范围

新手心法：90% 的场景选“Allow once”——对每一次命令保留判断权。

7.4 四类高风险命令——必须识别

有些命令一旦跑错几乎救不回来，必须学会一眼识别。

危险类 1：删除类

```
rm -rf 路径      # 强制递归删除（删文件夹和里面所有东西）
rm 文件          # 删单个文件
del 文件         # Windows 删
```

⚠ 关键字：rm、del、-rf（强制递归）

看到就要思考：删的范围对吗？删错了能恢复吗？

危险类 2：覆盖类

```
mv A B          # 如果 B 已存在，会被 A 覆盖
cp A B          # 同上（某些场景）
> 文件          # 把内容写到文件，会覆盖原内容
```

⚠ 关键字：> 单箭头（覆盖） vs >> 双箭头（追加）

危险类 3：提权类

```
sudo ...        # 用管理员权限跑（Mac）
Run as Admin ... # Windows 提权
```

⚠ sudo 开头的任何命令都要高度警惕——影响范围可能是整个操作系统。

危险类 4：下载 + 执行

```
curl https://xxx.com/script.sh | bash
wget xxx | sh
```

⚠ 这种模式：从网上下一段脚本直接执行。风险极高——你不知道脚本里写了什么。

原则：看到 curl ... | bash 或 wget ... | sh 先 Deny，要求 Claude 先把内容下下来让你看一遍再决定。

一个不完全清单（安全感参考）

命令	大致安全度	说明
<code>ls</code> / <code>pwd</code> / <code>cat</code> / <code>head</code> / <code>tail</code>	✅ 只读	看东西，不改
<code>mkdir</code> / <code>touch</code>	✅ 新建	新建文件夹/空文件
<code>cd</code>	✅ 切换	只改“当前位置”
<code>cp</code> 到新位置	🟡 一般安全	复制通常无害，但目标存在会覆盖
<code>mv</code> 到新位置	🟡 一般	移动，覆盖风险
<code>git add</code> / <code>git commit</code>	🟡 版本控制	改 git 状态但不丢数据
<code>rm</code> / <code>del</code>	⚠️ 高风险	永久删除
<code>sudo</code> 开头	⚠️ 高风险	提权
<code>curl xxx bash</code>	🔴 极高风险	从网下载执行

7.5 权限设置在哪改

临时改（本次会话）

在 Claude Code 输入框打 `/permissions`，按指引选。

永久改（所有会话）

设置文件在 `~/.claude/settings.json`（Mac）或 `C:\Users\你的名字\.claude\settings.json`（Windows）。不熟悉 JSON 格式就别手改——容易写错一个逗号整个文件失效。

💡 更安全的做法：让 Claude 自己帮你改这个文件。你跟 Claude 说：“帮我把 `ls` 命令加进自动允许的白名单”，它会按格式帮你改。

具体配置项第 18 章和第 25 章会讲。

常用白名单项

刚开始用时可以把这些命令加白名单省去不停确认：

- `ls`、`pwd`、`cat`、`head`、`tail`（只读）
- `git status`、`git diff`、`git log`（只读 git 操作）

不要加白名单：

- `rm`、`mv`、`cp` 等改动类
- `sudo` 开头任何
- `curl`、`wget` 下载类
- `git push`、`git reset`（影响版本历史或远程）

本章小结

- Claude Code 的“跑命令”能力是它和 ChatGPT 最大的差异之一，但必须通过权限系统
- 新手用三档：询问模式（默认）/ 自动接受编辑模式 / 计划模式
- 永远不要用 `--dangerously-skip-permissions` / `bypassPermissions`
- 读权限弹窗：命令 → 范围 → 描述，5 秒做决定
- 四类高风险命令：删除（`rm`）/ 覆盖（`>`）/ 提权（`sudo`）/ 下载执行（`curl` | `bash`）
- 新手心法：90% 的场景选 “Allow once”

动手任务

任务 1：体验询问模式的权限弹窗（5 分钟）

步骤：

1. `cd ~/Desktop/ccguide-练习`，`claude` 启动（默认询问模式）
2. 输入：`列出当前文件夹所有文件的大小，按大到小排`
3. Claude 会弹出权限框（它要跑 `ls -ls` 或类似命令）
4. 看清命令 → 选 `Allow once`
5. 看输出

成功标志：你成功决定了一次权限、看到了结果。

任务 2：切到计划模式探索（10 分钟）

步骤：

1. 退出当前 Claude Code（`Ctrl+C` 两次）

2. 重新启动： `claude --permission-mode plan`
 3. 输入： 帮我整理一下当前文件夹——给出整理方案
 4. Claude 会分析并用文字输出方案，但不会真的改任何东西
 5. 看方案 OK 后， `/permissions` 切回默认模式，让它执行
- 成功标志：计划模式下 Claude 不会动任何文件，只输出建议。

任务 3：故意 Deny 一次（5 分钟）

步骤：

1. 默认模式下，输入： 帮我清空下载文件夹，删掉所有内容
2. Claude 弹出 `rm` 类命令的权限框
3. 看清命令 → 选 Deny → 告诉它：“先列出要删的文件清单，别删”
4. Claude 改为只跑 `ls` 列表

成功标志：你熟悉了 Deny + 反馈的流程；学会“先看清单、再决定删不删”的好习惯。

如果你卡住了

症状 A：权限弹窗太频繁，每点几下就弹一次

- 原因：询问模式对每个文件/命令独立问。
- 解决：
 - 如果这次任务确信都安全 → 选 `Yes, and allow similar for this session`
 - 任务结束后不要保留这种宽松模式进入下一个任务

症状 B：命令看不懂，不知道选 Yes 还是 No

- 原因：命令太长或不熟悉。
- 解决：选 Deny，告诉 Claude：“用自然语言描述一下你想做什么”；它说人话解释后你再决定

症状 C：不小心 Yes 了危险命令

- 原因：手快或没看清。
- 解决：
 - 立即 `Ctrl+C` 中断（来得及就能中断）
 - 中断后立刻检查：被改动的文件是否还在、git 能否回滚、备份能否恢复
 - 下次刻意放慢节奏

主线结束。下面支线讲自动接受模式的使用场景，和白名单的建立。

🌿 【支线】—— 可选深入（学有余力再看）

🌿 支线 7.A：什么时候值得用自动接受模式 (acceptEdits)

这段讲什么：acceptEdits 权限模式的合理使用场景——不是越早开越好，但确实有合适的时机。什么时候回头读：用熟了一两周、开始觉得反复点 Yes 很烦、能识别风险了。

开启方式

启动时：claude --permission-mode acceptEdits 会话中切换：/permissions → 选 accept edits

它做什么、不做什么

操作	默认模式	自动接受模式
改文件（Edit/Write）	弹 diff 问	自动通过，不问
跑命令（Bash）	弹权限框问	仍然问
建文件 / 删文件	弹权限框问	仍然问（因为删是不可逆风险操作）

适合的场景

- 重构 / 批量改文档——你已经给了清晰指令，Claude 会改几十处，都是预期间的
- 明确范围的迭代——“把这个文件里所有 ‘foo’ 改成 ‘bar’”这类
- 你已经做了 git commit / 备份——改坏了能快速回滚

不适合的场景

- ❌ 第一次用 Claude Code——你不熟悉它的行为

- **✗ 重要项目第一次碰——风险过高**
- **✗ 你累了 / 分心了——审核能力下降**

⚠ 切回默认模式的时机：任务结束时立刻切回，别让 acceptEdits 跨会话存活。

支线 7.B：建一个私人白名单

这段讲什么：把常用的只读命令加进“以后不问”的白名单，减少打断。什么时候回头读：你已经用了至少 1-2 周、识别命令风险有了直觉。

思路

~/.claude/settings.json 里有一个 permissions 块，可以定义“以后自动允许”的命令模式（叫 allowList）。

格式（示例，以 Claude Code 当前版本为准，查官方文档）：

```
"permissions": {
  "allow": [
    "Bash(ls:*)",
    "Bash(pwd)",
    "Bash(cat:*)",
    "Bash(git status)",
    "Bash(git diff:*)",
    "Bash(git log:*)"
  ]
}
```

Bash(命令:参数模式) 表示允许这条命令 + 参数匹配时自动通过。

我推荐的最小白名单（零基础版）

只加这 6 条，都是只读：

```
Bash(ls:*)
Bash(pwd)
Bash(cat:*)
Bash(head:*)
```

```
Bash(tail:*)  
Bash(git status)
```

我强烈建议不要加入白名单的：

- **✗** `rm` / `del` （删）
- **✗** `mv` / `cp` （移动/复制——可能覆盖）
- **✗** 任何 `sudo` 开头
- **✗** `curl` / `wget` （下载）
- **✗** `npm install` / `pip install` （装包）
- **✗** `git push` / `git reset` / `git rebase` （影响历史或远程）

怎么改

推荐方式：让 Claude 帮你改。对 Claude 说：

```
帮我在 ~/.claude/settings.json 里加一个最小白名单，包括：  
ls / pwd / cat / head / tail / git status。  
加之前给我看改动 diff。
```

这样不会自己手改出 JSON 语法错误。

写在最后

白名单省事但要克制——哪怕每天少 20 次确认，一年也就少 7000 次确认；但只要有 1 次本来该 Deny 的你没看到，损失可能抵消一年的省时。默认模式的“累”是一种保护。

至此第 7 章完成。下一章讲整个对话的工作节奏——交互循环七步法，帮你把单点操作串成稳定的工作流。

第 8 章：交互循环七步法

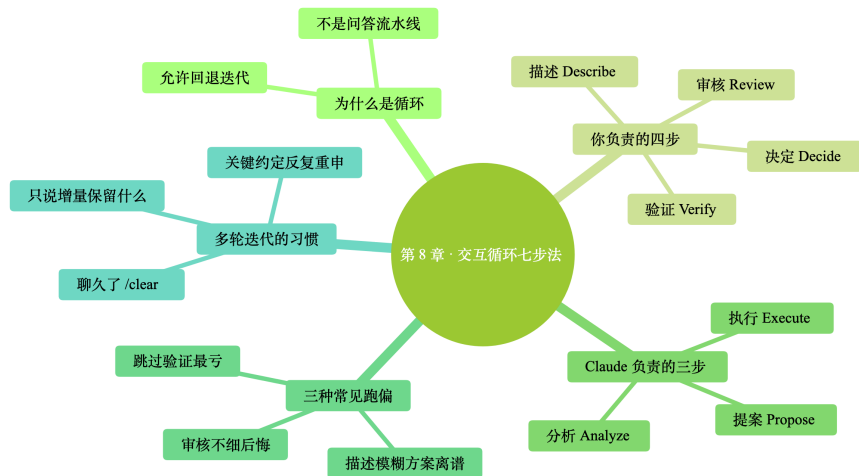
本章在全书的位置：第二部分 · 第 8 章 / 预估阅读+动手时长：主线 25-30 分钟 + 支线 10 分钟

前置章节：Ch 5（读文件）+ Ch 6（改文件）+ Ch 7（权限）

学完能做什么（主线）：你会知道一次“和 Claude Code 对话”不是一次简单问答，而是一个 7 步循环。每一步你都有该做的事，做对了任务就顺；做错了哪一步、结果就跑偏在哪。

开场三问

- 你会遇到什么问题：和 Claude 聊得好好的，但做出来的结果离你想要的总差那么一点——不知道是哪一步出问题了。
- 不读这章会踩什么坑：盲目问、盲目 Yes，结果反复修、反复改、一个任务拖一小时才做完。
- 读完你会多会什么事：把一次任务当成一个清晰的七步循环来走——你在每步都清楚自己该做什么、Claude 该做什么，效率翻倍。



🎯 【主线】—— 本章必读核心

8.1 为什么要讲“交互循环”

你可能觉得用 Claude Code 就是“问一句答一句”——那是 ChatGPT 的模型。Claude Code 不一样，它是一个循环：

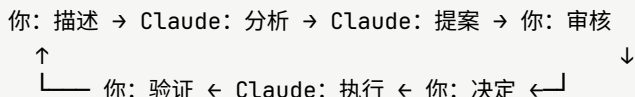
- 你不只是在问问题
- 它不只是在回答
- 它会分析、行动、观察结果、再决定下一步

这个循环每一步都需要你参与。理解清楚循环的 7 个步骤，你会明白：

- 为什么有时候 Claude 做得很准，有时候很偏
- 你在对话里的作用不是“问问题的人”，而是司机 + 审核员
- 问题出在哪步，修哪步

8.2 七步走一遍完整流程

先看一次完整的流程图：



展开讲，每一步名字和发生的事：

步骤	名字	谁在做	做什么
1	Describe 描述	你	告诉 Claude 你要什么（用 WHAT/ WHERE/HOW/VERIFY 模板更好）
2	Analyze 分析	Claude	读相关文件、理解上下文、思考方案
3	Propose 提案	Claude	给出 diff（要改的文件）或命令（要跑的事）
4	Review 审核	你	看 diff / 看命令，判断合不合理
5	Decide 决定	你	Yes / No / Edit 三选一
6	Execute 执行	Claude	按决定真正改文件 / 跑命令
7	Verify 验证	你	看结果对不对（打开文件看、跑测试、检查效果）

重点：7步里你占4步（描述、审核、决定、验证）。Claude 只占3步（分析、提案、执行）。

💡 这就是“你永远在开车”的意思——方向盘（决定权）始终在你手上。

8.3 每一步你的职责

步 1: Describe (描述)

你的任务：把想要的说清楚。

好的描述：具体、结构化、有成功标准 坏的描述：“帮我搞一下”、“优化一下”、“修复一下”

用第 5 章的 WHAT / WHERE / HOW / VERIFY 模板。

步 2: Analyze (分析) —— Claude 自己做, 但你要观察

Claude 可能会:

- 读几个文件确认当前状态
- 告诉你它理解到了什么

- 问你澄清问题（“你是指 X 还是 Y？”）

你的观察重点：它理解对了吗？如果它复述出来的需求和你想的不一样，立刻打断（Ctrl+C 或直接说“不对，我要的是……”），别等它跑完才发现方向错了。

步 3：Propose（提案）—— Claude 给方案

Claude 会拿出 diff（要改的文件对比）或弹权限框（要跑的命令）。

步 4：Review（审核）—— 你的主场

用第 6 章的三档审核：粗看 / 细审 / 跑测试。

常见的审错：

- 只看“改了什么”不看“还改了什么不该改的”
- 相信 Claude 说的“只改了 A”——它可能顺手动了 B
- 没看范围，结果动到了你不想动的文件

步 5：Decide（决定）

三选一：

- Yes：全盘接受
- No + 反馈：告诉它哪里不对
- Edit（如果版本支持）：手动调整它的方案

⚠ 最容易犯的错：看完 diff 觉得哪里别扭但说不清，懒得挑毛病直接 Yes——这往往是后面出坑的起点。不确定就 No，没坏处。

步 6：Execute（执行）—— Claude 真做

Claude 按你的决定改文件 / 跑命令。这一步你不需要做什么，等它报结果。

步 7：Verify（验证）—— 最容易被跳过的一步

Claude 说“改完了”≠改对了。你必须自己确认：

- 改了文件 → 切图形界面打开看一眼，或让 Claude 再 @文件名 读出来对比
- 跑了命令 → 看输出有没有报错；看预期的效果真发生了吗
- 改了多文件 → 挑几个看，不要 100% 相信它的“全部改完”报告

验证不通过怎么办：直接进入下一轮循环——从步 1（描述什么不对）开始。

8.4 三种常见跑偏 + 怎么拉回来

跑偏 A：步 1 描述模糊，步 3 提案离谱

症状：你让“帮我整理一下”，Claude 整理出来的东西完全不是你要的。拉回：回到步 1——补充具体要求：范围、方式、成功标准。不要让 Claude 继续猜。

跑偏 B：步 4 审核不仔细，步 7 才发现问题

症状：diff 时没细看，接受后打开文件才发现“咦，这段怎么没了？”拉回：

1. 立刻 `/rewind` 或 `Esc Esc` 回到改前（第 9 章详讲）
2. 下次同类任务升级审核档位——从粗看升到细审

跑偏 C：没走步 7（跳过验证）

症状：Claude 说“已完成”，你信了，过两天用那个文件发现不对。拉回：养成“每次 Yes 后扫一眼结果”的肌肉记忆。30 秒的验证省后面 30 分钟的救火。

8.5 把循环刻进肌肉——一个完整演示

一次“整理会议纪要”任务的完整七步：

步	谁做	内容
1	你	“@会议纪要/ 把里面的 5 份笔记合并成一份摘要，每份笔记保留 3 条要点，按日期倒序。放到 <code>~/Desktop/摘要.md</code> ”
2	Claude	读 5 份笔记，告诉你：“读完了，我看到日期分别是 …，我打算这样合并 …”
3	Claude	生成 diff：新建 <code>摘要.md</code> + 展示内容大纲
4	你	粗看：大纲结构对、日期顺序对 ☒ ；想了想某份笔记想保留 4 条不是 3 条
5	你	选 No：“第三份笔记‘2026 年战略’保留 4 条要点”
—	—	回到步 3，Claude 重出 diff
4'	你	看新 diff——OK
5'	你	Yes
6	Claude	真创建 <code>摘要.md</code>
7	你	图形界面打开 <code>摘要.md</code> 浏览一遍——确认 OK

整个任务 3-5 分钟。如果跳过步 4 的审核、步 5 的 No 修正、步 7 的验证——你可能得到一份格式不对或内容错漏的文件，后面再修反而更花时间。

本章小结

- Claude Code 的一次任务是七步循环，不是简单问答
- 你占 4 步（描述 / 审核 / 决定 / 验证），Claude 占 3 步（分析 / 提案 / 执行）
- 你永远在开车——方向盘一直在你手上
- 三种跑偏：描述模糊 / 审核不仔细 / 跳过验证——按步骤回头修
- 验证最容易被跳，但省时最多——每次 Yes 后 30 秒扫一眼

动手任务

任务 1：用七步法完成一次小任务（15 分钟）

找个你真实想做的小任务（300 字内能描述完），按七步走一遍：

步骤：

1. 写下你的描述（至少三行，符合 WHAT/WHERE/HOW/VERIFY）
2. 启动 Claude Code，把描述丢进去
3. 观察它分析阶段说了什么——理解对了吗？
4. 看它的 diff / 命令
5. 做决定（Yes / No / 先 No 后 Yes）
6. 让它执行
7. 验证——打开结果看或跑一下

成功标志：你走完七步、每步都有意识地停顿确认了一下。

任务 2：故意让它跑偏，然后拉回（10 分钟）

步骤：

1. 故意给一个模糊指令，比如：“帮我改一下 @笔记.txt ”
2. 看 Claude 生成的 diff——十有八九不是你想要的
3. 识别这是“跑偏 A”（描述不够具体）
4. 选 No，补充具体要求（“我要改的是：把时间格式统一成 24 小时制，其他别动”）
5. 看新 diff，Yes

成功标志：你体验到“模糊描述的代价”以及“No + 反馈”的修复路径。

如果你卡住了

症状 A：每次都要走完 7 步好累

- 原因：简单任务确实不用那么正式。
- 解决：简单任务（改错字、加个标点）可以压缩到 3-4 步（描述→审核→Yes→瞄一眼结果）。复杂任务才需要完整 7 步。

症状 B：步 2 Claude 分析时我看不懂它在干嘛

- 原因：分析阶段 Claude 可能在读文件、查资料，会刷一堆“工具调用”信息。
- 解决：不必每条都看，等它输出总结再深入看。总结不对劲就打断。

症状 C：步 7 验证时发现不对，不知道怎么回到步 1

- 原因：你不知道怎么回退。
- 解决：/rewind 或 Esc Esc（第 9 章讲）。或者直接告诉 Claude：“刚才那个改动不对，帮我回滚，然后按我新的要求再做”。

主线结束。下面两条支线讲“为什么叫循环”、以及多轮迭代的小技巧。

 【支线】—— 可选深入（学有余力再看）

支线 8.A：为什么叫“循环”，不是“流程”

这段讲什么：为什么强调是 loop（循环）而不是 pipeline（流水线）。什么时候回头读：你好奇这个设计选择的背后思想。

流水线：A→B→C→D，从头走到尾，一次过，结束。循环：走到某一步发现不对，退回任意前面步骤，继续迭代。

Claude Code 设计成循环是因为：

- 现实复杂：一次性描述清楚一个需求几乎做不到
- 人会改主意：走到一半发现“其实我要另一个方向”

- AI 会犯错：它的第一个方案不一定是最好的

循环的意义：不要追求一次做对，追求几次迭代后做对。

这也是为什么 `/rewind` / `Esc Esc` / `No` + 反馈 这些“回退”机制存在——它们让循环可以“倒转”，不只是往前冲。

支线 8.B：多轮迭代的小技巧

这段讲什么：当一个任务需要多轮调整时，怎么让对话保持效率。什么时候回头读：你在跟 Claude 打磨一个东西（文档、代码、设计）需要反复修改时。

技巧 1：每轮迭代说“保留 X，改 Y”

不要每次都重新描述全部需求。每轮反馈只说增量：

保留上一版的结构不变，只改第二段的语气——更直白一点。

技巧 2：累计到一定程度就 `/clear` 重来

迭代超过 5-6 轮后，上下文里全是旧版本的痕迹，Claude 容易被干扰。这时：

`/clear`

然后把最终需求 + 当前版本内容一起发给它，相当于新对话。干净高效。

技巧 3：让 Claude 记录“已确定的约定”

对大任务：

到目前为止我们已经确定了：

1. 文档用 Markdown 格式
2. 每章不超过 2000 字
3. 每章末尾加摘要

请记住这些。下面继续写第三章。

把已达成共识显式列出来，避免 Claude 在迭代中漂移。

技巧 4：中间阶段保存

多轮迭代中，满意的中间版本可以提前 `git commit`（或复制一份到备份文件夹）。后面改坏了能回到任意中间点。

下一章讲 Claude Code 的“安全网”—— `/plan`（先规划不动手）+ `Esc Esc`（快速撤销）+ `/rewind`（精确回退）。这三个在手，你就不怕改坏。

第 9 章：计划模式与撤销——Claude Code 的安全网三件套

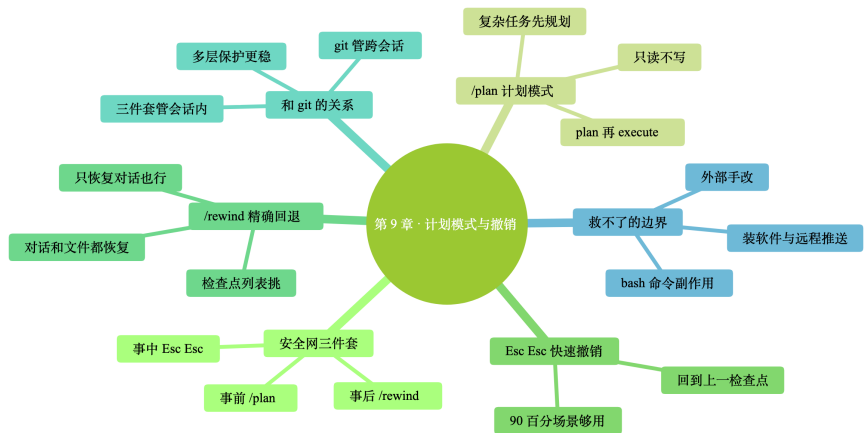
本章在全书的位置：第二部分 · 第 9 章 / 预估阅读+动手时长：主线 25-30 分钟 + 支线 10 分钟

前置章节：Ch 7（权限）+ Ch 8（交互循环）

学完能做什么（主线）：熟练使用 Claude Code 的三个“救命”机制——`/plan`（只规划不动手）、`Esc Esc`（一键撤销单步）、`/rewind`（精确回到历史检查点）。改错了不慌，还能让复杂任务有计划地推进。

开场三问

- 你会遇到什么问题：复杂任务不知道怎么下手——直接让 Claude 改怕改错；改到一半发现方向错了不知道怎么回到起点。
- 不读这章会踩什么坑：走一步算一步，踩坑回不去——只能从头重做；或者害怕踩坑所以啥都不敢让它做。
- 读完你会多会什么事：对任何复杂任务先 `/plan` 规划、出事 `Esc Esc` 一键回退、想精确回到历史某个时刻用 `/rewind`。这三个连用，你的“改错焦虑”基本消除。



🎯 【主线】—— 本章必读核心

9.1 安全网三件套——先认一下

Claude Code 给了你三个层级的保护：

工具	什么时候用	保护范围
<code>/plan</code> （计划模式）	任务开始前	让 Claude 只规划不动手——根本不给它出错的机会
<code>Esc Esc</code> （两次 Esc）	刚做完一步发现不对	一键撤销最近一个操作
<code>/rewind</code> （精确回退）	想回到更早的某个时刻	按检查点列表挑一个时间点回退（对话 + 文件都恢复）

三层保护的关系：

最前面： <code>/plan</code>	→ 尽量不让出事
中间： <code>Esc Esc</code>	→ 刚出事立刻救
事后： <code>/rewind</code>	→ 已经过了几步也能救

下面一个个讲清楚。

9.2 /plan 计划模式——让 Claude 只看不动

是什么

/plan 是一种权限模式（第7章提过）——在这个模式下：

-  Claude 可以读文件、分析、思考方案、给你方案
-  Claude 不能改任何文件、不能跑会改东西的命令
它只能说话，不能动手。

什么时候值得先 /plan

- 复杂任务：涉及多文件、多步骤、你没有把握
- 陌生项目：你自己都没看过的代码/文件夹，先让它探查
- 高风险改动：改生产配置、动重要数据前
- 你拿不准方案：不知道是方案 A 好还是方案 B 好，让它两个都分析

 反面：简单明确的改动（“把这个文件里'foo'全换成'bar'"）不用 /plan，直接做就行。

怎么用

方式 A：启动时进入

```
claude --permission-mode plan
```

整个会话都是计划模式。

方式 B：会话中切换

对话里打：

```
/plan
```

切进计划模式。

方式 C：切回来做事

计划结束、方案定了后：

```
/permissions
```

切回默认模式（acceptEdits 也行），让 Claude 执行方案。

一个典型的 plan → execute 流程

（计划模式下）

你：@src/ 帮我看这个项目整体架构，分析一下如果我想加一个"用户反馈"功能，要改哪些文件

Claude：（读文件、分析）我看到这个项目是 ... 要加"用户反馈"，建议改以下文件：

- A.tsx（新增表单）
- B.ts（API 接口）
- C.md（文档更新）

步骤：...

你：方案看起来可行。把 A 和 B 按这个方案先做，C 我自己来。

（切回默认模式）

/permissions → 选默认

你：按刚才的方案开始改 A 和 B

Claude：（生成 diff，正常审核循环）

这种“先规划后执行”的流程叫 plan → execute，本书后面多处推荐。

9.3 Esc Esc——快速撤销最近一步

是什么

连接两次 Esc 键（键盘左上角 Escape）——一次、松开、再一次——触发快速撤销。

作用：回到上一个检查点（通常是你做最近一次改动之前的状态）。

什么时候用

- 刚让 Claude 改了个文件，打开看发现改坏了
- 刚跑了个命令，效果不对
- 你刚发了条消息但想反悔（退回到发之前）

一个典型场景

你: @config.json 把 port 从 8080 改成 9090
 Claude: [生成 diff] ...
 你: Yes
 Claude: 改好了。
 你: (打开文件看) 哦不, 我意思是 port 不动, 改的是 host—赶紧撤销

→ 按两次 Esc

你: 回到改之前了。重新说: @config.json 把 host 从 localhost 改成 127.0.0.1

一次 Esc Esc 省了你手动恢复的麻烦。

和 /rewind 的区别

Esc Esc = “回到上一步” (快) /rewind = “从历史检查点列表里挑一个” (灵活)
 日常 90% 场景 Esc Esc 就够了。

9.4 /rewind——精确回到历史检查点

是什么

在对话里打:

```
/rewind
```

会弹出一个检查点列表——Claude Code 在对话过程中自动记录的“状态快照”。每个检查点通常对应:

- 你发出了一条消息
- Claude 完成了一次工具调用 (读文件、改文件、跑命令)

你用方向键选一个检查点, 回车确认。

你能恢复什么

/rewind 回退的是 Claude Code 会话状态:

- ☒ 对话历史 (消息会被回退到那个时刻)
- ☒ Claude 在本次会话中改过的文件 (恢复到检查点时的内容)

- **✗** 不会影响你手动（在图形界面或其他终端）改过的文件
- **✗** 不会影响 Claude 跑命令改过的系统设置（比如装的软件）

选“恢复代码 + 对话”还是“只恢复对话”

有些版本的 `/rewind` 允许你选：

选项	效果
恢复对话 + 文件	对话和 Claude 改过的文件都回到检查点
只恢复对话	文件不动，只把对话历史回退（适合你想“保留改动但换个思路重来”）

新手默认选“恢复对话 + 文件”。

一个典型场景

[30 分钟前] 你让 Claude 改了 A、B、C 三个文件

[15 分钟前] 你又改了 D、E、F

[现在] 你发现从改 D 开始方向就不对了，想退回到只改 A/B/C 的状态

→ `/rewind`

→ 列表里找到“改 D 之前”的那个检查点

→ 回车确认

现在：A/B/C 的改动保留、D/E/F 的改动消失、对话回到改 D 前的瞬间。

你可以基于那个时刻重新决定怎么继续。

9.5 三件套和 git 的关系

如果你已经在用 git（代码版本控制），可能会问：“有 git 了还要这三个吗？”

答案：都要。它们是不同的层级的保护。

工具	作用范围	粒度
Claude Code 安全网	一次 Claude Code 会话内	按“消息级”——细到每一条你发的消息
git	你整个项目的版本历史	按“提交级”——你 commit 一次就是一个节点
编辑器 Ctrl+Z	单个文件在单次编辑中的修改	字符级——最细
系统备份（Time Machine / 文件历史）	全盘	按天/小时的快照

搭配建议：

- 1. 开始任务前 `git commit` 一下 → 有个“干净基线”
 - 2. 任务中用 `/plan` / `Esc Esc` / `/rewind` → 在 Claude Code 会话内快速回退
 - 3. 任务完成后 `git commit` → 新的“干净基线”
 - 4. 过几天回头看发现问题 → `git revert` 或从备份恢复
- 这样你有四层保护，几乎不可能“永久改坏”一个文件。

💡 不懂 git 也没关系——Ch 25（团队协作基础）会从零讲 git 基础。现在记住：Claude Code 的三件套是“会话内的即时回退”，git 是“跨会话的长期版本管理”。

本章小结

- 安全网三件套：`/plan`（不让出事）+ `Esc Esc`（立刻救）+ `/rewind`（事后救）
- `/plan` 计划模式：只读不写，适合复杂 / 陌生 / 高风险任务；`plan` → `execute` 是推荐流程
- `Esc Esc`：回退最近一步，90% 日常场景够用
- `/rewind`：按检查点列表挑时间点回退，对话 + 文件都恢复
- 和 git 配合：三件套管会话内、git 管跨会话，多层保护

动手任务

任务 1：用 plan 模式规划一个任务（10 分钟）

步骤：

1. 进 ccguide-练习，`claude --permission-mode plan` 启动
2. 输入：
@笔记.txt 我想把这份笔记改造成 GTD (Getting Things Done) 风格的任务管理——规划一下改造方案，不要改文件
3. Claude 会给你一个方案（只说不做）
4. 看方案：`/permissions` 切回默认
5. 说“OK，按方案开始”
6. 正常审核 diff

成功标志：你体验了“先规划再执行”的流程，并且计划模式下确实没动文件。

任务 2：体验 Esc Esc（5 分钟）

步骤：

1. 默认模式下，让 Claude 随便改一下 笔记.txt （比如“把标题改成大写”）
2. 审核 → Yes → 改完
3. 按两次 Esc（一次、松开、再一次）
4. 看提示：回退成功
5. 切图形界面打开 笔记.txt 确认回到了改之前

成功标志：Esc Esc 撤销成功，文件回到未改状态。

任务 3：用 /rewind 选一个更早的检查点（5 分钟）

步骤：

1. 在 Claude Code 里做 3-4 次改动（不用复杂，就是让它改几次文件）
2. 打 `/rewind`
3. 看弹出的检查点列表（应该有 3-4 条）
4. 选一个比“最近一次”更早的检查点（比如第 2 次改动之前的）
5. 确认

成功标志：文件状态和对话都回到了你选的那个时刻。

如果你卡住了

症状 A：`/plan` 模式下让它改东西它拒绝了

- 原因：这是计划模式的预期行为，它就是“不给改”。
- 解决：`/permissions` 切到默认或 `acceptEdits` 模式

症状 B：Esc Esc 按了没反应

- 原因：按太慢（中间间隔太长会被认为是两次独立按键）、或当前没有可回退的操作
- 解决：连续按快一点；或用 `/rewind` 从列表选

症状 C：`/rewind` 恢复后文件还是错的

- 原因：那个文件被你在 Claude Code 外面（图形界面、其他程序）改过——`/rewind` 不管外面的修改
- 解决：手动恢复，或从 git / 备份拿

症状 D：`/rewind` 说没有检查点

- 原因：刚开始对话没几步，还没生成足够多的检查点
- 解决：这个会话暂时没救，用 git / Ctrl+Z / 备份

主线结束。下面支线讲 `plan` → `execute` 当默认 workflows、以及 `/rewind` 的边界。

🌿 【支线】—— 可选深入（学有余力再看）

🌿 支线 9.A：把 `plan` → `execute` 当默认 workflow

这段讲什么：一个值得养成的习惯——几乎所有不那么简单的任务都先 `/plan`。什么时候回头读：你用了几周、开始做更复杂任务时。

我推荐的“默认升级”

早期（前 1-2 周）：大多数任务直接做，踩坑后学习。

中期（2-4 周）：稍复杂任务（3 步以上、涉及多文件）默认先 `/plan`：

(启动就进 plan 模式)

```
claude --permission-mode plan
```

然后：

@目标文件/文件夹 分析一下，告诉我要做 X 该怎么做

看 Claude 方案 → 不满意就让它调整方案 → 满意后：

/permissions → 切回 default

开始做

好处

- 避免“做一半发现方案错了”的浪费
- 你有机会中途纠偏（用嘴改方案比用 diff 改实际文件快 10 倍）
- 复杂任务的“整体感”更强

代价

- 多花 3-5 分钟讨论方案
- 简单任务走这一圈显冗余

💡 判断标准：任务预计需要 Claude 做 3 步以上（多次改文件或跑命令），先 `/plan`。1-2 步的直接做。

支线 9.B：/rewind 的边界——它能和不能救什么

这段讲什么：`/rewind` 具体能恢复什么、不能恢复什么。什么时候回头读：你试图 `/rewind` 但发现没救回来想搞清楚原因。

`/rewind` 能救的

- ☒ Claude Code 本次会话中发出的消息（回退到那个时刻）
- ☒ 本次会话中 Claude 用 Edit/Write 工具改过的文件（恢复到检查点的内容）

-  本次会话中 Claude 新建的文件（会被删除，回到“未建”状态）

/rewind 救不了的

-  你在图形界面 / 其他终端 改过的文件
-  Claude 通过 `bash` 命令改过的东西（因为 `bash` 命令的结果 Claude Code 不知道该怎么回退）——比如它跑了 `rm file.txt`，`/rewind` 不会把文件变回来
-  Claude 安装的软件 / 改的系统设置
-  已经被 `git commit` / `push` 的东西（那是 `git` 的事）
-  已经关闭会话后再打开——之前的检查点不一定还在

边界清楚了怎么办

高风险操作的保护策略：

操作类型	保护方式
Claude 改文件（Edit/Write）	<code>/rewind</code> 就能救
Claude 跑 <code>rm</code> 、 <code>mv</code> 等命令	<code>/rewind</code> 救不了 → 操作前手动备份 / <code>git commit</code>
装软件、改系统设置	手动记录可逆的步骤
远程操作（ <code>git push</code> 、调 API）	操作前就要特别小心——发出去的不一定收得回

心法

`/rewind` 是便利工具而非终极保险。重要场景的终极保险永远是：

1. 预防：`/plan` 模式先规划
2. 备份：`git commit` / 手动复制一份
3. 分步小心：一步一步做、每步验证——而不是一口气 10 步然后指望回退

下一章讲这一部分的最后一个话题——信任校准：什么时候该相信 Claude、什么时候必须深度审核。

第 10 章：信任校准——什么时候信，什么时候核对

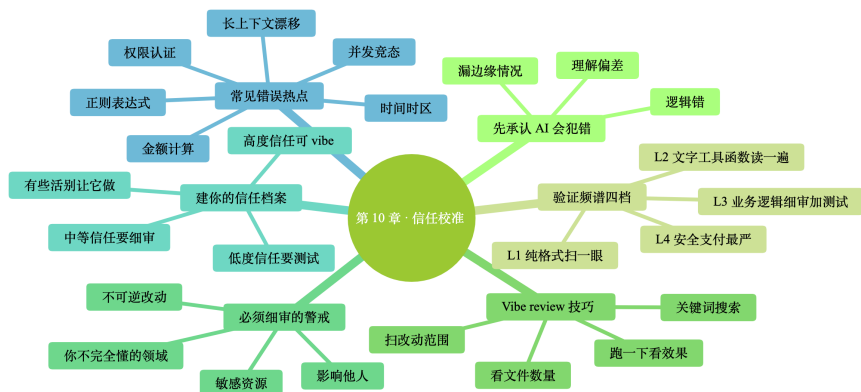
本章在全书的位置：第二部分 · 第 10 章 / 预估阅读+动手时长：主线 25-30 分钟 + 支线 10 分钟

前置章节：Ch 6 (diff 审核) + Ch 8 (交互循环)

学完能做什么（主线）：建立“信任校准”能力——面对不同类型的改动，能快速判断该用多深的审核、哪些必须逐字检查、哪些可以扫一眼通过。这是把 Claude Code 用熟、不翻车的核心判断力。

开场三问

- 你会遇到什么问题：前几章让你“审核 diff、走完七步、用安全网”——但审核到什么程度才够？全部细读太累，全部粗看又怕翻车。
- 不读这章会踩什么坑：两头极端——要么过度谨慎（每次都逐字审、累到不想用）、要么过度信任（盲目 Yes 然后翻车）。
- 读完你会多会什么事：按改动风险分档检查，该粗看粗看、该细审细审、该跑测试跑测试——稳步用熟 Claude Code 且基本不翻车。



【主线】—— 本章必读核心

10.1 先承认一个事实：AI 确实会犯错

很多教程避开这个话题，本书正面讲清楚：

🔪 AI 生成的内容和改动，出错率比你熟练的人写的高。

不是故意打击信心——而是你必须建立正确预期才能用好它。具体来说：

- 逻辑错误：AI 写的代码里逻辑错误比人类写的多约 1.7 倍
- 安全漏洞：涉及权限/认证/加密时，AI 代码出现漏洞的概率更高
- 边缘情况：AI 容易漏处理“空输入”、“超长数据”、“特殊字符”这类边缘
- 理解偏差：你说 A，它可能理解成 B，做出来是 C

💡 但这不代表 AI 不能用——只代表你不能盲目信。下一节讲怎么分情况校准信任。

10.2 验证频谱——按改动风险选检查深度

不是所有改动都要同样深度审核。用下面这个“频谱”决定：

改动类型	典型例子	建议深度	时间预算
L1 纯格式 / 配置	缩进、空行、JSON 字段排序、文件重命名	扫一眼	10-30 秒
L2 文字类 / 工具函数	改文案、重命名变量、小的帮手函数	读一遍 + 跑一下看效果	1-2 分钟
L3 业务逻辑	改功能、加判断条件、调接口参数、合并数据	细读 diff + 跑测试 + 想 3 个边缘情况	5-15 分钟
L4 安全 / 支付 / 权限相关	认证、加密、支付、用户权限、敏感数据处理	最严 + 静态检查工具 (如有) + 让第二个人 review	15-30 分钟

怎么对号入座

面对一个 diff 或命令，先问自己两个问题：

1. 改的是什么类型？参考上面的 L1-L4
2. 改坏了能不能快速恢复？能（有 git commit、有备份、不影响别人）→ 可以降一档；不能（影响生产、发给了客户、改了用户数据）→ 升一档

10.3 Vibe review——中间策略

很多新手两极化：要么逐字检查，要么完全不看。你需要学会 **vibe review**（凭“感觉”快速审）——不是偷懒，是高效。

vibe review 的动作

30 秒到 1 分钟流程：

1. 瞄一眼 diff 有几个文件——和你预期的范围一致吗？
2. 扫改动范围——动的地方是你让它动的吗？有没有“额外惊喜”（它顺手改了别的）？
3. 关键词搜索——如果是代码：有没有 `TODO`、`FIXME`、`any`、被注释掉的片段；如果是文档：有没有错别字、重复段落

4. 跑一下看效果——打开文件快速浏览；跑个测试看有没有报错
感觉都对 → Yes 有任何“怪怪的”感觉 → 升级到细审或 No 让它澄清

vibe review 能处理什么、不能处理什么

✅ 能处理：80% 的日常任务——改文档、整理文件、重命名、加小功能 ❌ 不能处理：涉及安全、支付、用户数据的任务——永远 L4 深度审核

10.4 必须细审的几类场景（警戒清单）

看到下列情况时，不能 vibe review，必须细审：

警戒 1：触及敏感资源

- 密码、API key、证书、用户隐私信息
- 支付接口、账单、退款逻辑
- 权限判断（谁能看什么、谁能改什么）
- 数据库写入、删除、更新

警戒 2：不可逆的改动

- `rm` 删文件（尤其批量）
- `git push --force`、`git reset --hard`
- 覆盖性命令（`mv A B` 当 B 已存在）
- 数据库 `DROP`、`DELETE`

警戒 3：影响他人的改动

- 提交代码到团队共享仓库
- 改生产环境配置
- 发邮件、发 Slack 消息
- 对外发布内容

警戒 4：你不完全懂的领域

- 让 Claude 写一段你不懂语言的代码（比如你是 Python 用户，它给你生成 Bash 脚本）
- 改你没接触过的工具配置（比如 nginx、docker）
- 涉及你不熟悉的协议、API

🚨 警戒清单里的任何一条出现 → 强制升级到 L4，哪怕任务看着简单。

10.5 建立你自己的“信任档案”

信任不是一个固定值。对不同类型的任务、不同的代码库，Claude 的靠谱度不同。记下来，形成你自己的信任档案：

怎么记

用一个简单的笔记文件（或记在 CLAUDE.md 里，第 14 章讲）：

```
# 我对 Claude 的信任档案

## 高度信任 (可 vibe review)
- 改文档文案、整理笔记
- 文件重命名、整理分类
- 简单的 Python / JavaScript 小脚本
- 生成 Markdown 表格

## 中等信任 (细审)
- 修 bug (它容易改多余的地方)
- 加新功能 (它容易漏边缘情况)
- 数据清洗 (易出错，需跑样例验证)

## 低度信任 (严审 + 测试)
- 认证 / 加密相关代码
- 数据库 schema 变更
- 正则表达式 (经常写错)
- 金额 / 时间 / 时区处理

## 不用 Claude 做的
- 删除重要文件 (宁可手动)
- 修改生产配置 (Claude 给建议，我自己动手)
- 写法律 / 合同 / 医疗 / 金融专业内容
```

这个档案怎么更新

- 每次 Claude 犯错，记下是哪类任务——下次降低信任
- 每次它表现很好，也可以记下——以后类似任务可以省审核时间
- 每 2-3 个月回头看，随着 Claude 版本升级、你自己熟悉度提升，重新评估

本章小结

- AI 会犯错，出错率比熟手高——不能盲目信
 - 验证频谱 L1-L4：按改动风险选审核深度（纯格式扫 10 秒、业务逻辑细审 15 分钟、安全相关更严）
 - Vibe review：30-60 秒的“凭感觉快扫”——能处理 80% 日常任务
 - 必须细审的四类警戒：敏感资源 / 不可逆 / 影响他人 / 你不完全懂的领域
 - 建你自己的信任档案——哪类任务可以放手、哪类必须严审
-

动手任务

任务 1：给过去做过的改动分级（10 分钟）

回顾你前几章已经让 Claude 做过的改动，用 L1-L4 给每一件事打分：

- 改 笔记.txt 加编号 → L?
- @日程.md 改 12 小时制 → L?
- Ch 6 任务里让它精简文字 → L?

成功标志：你能对照 10.2 节的频谱表做出判断。

任务 2：实战一次“先判断后审核”（15 分钟）

步骤：

1. 找一个你真实想做的任务（任何类型）
2. 动手前先想：这是 L 几？需要多深审核？
3. 用 Claude Code 做
4. 按你之前判断的档次审核
5. 做完后反思：档次判断对吗？如果重来，还是这个档吗？

成功标志：你开始有意识地在“做”之前先“判断审核深度”，而不是每次都一刀切。

任务 3：起草你的信任档案（10 分钟）

新建一个文件 `~/Desktop/ccguide-练习/信任档案.md`。照着 10.5 节的模板，写下你目前对 Claude 的信任分级——至少 3 项高度信任 + 3 项需要严审。

成功标志：你有了一份个人化的信任档案，以后做类似任务时能直接查。

如果你卡住了

症状 A：判断不出改动是 L 几

- 原因：前期对各类任务风险没经验。
- 解决：新手倾向升一档——L2 的东西当 L3 来审。踩几次坑后判断力会准。

症状 B：vibe review 失误了，放走了错误

- 原因：vibe 能力还没建立。
- 解决：记下这次失误的类型（比如“加条件语句类的 vibe 审漏了”），下次同类任务升级到细审 1-2 个月，直到 vibe 重新稳定。

症状 C：对所有东西都想升档到最严，审不过来

- 原因：过度谨慎。
- 解决：量化风险——改坏了影响是“我自己多花 5 分钟修”vs “客户损失 1 万块”？两个审核深度当然不同。

主线结束。下面两条支线讲“AI 最容易出错的典型场景”和“团队里怎么统一审核标准”。

 【支线】—— 可选深入（学有余力再看）

支线 10.A：AI 最容易出错的几个典型场景

这段讲什么：具体哪些情况下 AI 特别容易翻车，提前知道就能针对性防御。
什么时候回头读：你踩过坑、好奇是不是有规律可循。

错误热点 1：时间和时区

- “今天”/“明天”/“三天后”——AI 有时不知道“今天”是几号
- 时区换算——UTC / 北京时间 / 纽约时间，经常算错
- 闰年、月底日期——2024-02-29、1 月 31 日加一个月

防御：给它一个明确的日期（“2026-04-19”），或让它先说今天是几号再算。

错误热点 2：金额和计算

- 浮点数精度 ($0.1 + 0.2 \neq 0.3$)
- 货币单位混淆 (分 vs 元)
- 百分比 vs 小数
- 汇率换算

防御：给具体例子验证 (`100.55` 应该得出 `150.82`)，让它写测试跑一下。

错误热点 3：正则表达式

AI 写正则经常漏边缘情况。

防御：让它写正则时必须附带测试用例：能匹配的例子 + 不能匹配的例子，跑一下验证。

错误热点 4：权限 / 认证

- 谁能访问什么资源
- Token 刷新逻辑
- 多用户隔离

防御：这类必须人类严审 + 专门的安全测试，不要靠 vibe review。

错误热点 5：并发和竞态

- 同时改一个文件
- 异步操作的顺序
- 锁机制

防御：这是专业领域，普通用户碰到就 `/plan` + 找懂的人 review。

错误热点 6：长文档上下文遗忘

- 改长文档时，AI 可能忘记前面讨论过的约定
- 同一对话聊太久，前面的设定“漂移”

防御：关键设定定期重申；对话长了用 `/compact` (第 12 章讲) 或 `/clear` 重启。

支线 10.B：团队里怎么统一审核标准

这段讲什么：如果你要和同事一起用 Claude Code，怎么避免每个人标准不一致。什么时候回头读：你公司 / 团队开始大规模用 Claude Code 时。

问题

不同人的“什么该细审”不一样——马力觉得一眼过的东西，李四可能会翻车。

解决方案 1：写一份团队审核约定

放在项目 CLAUDE.md 或专门的文档里，例：

```
# 团队 Claude Code 审核约定

## 必须细审 (至少 L3)
- 任何涉及 `src/auth/` 目录的改动
- 任何涉及支付 (`src/payment/`)、用户数据 (`src/user/`) 的改动
- 数据库迁移 (`migrations/`)
- 对外 API 接口变更

## 可 vibe review (L1-L2)
- 改 README / 文档
- 前端样式调整
- 测试文件 (已有测试的增补)

## 绝对不用 Claude 做的
- 数据库 DELETE / TRUNCATE
- 生产环境部署脚本
- 删除任何含有用户数据的文件
```

解决方案 2：强制 code review

Claude 生成的改动，在 git commit message 里标记（比如加 [AI] 前缀），团队成员 review 时格外留意。

解决方案 3：自动化检查

对高风险目录加 git hook / CI 检查——敏感目录改动必须多人 review、必须跑完整测试。

（这些都属于进阶团队协作，Ch 25 会再讲一些基础。）

恭喜——你已经完成第二部分（日常使用）全部 6 章。

你现在具备的能力：

- 读文件（@ 引用 + WHAT/WHERE/HOW/VERIFY 模板）
- 改文件（diff + 三档审核）

- 跑命令（权限机制 + 危险识别）
- 工作节奏（交互循环七步法）
- 安全网（/plan、Esc Esc、/rewind）
- 判断力（信任校准）

下一部分（第三部分：深度概念）开始：隐私与数据、上下文原理、三层记忆、CLAUDE.md、模型与成本——这些是让你从“会用”到“用得溜”的关键。

第三部分 · 深度概念

第 11 章：你的数据去哪了——隐私与数据流

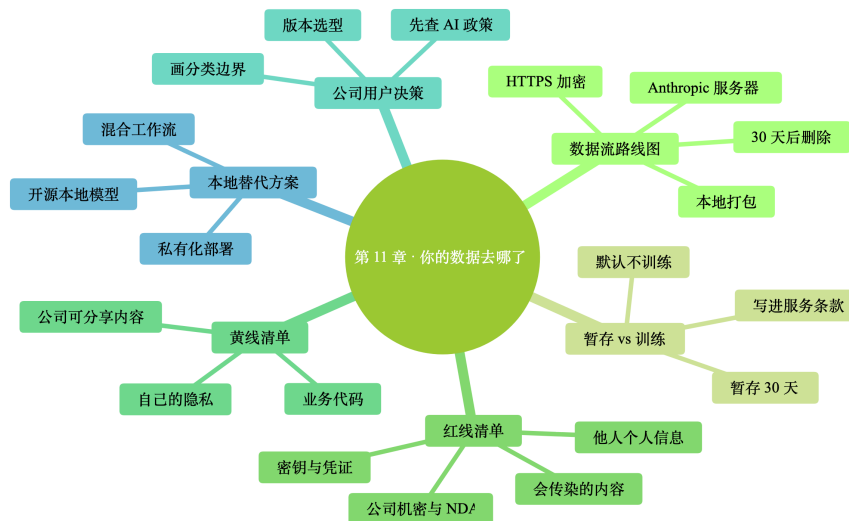
本章在全书的位置：第三部分 · 第 11 章 / 预估阅读+动手时长：主线 20-25 分钟 + 支线 10 分钟

前置章节：Ch 4（Claude Code 是什么）+ Ch 7（权限机制）

学完能做什么（主线）：搞清楚你发给 Claude 的文字、@ 引用的文件、! 跑的命令，它们去了哪、停留多久、会不会被人看、会不会用来训练模型。从此用 Claude Code 时心里有底。

开场三问

- 你会遇到什么问题：工作文件里有客户名单、商业数据、个人笔记——交给 Claude 会不会泄露？公司会不会不允许用？
- 不读这章会踩什么坑：要么过度恐慌（什么都不敢给它看）、要么完全不设防（把密码、合同一股脑塞进去）。
- 读完你会多会什么事：能画出“一次 Claude 对话的数据流路线图”，知道什么能放、什么必须脱敏、什么绝对不能放；能回答同事或老板“这个工具安全吗”。



【主线】—— 本章必读核心

11.1 先看一张图：一次对话的数据去了哪

你在终端敲的每一句话，会经过下面这些环节：

你的电脑（本地）

1. 你在终端打字 → Claude Code CLI 收到
2. CLI 读取你 @ 引用的文件内容
3. CLI 读取项目里的 CLAUDE.md
4. CLI 把“你的话 + 文件内容 + 说明”打成一个包

HTTPS 加密（类似银行网银）

Anthropic 服务器（美国，可能也有其他区域）

5. 大模型 (Claude) 处理这个包，生成回复
6. 服务器把回复发回你的电脑
7. 日志在 Anthropic 这边**最多保留 30 天**之后删除（企业版有单独约定）
8. 这些内容**不用来训练模型**（2024-05 起的消费者用户可选；API / 企业版默认不训练）

关键事实要记住 5 条

1. 传输过程是加密的——像你登录银行网银，中间没人能偷看。
2. Anthropic 服务器会暂存你的内容——最多 30 天（用于审查滥用、调试），到期自动删除。
3. 默认不用来训练模型——API / Claude Code 用户默认就不训练；消费者 Claude.ai 用户 2024-05 起也可以在设置里关。
4. 文件读取发生在你本地，但内容会上传——@笔记.md 这个操作是 CLI 读完本地文件，把内容打进发给 Anthropic 的包里。文件本身不会被远程复制保存，但文件内容必然进入那个 30 天的暂存。
5. ! 跑的命令结果也会上传——你让 Claude 跑 `ls`、`git log`，结果会回显给 Claude 当作上下文。所以跑 `cat ~/.ssh/id_rsa` 这种事等于把密钥交给 Anthropic。

11.2 “不训练模型”到底是什么意思

很多人对“训练模型”概念模糊。说人话：

- 训练：把你的内容永久学进 AI 的“大脑参数”里。训练过的内容取不回来，未来某用户提问时可能间接“想起”你的内容。
- 暂存：只是放在服务器上日志里，30 天内能用于排查滥用、bug 调查，过期删除，不会进 AI 的大脑。

Claude Code 默认行为：

-  暂存 30 天（用于滥用排查）
-  不训练（你发的东西不会成为 AI 未来知识的一部分）

这是 Anthropic 对 API / Claude Code / 企业版用户的合同承诺，不是“应该会、大概不”——写在服务条款里。

✖ 这不等于零风险：30 天暂存期内，如果 Anthropic 内部排查或被司法机关合法调阅，你发过的内容理论上可被访问。下面讲哪些东西真的不能放。

11.3 红线清单：这些东西绝对别发给 Claude

无论工具多靠谱，以下类型的内容就不应该出现在任何云端 AI 里（包括 ChatGPT、Gemini、Claude 都一样）：

红线 1：凭证和密钥

- API key、token、密码、证书私钥（`~/.ssh/id_rsa`、`.env` 里的密钥）
- 数据库连接字符串（`mysql://user:password@host/db`）
- OAuth refresh token、Cookie、JWT
- 加密货币私钥、助记词

✖ 不能做：@.env 帮我加个新字段 —— .env 里的密钥就全传过去了 ✔ 改这样：把 .env 里的值替换成 `<REDACTED>`，或者单独把字段名告诉 Claude，不给值

红线 2：他人的个人信息

- 客户 / 用户的身份证号、手机号、家庭住址、银行卡号
- 身份证、护照、驾照、病历的扫描件 / 照片
- 未脱敏的真实员工档案 / 客户名单

✖ 不能做：@客户名单.xlsx 帮我整理成邮件分组 ✔ 改这样：先把名单里的身份证号、手机号替换成 `<ID_1> <ID_2> ...`，处理完再手动把原值填回去；或用假数据跑通流程，再自己套真数据

红线 3：公司机密 + 法律限制

- 商业合同、报价单、未公开的财务报表
- 核心代码（涉及核心算法、专利、商业模型）
- 保密协议（NDA）覆盖的内容
- 医疗、法律行业涉及个人隐私的档案（很多国家有明确法规）

✖ 不能做：直接把合同 PDF 交给 Claude 分析 ✔ 改这样：先问公司 IT / 法务部是否允许；或企业版 + 自建代理；或本地用开源模型

红线 4：会传染的内容

有些内容你以为是你的，其实别人的权利也掺在里面：

- 朋友发你的聊天截图（泄露对方隐私）
- 第三方付费资料（知识付费课件、订阅制数据）
- 公司下发的内部文档（对你免费，但对 Anthropic 不免费）

判断标准：把这个内容发给 Claude，等于把它交给了一家美国公司的服务器（哪怕只暂存 30 天）。你有权这样做吗？

11.4 黄线清单：要小心的内容（可以发但先处理）

红线之外，还有一批“不是绝对不行，但要多想一步”的：

黄线 1：自己的隐私信息

- 自己的身份证号、手机号、地址
- 自己的私人笔记、日记
- 自己的聊天记录、邮件

原则：你愿意承担它暂存 30 天的风险，就可以发。但注意上下文越长越难控制——这类信息发完后下次 `/clear`，别让它长期留在会话里。

黄线 2：公司内部可分享的内容

- 公开的产品资料、技术博客草稿
- 内部但非机密的工作流程、会议纪要
- 同事间能随意转发的邮件

原则：查一下公司 IT 政策。很多公司已经有明确的“AI 使用规范”——允许用 Claude / ChatGPT 处理哪些等级的内容。没有政策就问 IT/法务一次。

黄线 3：代码和项目

- 非核心算法的业务代码
- 没特别机密的配置文件
- 公开 GitHub 仓库的内容（这类基本没问题）

原则：如果代码开源或准备开源——随便发。如果是公司闭源商业代码——企业版 / 私有化部署更安全，个人版先问一下。

11.5 公司 / 团队用户的决策清单

如果你是在公司里用 Claude Code，走一遍下面这张清单：

① 公司有没有 AI 使用政策？

- 有：严格按公司政策（可能限制特定目录、特定数据类型）。
- 没有：先问一次（IT、法务、合规），别自己判断。很多踩坑的人是“没问就用”。

② 用的是哪个版本？

版本	数据暂存	训练模型	适合谁
个人 Claude Code	30 天	不训练	个人项目、开源、学习
Anthropic API（带自己的 key）	30 天	不训练	小团队、技术用户
Claude for Work / Enterprise	可定制（甚至 0 天）	不训练	公司、敏感数据
私有化部署（某些大客户）	数据不离开公司	不训练	金融、医疗、政府

如果公司涉及敏感数据，争取升级到企业版。

③ 哪些任务可以用、哪些必须本地处理？

画一张自己的分类表：

我的 Claude Code 使用边界

可以用（低风险）

- 整理个人笔记、写博客草稿
- 改开源项目代码
- 处理公开资料、翻译公开文档

先脱敏再用（中风险）

- 整理含个人数据的表格（先替换关键字段）
- 改公司内部文档（确认非机密后）

本地工具代替（高风险）

- 处理合同 / 财报 / 客户名单原始数据
- 改涉密代码
- 处理身份证、医疗档案

绝不交给任何云端 AI

- 密钥、密码、数据库凭证
- 未脱敏的身份证 / 银行卡
- NDA 覆盖的文档

本章小结

- 一次对话的数据会经过：本地 → HTTPS 加密传输 → Anthropic 服务器（美国）→ 30 天暂存 → 删除
- Claude Code 默认不训练模型——你发的内容不会进 AI 的大脑
- 红线：密钥、他人个人信息、公司机密、NDA 内容——绝不发
- 黄线：自己的隐私、公司内部非机密、业务代码——处理后或问过再发
- 公司用户先问 IT/法务；涉密场景争取企业版或私有化部署

动手任务

任务 1：画你自己的“使用边界”表（15 分钟）

步骤：

1. 新建 `~/Desktop/ccguide-练习/使用边界.md`
2. 照 11.5 的模板，列出“你自己工作里的”4 类：可以用 / 先脱敏 / 本地处理 / 绝不上传
3. 每类至少写 3 项具体任务

成功标志：看着这张表就能决定“这个任务能不能用 Claude”。

任务 2：查你公司的 AI 政策（10 分钟，如果在公司用）

步骤：

1. 搜索公司内网 / 邮件存档“AI 使用” / “Claude” / “ChatGPT”关键词
2. 如果找不到明确政策，发一封邮件给 IT/合规：“我想用 Claude Code 处理 XXX 类型任务，公司是否允许？”
3. 把回复存成 `公司AI政策.md`

成功标志：你有了明确的“上班可以用 / 不可以用”边界。

任务 3：脱敏练习（10 分钟）

步骤：

1. 找一个含敏感信息的真实文件（自己的通讯录、一份旧合同、一个 `.env`）
2. 复制一份（别改原件），手动把每个敏感字段替换成占位符（`<NAME_1>`
`<PHONE_1>` `<KEY_REDACTED>`）
3. 记下你做脱敏的步骤，下次类似文件复用

成功标志：你掌握了“先脱敏再给 Claude”的基本流程。

如果你卡住了

症状 A：我的老板担心数据泄露，不让我用

- 原因：合理——尤其涉及敏感行业。
- 解决：给老板看本章 11.1 的数据流图 + Anthropic 的隐私承诺链接；申请企业版试点；从非敏感任务开始建立信任。

症状 B：不知道自己的任务算不算敏感

- 原因：标准模糊。
- 解决：默认当作敏感处理——要么脱敏再用，要么问一次。“保守一次”的代价远小于“泄露一次”。

症状 C：不小心把密钥粘进去了怎么办

- 原因：手滑或忘了检查。
- 解决：立刻去那个服务作废这个密钥（GitHub、云服务、API 平台都有“revoke”按钮）；重生一个新的；别依赖“删除对话”——密钥已经传出去了。

主线结束。下面支线讲“数据保留的细节”和“本地化替代方案”。

 【支线】—— 可选深入（学有余力再看）

支线 11.A：数据保留的几个细节

这段讲什么：30 天暂存、不训练——背后更准确的技术细节。什么时候回头读：合规审查时，或要给老板 / 法务讲清楚时。

保留 30 天的内容具体是什么

- 你发的所有 prompt（输入）
- Claude 返回的所有回复
- Claude Code 使用的元数据（工具调用日志、错误信息）

这些数据目的是“滥用检测”和“服务质量”——比如有人用 API 做违法事，Anthropic 能追溯。

企业版怎么更严

- 零保留（Zero retention）：数据过一下就扔，不暂存（某些套餐可配）
- 区域化部署：数据只在特定区域处理（欧盟、亚太），不跨境
- 审计日志自控：你可以自己保存、自己管理

如果你很在意

- 消费者 Claude.ai 用户：设置 → Privacy → 关闭“Help improve Claude”（但这个开关不影响 Claude Code/API）
- Claude Code / API 用户：默认就不训练，无需额外操作
- 企业级：签合同同时明确要求 Zero Retention / 私有化

一件事要知道

“不训练”的承诺是写在服务条款里的正式条款。Anthropic 如果违反，是合同违约——不是一句“我们原则上不这么做”。

支线 11.B：真的不敢上传，有没有本地替代

这段讲什么：如果你要处理的内容实在敏感，本地跑 AI 有哪些路。什么时候回头读：确定了有内容不能云端处理时。

方案 1：本地开源大模型

用 Ollama / LM Studio 在本机跑开源模型 (Llama、Qwen、DeepSeek、Mistral 等)，完全离线——你发的内容不出你的电脑。

- 优点：数据 100% 留在本地；长期免费
- 缺点：智能度不如 Claude（参数量差好几倍）；需要 16GB+ 内存；Mac 需要 M 系列芯片才跑得动

适合：写文档草稿、简单整理、不需要顶尖推理的任务

方案 2：混合工作流

- 敏感部分本地做：原始数据用本地工具 / 本地模型处理
- 结构化 / 脱敏后交 Claude：生成模板、做最终润色

例：合同审阅——原文本地看；只把条款提纲（去掉具体数字和名字）交给 Claude 让它检查逻辑。

方案 3：私有化部署

大公司可以从 Anthropic 采购企业版 / AWS Bedrock / GCP Vertex AI 托管的 Claude，数据走公司账户和 VPC，不经过公共 Anthropic 端点。

这需要公司层面采购，个人用户用不到。

一个实用建议

对大多数零基础读者来说：

- 日常工作任务（整理文件、写文档、翻译）—— Claude Code 完全够安全
 - 真正涉密的少数场景——本地 + 手工 仍然是最稳的选择
 - 不要因为“听说 AI 泄露”就全面拒绝，也不要因为“好用”就什么都上传
- 用分档策略（11.4 / 11.5 的表），对不同风险做不同处理。

下一章：第 12 章“Claude 为什么会变糊涂”——讲上下文。这是本书技术最硬的一章，但我会从“一个熬夜的人为什么想不清楚”这个比喻开始讲起。

第 12 章：Claude 为什么会变糊涂

——上下文原理

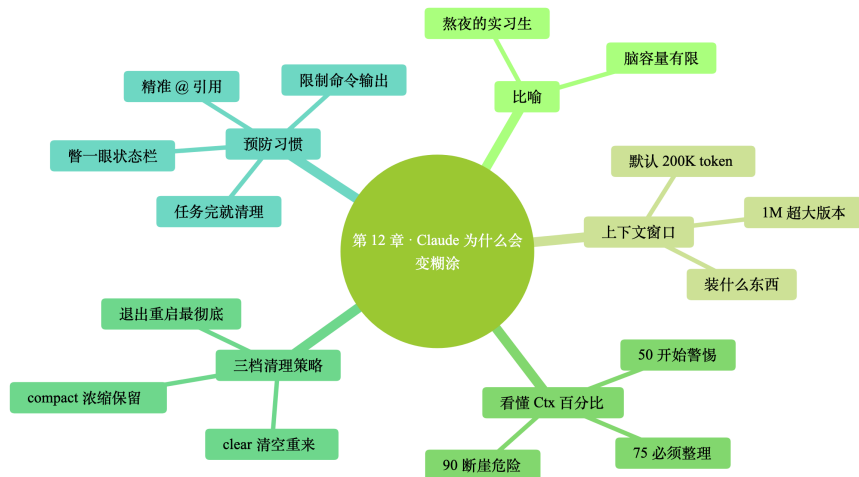
本章在全书的位置：第三部分 · 第 12 章 / 预估阅读+动手时长：主线 30-40 分钟 + 支线 15 分钟

前置章节：Ch 3（第一次对话）+ Ch 8（交互循环）

学完能做什么（主线）：搞清楚“上下文（context）”是什么、为什么聊久了 Claude 会开始糊涂；学会读状态栏里的 Ctx%；掌握 `/compact`、`/clear`、新开一个会话的三档策略，把 Claude 的“脑容量”管理起来。

开场三问

- 你会遇到什么问题：聊到后面 Claude 开始答非所问、忘掉你前面说的设定、反复改错同一个地方——感觉它“变笨了”。
- 不读这章会踩什么坑：硬往一个会话里塞东西，越用越糟；或乱 `/clear`，把还需要的上下文也清掉了。
- 读完你会多会什么事：看一眼状态栏就知道该不该整理记忆；知道什么时候用 `/compact`、什么时候 `/clear`、什么时候直接退出开新会话。



🌀 【主线】—— 本章必读核心

12.1 比喻先行：一个熬夜的人

想象你身边有一个很聪明的实习生，叫 Claude。他脑容量有限（就像人的短期记忆），能同时记住的事情大概相当于一本小册子的内容。

一天开始时（对话刚开启），他头脑清晰，读什么都记得。

你不停和他对话、让他读文件、让他跑命令——这些内容一条一条堆进他脑子里。小册子越写越满。

满了之后会发生什么：

- 想不清楚：前面讲过的细节想不起来
- 答非所问：问 A 回答 B，因为“B”刚好在记忆里更靠前
- 重复犯错：你前面纠正过的事，他又犯一遍
- 任务漂移：原本让他做 X，结果他做着做着变成做 Y

这不是 Bug，不是 Claude 变蠢了——是“脑容量”这个客观限制到头了。

学会管理这个容量，你就解锁了 Claude Code 的大半。

12.2 用技术名词说：上下文窗口（context window）

上面那本小册子，技术上叫上下文窗口。所有大模型都有这个限制，Claude 也不例外。

窗口里装了什么

- 你说的每一句话
- Claude 的每一句回复
- 你 @ 引用的文件的全部内容（不是摘要，是原文）
- 你让它跑的命令的完整输出（`git log` 如果几千行，全都进去）
- CLAUDE.md 的内容（每次启动都加载）
- Claude 自己的“系统提示”（Anthropic 给它的内部指令）
- 工具描述（能用哪些工具）

这个窗口多大

Claude 各模型的上下文窗口（以 token 为单位，1 token $\approx$ 0.7 个汉字或 1 个英文单词）：

模型	窗口大小	大致相当于
Sonnet 4.6	200,000 token	约 15 万汉字 / 一本中篇小说
Opus 4.7（默认）	200,000 token	同上
Opus 4.7（1M 上下文版）	1,000,000 token	约 75 万汉字 / 一部长篇巨著
Haiku 4.5	200,000 token	约 15 万汉字

听起来好像很多？别高兴太早——

- 你 @ 一个 50 万字的 PDF，直接爆
- 让它跑 `find / -name "*.md"`，几千个文件路径塞满
- 对话 3 小时不清理，来回几十条消息也满了

容量限制是客观物理限制，不是 Anthropic 抠门。

12.3 Claude Code 的状态栏：看懂 Ctx%

Claude Code 会在界面底下显示当前上下文使用率（简称 Ctx%）。

打开 Claude Code，注意看窗口底部状态栏（不同版本位置略有差异，但都能找到一个像 Ctx: 34% 或 34% 的数字）：

```
● Claude Opus 4.7    Ctx: 34%    ~/projects/ ...
```

这 34% 就是“小册子已经写了 34%，还剩 66% 空间”。

三档信号

Ctx 范围	状态	建议动作
0-50%	清醒	继续用，不用管
50-75%	开始满	准备整理思路（ /compact 或转折点收尾）
75-90%	明显糊涂	立刻 /compact 或 /clear——再往上性能断崖
90%+	危险区	模型开始“遗忘早期消息”、“答非所问”；马上 /clear 或新开会话

不同任务消耗速度不同

- 纯对话（只打字不读文件）：消耗最慢，可能聊几小时才到 50%
- 大量读文件（@ 多个大文件）：消耗极快，@ 一个几万字的 PDF 就可能瞬间从 10% 跳到 60%
- 跑命令输出很长（git log -n 200、npm install 的日志）：同样快速吃容量
- 上下文里存在一个长 diff：持续占位

12.4 三种清理策略：/compact、/clear、新会话

当 Ctx 逼近警戒线，你有三个工具：

策略 A： /compact ——让 Claude 自己整理思路

做什么：Claude 把当前对话浓缩成关键点（去掉冗长的中间步骤和废话），然后用这个摘要替换掉原始对话。

效果：Ctx 大幅下降（比如 70% → 20%），但保留核心共识和决策。

什么时候用：

- 任务还在进行中，你不想从头开始讲
- 对话里有你反复纠正的设定、达成的共识——这些要保留

怎么用：

你：/compact

(Claude 花几秒生成摘要，你会看到一段"已压缩对话"的提示)

你：继续我们刚才的活

注意：

- /compact 是有损压缩——细节会丢。如果某个具体变量名、具体数字后面还要用，最好自己在压缩前记下来。
- 紧急情况下，/compact 只救你一次。第二次又涨到 70% 时，建议直接 /clear 。

策略 B：/clear ——清空全部，从头开始

做什么：把当前对话完全抹掉，Claude 像刚启动一样。

效果：Ctx 回到接近 0%（只剩 CLAUDE.md 和系统提示）。

什么时候用：

- 任务已经做完，开始下一个任务
- 对话已经严重跑偏，整理也救不回来
- 做的事和前面完全没关系

怎么用：

你：/clear

(Claude 重置，下一条消息开始就是全新对话)

注意：清掉就拿不回来了。如果对话里有重要的“不想重新讨论”的决策，要么先 /compact，要么在 CLAUDE.md 里记下来。

策略 C：退出 + 新会话

做什么：完全退出 Claude Code (Ctrl+C 两次 或 /exit)，重新 claude 启动。

效果：比 /clear 更彻底——连临时文件、工具状态都重置。

什么时候用：

- 换了一个完全不同的项目（跨 workspace）
- Claude 出现诡异行为，/clear 也救不回来
- 一天结束，收工

三档速查表

情况	用哪个
任务正干着，只是嫌 Ctx 高	<code>/compact</code>
任务告一段落，开始做下一件	<code>/clear</code>
换项目 / Claude 卡住了	退出 + 重启

12.5 预防比清理更重要：省 Ctx 的 5 个习惯

清理是“病了再治”。预防才能让你更久保持清爽状态。

习惯 1：@ 引用时用具体路径，不 @ 整个文件夹

❌ @整个项目/ → 可能上传几百个文件 ✅ @src/components/Button.tsx → 只传你需要的

习惯 2：大文件先问“摘要”再决定要不要细看

对于可能很长的文件：

你：给我 @长文档.md 的 5 点摘要
Claude：（生成摘要）
你：第 3 点让我看看原文对应段落

比直接 @长文档.md 帮我改某段 省 Ctx。

习惯 3：跑命令限制输出

- `git log -n 10` （不是 `git log`，不设限制就全输出）
- `ls src/` （不是 `ls -R /` 递归列出整个目录）
- `cat 短文件.txt` （不是 `cat` 一个 10MB 的日志）

习惯 4：每个任务完成就 `/clear`

养成习惯：完成一件任务 → `/clear` → 开始下一件。不要一个会话跨三天做十件事。

习惯 5：定期检查 Ctx 数字

每几轮对话瞥一眼状态栏。看到 > 50% 就开始想“这个任务还有多远”，提前规划是 `/compact` 还是做完就 `/clear`。

本章小结

- 上下文 = Claude 的“短期工作记忆”，所有对话 / 文件 / 命令输出都占用它，满了会变糊涂
- Claude Code 状态栏的 Ctx% 实时显示使用率；50% 警惕、75% 必须整理、90% 断崖危险
- 三档清理：`/compact`（浓缩保留）→ `/clear`（全清重来）→ 退出重启（最彻底）
- 清理策略不对 → 要么还是糊涂（该清没清）、要么丢了重要共识（清过头）
- 预防 > 清理：精准 @、限制命令输出、每个任务完成就 `/clear`

动手任务

任务 1：观察 Ctx 变化（10 分钟）

步骤：

1. 启动 Claude Code，记下启动时的 Ctx 值（通常 1-5%）
2. @ 引用一个你手头的 Markdown 或 txt 文件（越长越好）→ 记下新的 Ctx
3. 让它跑一个 `!ls -la ~/Desktop` → 记下 Ctx
4. 来回聊 5-10 轮普通对话 → 记下 Ctx
5. 运行 `/compact` → 记下 Ctx
6. 运行 `/clear` → 记下 Ctx

成功标志：你现在直观感受到“什么操作消耗多、什么消耗少”。

任务 2：实战一次“长任务 + 主动压缩”（15 分钟）

步骤：

1. 找一个你真实想做的稍长任务（读 3-4 个文件 + 写一份报告）
2. 做到一半瞥一眼 Ctx——估计在 40%-60%？
3. 主动运行 `/compact`，观察 Claude 生成的摘要
4. 继续做完剩下的部分
5. 结束后 `/clear`

成功标志：你体验了“任务中段主动整理”的节奏，不再等到 90% 断崖才慌。

任务 3：建立你自己的 Ctx 纪律（5 分钟）

在你的个人 Claude Code 使用笔记（没有就建一个
~/Desktop/ccguide-练习/我的使用笔记.md）里写下：

我的 Ctx 纪律

- [] 每次启动后先看一眼状态栏
- [] 做大于 3 个文件的任务前，估计是否要 /compact
- [] Ctx > 50% 开始“收尾”思考
- [] Ctx > 75% 强制 /compact 或 /clear
- [] 每完成一个独立任务就 /clear

成功标志：你有了一份能贴在工作流里的个人约定。

如果你卡住了

症状 A：状态栏找不到 Ctx 数字

- 原因：不同版本位置不同；或你的终端太窄，状态栏被截断。
- 解决：把终端窗口拉宽；或运行 /status 命令直接查看（会显示当前模型、Ctx、其他状态）。

症状 B：/compact 后它忘了关键事情

- 原因：压缩是有损的，关键细节可能没保留。
- 解决：压缩前自己记下“绝不能丢的 3 件事”（比如具体的路径、数字、约定），压缩后再把这 3 条贴回去告诉它。

症状 C：对话没多久 Ctx 就到 70% 了

- 原因：你 @ 了大文件或跑了长输出命令。
- 解决：别让 Claude 读整个大文件。改成：让它先看目录，你再 @ 具体哪一部分；命令加 -n 20 之类限量参数。

主线结束。下面支线讲“上下文的内部机制细节”和“长上下文模型（1M 窗口）的适用场景”。



【支线】—— 可选深入（学有余力再看）

支线 12.A: Token 是什么——再深一层

这段讲什么：上文多次提到 token，这里说清楚 token 到底是什么、为什么不是“字符数”。什么时候回头读：好奇为什么一个汉字不等于一个字符、或想估算自己文件要多少 token 时。

什么是 token

Token 是模型处理文本的最小单位，但它不等于字符，也不等于单词。

大致规律：

- 英文：约 1 个单词 = 1 token；`hello world` $\approx$ 2 token
- 汉字：约 1 个汉字 = 1.5-2 token（Claude 的中文分词偏高）
- 代码 / 特殊符号：变化很大，一段 JSON 常见的 `"key":` 可能占 3-4 token

实用估算

给你一个拍脑袋公式：

- 1,000 汉字 $\approx$ 1,500-2,000 token
- 10,000 英文单词 $\approx$ 12,000-14,000 token
- 一份 1 万字的 Markdown $\approx$ 15,000-20,000 token

所以 200K token 窗口：

- 纯中文——大概能塞 10-12 万汉字
- 纯英文——大概能塞 14-16 万单词

为什么在意 token

- Ctx% 用的就是 token 比例
- Anthropic 按 token 计费（见第 15 章）
- `/compact` 的本质：把原始几万 token 的对话压缩成几千 token 的摘要

怎么查一段文字多少 token

- Anthropic 有官方 [Token Counter](#)
- 大多数情况不用精确算——拍脑袋估就够

支线 12.B：长上下文模型（1M 版本）什么时候值得用

这段讲什么：Claude Opus 4.7 有一个 1M 上下文窗口版本——5 倍于默认的 200K。什么时候值得切过去？什么时候回头读：你发现 200K 总不够用，在做超大任务时。

默认 200K 够不够用

对大部分日常任务：

- 改几个文件、读几个 PDF、写几篇文档——200K 远超需求
- 一个中等项目的全部代码——基本能塞进 200K

什么场景 200K 真不够

- 读整本书 / 整套财报分析——一本 30 万字小说 = 约 50 万 token
- 跨整个大代码库做重构分析（几百文件 + 历史日志）
- 长对话连续几小时不想分段

这时 1M 版本有价值。

1M 版本的代价

- 慢：上下文越长，每轮回复越慢
- 贵：按 token 计费，用满 1M 一次对话的成本是默认版本的 5-10 倍（见 Ch 15）
- 注意力分布：上下文太长时，模型对中间部分的关注度会下降——这叫“lost in the middle”——所以 1M 不意味着“塞进去它就全记得”

切换办法

在 Claude Code 里运行：

```
/model
```

会弹出可选模型列表，其中有“Opus 4.7（1M context）”选项。点击切换。

经验法则：

- 日常任务用 默认 Opus 或 Sonnet（200K 够用、快、便宜）
- 偶尔遇到“就这一次需要特别大”的任务再临时切 1M，做完切回去

支线 12.C: /compact 的工作原理

这段讲什么：/compact 到底是怎么压缩的？什么时候回头读：你想搞清楚它保留什么、丢什么时。

压缩过程

1. Claude 读完整个当前对话
2. 生成一个结构化摘要，大致包括：
 - 总目标是什么
 - 已完成的关键步骤
 - 重要的决策和约定
 - 还未完成的子任务
 - 关键文件路径和符号名
3. 用这个摘要替换掉原始对话历史
4. 上下文从“几万 token 长对话”→“几千 token 摘要”→ Ctx% 大幅下降

保留什么、丢什么

✅ 通常保留：

- 最终决策和约定
- 你反复强调的要求
- 还没做完的事

❌ 通常会丢：

- 你纠正过程中的曲折细节
- 早期被否决的方案
- 冗长的命令输出
- 代码的完整内容（会变成“已读过 XX 文件”的记号）

怎么让压缩更聪明

压缩前对 Claude 说：

你：在 /compact 之前，请先列出你打算保留的 5 个关键点，让我确认有没有遗漏。

Claude：好的，我打算保留：

1. ...

2. ...

...

你：（检查） 2 点里别漏了 X 的决定。（确认后）OK, /compact

这样压缩质量高很多。

下一章：第 13 章“三层记忆”——会话记忆 / 自动记忆 / 项目记忆 (CLAUDE.md)，什么信息放哪层，怎么搭建一个让 Claude 越用越懂你的系统。

第 13 章：三层记忆——会话 / 自动记忆 / CLAUDE.md

本章在全书的位置：第三部分 · 第 13 章 / 预估阅读+动手时长：主线 25-30 分钟 + 支线 10 分钟

前置章节：Ch 12（上下文原理）

学完能做什么（主线）：搞清楚 Claude Code 有三层记忆系统——短期（会话）/ 中期（自动记忆）/ 长期（CLAUDE.md），以及 Skills 的“按需加载”是第四种机制。知道什么信息放哪层，不混用、不浪费、不丢失。

开场三问

- 你会遇到什么问题：你告诉过 Claude 一件事，过几天新对话它又不知道；或者把所有东西都塞进 CLAUDE.md，结果越写越长、每次启动慢。
- 不读这章会踩什么坑：信息“放错层”——临时的放到永久里（CLAUDE.md 膨胀）、长期的只放会话里（下次就忘）。
- 读完你会多会什么事：三层各有专用场景，你能一眼判断“这个信息该放哪里”，让 Claude 越用越懂你。



🎯 【主线】—— 本章必读核心

13.1 三层记忆是什么——先看比喻

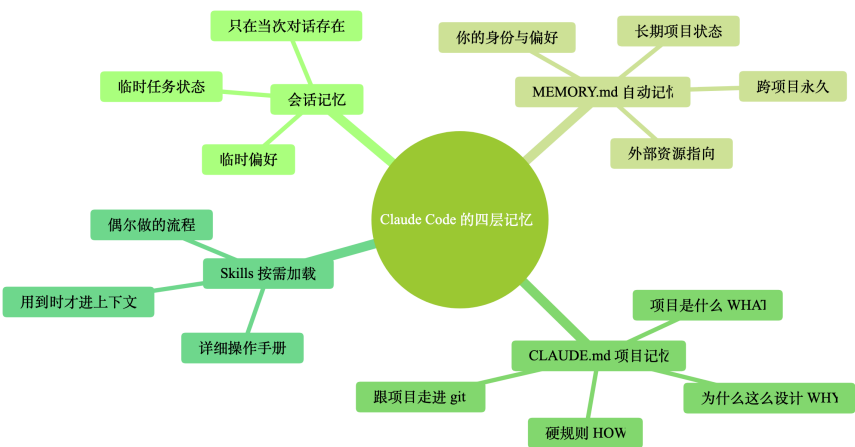
想象你有一个助理实习生：

- 短期工作记忆：今天上班期间你交代的事、他看过的文档。下班回家就全忘（会话记忆）
- 他的工作笔记本：他自己决定记下来的要点，明天还能翻（自动记忆 / MEMORY.md）
- 你贴在他桌面的岗位说明书：每天上班第一眼就看，关于这份工作的铁规矩（CLAUDE.md）

再加一个按需取用的专业手册柜：工位旁边柜子里放了一排手册，平时不占桌面，有特定任务时他才取出来翻（Skills 技能包）。

Claude Code 对应得一模一样：

层	Claude Code 里叫什么	保存时长	每次启动自动加载?
1	会话记忆	当前会话内	—
2	自动记忆 (MEMORY.md)	跨会话永久	✅ 每次启动加载
3	项目记忆 (CLAUDE.md)	跨会话永久	✅ 每次启动加载
4	Skills (技能包)	跨会话永久	❌ 按需加载（触发时才进上下文）



13.2 第一层：会话记忆

是什么：一次 `claude` 启动到 `/exit` 或 `/clear` 之间的所有内容——你说的话、Claude 的回答、读过的文件、命令输出、形成的共识。

存在哪里：存在 Claude 的短期工作记忆里，也就是第 12 章讲的上下文窗口。

什么时候消失：

- 运行 `/clear`
- 退出 Claude Code
- 电脑关机、程序崩溃
- Ctx% 爆了之后自动丢弃最早的内容（模型开始“遗忘”前面的）

会话记忆适合放什么

- 当前任务的中间状态——正在改哪个文件、已经走到哪一步
- 刚临时说的偏好——“这次结果用 JSON 格式”

- 探索性的对话——你在和 Claude 一起思考，还没定型

不适合放什么

- 以后多次会用的约定——“我的代码风格偏好是 XX”这种说一次、以后都适用的事
 - 项目的硬规定——“这个项目测试必须过、不能用某某库”
 - 你自己的长期身份信息——“我是做产品运营的”这种每次都适用的
- 这些放错地方就会在下次 `/clear` 后全消失。

一个常见误会

很多人以为“Claude 聊过的事以后都会记得”——完全不是。默认情况下，每次新会话它完全重置，只知道 CLAUDE.md 和 MEMORY.md 里的内容。

13.3 第二层：自动记忆 (MEMORY.md)

是什么：Claude Code 新版引入的跨会话持久记忆。Claude 在对话中自动判断什么值得记下来，写进一个 `MEMORY.md` 文件里。下次启动时自动读回来。

放在哪

不同系统路径略有不同，但大致：

```
Mac/Linux: ~/.claude/projects/<项目路径hash>/memory/MEMORY.md
Windows: %USERPROFILE%\.claude\projects\<项目路径hash>\memory\MEMORY.md
```

你可以直接打开这个文件看 Claude 给你记了什么。

Claude 会自动记什么

四种常见类型：

1. 用户信息 (user) ——你是谁、做什么工作、用什么语言、有什么偏好
2. 反馈 / 规矩 (feedback) ——你纠正过它的地方、你明确表达过的偏好
3. 项目状态 (project) ——项目现在在做什么阶段、下周截止
4. 外部引用 (reference) ——“相关内容在 Linear 的 XXX 看板”

自动记忆适合放什么

- 你的身份 / 职业 / 工作场景：跨项目都适用
- 反复纠正 Claude 的偏好：不记下它下次还会犯同样错

- 当下的项目状态：你手头的项目阶段、暂停点
- 外部资源指向：信息的准确原始来源在哪

你可以主动操作它

- 让它记：直接说“记住：我喜欢结果用 markdown 表格”——Claude 会写进 MEMORY.md
- 让它忘：说“忘掉之前记的我喜欢咖啡”——Claude 会删对应条目
- 手动编辑：打开 MEMORY.md 直接改、删

一个关键差异

MEMORY.md 是“Claude 对你的记忆”，CLAUDE.md 是“你对 Claude 的指令”——方向相反。

维度	MEMORY.md	CLAUDE.md
谁写	Claude 自己写（你也能编辑）	你自己写
记的是	关于你的事实	关于项目 / 团队的规则
范围	通常跨项目（在 ~/.claude 下）	绑定某个项目（在项目根目录）
更新频率	日常自动更新	偶尔手动维护

13.4 第三层：项目记忆（CLAUDE.md）

是什么：一个放在项目根目录的 `CLAUDE.md` 文件，Claude Code 每次在这个项目里启动时都读一遍。

存在哪里：`项目根目录/CLAUDE.md`，你自己创建、自己维护、和代码一起进 git。

谁能看到：团队所有人（跟项目走），不是只给你看。

CLAUDE.md 适合放什么

- 项目是什么（WHAT）：这个仓库做什么、核心概念
- 为什么这么设计（WHY）：关键决策背景——这个文件的灵魂
- 硬规则（HOW）：测试怎么跑、提交前必做什么、哪些不能碰
- 路径地图：哪类代码在哪个目录、重要文件在哪

不适合放什么（重要反模式）

❌完整的代码风格指南 → 让 Claude 读代码就知道了 ❌数据库 schema 全文 → 放在单独文档，需要时指引它去看 ❌10 条代码片段“供参考” → 这不是说明书，是代码——放代码里 ❌命令堆砌（“可以用 A 也可以 B 也可以 C”） → 给一个推荐就够 ❌用 `/init` 一键生成的一大坨 → 第 14 章专门讲为什么这是反模式

一个关键原则：Pointers > Copies

Pointers（指路）：告诉 Claude “这个信息在哪里可以找到”——比如“测试策略见 docs/testing.md” Copies（复制）：把 docs/testing.md 的全部内容复制到 CLAUDE.md 里

永远优先 Pointers。因为：

- CLAUDE.md 每次启动都加载（占 Ctx 名额）
- 内容在两个地方容易不同步
- 维护麻烦

详见第 14 章。

13.5 第四层（特殊）：Skills——按需加载

是什么：预先写好的一整套操作流程，平时不占上下文，只在你调用它时才加载。

存在哪里：

- 全局 Skills: `~/.claude/skills/<skill-name>/`
- 项目 Skills: `.claude/skills/<skill-name>/`

和 CLAUDE.md 的关键分工

维度	CLAUDE.md	Skills
加载时机	每次启动都加载	被调用时才加载
占 Ctx 预算	每次都占	只在用到时占
内容性质	全局规则、硬约束	特定任务的操作手册
怎么触发	自动（启动就读）	用户说“用 XX 技能”、或 Claude 判断需要

例子对比

- CLAUDE.md 合适：

- “这个项目用 Python 3.11”
- “测试必须 100% 通过才能 push”
- “所有新 API 放在 `src/api/v2/`”

- Skills 合适：

- “如何做一次数据库迁移”（只有偶尔用到）
- “如何发布新版本到 npm”（发布时才用）
- “特定格式的报告模板”（写某类报告时用）

为什么这个分工重要

CLAUDE.md 每次启动都加载意味着：

- 每条字都占用你的 Ctx 窗口
- 越大越让 Claude 每次启动变慢
- 内容越杂，Claude 越可能抓不住重点

而 Skills 只在需要时加载：

- 不用时 0 成本
- 可以很长、很详细
- 不同任务用不同 skill，互不干扰

经验法则：CLAUDE.md 要短而精，一次性多 / 偶尔性的流程放 Skills。

Skills 具体怎么用

第 20 章会完整讲 Skills 入门。这里你只需要知道它存在、而且和 CLAUDE.md 是分工关系。

13.6 快速决策表：这条信息放哪层

面对一条新信息，过下面这张表：

这条信息的性质	放哪层
只是这次对话临时要用的	会话（说一下就行，不存）
关于“我是谁、我喜欢什么”的跨项目偏好	MEMORY.md（让 Claude 记）
项目的硬规则，团队每个人都要知道	CLAUDE.md（写进去，进 git）
一个复杂但只偶尔做的操作流程	Skills（建一个技能包）
一段别人已经写好的长文档	Pointer（CLAUDE.md 写“详见 XXX”，不复制）

本章小结

- Claude Code 有三层常驻记忆 + 一层按需记忆：会话 → MEMORY.md → CLAUDE.md + Skills
- 会话（只在当次对话里存在）、MEMORY.md（Claude 自动记你的偏好，跨项目）、CLAUDE.md（你手动写项目规则，跟项目走）、Skills（按需加载的完整操作手册）
- 放错层的代价：临时的放永久层→膨胀；长期的只放会话→每次重头讲
- MEMORY vs CLAUDE：一个是“Claude 记你”，一个是“你指令 Claude”
- Skills vs CLAUDE.md：前者按需加载、后者每次加载——偶尔用的流程放 Skills，常驻规则放 CLAUDE.md

动手任务

任务 1：看看 Claude 给你记了什么（5 分钟）

步骤：

1. Mac 打开终端，运行：

```
open ~/.claude/projects/
```

2. 进去能看到以“项目路径 hash”命名的子文件夹，找一个最近用过的
3. 进去找 `memory/MEMORY.md`，打开看

成功标志：你看到了 Claude 给你记的条目——或者什么都没有（如果你一直没让它记过什么）。

任务 2：主动让 Claude 记一条（5 分钟）

步骤：

1. 启动 Claude Code（找一个你经常用的项目）
2. 输入：记住：我喜欢回答先给结论再给步骤，不要长篇大论。
3. Claude 会确认已记录
4. 打开 MEMORY.md 验证——应该出现一条新条目

成功标志：MEMORY.md 里出现了这条偏好。

任务 3：分辨三层的练习（10 分钟）

下面 8 条信息，每条判断应该放哪层：

1. “这次对话请用英文回复”
2. “我是一名医生，写的文档是临床相关，请避免太技术的术语”
3. “本项目用 Python 3.11，测试框架是 pytest”
4. “这个仓库的所有提交消息必须以 feat: fix: 等前缀开头”
5. “我正在调试的那个登录 bug 暂时搁置了，下周再看”
6. “如何给这个项目做版本发布：7 步流程”（偶尔做）
7. “我更喜欢看图表而不是纯文字描述”
8. “这段代码：function add(a,b) { return a+b }”

把你的答案写成 ~/Desktop/ccguide-练习/记忆分层练习.md。

参考答案（做完再看）：

1. 会话 — 临时性偏好
2. MEMORY.md — 跨项目、关于“你”的
3. CLAUDE.md — 项目硬规则
4. CLAUDE.md — 项目硬规则
5. MEMORY.md — 项目状态（你的任务状态）
6. Skills — 偶尔做的复杂流程
7. MEMORY.md — 跨项目偏好
8. 哪一层都不应该放 —— 代码写进代码文件，不是记忆里

成功标志：8 题答对 6 题以上——你已经建立了分层直觉。

如果你卡住了

症状 A：找不到 MEMORY.md 文件

- 原因：Claude 还没写过——需要你对话中说过“记住…”之类的触发语；或者你的 Claude Code 版本太旧不支持。
- 解决：升级到最新版；主动让它记一次（任务 2）再看。

症状 B：MEMORY.md 里记的东西不对 / 过时

- 原因：Claude 判断失误，或者你的偏好变了。
- 解决：直接打开这个文件手动编辑——删掉错的条目、改掉过时的。MEMORY.md 就是一个普通 markdown 文件。

症状 C：下次打开新会话还是记不住

- 原因：(a) 你在不同项目目录打开，MEMORY.md 是按项目存的；(b) 你可能用的是不带自动记忆的老版本。
- 解决：(a) 确认在同一个工作目录启动；(b) 升级 Claude Code。

主线结束。下面支线讲“MEMORY.md 的高级管理”和“全局 vs 项目 CLAUDE.md”。



【支线】—— 可选深入（学有余力再看）



支线 13.A：高效管理你的 MEMORY.md

这段讲什么：MEMORY.md 怎么保持干净、不让它越记越乱。什么时候回头读：用了几个月后，发现记忆里塞了一堆过时的东西。

常见问题

- 旧项目状态没更新（还写着“上周在做 X”，实际早完成了）
- 重复条目（同一个偏好记了三次）
- 临时情境被当作永久偏好（某次生气说“别再给我 emoji”，被永久记成“禁用 emoji”）

维护节奏

每 1-2 个月：打开 MEMORY.md 扫一遍，做三件事：

1. 删过时的（项目已结束 / 你已经不那么偏好）
2. 合并重复的
3. 更新项目状态

手动编辑的注意

- Claude Code 新版本使用两层结构：MEMORY.md 是索引，详情分拆到各自小文件
- 所以编辑时：改索引 (MEMORY.md) 和改详细内容 (对应的小文件) 都要做
- 具体结构打开看一眼就懂——就是标准 markdown

让 Claude 帮你整理

你可以直接让 Claude：

你：帮我检查 MEMORY.md，找出过时或重复的条目，给出删减建议。

Claude：（扫描后列出）

你：（确认）OK 按这个建议改

Claude 会用 Edit 工具帮你清理。

支线 13.B：全局 CLAUDE.md vs 项目

CLAUDE.md

这段讲什么：CLAUDE.md 其实可以放在多个位置，每个位置作用不同。什么时候回头读：想建立“跨所有项目”的个人编程规范时。

两种 CLAUDE.md

1. 项目 CLAUDE.md：<项目根>/CLAUDE.md ——跟项目走、进 git、团队共享
2. 全局 CLAUDE.md：~/claude/CLAUDE.md ——你所有项目都生效

加载顺序

Claude Code 启动时两者都读，项目的叠加在全局之上（如有冲突以项目为准）。

什么放全局

- 你跨所有项目的硬偏好：“我写代码习惯短变量名”、“解释代码前先说大概思路”
- 你不希望每个项目都要重写一遍的约定

什么放项目

- 只这个项目有的规则：技术栈、测试规范、目录结构
- 团队共享的约定：代码风格、提交规范

两个层次的例子

```
# ~/.claude/CLAUDE.md （全局，只给你看）
```

```
## 我的偏好
```

- 回答先结论后细节
- 写代码加类型标注（Python 用 type hints、TypeScript 用 type）
- 不给我 emoji

```
# 项目根/CLAUDE.md （项目，团队共享）
```

```
## 项目：XXX 后端
```

```
## 技术栈
```

- Python 3.11 + FastAPI
- 测试：pytest，必须 100% 通过才能 push

```
## 硬规则
```

- 所有 API 在 src/api/v2/ 下
- 数据库迁移用 alembic

两者叠加后，Claude 同时知道你的个人偏好 + 项目规则——互不冲突。

小陷阱

- 全局 CLAUDE.md 不进 git——它是你本地的，别把它跟项目混在一起

- 跟别人 pair programming 时，注意你的全局设定可能让 Claude 给出“只你习惯”的答案——关键决策时互相确认

下一章：第 14 章“写好你的 CLAUDE.md”——这是全书杠杆最高的一章。很多老手建议“一装好就跑 `/init`”——这是反模式。下一章讲为什么，以及该怎么写。

第 14 章：写好你的 CLAUDE.md—— 全书最高杠杆的一章

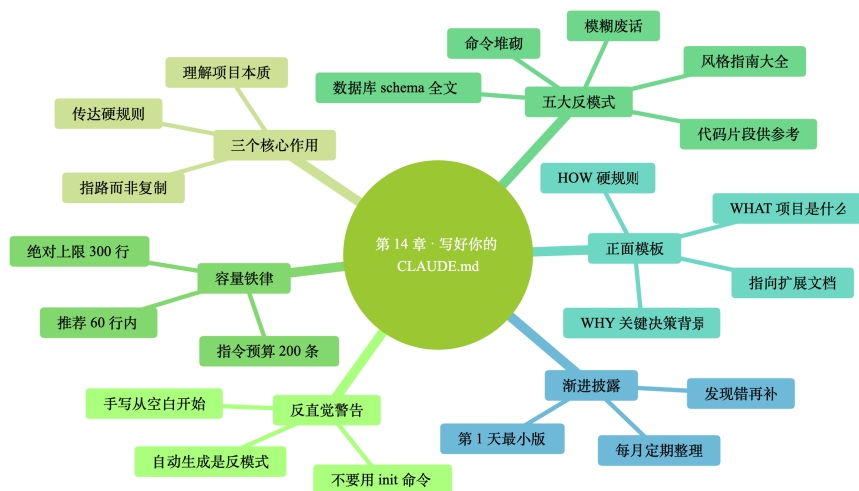
本章在全书的位置：第三部分 · 第 14 章 / 预估阅读+动手时长：主线 35-45 分钟 + 支线 15 分钟

前置章节：Ch 12（上下文原理）+ Ch 13（三层记忆）

学完能做什么（主线）：学会写一份精准有效的 CLAUDE.md——不是越长越好、不是堆砌命令、不是 `/init` 一键生成。会挑内容、懂取舍、懂维护节奏。写出来的 CLAUDE.md 能让 Claude 在你的项目里真的越用越准。

开场三问

- 你会遇到什么问题：Claude 每次在你的项目里回答都像“第一次听说”——同样的约定要反复讲。
- 不读这章会踩什么坑：要么根本不写 CLAUDE.md（长期遭受重复回答折磨）；要么乱写 CLAUDE.md（写成 500 行巨型文档、实际效果反而下降）。
- 读完你会多会什么事：能判断“哪条信息值得进 CLAUDE.md、哪条不值”；能写出 < 60 行就让 Claude 懂项目的说明书；能维护它不让它膨胀。



【主线】—— 本章必读核心

14.1 第一反直觉警告：不要用 `/init` 一键生成

Claude Code 自带一个命令：`/init`。

你在项目目录里跑 `/init`，Claude 会自动扫描代码，生成一份巨型 CLAUDE.md——列出所有文件、所有技术栈、所有命令、甚至一些无关的注释。

听起来很方便。99% 的博客、视频教程告诉你“装好 Claude Code 第一件事就跑 `/init`”。

本书告诉你：别这么做。

为什么 `/init` 是反模式

1. 膨胀：自动生成的 CLAUDE.md 通常 300-1000 行——太长，Claude 注意力会分散
2. 堆垃圾：它会把 `node_modules` 目录下的一些奇怪内容也写进去；把几十条可以用但没用的命令都列一遍

3. 没有“为什么”：只有“是什么”——Claude 知道“项目用 Express”，但不知道“为什么选 Express 而不是 Fastify”——这才是判断力所在
4. 你不理解就不能维护：自动生成的你自己都没通读过，之后更新怎么办？谁都不会改的 CLAUDE.md 几个月后就过时

正确做法

手写。从一个几乎空白的 CLAUDE.md 开始，随着项目进展、随着你发现 Claude 反复犯同一错误，再往里补内容。

每一条都是你自己觉得值得记的。这份文档是活的，跟着项目长大。

📌 本书口号：“CLAUDE.md 是写出来的，不是生出来的。”

14.2 CLAUDE.md 的三个核心作用

先搞清楚这份文档到底要干什么：

作用 1：让 Claude 理解项目本质（WHAT + WHY）

WHAT：这个仓库做什么？核心概念是什么？WHY：为什么这么设计？关键决策的背景是什么？

读完 CLAUDE.md，Claude 应该能一句话说出“这个项目是什么”。

作用 2：传达硬规则（HOW 的一部分）

不可违反的硬约定：

- 测试必须 100% 过才能 push
- 所有 API 必须有文档
- 不能直接改 main 分支

这些不是建议，是红线。

作用 3：路径地图（where to look）

告诉 Claude 哪里去找更详细的信息：

- “测试策略的完整规范见 docs/testing.md ”
- “数据库 schema 在 db/schema.sql ”

不复制内容，只给指针（Pointers > Copies）。

14.3 容量铁律：越短越好

容量越大 $\neq$ 效果越好。反过来：容量越大，效果越差。

为什么

- CLAUDE.md 每次启动都全量加载进上下文（见 Ch 12）
- 越长，Claude 越难抓住重点
- 指令之间可能互相冲突或语义重叠，Claude 判断不出该听哪个
- 维护负担指数上升

三条硬线

- 绝对上限：300 行。超过 300 行，必然存在可以砍的，本章教你怎么砍。
- 推荐上限：60 行。小项目、个人项目，60 行能表达清楚。
- 指令预算：150-200 个指令性语句（“必须…”、“不要…”、“优先…”）。超过这个数量，Claude 记不住，自己就会“取舍遗忘”。

这是反直觉的

很多人觉得“我把所有规矩都写进去才放心”。实际效果相反——Claude 面对一份 500 行的文档，会抓住表面几条、忽略中间大部分。

写少 = 让重要的被看见。

14.4 反模式清单：这些东西不要写进 CLAUDE.md

下面这些是最常见的“错误内容”。你的 CLAUDE.md 里一条都不应该有：

反模式 1：完整的代码风格指南

❌ 不要写：

代码风格

- 变量名用 camelCase
- 函数名用动词开头
- 每个函数必须有 JSDoc
- 缩进用 2 空格
- 分号必须有
- 字符串用单引号
- ...（还有 30 条）

为什么不好：

- Claude 读代码就能看出风格
- 真正想强制，用 ESLint / Prettier 等工具
- 30 条风格规则会把重要指令淹没

✅ 改成：

代码风格

- 跟项目现有风格（跑 `npm run lint` 自检）
- ESLint 配置在 `.eslintrc`

反模式 2：数据库 schema 全文

❌ 不要写：

数据库表结构

users 表：

```
id INTEGER PRIMARY KEY
email VARCHAR(255) NOT NULL UNIQUE
name VARCHAR(100)
created_at TIMESTAMP
updated_at TIMESTAMP
... (30 张表、500 行)
```

为什么不好：

- schema 会变，两个地方保持同步很难
- 这是代码类数据，应该从代码里读

✅ 改成：

数据库

- schema 定义见 `db/schema.sql`
- 迁移用 alembic，迁移文件在 `alembic/versions/`

反模式 3：代码片段“仅供参考”

❌ 不要写：

示例：创建用户

```
```python
```

```
def create_user(email, name):
 user = User(email=email, name=name)
 db.add(user)
 db.commit()
 return user
```

当你要创建时请照此风格。

**\*\*为什么不好\*\*：**

- 这**\*\*就是代码\*\***——应该进代码，不是文档
- 代码改了，CLAUDE.md 还留着旧的

✅ 改成：

```
```markdown
## 创建新用户
- 参考 `src/services/user.py` 里现有的 create_user 函数
```

反模式 4：命令堆砌

❌ 不要写：

```
## 常用命令
- 启动: npm run dev 或 npm start 或 yarn dev 或 yarn start
- 测试: npm test 或 npm run test 或 jest 或 jest --watch
- 构建: npm run build 或 npm run prod 或 webpack
```

为什么不好：

- 多个“或”让 Claude 猜——给它一个推荐
- 这些命令 Claude 看 package.json 就知道

✅ 改成：

```
## 常用命令
- 启动开发: `npm run dev`
- 跑测试: `npm test`
- 构建生产版: `npm run build`
```

反模式 5：写进去谁也不看的规矩

❌ 不要写：

提交规范

- 每次提交必须写清楚
- 要有意义
- 不要提交无关的内容
- 要检查代码质量
- 测试要跑过

为什么不好：

- 太模糊，Claude 根本无法判断“有意义”
- 是给人看的废话，不是给 AI 的指令

✅ 改成：

提交规范

- 提交消息以 `feat:` `fix:` `docs:` `refactor:` `test:` 之一开头
- 单次提交改动 < 400 行（大改拆成多个）

要点：CLAUDE.md 里每一条都要足够具体、机器可执行——“< 400 行”是机器能检查的，“要有意义”不是。

14.5 正面模板：一个好的 CLAUDE.md 长什么样

下面是一份真实能用的 CLAUDE.md 模板（55 行）：

项目：XXX 后端 API

WHAT（这是什么）

一个为 XXX 产品的移动端提供数据的后端 API。Python 3.11 + FastAPI + PostgreSQL。

目前服务 10 万 DAU，核心模块是订单和推送。

WHY（关键决策背景）

- ****选 FastAPI 不选 Django****：团队偏好异步 IO；Django 的 ORM 对现有遗留 schema 不友好

- ****暂不拆微服务****: 规模没到, 单体够用
- ****用 Redis 不用 Memcached****: 已经用 Redis 做队列, 不想引入第二个缓存

HOW (协作时的硬规则)

不能违反

- 任何改动必须跑 `pytest tests/` 全绿才能 push
- 数据库改动必须用 alembic 迁移, ****禁止直接 SQL 改表****
- 新 API 必须放在 `src/api/v2/`, ****不碰**** `src/api/v1/` (遗留兼容)
- 涉及钱的逻辑 (订单金额、退款) ****必须写测试且让另一个人 review****

目录地图

- API 路由: `src/api/v2/`
- 业务逻辑: `src/services/`
- 数据模型: `src/models/`
- 测试: `tests/`, 跟源码同结构

常用命令

- 起服务: `uvicorn src.main:app --reload`
- 跑测试: `pytest tests/ -v`
- 迁移生成: `alembic revision --autogenerate -m "xxx"`

扩展信息的位置 (不复制在这里)

- 详细测试策略: `docs/testing.md`
- 部署流程: `docs/deployment.md`
- 数据库 schema: `db/schema.sql`
- API 接口文档: 启动服务后访问 `/docs`

正在进行的阶段

****2026-04-19****: 重构订单模块的并发处理, 禁止对 `src/services/order.py`

做大重构 (会有冲突)。小 bugfix 可以。

为什么这份模板有效

- 55 行，远低于 300 行上限
- WHAT / WHY / HOW 三段骨架——清晰
- WHY 讲“为什么这么选”——Claude 能据此做判断，不只是“按规则做事”
- 硬规则用明确语言（“必须”、“禁止”）——不给 Claude 解释空间
- Pointers 指向详细文档，不复制内容进来
- “正在进行的阶段”告诉 Claude 当下的警戒线——随项目更新

14.6 渐进披露原则：慢慢加、不要一次堆

你的 CLAUDE.md 不是一次写完的。正确节奏：

第 1 天：写最小版本

只写：

- 项目是什么（1-2 句）
- 3-5 条最硬的规矩
- 目录结构提要

总共 20-30 行。

第 1 周：补你观察到的

每次发现 Claude 犯错——问自己：“这是不是因为我没告诉它？”

- 是 → 加一条到 CLAUDE.md
- 不是（它就是不懂 / 它粗心）→ 不要加

第 1 个月：定期整理

- 扫一遍：有没有过时的条目？
- 扫一遍：有没有因项目演进变得错的条目？
- 扫一遍：能不能合并相似条目？

原则：每加一条要能说清楚它解决了什么问题

不能说清就不要加。很多 CLAUDE.md 越写越长，就是因为每条“万一有用”的都加进去。

本章小结

- 不要用 `/init` ——那会生成一份 300+ 行的垃圾文档。手写、从空白开始。
- CLAUDE.md = WHAT + WHY + HOW + Pointers（项目本质 + 关键决策 + 硬规则 + 指路）
- 容量铁律：推荐 < 60 行，绝对不超过 300 行；< 200 条指令
- 五大反模式：代码风格大全 / schema 全文 / 代码片段 / 命令堆砌 / 模糊废话——一条都别写
- 渐进披露：从小版本开始，发现 Claude 犯错才补条目；每月整理；每条都要能说清解决了什么问题

动手任务

任务 1：评估一份 CLAUDE.md（可选找一个公开项目，10 分钟） 步骤：

1. 在 GitHub 随便搜 `filename:CLAUDE.md`，随便找一个看起来像模像样的项目
2. 打开它的 CLAUDE.md，用本章 14.4 的反模式清单对照扫一遍
3. 数：多少行？是否 < 300？有没有硬规则？有没有 WHY？

成功标志：你能准确指出这份 CLAUDE.md 的 3 个优点和 3 个可改进点。

任务 2：给你当前一个项目写 CLAUDE.md v0.1（30 分钟） 步骤：

1. 选一个你手头的项目（即使是“整理照片”这种非代码项目也行）
2. 不要用 `/init`。新建 `CLAUDE.md`，复制 14.5 的模板
3. 填进你的项目信息：
 - WHAT：1-2 句
 - WHY：3 个关键决策 + 原因
 - HOW：5 条硬规则
 - Pointers：至少 2 个指路
4. 控制在 60 行内
5. 保存、启动 Claude Code、问它一个项目相关的问题，看回答质量

成功标志：你有了一份真正自己懂的 CLAUDE.md，Claude 的回答比之前更准。

任务 3：用 5 天养成“发现问题就改 CLAUDE.md”的习惯（跨天，持续 5 天）

步骤：

1. 连续 5 天，每天用 Claude Code 处理你的项目至少 15 分钟
2. 每次 Claude 犯错或让你重复解释时，立刻问自己：“这是不是该进 CLAUDE.md？”
3. 是 → 立刻加一条（加完保存）
4. 5 天后回看 CLAUDE.md，看增加了多少条

成功标志：你的 CLAUDE.md 比 5 天前多了 3-8 条——每条都能说清当初为什么加。这是 CLAUDE.md 正确的成长方式。

如果你卡住了

症状 A：不知道项目的“WHY”该写什么

- 原因：这是设计决策背景——你做项目时的考虑。
- 解决：问自己 3 个问题——“为什么选这个技术栈？为什么这么分目录？为什么有这些硬规则？”——答案就是 WHY。

症状 B：我的项目是非编程项目（整理文件、写文档）

- 原因：本章示例偏代码项目，但 CLAUDE.md 同样适用。
- 解决：WHAT = “这是什么项目、整理什么内容”、WHY = “为什么这么分类”、HOW = “命名规则、保存位置、不能碰的文件”。原理完全一样。

症状 C：写完 CLAUDE.md，Claude 的表现没明显变好

- 原因：要么条目太模糊 Claude 用不上，要么写的东西本来它就知道（读代码就懂）。
 - 解决：删一半——只留 5 条最特殊、最违反直觉的规则。观察效果。少而精 > 多而杂。
-

主线结束。下面支线讲“团队场景的 CLAUDE.md”和“项目 CLAUDE.md 版本管理”。



【支线】—— 可选深入（学有余力再看）

🌿 支线 14.A：团队场景下的 CLAUDE.md

这段讲什么：多人协作时 CLAUDE.md 的组织方式。什么时候回头读：你的项目开始有多个人用 Claude Code 时。

问题：不同人的习惯冲突

马力加一条“函数必须有文档字符串”，小红觉得“短函数没必要”。两个人来回改 CLAUDE.md 就打架了。

解决方案 1：CLAUDE.md 作为团队共识

把 CLAUDE.md 当作代码一样进 git，改动走 PR review：

- 每次改 CLAUDE.md 都提 PR
- 至少 1 人 review + 讨论
- merge 之后所有人 pull 下来就同步了

这样 CLAUDE.md 成了团队正式约定，不是某个人的偏好。

解决方案 2：分层——全局 + 项目

- 项目 CLAUDE.md（共享、进 git）：所有人必须遵守的硬规则
- `~/.claude/CLAUDE.md`（个人、不进 git）：每个人自己的偏好

这样个人偏好不污染团队约定。

解决方案 3：放在 CLAUDE.md 里的“团队规范”文件夹

有的项目会把 CLAUDE.md 拆成主文件 + 多个子文件（当项目很复杂时才用）：

```
CLAUDE.md           # 入口，指向子文件
.claude/
  testing.md        # 测试规范
  api-design.md     # API 设计规范
  deployment.md     # 部署规范
```

CLAUDE.md 里写：

```
## 深入话题（按需看）
```


- 测试规范：`.claude/testing.md`
- API 设计：`.claude/api-design.md`

Claude 会在需要时去读那些文件——但不会一启动就全加载（靠 Skills 机制实现按需加载）。

一个经验

90% 的项目不需要分层。一份 60-150 行的 CLAUDE.md 走完全程。只在大型团队 + 复杂项目才需要拆文件。

支线 14.B: CLAUDE.md 的版本管理

这段讲什么：CLAUDE.md 作为“活文档”的演化怎么追踪。什么时候回头读：你了解“这条规则是什么时候加的、当时为什么加”时。

把 CLAUDE.md 当作代码

- 进 git，提交时附带说明：

```
git commit -m "CLAUDE.md: add rule against direct SQL
migrations after incident with prod schema drift"
```

- 未来想改动时，看 `git log CLAUDE.md` ——所有演化一目了然

删 vs 标记过时

有一条旧规则不再适用——直接删好，还是标记“已废除”好？

一般直接删——CLAUDE.md 是 Claude 的指令，留着会让它混淆。删除历史留在 git 里就好，不放到当前版本。

每条规则的“诞生故事”

理想情况下，每加一条规则都在 git 提交消息里写一句：这条解决了什么具体事故或问题。

半年后回来看，你一眼能想起“哦，这条是当年 XX 事故后加的”——还有效就保留，已经不必要就删。

和“渐进披露”原则配合

- 主线 14.6 讲“渐进加”——从小到大、按需增长
- 本支线讲“怎么追踪演化”——让每次增长都可追溯、可回退

两者结合：CLAUDE.md 是一份活的、跟项目一起长的、可审计的团队共识。

下一章：第 15 章“模型与成本”——讲 Sonnet / Opus / Haiku 怎么选、`/status` 怎么读、大致要花多少钱、怎么省——给你一份可控的预算预期。

第 15 章：模型与成本——怎么选、怎么算、怎么省

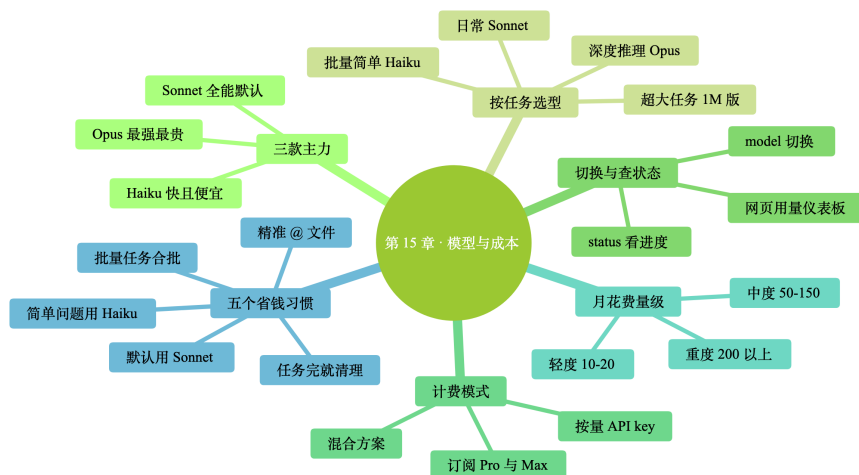
本章在本书的位置：第三部分 · 第 15 章 / 预估阅读+动手时长：主线 25-30 分钟 + 支线 10 分钟

前置章节：Ch 12（上下文原理，搞懂 token）

学完能做什么（主线）：知道 Claude 有几款模型（Haiku / Sonnet / Opus），各自擅长什么、贵多少；会读 `/status` 和使用量；知道大致一个月要花多少钱；掌握 5 个省钱的小习惯。

开场三问

- 你会遇到什么问题：听说用 AI 会花不少钱，担心用着用着月底收到大账单；不知道什么场景该用贵模型、什么场景便宜的就够。
- 不读这章会踩什么坑：一律用最贵的模型（浪费）；或一律用最便宜的（关键任务翻车）。
- 读完你会多会什么事：对每个任务能快速判断“这个用哪款模型合适”；心里有一份月度预算预期，不会被账单吓到。

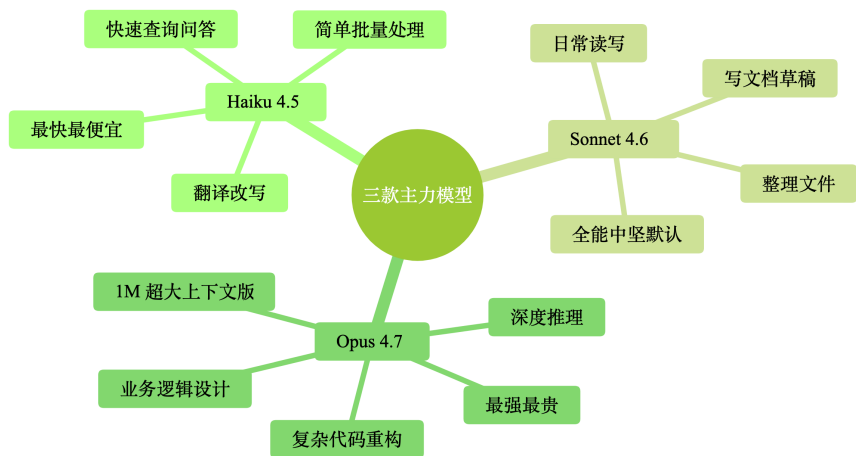


🎯 【主线】—— 本章必读核心

15.1 三款模型——一句话对比

Anthropic 目前主力三款模型，像三个不同岗位的实习生：

模型	比喻	速度	智力	价格
Haiku 4.5	干活很快的助理	最快	入门	最便宜
Sonnet 4.6	全能中坚	中等	强	中等
Opus 4.7	顶尖思考者	最慢	最强	最贵



记住规律：

🔑 越智能 = 越慢 = 越贵——三者是绑定的，没有“又聪明又便宜又快”的模型。

Opus 4.7 和 Opus 4.7 (1M 上下文版)

Opus 有两个版本：

- 默认 Opus 4.7：200K 上下文窗口
- Opus 4.7 (1M)：1M 上下文窗口（5 倍大），更贵

除非你真的在做超大项目（见 Ch 12 支线 B），日常用默认版就够。

15.2 怎么选——按任务类型

场景 A：日常读写、整理文件、写文档草稿

→ Sonnet 4.6

理由：够聪明 + 不慢 + 不贵。大部分日常任务它就是最优解。

场景 B：需要深度推理的复杂任务

例子：

- 大段代码重构，要一次性想清楚多文件的影响

- 复杂的业务逻辑设计
- 分析长文档、生成深度洞察

→ Opus 4.7

理由：这是它的强项。用 Sonnet 可能似是而非，用 Opus 能真的理解。

场景 C：简单批量处理、快速查询

例子：

- 批量改几十个文件的格式
- 简单的问答（“这个命令什么意思”）
- 翻译、简单改写

→ Haiku 4.5

理由：任务对智力要求不高，选快的 + 便宜的，体验更好。

场景 D：长文档 / 大项目

→ Opus 4.7 (1M)（仅在默认 200K 不够时）

新手最简经验

如果你不想记这么多：

- 默认用 Sonnet 4.6——覆盖 80% 场景
- 遇到卡住或深度推理用 Opus 4.7
- 简单速查用 Haiku

15.3 怎么切换——`/model` 命令

在 Claude Code 里随时可以换模型：

```
/model
```

会弹出一个列表，显示可用模型（大致如下）：

```
Select a model:
1. Claude Sonnet 4.6 (默认)
2. Claude Opus 4.7
3. Claude Opus 4.7 (1M context)
4. Claude Haiku 4.5
```

选一个，后续对话就用新模型。

切换是即时生效的

- 不需要重启
- 不影响已有上下文——当前对话继续，只是“处理这句话的大脑”换了
- 可以中途切换——任务前期用 Opus 想清楚策略，后期执行细节换 Sonnet

一个实用技巧：Opus 规划 → Sonnet 执行

(启动，默认 Sonnet)

/model → 选 Opus

你：分析 @这个项目/ 的代码结构，规划一个模块拆分方案

Opus：(给深度方案)

你：OK，方案我同意。

/model → 选 Sonnet

你：按刚才方案一步步执行，先改 src/foo.py

Sonnet：(执行)

规划用 Opus（深度思考，值回票价），执行用 Sonnet（快且便宜）。

15.4 怎么查用了多少——/status 和账单页

在 Claude Code 里查

/status

会显示：

- 当前模型
- 当前会话用了多少 token
- 当前 Ctx%
- 其他状态信息

这是单次会话的实时情况。

在网页后台查历史

Anthropic 官网 console.anthropic.com (或 Claude.ai 账户设置) 有用量仪表板：

- 按天的 token 消耗
- 按模型分的费用
- 订阅计划剩余额度

两种付费模式

Claude Code 目前有两种：

模式	怎么计费	适合谁
Claude Pro / Max 订阅	月费，有每日用量上限	重度个人用户（20-200 美金/月级别）
API Key 按量付费	按每 1M token 计价	轻度用户 / 公司 / 和别的工具共用

大致价格（2026 年水平，可能随时调整）

下面是按 API 算的大致单价，供参考。订阅制等于把这些用量打包成月费。

模型	输入（每 1M token）	输出（每 1M token）
Haiku 4.5	~\$1	~\$5
Sonnet 4.6	~\$3	~\$15
Opus 4.7	~\$15	~\$75
Opus 4.7 (1M)	~\$30	~\$150

官方最新价格请查：anthropic.com/pricing——本书出版后价格会变。

15.5 一个月大概要花多少——量级感

没做过 AI 工具的人对“一个月多少钱”没概念。给你三类参考：

轻度用户：每天 10–30 分钟

例子：偶尔整理文件、写几段文字

- 月均 token 消耗：3-5M（约 500 万 token）
- 费用：\$10-20 / 月
- 建议方案：Claude Pro（\$20/月）或按量 API

中度用户：每天 1–2 小时

例子：日常工作里频繁用它整资料、改文档、做小脚本

- 月均 token 消耗：30-50M

- 费用：\$50-150 / 月
- 建议方案：Claude Max（更高限额，\$100-200/月）或按量

重度用户：每天 3+ 小时的专业开发

例子：全职写代码、大量用 Opus 做复杂重构

- 月均 token 消耗：100-300M
- 费用：\$200-800 / 月
- 建议方案：Max 订阅 + 按量混用；或公司报销

注意：Ctx 爆满情况下费用翻倍

上下文越长，每轮对话的“输入”部分越大。一个 180K 上下文的对话，每轮输入就烧 18 万 token，Opus 按 15 美元/1M 算——每轮光输入就 2.7 美元。

所以管好上下文（/clear、/compact）直接就是省钱。

15.6 五个省钱习惯

习惯 1：默认 Sonnet，不默认 Opus

很多新手一上来就切 Opus——10 倍贵，大部分任务没必要。

规则：用 Sonnet 试，卡住了再切 Opus。

习惯 2：任务完成 /clear

（Ch 12 已讲）每多一轮对话，之前的积累都要重新发一次。清完重来比一直续着便宜。

习惯 3：精准 @，不 @ 整目录

- @src/Button.tsx → 可能 2K token
- @src/ → 可能几十 K 甚至 100K+ token

每次花 5 秒想清楚“真需要的是哪个文件”。

习惯 4：简单问题用 Haiku

- “这个命令什么意思” → Haiku 够用
- “这段错误是什么原因” → Haiku 够用
- “翻译这段” → Haiku 够用

这类任务用 Sonnet 或 Opus 是浪费。

习惯 5：批量任务合批

不要一次性发 10 条“改一下这个文件的这段”。合并成一次任务：

❌ 差：

```
你：改 a.py 的 X
(Claude 回答)
你：改 b.py 的 Y
(Claude 回答)
你：改 c.py 的 Z
...
```

✅ 好：

```
你：我有三个改动要做，一次说完。@a.py 改 X；@b.py 改 Y；@c.py 改 Z。
```

第二种只用一次“大脑循环”，前者用三次，后者更便宜也更快。

本章小结

- Claude 有三款主力：Haiku（快便宜）、Sonnet（中坚，日常默认）、Opus（最强最贵）——越强越慢越贵
- 选择原则：日常 Sonnet；深度推理或卡壳切 Opus；简单批量用 Haiku；超大任务 Opus (1M)
- `/model` 切换模型；`/status` 看当前会话；网页后台看历史用量
- 月花费量级：轻度 \$10-20、中度 \$50-150、重度 \$200+——订阅或按量二选一
- 五个省钱习惯：默认 Sonnet / 任务完成 `/clear` / 精准 `@` / 简单问题用 Haiku / 合批

动手任务

任务 1：跑一次 `/status` 熟悉界面（3 分钟）

步骤：

1. 启动 Claude Code
2. 运行 `/status`
3. 记下你看到的所有信息——模型、Ctx%、本次用量

成功标志：你现在知道随时用 `/status` 查进度。

任务 2：用同一个问题对比 Sonnet 和 Opus（10 分钟）

步骤：

1. 准备一个稍有深度的问题（例：“给我的 @一个项目/ 提三个可以重构的点”）
2. 先用 Sonnet 问一遍，记录回答
3. `/model` 切 Opus，同一个问题再问一遍
4. 对比：Opus 的回答是不是真的深一层？速度慢了多少？

成功标志：你对“什么时候 Opus 值钱、什么时候不值”有了直觉。

任务 3：查你过去 30 天的用量（5 分钟，需要已用过几天）

步骤：

1. 打开 console.anthropic.com 或 Claude.ai 账户页
2. 找到“Usage”/“用量”页
3. 看上月总花费

成功标志：你对自己的实际花费水平有了数字，不再凭感觉。

如果你卡住了

症状 A： `/model` 列表里看不到 Opus 或 Haiku

- 原因：(a) 账户没开对应权限；(b) Pro 用户某些高级模型要 Max 订阅。
- 解决：查账户订阅等级；或用 API key 用量付费模式（通常所有模型都可用）。

症状 B：钱花得比预期快很多

- 原因：(a) 一直用 Opus；(b) 上下文总是满的；(c) @ 了大文件 / 大目录。
- 解决：(a) 切 Sonnet；(b) 养成 `/clear` 习惯；(c) 精准 @。

症状 C：遇到“rate limit”报错

- 原因：短时间内用太多——达到每分钟 / 每小时额度上限。
- 解决：等 1-5 分钟；或暂时切到 Haiku；或升订阅等级。

主线结束。下面支线讲“按量 vs 订阅怎么选”和“公司报销的一点建议”。



【支线】—— 可选深入（学有余力再看）

支线 15.A：按量付费 vs 订阅——怎么选

这段讲什么：Claude Pro/Max 订阅 vs 自备 API key 按量计费，哪种更划算。
什么时候回头读：你第一个月账单出来后想优化时。

订阅 (Claude Pro / Max)

优点：

- 月费固定，不担心突然大账单
- 额度用不完也不退，但心理负担小
- 升级模型选择方便 (Max 能用最高档的)

缺点：

- 有每日额度——用多了会被限速
- 额度用不完当月清零，波动大的用户不划算

适合：稳定高频用户（每天都用 1 小时以上）

API key 按量

优点：

- 真正按量算钱——用多少付多少
- 可以用第三方工具共享同一个 key
- 公司场景便于报销（拿到发票）

缺点：

- 有月底大账单的风险
- 每个模型的价格要自己比

适合：

- 用量波动大的用户（有时一天几小时，有时一周不用）
- 小团队共享
- 公司场景

混合方案

高阶玩法：订阅 + API key 两份，视情况切换：

- 日常用订阅（用额度）

- 订阅额度用完 / 特殊场景用 API key

这种配置复杂，一般人不需要。先订阅一个月看看自己的模式再决定。

支线 15.B：团队 / 公司场景的一点建议

这段讲什么：打算给团队用、或公司报销时的几个注意点。什么时候回头读：你要为团队采购 Claude 时。

建议 1：查清楚公司 AI 政策

(Ch 11 讲过) 很多公司对 AI 工具有采购流程，别自己买了之后报销碰钉子。

建议 2：选择 Claude for Work / Enterprise

如果规模大 (10 人以上)：

- 支持 SSO 登录
- 可配置数据保留策略 (0 天、30 天等)
- 团队统一计费
- 更严的合规约定

比每个员工自己买订阅规整得多。

建议 3：团队共享不共享？

共享一个 API key 给所有人用：

-  省管理成本
-  用量监控困难，谁花多了看不清

每人一个 key / 账号：

-  用量清晰
-  方便审计
-  管理复杂

推荐：每人一个账号，但统一结算到公司 (Anthropic 企业版就是这么设计的)。

建议 4：预算控制

- Anthropic 有每月支出上限可以设——防止爆单

- 建议公司级账户设 \$X/月 上限，超了自动停
- 个人账户也可以设 \$50 / \$100 这种保护线

第三部分（深度概念）到此完成。

你现在已经掌握了：

- Claude Code 的数据去哪了、什么能发什么不能（Ch 11）
- 上下文为什么会满、三种清理策略（Ch 12）
- 三层记忆系统，每层放什么（Ch 13）
- 怎么写一份让 Claude 真的变准的 CLAUDE.md（Ch 14）
- 怎么选模型、怎么算成本、怎么省（Ch 15）

****下一部分（第四部分：避坑与判断力）****开始：10 大常见错误、什么时候该停下来、常用命令速览、输入技巧——把你从“会用”推到“用得不踩坑”。

💡 想专门了解 Opus 4.7 相对 4.6 的变化、新的 effort 档位、分词器变化、prompt 微调建议、5 个新手坑——直接读 [附录 J · Claude Opus 4.7 新指南](#)。本书以 4.7 为基线写作，附录 J 把 4.7 的所有差异集中讲了一遍。

第四部分 · 避坑与判断力

第 16 章：新手 10 大错误——看完能避开的坑

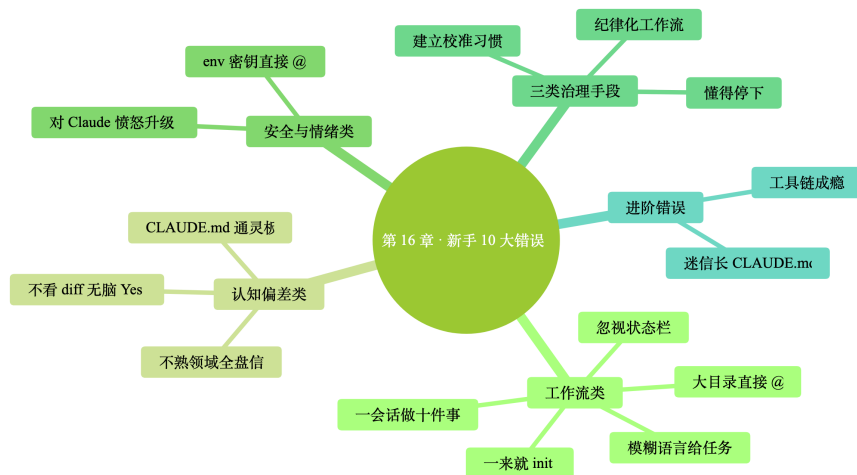
本章在本书的位置：第四部分 · 第 16 章 / 预估阅读+动手时长：主线 25-30 分钟 + 支线 10 分钟

前置章节：读过 Part 1-3 再看，代入感最强

学完能做什么（主线）：对零基础用户最常犯的 10 个错误有免疫力。看见每个错就能联想起“本章讲过”——在踩之前先绕开。

开场三问

- 你会遇到什么问题：按说该会用了，但总在同一些地方栽跟头；踩一次印象深，但下次又没防住同类。
- 不读这章会踩什么坑：被动踩坑——只有事故发生后才学到。
- 读完你会多会什么事：看到这 10 类情境就条件反射警觉——踩过没踩过都能绕过去。



🎯 【主线】—— 本章必读核心

16.1 错误 1：一来就跑 `/init`

场景：第一次打开 Claude Code，网上教程说“先 `/init` 生成项目说明书”。一跑，生成了一份 400 行的 CLAUDE.md。

翻车：

- 这份 CLAUDE.md 你自己没通读过
- 每次启动它都占你 Ctx
- Claude 抓不到重点——规则太多互相淹没

正确做法（详见 Ch 14）：

- 手写，从空白或 20-30 行模板开始
- 发现 Claude 重复犯错时，再补条目
- 容量控制在 < 60 行

16.2 错误 2：把 `.env` 或密钥文件 @ 进来

场景：想让 Claude 帮你加一个环境变量配置，直接 @`.env` 帮我加一个 `REDIS_URL`。

翻车：

- `.env` 里已有的密钥（数据库密码、API key）全部上传到 Anthropic 服务器（30 天暂存）
- 虽然不训练模型，但密钥已经离开你的电脑
- 补救：立刻去每个服务 `revoke` 那些密钥，重新签发

正确做法（详见 Ch 11）：

- `.env` 里放敏感值的文件永远不 @
- 想让它改，复制一份脱敏版（值改成 `<REDACTED>`），改完再把真值填回去
- 或只告诉 Claude 字段名，不给值

16.3 错误 3：不看 diff 就无脑 Yes

场景：让 Claude 改一个文件，它说“改好了”，你 Enter Enter 下去没细看。

翻车：

- 它**“顺手”多改了几个你没让它改的地方**——常见
- 新增了一段你不需要的“防御性代码”
- 删掉了你原来有意保留的注释

正确做法（详见 Ch 6、Ch 10）：

- 即使 `vibe review`，也要扫一眼 diff 文件数 + 改动范围
- 看到“顺手改了 X”立刻 No，让它只做你要求的事
- 高风险场景（auth / 支付 / 数据库）强制细审

16.4 错误 4：在一个会话里做 10 件不相关的事

场景：早上 9 点起一个 Claude Code 会话，整理照片；10 点改博客；11 点看代码；12 点写周报……全在同一个会话里。

翻车：

- Ctx 越来越高，到下午 Claude 答非所问
- 不同任务的上下文互相干扰（改博客时它还在聊早上的照片文件名）
- 你纠正一次、它好一会，漂移不止

正确做法（详见 Ch 12）：

- 每完成一件独立任务就 `/clear`
- 关系近的连贯任务可以续着，跨主题就清
- 一个会话聚焦一件事

16.5 错误 5：文件太大 / 目录太大直接 @

场景：`@整个项目 / 帮我看看哪里可以优化`——整个项目几百个文件。

翻车：

- 几万 token 瞬间灌进上下文——Ctx 直接 80%+
- Claude 看得太杂，回答泛泛而谈
- 比你自已打开 IDE 看还低效

正确做法（详见 Ch 5、Ch 12）：

- 先让 Claude 看目录结构（`ls / tree`），再由你指定具体哪个文件细看
- 一次 `@` 不超过 3-5 个文件
- 大文件先要摘要再细看关键段

16.6 错误 6：让 Claude 干完全不懂的活然后全盘信

场景：你是 Python 开发者，让 Claude 写一段 Bash / SQL / 正则——从没学过，看它写完就直接拿去用。

翻车：

- Claude 写这些领域容易翻车（尤其正则和 SQL）
- 你不懂 = 你检查不了——出错了你都看不出
- 搞坏数据库 / 删错文件，事后才发现

正确做法（详见 Ch 10）：

- 跨自己不熟的领域 → 强制 L4 审核
- 让 Claude 附带测试 / 样例验证（正则给 3 个能匹配 + 3 个不能匹配的字符串）
- 或找一个懂的人 review
- 不要在“不熟 + 不可逆”的交集上让 Claude 出手（例：直接改生产库）

16.7 错误 7：用模糊语言给任务

场景：`帮我优化一下这段代码`。什么叫优化？速度？可读性？内存？

翻车：

- Claude 猜一个方向去做，通常不是你想要的
- 它做完你说“不是这个意思”，重做一遍——Ctx 双倍消耗，你也烦

正确做法（详见 Ch 5 WHAT/WHERE/HOW/VERIFY）：

- WHAT：具体改什么
- WHERE：哪个文件哪段
- HOW：怎么改（如果能想到）
- VERIFY：怎么验证改对了

例：❌“优化 @util.py” → ✅“@util.py 里的 parseDate 函数太慢（测过 500ms），请改成不用正则的实现，改完我会跑 pytest tests/test_util.py 验证”

16.8 错误 8：忽视状态栏

场景：你根本没看过 Claude Code 状态栏——甚至不知道有 Ctx% 这个东西。

翻车：

- Ctx 90%+ 你没察觉，然后 Claude 开始“无中生有”（幻觉）
- 用的哪个模型自己都不知道——该用 Sonnet 时用了 Opus（贵）或反过来（笨）
- 账单超出预期

正确做法（详见 Ch 12、Ch 15）：

- 养成每几轮瞄一眼状态栏的习惯
- 不清楚就 /status 看一眼
- Ctx > 50% 开始警觉、> 75% 主动整理

16.9 错误 9：对 Claude 的错误愤怒升级

场景：Claude 改错了，你生气：“我都说过多少次了！”下一句 Claude 继续错。你更生气、措辞更重、加大写加感叹号。

翻车：

- 愤怒的提示词里反而信息更少、更模糊
- 你在和“一个没人格的语言模型”对话——它不会“改正态度”，只会继续按猜测做
- 越烦躁越出错，掉进死循环

正确做法（详见 Ch 17）：

- 感觉到“第三次纠正同一个错”时——停
- `/clear` 重启；或 `/rewind` 回到错之前
- 用更具体 + 冷静的语言重讲一遍：“X 的要求是……，不是你理解的那样”
- 实在不行，自己动手——AI 不是万能

16.10 错误 10：把 CLAUDE.md 当“通灵板”

场景：CLAUDE.md 里写“Claude 必须像资深工程师一样思考”、“优雅地处理所有边缘情况”、“写出高质量代码”。

翻车：

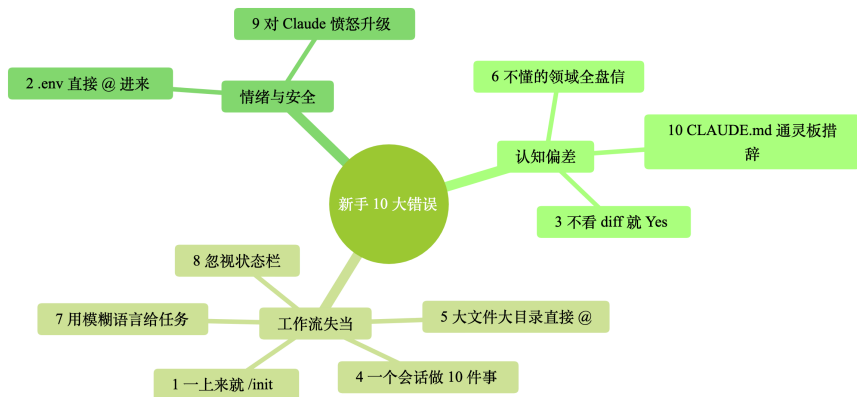
- 这些对 Claude 没有任何意义——它不知道“资深”、“优雅”、“高质量”具体什么标准
- 条目占位置但不 actionable
- 真正该写的具体规则被淹没

正确做法（详见 Ch 14）：

- 每条规则要可机械判断
- ❌ “写高质量代码” → ✅ “函数 < 50 行 / 单一职责 / 必须有测试”
- ❌ “处理好错误” → ✅ “每个外部调用必须 try/except，异常日志记到 `logger.error`”

本章小结

10 个错误归起来就 3 种模式：



- 认知偏差：以为 AI 比实际更聪明 / 更可靠（错误 3、6、10）
- 工作流失当：没管好上下文、没精准提问（错误 1、4、5、7、8）
- 情绪与安全：愤怒重试 / 敏感信息上传 / 在不熟领域踩雷（错误 2、6、9）

治理这三类就用三个手段：

- 建立校准习惯（Ch 10 信任档案 + L1-L4 审核）
- 纪律化工作流（`/clear` / 精准 `@` / WHAT-WHERE-HOW-VERIFY）
- 懂得停下（下一章）

动手任务

任务 1：对号入座（10 分钟）

回想过去 1-2 周你用 Claude Code 踩过的坑。对照本章 10 条，打勾你踩过的。
成功标志：至少识别出自己踩过 3 条——不是攻击你，是你现在有名字可用，下次能警觉。

任务 2：建立“避坑自检清单”（15 分钟）

新建 `~/Desktop/ccguide-练习/避坑清单.md`：

```
# 我的 Claude Code 避坑清单
```

```
## 动手前问自己
```

```
- [ ] 这个任务涉及敏感信息吗？需要脱敏吗？（错误 2）
```

- [] 我要 @ 的文件是精准的吗？（错误 5）
- [] 我的任务描述有 WHAT/WHERE/HOW/VERIFY 吗？（错误 7）
- [] 这是我熟悉的领域吗？不熟要升级审核档位（错误 6）

动手中警觉

- [] 状态栏 Ctx 多少？（错误 8）
- [] diff 改了哪些文件，和我预期一致吗？（错误 3）
- [] 跨任务了吗？该 /clear 了吗？（错误 4）

情绪信号

- [] 第三次纠正同一个错 → 停下来，/clear 或亲自动手（错误 9）

成功标志：这份清单贴你桌面 / 贴输入法候选 / 贴 CLAUDE.md 边上都行——肉眼能看见。

任务 3：给一个现成 CLAUDE.md 找错误 10（5 分钟）

打开你自己写过的或公开找的一份 CLAUDE.md，找出所有“通灵板式”措辞（“高质量”、“优雅”、“像专家”这类）。

成功标志：至少识别 3 条——把它们改成 actionable 的具体规则。

如果你卡住了

症状 A：一条都没踩过，觉得本章没用

- 原因：要么你用得还不够多、要么你已经很谨慎。
- 解决：当作备忘留着——用到 10 小时以上大概率会踩其中几条。

症状 B：10 条都踩过，觉得自己是笨蛋

- 原因：正常人的学习曲线就是这样。
- 解决：踩过不代表永远踩——踩过而能命名的错误，下次会本能警觉。

症状 C：不确定自己踩了哪一条

- 原因：错误之间有时重叠（错误 4 + 5 常一起发生）。
- 解决：不用精确归类——重要的是认出类型，能对照到本章的建议即可。

主线结束。下面支线讲“两个进阶错误（错误 11-12）”和“如何把这份清单当项目持续迭代”。

【支线】——可选深入（学有余力再看）

支线 16.A：两个进阶错误

这段讲什么：不在主线 10 条里，但用一阵子后容易遇到的两个坑。什么时候回头读：你已经用 Claude Code 超过一个月时。

错误 11：工具链成瘾——什么都想 Skill 化

用熟了之后开始疯狂写 custom Skills、Hooks、Subagents——每件小事都做一个工具。

翻车：

- 时间都花在建工具上，反而真正的工作没做
- 工具之间打架（Hook 触发了 Skill，Skill 又调 Subagent……）
- 维护负担指数上升

正确做法：

- 一个任务做过 5 次以上再考虑工具化
- 工具链先求少，每个工具独立可验证
- 定期砍掉不常用的工具

错误 12：迷信长 CLAUDE.md

做了一阵开始觉得 CLAUDE.md 越全越好——加到 500 行、1000 行，觉得这样 Claude 就肯定懂项目了。

翻车：

- Ch 14 讲过——长 CLAUDE.md 效果反而变差
- 注意力分散、规则冲突
- 每次启动慢

正确做法：

- 记住 60 / 300 行两条线
- 长了就砍：砍可 pointer 化的、砍含糊的、砍早已默认的
- CLAUDE.md 不是“保险”——不是条数越多越保险

支线 16.B：把错误清单当项目迭代

这段讲什么：本章 10 条是通用清单，但你有个人化的错误模式。什么时候回头读：踩过本章 10 条的大部分后。

建立“我的个人错误日志”

新建 `~/Desktop/ccguide-练习/个人错误日志.md`：

```
# 我在 Claude Code 上犯过的错

## 2026-04-15: 周报被 Claude "优化"掉了关键数据
- 原因：无脑 Yes，没审 diff
- 教训：vibe review 时至少扫一遍数字字段有没有动
- 类型：错误 3（无脑 Yes）

## 2026-04-18: 老板问我 Claude 花了多少钱，答不上来
- 原因：从没查过用量页
- 教训：每月 1 号看一次 console
- 类型：错误 8（忽视状态）
```

每月回顾

- 本月犯过哪些错
- 哪几类重复发生（对应本章哪一条）
- 针对高频错误加一条硬规矩到 CLAUDE.md 或避坑清单

这份日志的两个好处

1. 给自己留记忆——错过的不再错第二次
2. 如果团队用 Claude Code——汇总大家日志，形成团队规范

下一章：第 17 章“什么时候该停下来”——最务实的判断力训练。AI 不是什么都做，不是多给几次提示就能解决。知道什么时候停、改用别的方法，才是真正的熟手。

第 17 章：什么时候该停下来——不要和 AI 对抗

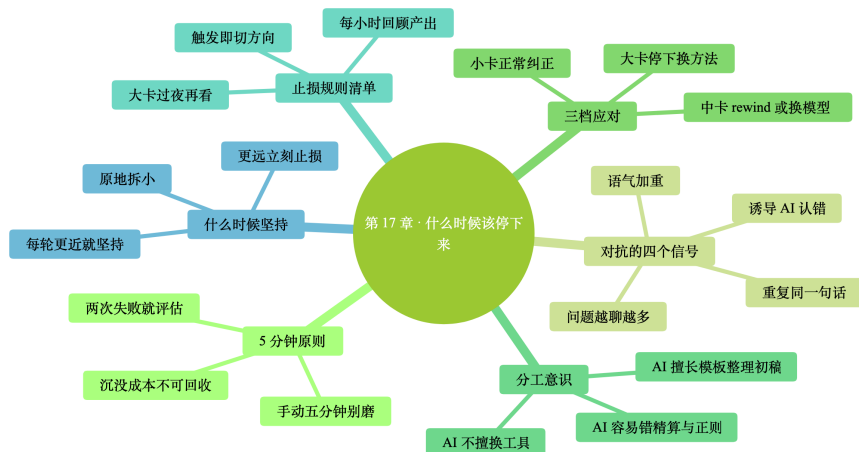
本章在全书的位置：第四部分 · 第 17 章 / 预估阅读+动手时长：主线 20-25 分钟 + 支线 8 分钟

前置章节：Ch 10（信任校准）+ Ch 16（10 大错误）

学完能做什么（主线）：掌握**“停”的判断力**——知道什么时候该放弃当前尝试、换方向、甚至放下 Claude 自己动手。这是区分“新手”和“熟手”的核心能力。

开场三问

- 你会遇到什么问题：陷入“再提示一次就行”的死循环——花 40 分钟修一个本来手动 5 分钟能完成的事。
- 不读这章会踩什么坑：沉没成本陷阱——已经和 Claude 来回 10 轮，不甘心放弃，继续下去更慢更贵。
- 读完你会多会什么事：给自己设“止损规则”，过线就强制切方向——该手动就手动、该重启就重启、该放下就放下。



【主线】—— 本章必读核心

17.1 核心原则：5 分钟原则

这是从老手社区里总结出来的一条铁律：

🔑 一个任务原以为 AI 能搞定，结果花了 30 分钟还没对——比你手动做慢。

具体规则：

5 分钟原则

- 第 1 次尝试不成功 → 调整提示词，再来 1 次
- 第 2 次还不成功 → 停下来：问自己“手动做要多久？”
- 手动 < 5 分钟 → 关掉 Claude 自己做
- 手动 > 20 分钟 → 继续用 AI 但换方向（换模型 / `/clear` / `/plan` / 拆小）

为什么是 5 分钟

- 手动做 5 分钟以内的事，你自己做确定可控——不会像 Claude 那样“顺手错改”
- 机会成本：再和 Claude 磨 10 分钟 = 你已经手动做了 2 次

一个真实场景

(任务：把 20 张截图按时间重命名)

第 1 次：你：给 ~/Desktop/截图/ 的文件按修改时间排序重命名为 01-20

Claude：(生成一个脚本，跑错，部分文件名乱了)

你：回退

第 2 次：你：@截图/ 列出来，然后按顺序重命名

Claude：(这次命名对了，但漏了 3 个隐藏文件)

你：..... 还是不对

【5 分钟原则触发】

手动做要多久？我自己用 Finder 点一下排序、右键批量重命名，2 分钟。

→ 关掉 Claude，自己做，2 分钟搞定。

17.2 “和 AI 对抗”的四个信号

你正在和 AI 对抗时，会出现以下信号。看到任何一个就要警觉：

信号 1：你开始用加重语气

“不是那样”、“我已经说了”、“为什么还错”——语气升级但信息没升级。

背后：你以为 AI 会因为你“生气”而改正。它不会——它只是个语言模型，没有“被骂会认真”的反射。

对策：深呼吸。换冷静+具体的语言重说一遍，或者直接 `/clear` 重新开。

信号 2：你在第 N 次说同样的话

“再说一次——X 需要...”。你意识到自己在重复，Claude 没吸收。

背后：

- 可能 Ctx 爆了 (Ch 12) ——前几轮已被“挤掉”
- 可能提示词本身模糊——它每次理解都不同

- 可能是领域 Claude 处理不好（参见 10.A 错误热点）

对策：

- Ctx 高 → `/compact` 或 `/clear`
- 提示词模糊 → 把关键要求写进 CLAUDE.md 或提示词最前面的加粗段
- 领域困难 → 换方法（换模型 / 分解任务 / 人工介入）

信号 3：问题越聊越多

原来是 1 个 bug，修到第 5 轮变成 3 个 bug。Claude 在“修”的过程中引入了新问题。

背后：

- 它改了你没让它改的地方（常见）
- Ctx 里前后要求冲突，它“把旧要求忘了”

对策：

- `/rewind` 回到最早那个状态
- 重新开始，这次只让它做一件事
- 或者 `/clear` + 重新描述需求

信号 4：你在“诱导”AI 说出你想要的

“你确定这段没问题？”、“这样真的对吗？”——你希望它自己发现错误。

背后：你其实心里已经觉得有问题，但不想自己分析。

对策：

- 你自己审一遍代码/结果——比让 AI 自省更准
- 或直接描述你担心的点：“这段并发安全吗？如果两个请求同时…会怎样”——具体质疑，不是笼统怀疑

17.3 卡住时的三档应对

不同程度的“卡住”用不同招。

第 1 档：小卡（< 3 轮重试解决）

表现：Claude 小错一下，你纠正一下就对。

对策：

- 正常纠正、继续
- 不用动任何“大招”

第 2 档：中卡（3–5 轮还在同一个坎）

表现：反复错在同一个点，你感觉“绕不过去”。

对策（按顺序试）：

1. /rewind 回到错之前，换种提示方式重做
2. /compact 压一下——有时是 Ctx 满了让它糊涂
3. 换模型：用 Opus 重新想一次（可能是 Sonnet 这种特殊任务确实吃力）
4. 拆小任务：大步改小步——“先只做 A”、“做完再做 B”
5. 切 /plan 模式：让它先出方案你审，审过了再动手

第 3 档：大卡（5 轮以上没有进展）

表现：你已经调整了好几次提示词、换了模型，还是不对。

对策：

1. 停下 Claude Code——不要再试
2. 问自己 3 个问题：
 - 这个任务真的适合 AI 做吗？（有没有碰到 Claude 薄弱环节）
 - 我自己手动要多久？
 - 如果换一个不用 AI 的工具 / 方法，会不会更简单？
3. 给出三选一：
 - 亲自动手（如果手动 < 30 分钟）
 - 分阶段：AI 辅助 + 人工把关（AI 做大体，关键步骤人工亲自做）
 - 先放下，明天再说（有时睡一觉换个角度就解了）

17.4 人和 AI 各自擅长什么

搞清楚分工，才不会在 AI 短板上死磕。

AI 擅长（放手让它干）

- 模板化的大量工作：改 50 个文件的 import 语句、批量改变量名、统一格式
- 信息整理：整理笔记、提取要点、做摘要
- 写初稿：博客草稿、邮件草稿、文档大纲
- 翻译、改写、润色
- 你懂大意但懒得动手的小脚本
- 语法 / 语言错误检查

AI 容易出错（要严审）

- 精确数字 / 日期 / 货币计算（参见 10.A 错误热点 1-2）
- 正则表达式
- 并发 / 竞态条件
- 安全 / 加密
- 长文档前后一致性（第 100 页忘了第 1 页的设置）
- 极新的技术栈（训练数据里没见过的新版本）

AI 不擅长（换工具）

- 精确数值计算 → 用 Excel / Python 计算
- 大批量文件操作 → 直接用 Finder / 命令
- 实时数据 / 网络查询 → 用浏览器 / API
- 图片深度编辑 → 用 Photoshop / 专业工具
- 复杂的 GUI 操作 → 自己动手
- 需要物理世界反馈的任务 → 亲自来

记住：Claude 是你的助手，不是你的替身。助手干不了的事，你接回来干。

17.5 建立你自己的“止损规则”

把“什么时候停”变成一张清单，减少现场判断的负担。

推荐起始版（新建 `~/Desktop/ccguide-练习/止损规则.md`）：

```
# 我的 Claude Code 止损规则
```

```
## 必须停的时刻（任何一条触发都停）
```

- 同一个错被纠正超过 3 次 → /clear 或亲自动手
- 第 2 次尝试仍失败 → 评估"手动多久"
- 手动 < 5 分钟的任务 → 根本别上 AI
- Ctx > 85% → 强制整理或 /clear
- 自己开始用加重语气 → 深呼吸 + 切方向
- 做到一半出现新 bug → /rewind 回去

```
## 每小时给自己一个"回顾点"
```

- 这小时实际产出了什么？
- 花在 AI 上的时间值得吗？
- 要不要换方法？

兜底招

- 大卡超过 15 分钟 → 今天不做了，明天再说

把这份规则打印贴在电脑旁，或用一个贴纸提醒自己。

本章小结

- 5 分钟原则：手动 < 5 分钟的事别磨 AI；AI 尝试 > 2 次不成功就评估
- 对抗信号：加重语气 / 重复说同话 / 问题越聊越多 / 诱导回答——任一出现就警觉
- 三档应对：小卡（纠正继续）→ 中卡（rewind / compact / 换模型 / 拆小 / plan）→ 大卡（停下评估换方法）
- 分工意识：AI 擅长模板化、整理、初稿；不擅长精算、并发、安全、实时数据
- 建一份止损规则：让“什么时候停”变成自动触发，不靠现场判断

动手任务

任务 1：回顾过去一周最浪费时间的一次对话（10 分钟）

找出你最近一次“花了很久没搞定”的 Claude Code 尝试。复盘：

- 我在第几轮时应该停？
- 停了之后该换什么方法？（亲自动手 / 换模型 / /plan …）
- 这次总共浪费了多少时间？
- 如果当时知道本章内容，会省多少？

成功标志：你找到了自己的“典型浪费模式”——下次能认出来。

任务 2：建立你的止损规则（10 分钟）

按 17.5 的模板建 `~/Desktop/ccguide-练习/止损规则.md`。照抄也行，但要加至少 1 条你自己的个性化规则（根据任务 1 的复盘）。

成功标志：你有了一份属于你自己的止损清单。

任务 3：在下一个任务里真的执行“5 分钟原则”（下次做任务时）

步骤：

1. 下次用 Claude Code 时，出声念一遍：5 分钟原则
2. 第 2 次失败时，强制问自己“手动多久”
3. 真的手动做（哪怕你觉得“再来一次能行”）
4. 事后对比：手动 vs 继续磨，哪个更快

成功标志：你亲身体验一次“停下来自己做”更划算——把这个体验印进肌肉记忆。

如果你卡住了

症状 A：每次都想“再来一次就行”，停不下来

- 原因：沉没成本偏见——已经投入 10 分钟，不想白费。
- 解决：沉没成本是不可回收的——和“再花多久”无关。默念：已经花的钱不会因为再花一些就回来。

症状 B：不知道怎么判断“手动要多久”

- 原因：对手动流程不熟。
- 解决：估不准就强制试 1 分钟——如果 1 分钟内自己能跑出一半，说明手动快；跑不出来再回 AI。

症状 C：止损规则写了但没执行

- 原因：规则没有“可见性”——写完放文件夹里就忘了。
 - 解决：贴在显示器边上；或用手机定 30 分钟倒计时闹钟——响了就回顾进度。
-

主线结束。下面支线讲“什么时候坚持”（反面教材避免误杀）和“多人场景的止损”。

 【支线】—— 可选深入（学有余力再看）

支线 17.A：什么时候坚持——避免过度止损

这段讲什么：主线讲的是“什么时候停”，但也有“该坚持”的场景——不然会误杀好任务。什么时候回头读：你感觉“我是不是太快放弃了”时。

值得多试几次的场景

1. 第一次尝试——没有充分尝试前别下结论
2. 你自己描述不够具体——换一个精准提示词还没试过
3. 任务本身复杂——5-10 轮循环本来就正常
4. Claude 给的方向有希望——只是细节没对准
5. 你确实不会手动——止损成本为负（比 AI 还慢或根本不会做）

区别“坚持”和“对抗”

- 坚持：方向对、细节调、进度有（每轮离目标更近一点）
- 对抗：方向错 / 细节反复错 / 每次都“前进两步退三步”

判断标准：每轮过后离目标更近还是更远？

- 更近 → 坚持
- 原地 → 拆小 / 换方法
- 更远 → 立刻 `/rewind` 或 `/clear`

一个简单计时器

做任务时心里默数“进展感觉”：

- 第 1 轮：基线
- 第 3 轮：比第 1 轮近了吗？
- 第 5 轮：比第 3 轮近了吗？

如果第 5 轮比第 3 轮还远——止损信号确认，不是错觉。

支线 17.B：团队 / pair programming 场景

这段讲什么：两个人一起用 Claude Code 时，“停”的判断要怎么协作。什么时候回头读：你和同事一起用 Claude Code 时。

两个人一起更容易陷入“对抗”

- 两人都不想先喊停，以免显得“没耐心”
- 轮流提示，各自都有“这一次应该行”的乐观预期
- 时间不知不觉更长

建议规则

约定止损信号：

- 明确的止损触发器——比如“两人加起来尝试 3 次还不对就停”
- 一个人打开计时器——15 分钟闹钟
- 响了就强制讨论“继续还是换方法”

分工也要明

不要两个人都抢着敲提示词。约定：

- A 负责提示（主笔）
- B 负责审核 + 判断止损（旁观者视角）

旁观者更容易看出“我们陷进去了”——A 沉浸在提示词里，看不出整体。

下一章：第 18 章“常用快捷命令全览”——整本书出现过的各种斜杠命令（`/clear` `/compact` `/model` `/status` `/plan` `/rewind` ...）一次梳理，带使用场景。速查手册级别。

第 18 章：常用快捷命令全览——斜杠命令速查

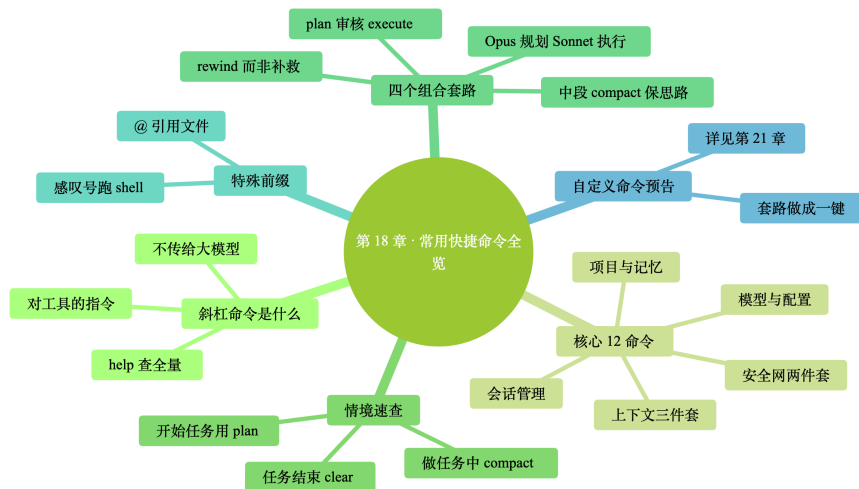
本章在全书的位置：第四部分 · 第 18 章 / 预估阅读+动手时长：主线 20-25 分钟 + 支线 5 分钟

前置章节：前面章节已出现的命令在这里统一整理

学完能做什么（主线）：一张图 + 一张表记住所有常用斜杠命令，按场景知道该用哪个。以后遇到新情境能直接反应出合适命令。

开场三问

- 你会遇到什么问题：命令散落在各章，记不全；遇到情境不知道有没有现成命令对应。
- 不读这章会踩什么坑：不知道工具箱里有什么工具 → 硬凑 → 费时。
- 读完你会多会什么事：看到情境能条件反射想到命令——效率翻倍的关键。



【主线】—— 本章必读核心

18.1 斜杠命令是什么

在 Claude Code 里，以 `/` 开头的一行就是斜杠命令（slash command）——不是提示词、不传给大模型，而是直接对 Claude Code 这个工具的指令。

怎么用

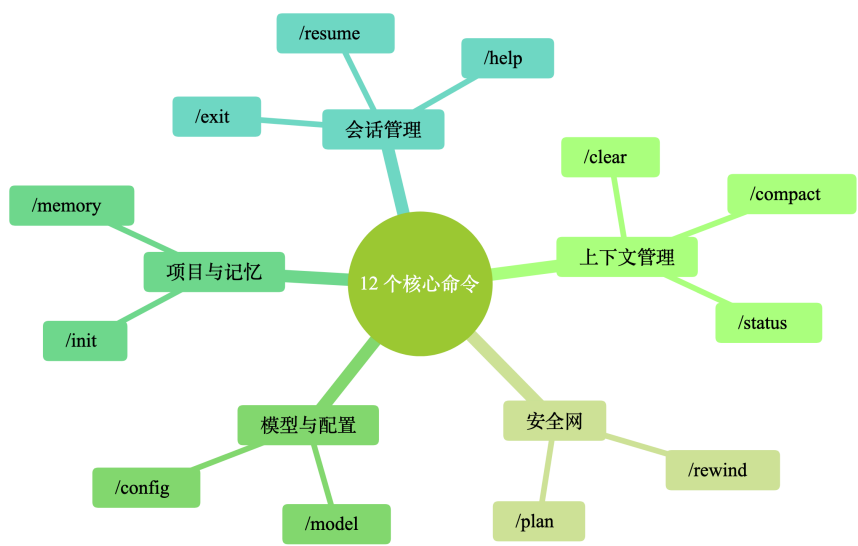
1. 在 prompt 提示符处直接敲 `/`
2. 会出现候选列表，打字筛选
3. 回车执行

怎么查全量

任何时候跑 `/help`，能看到当前版本所有可用命令（含自定义命令）。

18.2 核心 12 个命令——按场景分组

所有命令按用途分成 5 组。这 12 个覆盖 95% 的日常需要。



组 1：上下文管理（Ch 12）

命令	作用	什么时候用
<code>/clear</code>	清空当前会话	任务做完、切新任务
<code>/compact</code>	压缩当前会话成摘要	Ctx > 50% 但任务未完
<code>/status</code>	看当前状态（模型、Ctx%、用量）	任何时候想查进度

组 2：安全网（Ch 9）

命令	作用	什么时候用
<code>/plan</code>	进入计划模式（只出方案不动手）	开始一个需要先想清楚的任务
<code>/rewind</code>	精确回退到之前某个状态	走错一段后想回到分叉点

组 3：模型与配置 (Ch 15)

命令	作用	什么时候用
<code>/model</code>	切换模型 (Sonnet / Opus / Haiku)	任务难度变了、想省钱 / 求准
<code>/config</code>	看 / 改 Claude Code 配置	调偏好 (主题、自动保存等)

组 4：项目与记忆

命令	作用	什么时候用
<code>/init</code>	自动扫描项目生成 CLAUDE.md	不推荐 (见 Ch 14), 建议手写
<code>/memory</code>	打开记忆管理界面	想查 / 改 MEMORY.md 里 Claude 对你的记忆

组 5：会话管理

命令	作用	什么时候用
<code>/exit</code>	退出 Claude Code	关掉
<code>/resume</code>	恢复上次会话	不小心退了想回到前一次状态
<code>/help</code>	查看所有命令的说明	记不清用法时

18.3 一图看懂：什么情境用什么命令

遇到情境 → 用什么命令

【开始任务】

— 需要先想清楚策略

→ `/plan`

— 想换模型

→ `/model`

— 看看当前状态

→ `/status`

【做任务中】

```
|
├─ Ctx 快满了 (> 50%)      → /compact
├─ 做错了一步              → /rewind
├─ 想查 Claude 记得我什么 → /memory
└─ 想改偏好                → /config
```

【结束任务】

```
|
├─ 任务完成，下一件        → /clear
├─ 关闭 Claude Code        → /exit
└─ 明天想回到现在状态      → （退出即可）次日跑 /resume
```

【记不清命令时】

```
|
└─ /help
```

18.4 命令组合使用——“套路”

单个命令有用，组合起来威力更大。几个高频套路：

套路 1：plan → 审核 → execute

```
你: /plan
    我要给 @项目/ 加一个批量处理功能，先出方案。
Claude: (给计划)

你: (审计划，OK)
    退出 plan 模式，按方案走。
Claude: (执行)
```

适合：任何“动手前应该想想”的任务。

套路 2：Opus 规划 + Sonnet 执行

```
你: /model → Opus
    深度分析 @this_project/, 规划重构策略。
Opus: (给深度方案)
```


你：/model → Sonnet
 按刚才方案一步步做，先改 src/foo.py。
 Sonnet：(执行)

适合：复杂任务，想省 Opus 钱又想要深度思考。

套路 3：中段 compact 保持思路

(聊了 10 轮，Ctx 65%)
 你：/compact
 Claude：(压缩成摘要)

 (Ctx 降到 15%，继续)
 你：继续刚才的任务

适合：任务还要继续、但不想丢前面的共识。

套路 4：rewind 而不是补救

(Claude 改错了一个文件)

✘ 差做法：
 你：不对，你改多了，把 X 和 Y 改回来
 (Claude 再改一轮，可能又多错一个)

✔ 好做法：
 你：/rewind
 (直接回到出错前)
 你：换种方式：只改 X 的 foo 函数，不动其他

适合：任何 Claude 动作出错后的回退——比让它自己“修复”更干净。

18.5 ! 和 @ —— 严格来说不是斜杠命令，但属于本章
 这两个前缀不以 / 开头，但同样是 Claude Code 的“特殊输入”——效率关键。

! 快速跑 shell 命令

在 prompt 里以 ! 开头的那一行，Claude Code 会直接跑为终端命令，输出给 Claude 看。

最常用场景：

- `!ls` 看当前文件夹
- `!git status` 看 git 状态
- `!pwd` 看当前工作目录

和“让 Claude 跑命令”的区别：

- `!command`：你主动让它执行，绕过 Claude 判断，直接跑
- 不加 `!` 说“请跑 X 命令”：Claude 会在权限模式里判断是否安全再决定是否执行

什么时候用 `!`：

- 明确知道命令安全
- 快速补一句上下文（“这是我的 git status 给你看”）

@ 引用文件

这是 Ch 5 的主角——`@文件路径` 让 Claude 读这个文件。

小提示：

- `@` 之后按 Tab 会自动补全——不必手打完整路径
- 支持多个 `@`： `@a.md @b.md` 对比这两份

18.6 自定义命令的一个预告

除了系统内置命令，你也可以自己定义 slash 命令——例如 `/lint` 帮你跑代码检查、`/review` 帮你走一遍 review 流程。

具体怎么建第 21 章讲。本章只要知道：自定义命令和内置命令一样用，都以 `/` 开头。

本章小结

- 斜杠命令 = 对 Claude Code 这个工具的指令（不是给大模型的 prompt）
- 12 个核心命令按 5 组分：上下文管理 / 安全网 / 模型配置 / 项目记忆 / 会话管理
- 组合套路威力最大：plan→execute / Opus 规划→Sonnet 执行 / 中段 compact / 用 rewind 而不是补救
- `!` 跑 shell 命令、`@` 引用文件——最常用的两个“特殊前缀”
- 记不清时跑 `/help` —— 永远能查全

动手任务

任务 1：把 12 个命令都跑一遍（15 分钟）

步骤：

1. 启动 Claude Code
2. 按 18.2 表格每个命令都跑一次，观察效果（`/init` 跑完删掉生成的 `CLAUDE.md`，别保留）
3. 每个命令写 1 行自己的备注：这个干什么、我什么时候会用

成功标志：12 行备注——你对工具箱有了肌肉记忆。

任务 2：完整走一次“套路 1”（10 分钟）

步骤：

1. 选一个真实想做的小任务（5-10 分钟能完成的）
2. 执行：`/plan` → 描述任务 → 让 Claude 出方案 → 审 → 退出 plan → 让它执行 → 审 diff → Yes
3. 全程留意：你多感觉踏实了？

成功标志：你体验了 plan-first 工作流和之前“直接动手”的区别。

任务 3：贴一张“命令速查卡”（5 分钟）

把 18.3 的 ASCII 图（或你自己整理的版本）复制到你的日常笔记 / 显示器边的便签。

成功标志：眼睛扫一下就能找到对应情境的命令。

如果你卡住了

症状 A：斜杠命令敲不出来 / 没提示

- 原因：有的终端里 `/` 需要特殊处理；或你在提示符之外打字。
- 解决：确认在 Claude Code 的输入框内（不是在外面的终端）；逐字符敲，别粘贴。

症状 B：某个命令找不到

- 原因：Claude Code 版本不同，个别命令可能名字变了。
- 解决：先跑 `/help` 看当前版本全量列表；对不上就按 `/help` 给的为准。

症状 C：/rewind 没有“我想回到点”

- 原因：/rewind 的粒度是按消息，不是按字符；回得太远可能丢别的共识。
- 解决：把 /rewind 和 /clear 组合用——大问题先 /clear；小问题用 /rewind。

主线结束。下面支线讲“配置文件与个性化命令别名”。

 【支线】—— 可选深入（学有余力再看）

支线 18.A：自定义“常用套路”为一键命令

这段讲什么：如果你发现自己反复用同一种“套路”（比如 plan 开头 + 特定提示词），可以把它做成自定义命令。什么时候回头读：某个组合你用了 10 次以上。

一个简单例子

你发现自己经常这样开启一个任务：

```
/plan
需求：...
请严格遵守：
- 不改 src/legacy/
- 要用 pytest 做测试
- 最后给出 diff 摘要
```

这段开场白每次都一样——做成命令，以后只要打：

```
/start-task
```

就自动插入这段模板，你只要填需求。

怎么建

(详见第 21 章) 简要：在 `~/.claude/commands/` 或 `.claude/commands/`（项目级）里放一个 markdown 文件 `start-task.md`，内容就是你的套路模板。

哪些套路值得做成命令

- 你反复写一样的开头
- 启动任务前的检查清单
- 完成任务后的收尾（清理、commit、记录）
- 团队里大家都用的工作流

和 Skill 的区别

- Skill：给 Claude 的指令——“遇到 X 类任务用 Y 方法”；Claude 判断调用
- Custom command：给你自己的快捷键——主动敲触发

两者可以组合：命令触发、命令内容里调用 Skill。

下一章：第 19 章“输入技巧”——！和 @ 之外的效率技巧：截图输入、多行输入、历史、粘贴大段文字的正确方式。把输入效率榨干。

第 19 章：输入技巧——把效率榨干

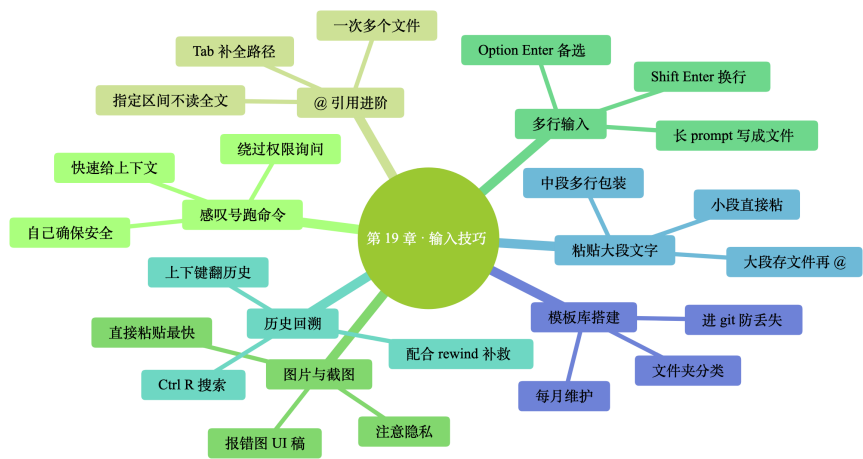
本章在全书的位置：第四部分 · 第 19 章 / 预估阅读+动手时长：主线 20-25 分钟 + 支线 8 分钟

前置章节：Ch 5 (@ 引用)、Ch 18 (命令速查)

学完能做什么 (主线)：掌握 6 个让输入速度翻倍的技巧——！快跑命令、@ 精准引用、图片/截图输入、多行输入、历史回溯、粘贴大段文字的正确方式。

开场三问

- 你会遇到什么问题：知道该说什么，但敲字打断思路——一来一回翻文件找路径、拼长段提示词、粘贴时排版乱掉。
- 不读这章会踩什么坑：效率瓶颈在输入而不在 Claude——你慢它也等着。
- 读完你会多会什么事：输入变成自动化动作，注意力全在“想什么”上。



 【主线】—— 本章必读核心

19.1 ! 快跑 shell 命令（已在 Ch 18 提过，这里说透）

用法

提示符处直接 **!** 开头敲命令：

```
!git status
```

效果：Claude Code 跑这个命令，输出进上下文让 Claude 看到。

为什么值得单独用

- 提供上下文：“先给 Claude 看 git 状态再讨论怎么 commit”——比你口述更精准
- 避免中断流程：不用切到另一个终端跑了再描述给 Claude

常用的 ! 命令

命令	干什么
<code>!ls / !ls -la</code>	看当前文件夹
<code>!pwd</code>	看当前在哪
<code>!git status / !git log -n 5</code>	看 git 状态 / 最近 5 次提交
<code>!cat 文件名</code>	把文件内容贴给 Claude（小文件可以这样，大文件用 @）
<code>!date</code>	让 Claude 知道“今天是几月几号”

和 Claude 主动跑命令的区别

- `!command`：你主动跑，绕过权限询问（但仍在 Claude Code 的沙箱里）
 - Claude 要跑命令：它会先说“我要跑 `x`，同意吗？”——走权限弹窗
- 用 **!** 的前提：这个命令你百分之百确定安全。

19.2 @ 精准引用——进阶用法

基础用法 Ch 5 讲过。三个进阶技巧：

技巧 1: Tab 自动补全

打 `@` 后开始打路径，按 `Tab`：

- 自动补全文件名
- 如果路径有多个匹配，再按 `Tab` 会轮流显示

省时：不用手打长路径、不会拼错文件名。

技巧 2: 引用多个文件

`@a.md @b.md` 对比这两份笔记的相同点和差异

Claude 会一次性读取多个。

注意：`@` 的文件越多 / 越大，Ctx 消耗越快（见 Ch 12）。一次 3-5 个是舒适区。

技巧 3: 引用文件的特定段（更精准）

不要直接 `@巨型文件.md`——先自己定位到关心的区间：

`@长文档.md 第 3 节"数据流"那段大概 100-150 行左右，帮我改得更简洁`

Claude 会直接定位到那段，不会把几万字全读进来。

（更准一点的话：有些版本支持 `@文件#锚点` 或 `@文件:50-80` 语法——查 `/help` 看你的版本是否支持。）

19.3 图片 / 截图输入

Claude 能看图——这是很多人不知道的利器。

怎么输入图片

方式 1: 直接粘贴（最快）

- 系统截图（Mac `Cmd+Shift+4`、Windows `Win+Shift+S`）
- 直接 `Cmd+V` / `Ctrl+V` 粘贴到 Claude Code 输入框
- Claude Code 会显示 `[image 1]` 之类的占位
- 发出去，Claude 看到图

方式 2: `@` 引用图片文件

`@~/Desktop/截图.png` 帮我看这张图里的错误信息

什么场景该用图

- 报错截图：你看不懂某个报错窗口——截图比打字描述准
- UI 设计稿：让 Claude 按稿子写代码
- 表格照片：把纸上的表格录进电脑
- 图表：让 Claude 解读数据图
- 手写笔记：转成电子版

注意事项

- 图片也占上下文（且占不少）——不要一次塞十张图
- 不要传含个人隐私的图（身份证、聊天截图等）——回 Ch 11 复习红线
- 图片识别有误读风险：手写字 / 模糊文字容易错——关键信息自己核对

一个真实场景

你：（截图了一个 Python 报错）

这个错是什么意思？怎么修？

Claude：（识别图里的 stack trace）

这是一个 `AttributeError`，在 line 42 的 ...

可能原因：...

修复方案：...

比你打字复述报错信息快 10 倍。

19.4 多行输入——长提示词的正确姿势

问题

复杂任务需要多行提示词：

我要做 XXX

要求：

1. ...

2. ...

3. ...

限制：...

默认按 Enter 就发了——你只能打到第一行就被迫发出。

解决

Shift+Enter（大部分终端）——换行不发送。

完整流程：

1. 敲第一行
2. Shift+Enter 换行
3. 敲第二行
4. Shift+Enter 换行
5. … 都敲完
6. Enter 发送

替代方式

某些终端不支持 Shift+Enter，试：

- Option+Enter (Mac)
- Alt+Enter (Windows / Linux)
- 或使用粘贴——在别处写完多行文字，整体粘进来

超长提示词：用文件

如果你要发一个几百字的复杂提示，最佳做法：

1. 在文本编辑器里把提示词写成一个 `.md` 文件
2. Claude Code 里 @prompt.md
3. 后面加一句“按上面这份说明做”

为什么这样更好：

- 你可以反复调整 prompt 文件，不怕每次重敲
- 可以复用——下次类似任务直接 @ 同一个文件
- 多人协作时能共享 prompt 模板

19.5 历史回溯

上 / 下键翻历史

在输入框里按 ↑：逐条翻看之前发过的消息。

什么时候用：

- 刚才发的那句话想改一下重发——按 ↑ 取回来改
- 翻“几轮前我具体说了什么”

- 复用之前的模板化提示

搜索历史

某些终端支持 **Ctrl+R** 搜索——打几个关键词，找到历史消息。

注意：历史只在当次会话内有效。 `/clear` 或退出后清空。

粘贴失败时的历史回溯

一种常见场景：

- 你粘贴了一大段文字
- Enter 发送时发现其中有一处错
- `/rewind` 回到发送前
- 按 `↑` 取回那段文字
- 改好后重发

比“从头写一遍”快很多。

19.6 粘贴大段文字的正确姿势

问题

你从网页 / 文档里复制一段话，粘进 Claude Code。结果：

- 格式乱了（换行错位 / 特殊字符）
- 太长，直接爆上下文
- 粘贴过程卡顿

解决方案

方案 A：小段（< 10 行）直接粘

- 正常 `Cmd+V` / `Ctrl+V`
- 发送前检查一下排版

方案 B：中段（10-100 行）用多行输入

- 先打一句引子：“以下是我要讨论的段落：”
- `Shift+Enter` 换行
- 粘贴
- Enter 发送

方案 C：大段（> 100 行 / 很长的文档）保存为文件再 @

- 新建 `~/Desktop/待处理.md`
- 把内容保存进去
- Claude Code 里 `@~/Desktop/待处理.md`

大段文字的两个陷阱

1. 意外的 Enter：粘贴时某些行结尾是 Enter，可能触发发送——先粘到单独的文本编辑器整理再二次粘到 Claude Code
2. 隐藏格式：从网页复制的文字可能带格式标记 (HTML)，Claude 会读到“干扰文本”——粘进 Markdown 文件先“洗”一遍，再 `@`

本章小结

- `!command` 快跑 shell 命令（你确定安全时），输出给 Claude 当上下文
- `@` 三进阶：Tab 补全 / 多文件 / 指定区间
- 图片 / 截图直接粘贴——报错、UI 稿、图表场景效率爆炸
- 多行输入用 `Shift+Enter` / `Option+Enter`；超长 prompt 用文件 `@`
- `↑` / `↓` 翻历史；大段粘贴的三档方案（`< 10` / `10-100` / `100+`）

动手任务

任务 1：把 6 个技巧每个试一次（15 分钟）

步骤：

1. 启动 Claude Code
2. `!ls` 跑一次 shell 命令
3. `@` + Tab 补全一个文件名
4. 截图一张图（任意）粘贴给 Claude，问它看到什么
5. `Shift+Enter` 发一条三行的消息
6. `↑` 翻历史调出刚才的消息
7. 粘贴一段 20 行左右的 Markdown

成功标志：6 个动作都能顺利完成——下次无需思考就能用。

任务 2：用“文件 @”代替长 prompt（15 分钟）

步骤：

1. 想一件你反复做的任务（如“给我的日报文字润色”）
 2. 把“完整要求 + 风格说明 + 禁忌”写成
~/Desktop/ccguide-练习/日报润色-prompt.md，至少 10 行
 3. 下次要做日报润色时：`@日报润色-prompt.md @今天的日报.md` 按模板走
- 成功标志：你有了第一份可复用 prompt 模板——以后类似任务一秒启动。

任务 3：截图流（5 分钟）

步骤：

1. 找一个你看不懂的报错窗口 / 一张数据图 / 一张别人的 UI 截图
2. 截图粘到 Claude Code 问它解读
3. 评估回答质量

成功标志：你意识到“打字描述错误永远不如截图给 Claude 看”。

如果你卡住了

症状 A：Shift+Enter 按下直接发送了

- 原因：某些终端仅认 Option+Enter 或 Alt+Enter。
- 解决：都试一下；或用“先写文件再 @”代替长提示词输入。

症状 B：粘贴图片没反应

- 原因：某些终端不支持图片粘贴。
- 解决：保存图片到本地文件夹再 @ 路径，效果一样。

症状 C：粘贴大段文字后 Claude 回复乱码 / 看不懂

- 原因：文字里有隐藏格式 / 特殊字符。
- 解决：先粘到 VS Code / 记事本等纯文本编辑器，保存成 .md 文件，再 @ 进来。

主线结束。下面支线讲“多光标、自动补全的高级用法”和“键盘快捷键收藏单”。

🌱 【支线】—— 可选深入（学有余力再看）

 支线 19.A：键盘快捷键收藏单

这段讲什么：Claude Code 里不只 @、!，还有一堆编辑快捷键。什么时候回头读：你用了一段时间后想提效时。

光标移动

键	效果
Ctrl+A	光标跳到行首
Ctrl+E	光标跳到行尾
Option+← / Option+→	跳一个词 (Mac)
Alt+← / Alt+→	跳一个词 (Windows)

删除

键	效果
Ctrl+U	删除从光标到行首的内容
Ctrl+K	删除从光标到行尾
Option+Delete	删一个词 (Mac)
Ctrl+Backspace	删一个词 (Windows)

会话控制

键	效果
Ctrl+C	中断当前运行 (Claude 思考中按它会停止)
Ctrl+C Ctrl+C (两下)	退出 Claude Code
Esc Esc	快速撤销 (见 Ch 9)

把这张表打印出来贴桌面

- 不用背——忘了就翻
- 用多了自然肌肉记忆
- 一周之后基本都会用

支线 19.B：输入模板库的搭建

这段讲什么：任务 2 只建了一个模板，展开讲如何系统化建“prompt 库”。什么时候回头读：你开始在多种任务上反复用同类提示时。

文件夹结构建议

```
~/Desktop/prompts/  
├── 写作/  
│   ├── 日报润色.md  
│   ├── 周报生成.md  
│   └── 邮件草稿.md  
├── 整理/  
│   ├── 截图重命名.md  
│   └── 笔记归档.md  
├── 编程/（如果你会点）  
│   ├── 代码审查.md  
│   └── 写测试.md  
└── README.md    # 一张目录索引
```

每个模板文件的格式

任务：日报润色

输入

我会给你一份原始日报（纯流水账）。

输出要求

- 保留所有真实事项
- 合并相近事项（同一天两次“改文档”合并）
- 每项 30 字以内
- 顺序按重要性，不按时间

禁忌

- 不要编造事项
- 不要加 emoji

风格示例

- ✕ "今天早上 9 点开会讨论了项目进度"
- ✔ "项目会议：Q2 目标确认"

怎么用

@prompts/写作/日报润色.md @今天的日报.md 按模板走

维护节奏

- 每次做完任务回头看：模板能不能更好？更新
- 每月扫一遍：哪些没用到的删掉
- 存进 git（或云盘同步）：防止丢失

这一套建立起来之后，日常任务的提示词成本降到接近零——你花 2 分钟填具体内容，模板把其余都包了。

第四部分（避坑与判断力）完成。

你现在具备的能力：

- 10 大典型错误的免疫力（Ch 16）
- 知道什么时候该停、什么时候坚持（Ch 17）
- 12 个核心斜杠命令 + 组合套路（Ch 18）
- 6 个输入效率技巧 + 模板库（Ch 19）

下一部分（第五部分：扩展能力）开始讲 Skills / Slash Commands / Subagents / Hooks / MCP 这 5 种扩展 Claude Code 的方式——每种用一章，结构统一：是什么 → 最简单的一个例子 → 什么时候值得用 → 进阶去哪里学。

第五部分 · 扩展能力

第 20 章：Skills 入门——可复用的操作手册

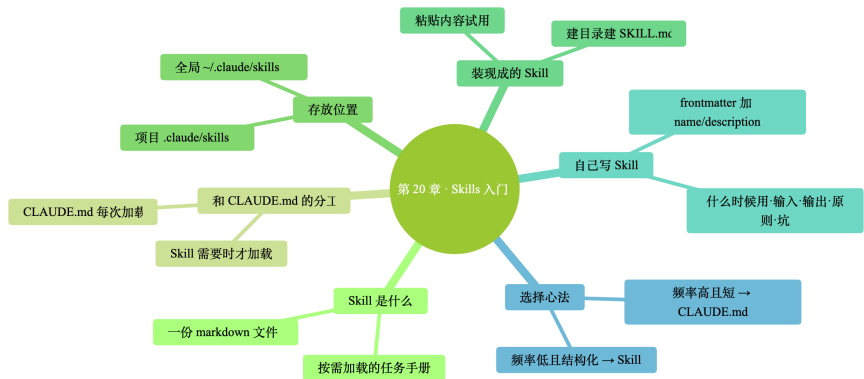
本章在全书的位置：第五部分 · 第 20 章 / 预估阅读+动手时长：主线 25-30 分钟 + 支线 10 分钟

前置章节：Ch 13（三层记忆）+ Ch 14（CLAUDE.md）

学完能做什么（主线）：理解 Skills 是什么、它和 CLAUDE.md 有什么分工；会装一个现成的 Skill；能写出你第一个最简 Skill（一份 markdown 文件）。

开场三问

- 你会遇到什么问题：有些任务你做得够多了，每次都要给 Claude 讲同一套流程——烦。
- 不读这章会踩什么坑：把这些“偶尔用但要讲一大通”的流程塞进 CLAUDE.md，导致 CLAUDE.md 膨胀。
- 读完你会多会什么事：会写第一个 Skill（把常用流程打包），Claude 只在需要时才加载——省 Ctx、清爽。



🎯 【主线】—— 本章必读核心

20.1 Skill 是什么

Skill = 一份预先写好的、针对特定任务的操作手册，以 Markdown 文件的形式存在，Claude 在需要时自动加载进上下文，用完丢弃。

一句话定义

🔑 Skill = 按需加载的任务手册。

和 CLAUDE.md 的关键区别（复习）

CLAUDE.md		Skill
加载	每次启动加载	按需加载
内容	全局规则、硬约束	特定任务的手册
适合什么	“这个项目永远这样”	“偶尔做 X 任务时这么做”

一个简单例子

假设你每月做一次“项目月报”。流程是：

1. 汇总本月所有提交
2. 整理关键成就 3-5 条
3. 列待办
4. 按固定模板排版

这个流程你一个月做一次——放 CLAUDE.md 太浪费（占 Ctx 却 99% 时间用不到）。

改成一个 Skill `monthly-report`：

- 平时不占 Ctx
- 你说“做月报”或 Claude 识别到类似需求时，它才加载进来
- 加载之后 Claude 按手册步骤做

Skills 能放什么

- 结构化的任务流程: 月报、代码审查、发版流程
- 有固定模板的工作: 邮件回复、会议纪要、周报
- 多步推理的任务: 从数据到报告的完整管线

Skills 不适合放什么

- 所有任务都适用的通用规则 → 放 CLAUDE.md
- 一次性任务 → 直接在 prompt 里讲就行
- 太复杂、步骤上百 → 拆成多个 Skill

20.2 Skills 存在哪里

两种位置 (和 CLAUDE.md 类似):

全局 Skills (只有你能用)

```
~/.claude/skills/<skill-name>/
```

项目 Skills (团队共享, 进 git)

```
<项目根>/.claude/skills/<skill-name>/
```

一个 Skill 的最简文件结构

```
~/.claude/skills/monthly-report/  
└─ SKILL.md          # 就一个文件, 其他都可选
```

一个 Skill 的最小构成 = 一个 **SKILL.md** ——就是这么简单。

20.3 第一个 Skill: 装一个现成的 (15 分钟)

先装别人写好的——比从零写省事。

第一步: 建目录

打开终端, 跑:

```
mkdir -p ~/.claude/skills
```

(只建一次, 之后所有 Skills 都放这下面)

第二步: 新建 Skill 目录

我们做一个“会议纪要整理”Skill。

```
mkdir -p ~/.claude/skills/meeting-notes
```

第三步: 建 SKILL.md

```
open -e ~/.claude/skills/meeting-notes/SKILL.md
```

(Windows 上
用 `notepad %USERPROFILE%\\.claude\skills\meeting-notes\SKILL.md`)
粘贴下面的内容:

```
---
name: meeting-notes
description: 把凌乱的会议录音转写 / 笔记整理成结构化会议纪要
---

# 会议纪要整理

## 什么时候用

用户给你一段会议记录（语音转写、实时笔记、碎片发言），要把它变成结构化纪要。

## 输入

用户通常会给你一段凌乱的文字——可能是：
- 讯飞 / 飞书会议的录音转写
- 自己边听边记的笔记
- 多人轮流发言的原文

## 输出格式（固定）

```markdown
会议纪要: <会议主题>

- **时间**:<日期>
```

- \*\*与会\*\*：<名字列表>
- \*\*主持\*\*：<名字>

### ## 关键决定

1. ...
2. ...

### ## 讨论要点

- 要点 1: ...
- 要点 2: ...

### ## 行动项

事项	负责人	截止
...	...	...

### ## 悬而未决

- ...

## 原则

- 忠于原文——不要编造没提到的决定
- 不确定的细节（比如截止日期没说）就标 <待确认>
- 参会名单如果原文没列清楚，只写“与会多人”，不要乱填
- 按“决定 > 讨论 > 行动 > 悬而未决”排序

## 常见坑

- 原文里有闲聊 / 跑题——剔除
- 同一件事多人重复讲——合并
- 一个人说“我觉得应该 X”但没达成共识——放“讨论要点”，不放“关键决定”

保存。

### 第四步：试一试

在 Claude Code 里打开一个对话，找一段会议原始记录（随便从邮件、聊天记录拼一段假的也行），说：

你：用 meeting-notes 技能整理下面的会议记录：  
（粘贴原始文字）

或者不指定技能，直接描述任务：

你：我有一段会议原始记录要整理成纪要，整理成结构化的样子。（粘贴）

Claude 会自动判断需要用 `meeting-notes` Skill，加载进来，按手册做。

**\*\*成功标志\*\***：输出严格按 SKILL.md 里的模板格式。

## 20.4 自己写一个 Skill (15 分钟)

写 Skill 没什么神秘的——\*\*就是一份 markdown 说明书\*\*。

### 步骤

1. **\*\*选一个你反复做的任务\*\***（周报、邮件回复、代码审查、截图重命名...）
2. 想清楚这个任务的"标准流程"— 输入是什么、输出什么样、有哪些规矩
3. 新建 `~/ .claude/skills/<名字>/SKILL.md`
4. 按下面结构写

### 推荐结构

```
```markdown
```

```
---
```

```
name: <skill-name>    # 英文、连字符连接, Claude 引用时用
```

```
description: <一句话说说它干什么>    # 这句是 Claude 判断"要不要用这个技能"的依据
```

```
---
```

```
# <标题, 人类可读>
```

```

## 什么时候用
(什么场景触发这个 Skill — 越具体越好, Claude 靠这个决定加载不加载)

## 输入
(用户会给你什么形式的原料)

## 输出格式
(你期望的最终产出是什么样—给模板)

## 原则
(不能违反的铁律)

## 常见坑
(历史上出过哪些错、怎么避免)

```

一个实例：周报生成

```

---
name: weekly-report
description: 把本周流水账整理成老板喜欢的精简周报
---

# 周报生成

## 什么时候用
用户说"整理周报"、"写周报"、或丢给你一堆本周做的事让你汇总。

## 输入
- 一堆碎事项（多半按时间顺序）
- 有些是已完成、有些在做

## 输出格式

```markdown
<姓名> 周报 (<日期范围>)

本周亮点 (1-3 条)

```



```
- ...

已完成
- 事项 1 - 一句话结果
- ...

进行中
- ... (标进度百分比)

下周计划
- ...

需要协助
- ... (没有就写"无")
```

## 原则

- 亮点永远是“有显著成果的事”，不是“我很努力做了 XX”
- 单项不超过 30 字，老板扫读友好
- “已完成”要有结果（“客户接入 3 家”），不能只写“做了对接”
- 不编造没做的事

## 常见坑

- 把所有做过的事都写成“亮点”——一条都不突出
- 用日期串起来的流水账 —— 按“已完成 / 进行中”分类更清晰

写完保存，跑一次看看效果。

### Claude 怎么知道什么时候用这个 Skill

两条路径：

1. \*\*用户明说\*\*： "用 weekly-report Skill"
2. \*\*Claude 自己判断\*\*： 看 `description` 字段 + `## 什么时候用` 描述，觉得匹配就加载

**\*\*关键\*\*:** `description` 要写准——这是 Claude 做决定的主要依据。

## ## 20.5 Skill vs CLAUDE.md 的选择心法

遇到一条"我想让 Claude 记住的流程", 按这张表判断:

情境	放 CLAUDE.md	写 Skill
-----	-----	-----
每次对话都相关	✓	
一个月才用一次		✓
简短硬规则 (1-2 行)	✓	
结构化流程 (多步骤、带模板)		✓
所有项目都适用	✓ (全局 CLAUDE.md)	✓ (全局 Skills)
步骤可能超过 50 行	✗ (太大)	✓

**\*\*核心判断\*\*:** \*\*频率高 + 短 → CLAUDE.md; 频率低 + 长 → Skill\*\*。

---

## ## 本章小结

- Skill = 按需加载的任务手册, 一份 markdown 文件
- 放在 `~/.claude/skills/<名>/SKILL.md` (全局) 或 `~/.claude/skills/<名>/SKILL.md` (项目)
- 核心字段: `name` + `description` + 「什么时候用 / 输入 / 输出 / 原则 / 常见坑」
- \*\*频率高 + 短 → CLAUDE.md\*\*;
- \*\*频率低 + 结构化 → Skill\*\*
- description 字段是 Claude 判断"要不要用"的主要依据——写准它

---

## ## 动手任务

### 任务 1: 装 meeting-notes Skill (10 分钟)

**\*\*步骤\*\***: 照 20.3 完整走一遍——建目录、建文件、粘贴内容、试一段假会议记录。

**\*\*成功标志\*\***: Claude 按模板整理出纪要, 结构和 SKILL.md 里定义的一致。

### 任务 2: 写你自己的第一个 Skill (20 分钟)

**\*\*步骤\*\***:

1. 选一个你反复做的任务 (从本书给的例子里挑一个: 周报、邮件、截图整理、笔记归档)
2. 照 20.4 的结构写
3. 放到 `~/ .claude/skills/<名>/SKILL.md`
4. 做一次真实任务, 用这个 Skill

**\*\*成功标志\*\***: 你有了第一个**\*\*自己\*\***的 Skill, 下次同类任务能一句话调用。

### 任务 3: 对照决策 (5 分钟)

看自己当前的 CLAUDE.md, 有没有内容其实更适合做 Skill?

典型信号:

- 一整段操作流程 (> 10 行)
- 用到不到每周一次的
- 带模板 / 输出格式的

把这类**\*\*搬出** CLAUDE.md 做成 Skill**\*\***。

**\*\*成功标志\*\***: CLAUDE.md 瘦下来 10 行以上。

---

## 如果你卡住了

**\*\*症状 A**: Claude 没自动识别 Skill, 每次都要我明说**\*\***

- 原因: description 太模糊, Claude 判断不出匹配。
- 解决: description 要**\*\*说人话 + 关键词\*\***——"把会议记录整理成纪

要"比"会议相关的工具"好用。

**\*\*症状 B: SKILL.md 里写的东西 Claude 没执行到位\*\***

- 原因: 规矩写得太软 ("尽量"、"建议"), Claude 理解有弹性。
- 解决: 改成**\*\*明确指令\*\***—"必须按这个模板"、"不得编造未提到的决定"。

**\*\*症状 C: 不知道 Skill 有没有真的被加载\*\***

- 原因: Claude Code 某些版本不显式告诉你。
- 解决: **\*\*看产出格式\*\***—如果输出严格符合 SKILL.md 里定义的结构, 说明加载了; 完全不像就是没加载。

---

主线结束。下面支线讲"Skill 的进阶元素"和"怎么分享 / 共享 Skill"。

---

> 🌿 **\*\*【支线】— 可选深入 (学有余力再看) \*\***

---

**## 🌿 支线 20.A: Skill 的进阶结构**

> **\*\*这段讲什么\*\***: SKILL.md 之外还能放什么。

> **\*\*什么时候回头读\*\***: 你写了 3-5 个 Skill 后想把某个做大时。

**### 多文件 Skill**

复杂 Skill 可以有多个文件:

```
~/claude/skills/code-review/ |—— SKILL.md # 入口 / 总览 |——
checklist.md # 审查清单 |—— common-bugs.md # 常见 bug 库 |——
examples/ # 示例 |—— good-review.md
```

SKILL.md 里**\*\*引用其他文件\*\***:

```
```markdown
## 审查流程

详细清单见 [checklist.md](./checklist.md)。
常见 bug 参考见 [common-bugs.md](./common-bugs.md)。
```

Claude 加载 SKILL.md 时会注意这些指针，需要时去读相关子文件。

给 Skill 加元数据

frontmatter 里可以加更多字段：

```
---
name: code-review
description: 代码审查助手
version: 0.3
author: "我"
tags: [review, quality]
---
```

这些元数据 Claude 也能看到，对它判断“这个 Skill 是不是最新版”有帮助。

Skill 里调用其他工具

SKILL.md 里可以明确指导 Claude 调用特定工具：

```
## 步骤
1. 用 `!git diff main` 获取变更
2. 用 @ 引用被修改的所有文件
3. ...
```

Claude 会照着做——相当于把操作流程固定住。

支线 20.B: Skill 共享 / 社区

这段讲什么：你的 Skill 可以分享给团队 / 公开。什么时候回头读：你写了好的 Skill 想让别人用时。

团队共享

把 Skill 放项目级目录: `<项目>/Claude/skills/<名>/`

进 git, 所有人 pull 下来就生效。

公开分享

- 发到 GitHub 单独仓库
- 别人 clone / 下载, 放进自己的 `~/.Claude/skills/` 即可

正在兴起的 Skill 社区

Anthropic 有官方 skill 目录、社区也在建立第三方集。搜 “claude-code skills” 或 “awesome claude skills”, 找到合适的 fork 一份到本地用。

共享时的注意

- 不要在 Skill 里硬编码你的路径 (`/Users/yourname/...`) ——用相对路径或占位符
- 不要在 Skill 里放密钥或个人信息
- 写清楚 Skill 需要的前提条件 (依赖哪些工具、哪些 CLAUDE.md 配置)

下一章: 第 21 章“自定义 Slash 命令”——Skill 是 Claude 的手册, 自定义 Slash 命令是给你自己的快捷键。两者怎么分工? 下一章讲。

第 21 章：自定义 Slash 命令——给你自己的快捷键

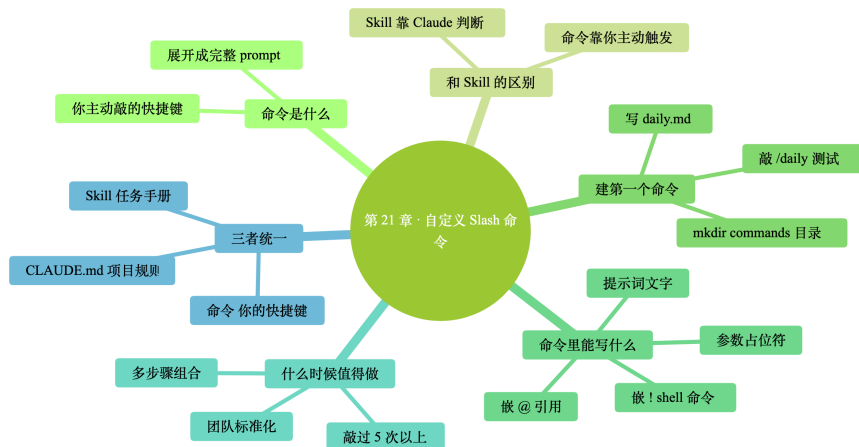
本章在全书的位置：第五部分 · 第 21 章 / 预估阅读+动手时长：主线 20-25 分钟 + 支线 8 分钟

前置章节：Ch 18（常用斜杠命令）+ Ch 20（Skills）

学完能做什么（主线）：理解自定义 slash 命令和 Skills 的分工；建出你第一个 `/xxx` 命令（10 行以内）；知道什么时候做成命令、什么时候做成 Skill。

开场三问

- 你会遇到什么问题：每次开启任务都要敲同样一长段 prompt——“请严格遵守：A、B、C，输出格式 D”。累。
- 不读这章会踩什么坑：明明可以一键触发的动作，还是每次手工复制粘贴 prompt。
- 读完你会多会什么事：把反复用的 prompt 变成 `/xxx` 一个命令——敲命令名 = 自动展开完整 prompt。



🎯 【主线】—— 本章必读核心

21.1 自定义 Slash 命令是什么

Ch 18 讲的 `/clear`、`/compact`、`/status` 等是 Claude Code 内置命令。你也可以自己建——以 `/` 开头，以你的自定义名字结尾：

- `/daily` — 自动生成一条今日开工清单
- `/review` — 启动代码审查模板
- `/ship` — 按你的发布流程一步步走
- `/focus` — 切换到“不打扰”模式（关掉某些提示）

一句话定义

🔑 自定义 Slash 命令 = 你给自己起的、以斜杠开头的、一敲就自动展开成完整 prompt（或触发特定动作）的快捷键。

和 Skills 的本质区别（重要）

维度	Skill	Custom Slash Command
触发方式	Claude 自动判断（看到符合场景就加载）	你主动敲命令
本质	给 Claude 的任务手册	给你的 prompt 快捷键
存在形式	SKILL.md + 相关文件	一个简短的 markdown
负责“做什么决定”的是谁	Claude	你

最简单的类比：

- Skill = Claude 的工具箱（它自己决定啥时候拿工具）
- Custom command = 你的键盘宏（你按键就触发）

两者也可以配合——命令触发后 prompt 里引用 Skill，最佳实践。

21.2 建你的第一个命令（10 分钟）

我们建一个 `/daily` 命令——“今天开工自动列清单”。

步骤 1：建目录

全局命令（所有项目可用）：

```
mkdir -p ~/.claude/commands
```

项目命令（只在这个项目可用）：

```
mkdir -p .claude/commands
```

新手先用全局。

步骤 2：建命令文件

文件名 = 命令名。做一个 `/daily` 命令就建 `daily.md`：

```
open -e ~/.claude/commands/daily.md
```

(Windows: `notepad %USERPROFILE%\.claude\commands\daily.md`)

步骤 3：写内容

粘贴：

```
---
```

```
description: 今日开工清单——列出需要做的事、优先级、大概时间预算
```

```
---
```

现在帮我列今天的开工清单。按以下步骤：

1. 先跑 `!git log --since="24 hours ago" --oneline` 看我昨天做到哪了
2. 跑 `!ls ~/Desktop/` 看桌面上有没有未归档的文件
3. 基于上面两步，问我 2-3 个问题帮你搞清楚今日重点
4. 最后输出一份今日清单：

```
```markdown
```

```
<日期> 清单
```

```
必须完成 (P0)
```

```
- [] ... (30-60 分钟)
```

```
建议完成 (P1)
```

```
- [] ... (15-30 分钟)
```

```
可以推迟 (P2)
```

```
- [] ...
```

原则：

- 每项带时间预算
- 不超过 5 项
- 严格按优先级排序

保存。

```
步骤 4：试一试
```

启动 Claude Code，在输入框敲：

/daily

**\*\*按回车\*\***—命令里的内容会自动传给 Claude，它就开始执行。

**\*\*成功标志\*\***：Claude 按步骤 1-4 做，最后出清单。

### 如果没识别

- 确认文件路径对：`~/ .claude/commands/daily.md`
- 确认文件以`.md`结尾
- 重启一次 Claude Code（某些版本要重启才扫描新命令）

## 21.3 一个命令里能干什么

自定义命令**\*\*本质上是一段 prompt 模板\*\***。里面可以：

### 能力 1：直接写文字提示词

```markdown

帮我把最后一条 commit 的变更摘要为 1 句话，用于 Slack 公告。

能力 2：嵌入 shell 命令

用 `!command` 让 Claude 跑命令（和 Ch 19 讲的一样）：

先跑 `!git status` 看当前状态，然后.....

能力 3：嵌入 @ 引用

读 `@CLAUDE.md` 和 `@docs/release.md`，按其中规则走下面步骤.....

能力 4：带参数（进阶）

命令支持占位符：

description: 给指定文件生成测试

为 ``{{1}}`` 生成一组单元测试。要求.....

使用时：

```
/gentests src/utils.py
```

`{{1}}` 会替换成 `src/utils.py`。

不同版本语法略有差异——打 `/help` 或官方文档看你这版具体语法。

能力 5：组合 Skill

用 `meeting-notes Skill` 整理我刚贴的会议记录，要求输出按 HR 格式。

Claude 会自动加载你的 `meeting-notes Skill` + 接你的追加要求。

21.4 什么时候值得做成命令

不是什么都该做成命令。值得做的信号：

✅ 值得做成命令

- 你敲过这段 prompt 超过 5 次
- 组合了多个步骤（shell 命令 + @ 引用 + 说明）
- 团队里每个人都要用同一套流程（放项目级 `.claude/commands/`，进 git）
- 为标准化工作流——比如 `/ship` 保证每次发布都走同一流程

❌ 不值得做成命令

- 一次性 prompt——下次可能用不到
- 只写一两句话的提示词——直接敲比打命令名还快
- 参数变化很大的任务——每次都要改命令内容不如直接写 prompt

21.5 命令、Skills、CLAUDE.md 三者统一对比

三个都是“让 Claude 行为可重用”的方式，分工如下：

| 维度 | CLAUDE.md | Skill | Custom Command |
|------|-----------------------|----------------|------------------------|
| 触发 | 每次启动自动加载 | Claude 判断需要时加载 | 你主动敲 <code>/xxx</code> |
| 适合频率 | 每次对话都用得到 | 偶尔用，有标准流程 | 反复用同样 prompt |
| 内容本质 | 项目规则 + 上下文 | 任务手册 | Prompt 模板 |
| 典型例子 | “这个 项目 用 Python 3.11” | “做代码审查的完整流程” | “/daily 今日清单” |
| 维护频率 | 项目变化时改 | 流程优化时改 | 使用中 发现需要调时改 |

三者联合的典型 workflow

```

用户: /daily
  ↓ (Custom Command 展开成完整 prompt)
Claude: 读 CLAUDE.md 知道项目背景
  ↓
Claude: 识别出"需要生成清单"场景 → 加载 daily-planning Skill
  ↓
Claude: 按 Skill 手册 + CLAUDE.md 规则 + Command 里的步骤执行
  ↓
输出: 一份清单

```

三者各司其职，互不冲突。

本章小结

- 自定义 Slash 命令 = 你主动敲的 prompt 快捷键
- 放在 `~/.claude/commands/<名>.md` (全局) 或 `.claude/commands/<名>.md` (项目)
- 文件结构: frontmatter (description) + 正文 (prompt 模板, 可带 `!shell / @引用 / {{参数}}`)
- Skill vs Command: 前者 Claude 自动调用、后者你主动触发
- 值得做的信号: 敲过 5 次以上、组合多步骤、团队标准化
- 命令、Skill、CLAUDE.md 三者联合是最强组合

动手任务

任务 1：建 `/daily` 命令（10 分钟）

步骤：照 21.2 完整走一遍。改 prompt 内容适应你自己的工作习惯（例：不关心 git？就删掉那步；关心邮件？加一步 `!mail` 或提醒检查邮箱）。

成功标志：敲 `/daily` 能跑出一份对你真有用的今日清单。

任务 2：再建一个你自己的命令（15 分钟）

步骤：

1. 回顾过去一周你反复敲过的 prompt
2. 选一个最频繁的（至少敲过 3 次）
3. 做成一个命令——起个 3-7 字的名字（例 `/email-reply`、`/review`）
4. 试用一次，发现问题就改

成功标志：你有了第二个命令，真实减少敲字量。

任务 3：对比三种扩展（10 分钟）

写出你目前三层扩展的分工：

```
# 我的 Claude Code 扩展地图

## CLAUDE.md（当前项目每次都带）
- ...

## Skills（按需加载，我已安装的）
- ...

## Custom Commands（我自己的快捷键）
- /daily
- /...
```

成功标志：看一眼这份地图，能说清楚每层分别干什么。

如果你卡住了

症状 A：敲 `/daily` 没反应或说“command not found”

- 原因：(a) 文件路径错了；(b) 文件名带空格 / 特殊字符；(c) 需要重启 Claude Code。
- 解决：重建文件确认路径；文件名只用英文小写 + 连字符；关掉 Claude Code 重开。

症状 B：命令展开了但 Claude 没按步骤做

- 原因：prompt 里的指令太软（“最好”、“可以”）。
- 解决：改成硬指令（“必须先跑 X”、“不得跳过步骤 Y”）。

症状 C：我想做带参数的命令但不会

- 原因：参数语法各版本略有不同。
- 解决：先跑 `/help` 看你这版的参数语法；或先用无参数版本——prompt 里留一个“...”，Claude 会追问你具体参数。

主线结束。下面支线讲“团队共享命令的最佳实践”。

🌿 【支线】—— 可选深入（学有余力再看）

🌿 支线 21.A：团队共享命令的最佳实践

这段讲什么：项目里放一组命令，所有人 pull 下来就能用。什么时候回头读：团队有 3 人以上一起用 Claude Code 时。

放在 `.claude/commands/`（项目级）

团队命令进 git，每个人 clone 之后就有。

```
<项目根>/
├── .claude/
│   └── commands/
│       ├── ship.md      # /ship: 发版流程
│       ├── review.md   # /review: 代码审查
│       └── daily.md     # /daily: 每日早会前汇总
```

```
└─ CLAUDE.md
└─ src/...
```

三类值得团队共享的命令

1. 流程类命令（保证一致性）

例： `/ship` — 发布流程

```
---
description: 标准发布流程——保证每次发版都走同一套步骤
---
```

按以下步骤走：

1. 跑 `!git status` 确认无未提交`
2. 跑 `!npm test` 全绿`
3. 跑 `!npm run build` 成功`
4. 生成 CHANGELOG 更新
5. 创建 tag
6. 推到远程
7. 发布到 npm

每步等我确认再进下一步。

价值：防止人手忘步骤——Claude 按清单推进。

2. 审查类命令

例： `/review` — 代码审查启动

```
---
description: 审查当前改动（基于 main 分支对比）
---
```

对比 main 分支，审查当前所有改动：

1. 跑 `!git diff main``
2. 按这 5 个维度评估：
 - 逻辑正确性
 - 测试覆盖

- 边缘情况
- 命名和可读性
- 潜在性能问题

3. 输出报告

3. 模板化输出命令

例： `/release-notes` — 生成发版说明

```
---  
description: 把上次 tag 到现在的改动生成 release notes  
---
```

1. 跑 `!git log <上次tag>..HEAD``
2. 按以下格式生成：

vX.Y.Z

新增

- ...

修复

- ...

破坏性变更

- ... (如有)

3. 语言统一、用户视角、不要技术细节

共享时的约定

- 命名一致： `/review` , `/ship` , `/release-notes` 这种动作化命名
- 每个命令有 `description`——队友看 `/help` 能知道命令干啥
- 命令内容写在 `markdown`——便于 PR review
- 大改走 PR：改命令 = 改流程，要经过讨论

个人命令 vs 团队命令

- 个人命令 (`~/.claude/commands/`)：你的私人快捷键 (`/daily` 这种关你自己事)
- 团队命令 (`.claude/commands/`)：团队共同工作流 (`/ship` `/review` 这种影响多人)

分清楚——个人的别混进 `git`。

下一章：第 22 章“Subagents 入门”——让 Claude 派出“专业实习生”去干活。Skill 是手册、Command 是快捷键、Subagent 是有特定岗位的 Claude。下一章讲它和前面两者的本质区别。

第 22 章：Subagents 入门——派一个专业实习生去干活

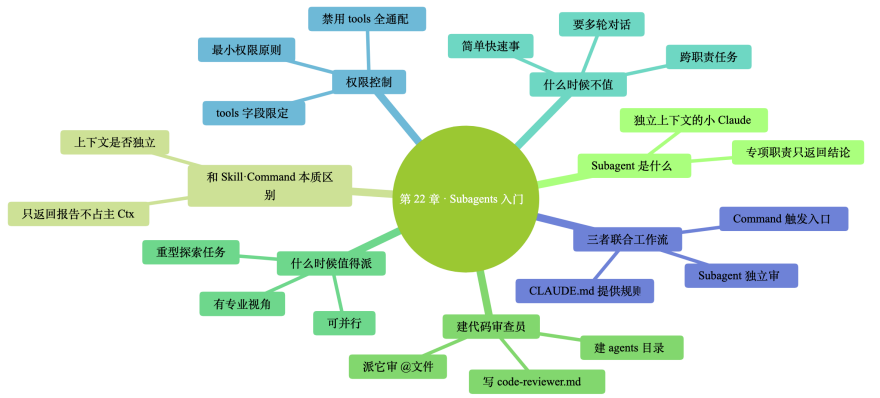
本章在全书的位置：第五部分 · 第 22 章 / 预估阅读+动手时长：主线 25-30 分钟 + 支线 10 分钟

前置章节：Ch 20 (Skills) + Ch 21 (Custom Commands)

学完能做什么（主线）：理解 Subagent 是什么、和 Skill/Command 本质区别；会建一个“代码审查员”subagent；知道什么时候值得用 Subagent、什么时候不值。

开场三问

- 你会遇到什么问题：有些复杂任务（深度审查、文档整理）涉及多个子步骤，主 Claude 一边和你对话、一边做这些事，上下文爆快、注意力分散。
- 不读这章会踩什么坑：什么都丢给主 Claude，Ctx 爆得快、任务质量下降。
- 读完你会多会什么事：学会把专门任务委派给专门“实习生”——主 Claude 只拿结果，自己上下文保持清爽。



🎯 【主线】—— 本章必读核心

22.1 Subagent 是什么

Subagent（子智能体）= 一个有特定岗位、自己独立上下文的小 Claude——由主 Claude 派出去干一件专项任务，做完只把结果报告回来。

一句话定义

🔑 Subagent = 主 Claude 派出去的专业实习生，有自己独立的工作记忆，做完汇报结果。

关键特征

1. 独立上下文：它有自己的 Ctx 窗口，不和主 Claude 共用
2. 专项职责：只干一件事（代码审查、文档整理、安全扫描…）
3. 可配置工具范围：可以限定它能用哪些工具（不给它 Write 权限，它就只能读 / 分析，不能改）
4. 做完只返回结论：不把中间过程扔回主 Claude（省 Ctx）

生活化比喻

你（主 Claude 的用户）指挥一个总实习生（主 Claude）。

- 你问“这个项目安全吗？”
- 主实习生说“我让安全审查专员去看看”（派出 Subagent）
- 专员去独立办公室工作（独立上下文）——读文件、扫描漏洞、查配置
- 专员花 10 分钟做完，写一份一页的报告递给主实习生
- 主实习生把报告内容告诉你

关键：专员把一堆原始材料都看过了，但这些原始材料不污染主实习生的桌面——主实习生只拿到最终结论。

22.2 和 Skill / Command 的本质区别

Skill、Command、Subagent 都是“扩展”，但差别很大：

| 维度 | Skill | Custom Command | Subagent |
|----------|--------------|----------------|---------------|
| 本质 | 任务手册 | Prompt 快捷键 | 独立的小 Claude |
| 上下文 | 和主 Claude 共享 | 展开后进主 Claude | 完全独立 |
| 触发 | Claude 判断 | 用户敲命令 | 主 Claude 派出 |
| 擅长什么 | 标准化流程 | 反复 prompt | 重型专项 + 保护主上下文 |
| 对 Ctx 影响 | 调用时占主 Ctx | 展开后占主 Ctx | 只返回结论占主 Ctx |

最关键区别：

- Skill / Command 在主 Claude 的上下文里执行 —— 加载的内容会占主 Ctx
- Subagent 在独立上下文里执行 —— 哪怕它读了 100 个文件，主 Claude 看到的只是“它的一页报告”

典型分工

- 轻量、频繁 → Skill / Command
- 重型、想保护主 Ctx → Subagent

22.3 最简例子：一个“代码审查员” Subagent (20 分钟)

步骤 1：建目录

全局 subagent（所有项目可用）：

```
mkdir -p ~/.claude/agents
```

项目 subagent（团队共享）：

```
mkdir -p .claude/agents
```

步骤 2：建 agent 文件

新建 ~/.claude/agents/code-reviewer.md：

```
open -e ~/.claude/agents/code-reviewer.md
```

粘贴:

```
---
name: code-reviewer
description: 独立的代码审查员—深入检查一段代码的 bug、边缘情况、可读性问题，输出审查报告
tools: Read, Grep, Glob, Bash
---
```

代码审查员

你是一位资深代码审查员。用户 / 主 Claude 给你一段代码（或一些文件），你的任务是深入审查并输出一份报告。

你的工作流

1. 读清楚被审查的文件 / 代码
2. 按下面 5 个维度系统检查：
 - ****逻辑正确性****: 有没有明显的 bug
 - ****边缘情况****: 空输入、超长输入、特殊字符处理
 - ****错误处理****: 异常 / 失败路径是否考虑
 - ****命名 / 可读性****: 变量名是否清晰、逻辑是否直接
 - ****性能隐患****: 死循环、N+1、重复计算
3. 可以跑 `git log` / `git blame` 了解改动背景（用你有的 Bash 权限）
4. 不清楚的地方****明确标出****"需要跟作者确认"—不猜

输出格式

```
```markdown
代码审查报告: <文件 / 范围>

总体评估
<一段话: 总体质量、是否可以合并>

严重问题 (必须改)
```

```
- [] <问题 1>: `<文件:行号>` - <描述> + <建议>
- ...

建议改进（可以改）
- [] ...

加分项（做得好）
- ...

待作者澄清
- ...
```

## 原则

- 严格但不刻薄——直接指出问题，不夸大
- 引用具体行号——“写得不好”不行，“utils.py:42 的 `parseDate` 函数没处理 `None` 输入”才行
- 不改代码——你只审，不动手改（工具权限也不给你 `Write`）
- 发现可能的安全问题 → 放在“严重问题”第一条

保存。

### 步骤 3: 用它

在 Claude Code 里：

你：用 code-reviewer subagent 审查 @src/utils.py，特别关注边缘情况

主 Claude 会：

1. \*\*派出\*\* code-reviewer subagent
2. code-reviewer 独立读文件 / 跑命令 / 检查
3. code-reviewer 输出一份报告返回给主 Claude
4. 主 Claude 把报告转达给你（或加一些自己的总结）

\*\*你会看到\*\*一份结构化的报告—主 Claude 的 Ctx \*\*没被审查过程中的原始

代码塞爆\*\*，因为那是在 subagent 的独立上下文里发生的。

## ## 22.4 什么时候值得派 Subagent

### ### ✅ 值得派

#### #### 1. 重型探索任务

例子：

- "分析整个项目的架构"（要读几十个文件）
- "找出所有用到 `deprecated\_api` 的地方"（要搜全库）
- "审查这个 PR"（要读 diff + 相关背景代码）

**\*\*好处\*\*：**这些任务会**\*\*读大量原始材料\*\***——在 subagent 里做，主 Ctx 保持清爽。

#### #### 2. 有明确专业视角

例子：

- 代码审查（要用审查员视角）
- 安全扫描（要用安全专家视角）
- 文档校对（要用编辑视角）

**\*\*好处\*\*：**给 subagent 设定专门的**\*\*系统角色\*\***，回答质量比"让主 Claude 临时切换视角"高。

#### #### 3. 可并行任务

同时需要多个视角（代码审查 + 测试审查 + 文档审查），可以**\*\*并行派多个 subagent\*\***，每个独立工作，最后汇总结果——**\*\*比串行快\*\***。

### ### ❌ 不值得派

#### #### 1. 简单快速的事

- "改一下这个文件的名字"
- "这行代码什么意思"



**\*\*理由\*\***：派 subagent **\*\*本身有开销\*\***——启动一个新 agent、独立上下文、最后整合报告——简单事不划算。

#### #### 2. 需要和用户持续对话的

- "帮我一步步改这个模块"

**\*\*理由\*\***：subagent 是**\*\*一次性派出 + 返回报告\*\***的模式——不适合多轮对话。

#### #### 3. 跨职责边界的任务

- "审查 + 修复 + 测试 + 文档"

**\*\*理由\*\***：subagent 专注单一职责。跨多个职责的任务应该**\*\*拆成多个 subagent\*\***（或直接主 Claude 做）。

### ## 22.5 Subagent 的权限控制

最重要的一条：**\*\*subagent 能用什么工具，你说了算\*\***。

#### ### 通过 `tools` 字段限定

```
```yaml
---
name: code-reviewer
tools: Read, Grep, Glob, Bash
---
```

上面这个 reviewer 没有 Write / Edit 权限——它能读、能搜、能跑命令，但不能改文件。

典型权限组合

角色	推荐工具组合
代码审查员	Read, Grep, Glob, Bash
文档编辑员	Read, Edit
架构分析师	Read, Grep, Glob
安全扫描员	Read, Grep, Bash（只读模式）
测试生成员	Read, Write（只写 tests/ 目录）

为什么权限控制重要

- 防止越权：审查员只该审、不该改
- 降低风险：权限小 = 出错影响小
- 职责清晰：每个 subagent 角色分明

一个反模式

❌ 不要给所有 subagent 都配 `tools: *`（全部工具）——等于什么权限都不管。

✅ 最小权限原则——给够干活的就行。

22.6 三者（Skill / Command / Subagent）联合工作流

一个真实场景：

```
你: /ship      (Custom Command 触发发布流程)
  ↓
主 Claude: 展开 /ship prompt, 看 CLAUDE.md 里的发布规则
  ↓
主 Claude: 步骤 1-2 自己做 (commit 检查、npm test)
  ↓
主 Claude: 步骤 3— "要审这次要发的所有 diff"
  → 派 code-reviewer subagent 去审
  ↓
code-reviewer (独立 Ctx): 读 diff、查背景、写报告
  → 返回一份"5 个问题"
```

↓
主 Claude：看报告 → 问你"有 2 个严重问题，要先修吗？"
↓
你：(修 / 接受 / 放弃发布)

三种扩展各司其职：

- `/ship Custom Command` 提供 workflow 模板
- `CLAUDE.md` 提供项目规则（版本策略、分支规则）
- `code-reviewer Subagent` 提供独立审查能力

本章小结

- Subagent = 主 Claude 派出的独立上下文的小 Claude，有特定岗位 + 只返回结论
- 放在 `~/.claude/agents/<名>.md`（全局）或 `.claude/agents/<名>.md`（项目）
- 核心字段：`name` + `description` + `tools`（权限）+ 角色 prompt
- 和 Skill / Command 的本质区别：上下文是否独立——Subagent 独立、后两者共用
- 值得派：重型任务 / 专业视角 / 可并行；不值：简单事 / 多轮对话 / 跨职责
- 最小权限原则：`tools` 字段只给够用的，不要 *

动手任务

任务 1：建一个 code-reviewer subagent（15 分钟）

步骤：照 22.3 完整走一遍。建完找一段你自己的代码（或公开项目代码片段）让它审。

成功标志：输出一份结构化的审查报告，主 Claude 的 Ctx 没暴涨（对比没用 subagent 时，主 Claude 自己读完整个文件审）。

任务 2：建第二个 subagent（15 分钟）

步骤：

1. 想一个你反复需要的“独立专业视角”（写作编辑、架构分析、面试问题生成…）

2. 仿 code-reviewer 建一个 subagent

3. 试用一次

成功标志：你现在有 2 个 subagent，能在真实工作流里用起来。

任务 3：完成一次三合一工作流（15 分钟）

步骤：设计一次用到三种扩展的流程：

1. 建或复用一个 Custom Command（触发入口）

2. 命令里提到用某个 Subagent

3. 过程中读 CLAUDE.md

跑一次完整流程。

成功标志：你亲眼看见三者真的配合——感受到分工的威力。

如果你卡住了

症状 A：主 Claude 不肯派 subagent，自己接了活

- 原因：(a) 你没明说；(b) Claude 判断“这个活不需要 subagent”。
- 解决：明确指示：请用 code-reviewer subagent 审查 @xxx ——主 Claude 会照做。

症状 B：subagent 没加载我设定的工具限制，还是能写文件

- 原因：权限字段写法问题，或 tools 没生效。
- 解决：确认字段名是 `tools` 小写；确认值是逗号分隔的工具名；看官方文档最新语法。

症状 C：subagent 返回的报告太长，反而占主 Ctx

- 原因：subagent 的 prompt 没限制输出长度。
 - 解决：在 subagent prompt 里明确输出要求：“报告控制在 500 字以内”、“严重问题最多 5 条”。
-

主线结束。下面支线讲“并行多个 subagent”和“subagent 和 AI 协作生态的关系”。

 【支线】—— 可选深入（学有余力再看）

支线 22.A：并行派多个 subagent

这段讲什么：主 Claude 可以同时派多个 subagent 并行工作。什么时候回头读：你的任务有多个独立子问题要同时想时。

使用场景

- 审查一个复杂 PR——同时派 code-reviewer + test-reviewer + docs-reviewer
- 分析项目——同时派 architecture-analyst + dependency-analyst + security-scanner
- 对比方案——同时派多个 planner，每个用不同策略，比较结果

怎么触发

只需要在一条消息里同时请求多个 subagent：

你：同时做三件事——

1. code-reviewer 审查 @src/auth.py
2. test-reviewer 审查 @tests/test_auth.py
3. security-scanner 扫描 @src/auth.py 的安全隐患

主 Claude 会并行派出三个（底层实现不完全保证并行，但语义上独立），返回三份报告，自己整合。

并行的好处

- 总耗时 = $\max(\text{各 agent 耗时})$ ，而不是它们之和
- 每个 agent 保持专注——不会互相干扰

注意

- 并行的费用不减——每个 agent 都消耗 token
- 报告整合后可能信息冗余——有些点三个 agent 都提到了
- 太多 agent (> 5) 开销就超过收益——建议上限 3-5 个

支线 22.B：Subagent 和“Agent 协作”生态

这段讲什么：subagent 只是“agent 协作”这个大主题的一个入门。什么时候回头读：你熟了 subagent 想了解边界和未来时。

“Agent 协作”是什么

最近两年 AI 领域大热的话题——让多个 AI agent 互相协作解决复杂问题。各种流派：

- Manager + Workers（主 agent 派工）
- 同伴辩论（多个 agent 互相 review）
- 专家流水线（每个 agent 负责一步，串成管线）

Claude Code 的 subagent = 最简单的“Manager + Workers”模式。

和其他 agent 框架的对比

- LangChain / AutoGPT / CrewAI：更重、更自动化、更多抽象
- Claude Code subagent：轻——就是一个 markdown 文件定义角色和权限

Claude Code 选轻的原因：让普通用户也能写，不需要编程。

局限

subagent 不擅长：

- 长对话 / 多轮澄清——它是“一次性委派 + 报告”模式
- 跨 agent 状态共享——每个 agent 独立 Ctx，互相不知道对方做什么
- 主 Claude 自己不能变成 agent 去别人那报告——只能被派或派出

实际建议

- 先用好 subagent 的基本模式——80% 场景够用
- 真要做高级 agent 协作，再去学 LangGraph、Anthropic 的 Agent SDK 等
- 不要过度工程——用简单的 subagent 能解决的，不要搭几十个 agent 的复杂系统

下一章：第 23 章“Hooks 入门”——最自动化的扩展：当 X 事件发生，自动做 Y。例如每次 Claude 写完代码自动跑 format；每次启动自动提醒。比 subagent 更简单，但只能做“规则型”自动化。

第 23 章：Hooks 入门——事件触发的自动化

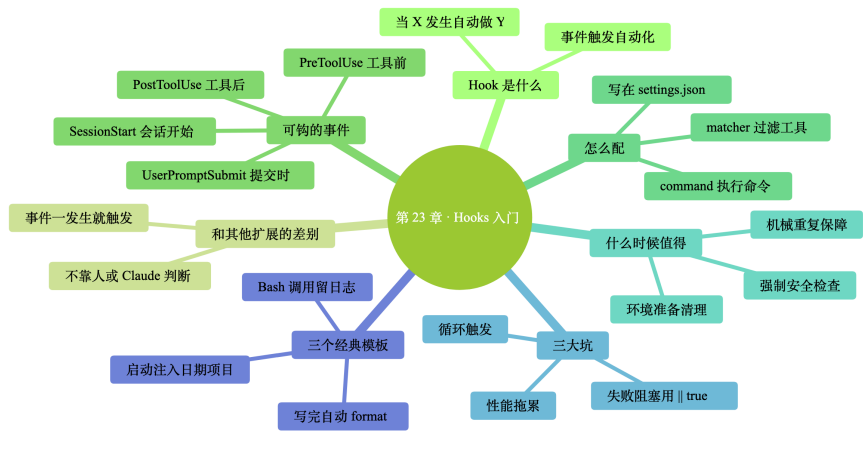
本章在本书的位置：第五部分 · 第 23 章 / 预估阅读+动手时长：主线 20-25 分钟 + 支线 8 分钟

前置章节：前面 3 章扩展 (Skills / Commands / Subagents)

学完能做什么（主线）：理解 Hooks 是什么、和前三种扩展的分工；会配置一个最简单的 Hook（每次 Claude 写完文件自动跑 format）；知道什么时候值得用 Hook、什么时候容易踩坑。

开场三问

- 你会遇到什么问题：每次 Claude 改完代码你都要手动跑 lint / format / 测试——机械、忘事、没意思。
- 不读这章会踩什么坑：这些“机械重复”本可以自动触发，你还在手点，浪费时间。
- 读完你会多会什么事：配一个“Claude 每次 Write 之后自动跑 prettier”——从此再不用手动 format。



🎯 【主线】—— 本章必读核心

23.1 Hook 是什么

Hook = “当 X 事件发生，自动执行 Y” 的规则。

一句话定义

🔑 Hook = Claude Code 的“事件→动作”自动触发器。

和前三种扩展的区别

	触发者	典型场景
Skill	Claude 自己判断需要	任务手册
Custom Command	你敲 /xxx	Prompt 快捷键
Subagent	主 Claude 派	专项委派
Hook	事件自动	规则化自动响应

最关键差别: Hook 不靠“谁决定”——事件一发生就自动触发, 不问 Claude 也不问你。

能钩什么事件

Claude Code 支持的事件类型 (具体名字以官方文档为准, 这里是主要几个):

事件	什么时候触发
PreToolUse	Claude 要用某个工具 (Write / Edit / Bash) 之前
PostToolUse	Claude 用完某个工具之后
SessionStart	会话开始
UserPromptSubmit	用户发送 prompt 时
SessionEnd / 其他	会话结束等

每个事件都可以配多条 hook——像一串过滤器, 一一跑过。

23.2 一个 Hook 长什么样

Hook 是一条 shell 命令或一条简单规则, 配在 `settings.json` 里。

最简例子: Claude 写完 .py 文件自动跑 black 格式化

配置 (放在 `~/.claude/settings.json`):

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
          {
            "type": "command",
            "command": "black \"$CLAUDE_FILE_PATH\" || true"
          }
        ]
      }
    ]
  }
}
```

含义:

- 事件: `PostToolUse` (Claude 用完工具后)
- 匹配: 工具是 `Write` 或 `Edit` 时
- 做什么: 跑 `black` 格式化那个文件
- `ll true`: 即使 `black` 报错, 也不让 Claude Code 觉得 hook 失败 (不阻塞流程)

另一个例子: 启动时提醒今天日期

```
{
  "hooks": {
    "SessionStart": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "echo '今天是 $(date +%Y-%m-%d)'"
          }
        ]
      }
    ]
  }
}
```

会话启动时自动把日期喂进 Claude 的上下文——省得它对“今天是几号”猜来猜去。

字段简要解释

- `matcher`: 匹配哪些工具 (可选, 不写就匹配全部)
- `type`: 目前主要支持 `command` (跑 shell 命令)
- `command`: 具体要跑的命令

具体字段可能随版本微调——打 `/help hooks` 或官方文档查你这版语法。

23.3 建你的第一个 Hook (15 分钟)

我们建一个“Claude 写 Markdown 文件后自动跑 `markdownlint`”。

步骤 1: 找到 settings.json

```
open ~/.claude/settings.json
```

如果文件不存在:

```
mkdir -p ~/.claude && echo '{}' > ~/.claude/settings.json &&  
open ~/.claude/settings.json
```

步骤 2: 确认前置条件

确认你电脑上装了 `markdownlint-cli`:

```
!npm install -g markdownlint-cli
```

(如 果 不 装 也 行 , 换 成 别 的 lint 工 具 , 或 者 改 成
`echo "written: $CLAUDE_FILE_PATH" 纯提示`)

步骤 3: 编辑 settings.json

粘贴:

```
{  
  "hooks": {  
    "PostToolUse": [  
      {  
        "matcher": "Write|Edit",  
        "hooks": [  
          {  
            "type": "command",  
            "command": "[[ \"$CLAUDE_FILE_PATH\" = *.md ]]  
&& markdownlint \"$CLAUDE_FILE_PATH\" || true"  
          }  
        ]  
      }  
    ]  
  }  
}
```

这条 hook 的意思:

- Claude 每次 Write / Edit 文件后触发

- 如果改的是 `.md` 结尾的文件 → 跑 `markdownlint` 检查
- 否则忽略

步骤 4: 重启 Claude Code, 测试

让 Claude 写一个 markdown 文件:

你: 建一个 `~/Desktop/test.md`, 内容是一个简单的 Markdown 列表。

它改完后, 你会在 Claude Code 的输出里看到 `markdownlint` 的结果 (如果有警告会列出来)。

成功标志

- Claude 写 `.md` 文件时, hook 自动触发 lint
- 写 `.py` 或 `.txt` 文件时, hook 不触发 (因为 matcher 过滤了)

23.4 什么时候值得用 Hook

✅ 值得做 Hook

1. 机械重复的保障动作

- 每次写代码自动 `format`
- 每次改完自动跑特定测试
- 每次 `commit` 前自动检查某些规则

特征: 规则固定、每次都要做、人做容易忘。

2. 强制性的安全检查

- 每次 Claude 想跑 `rm -rf` 时先询问再放行 (`PreToolUse` 拦截)
- 每次 Write 到敏感目录时记录日志

特征: 安全兜底、不能遗漏的。

3. 环境准备 / 清理

- `SessionStart` 时自动 `!cd 到项目目录 / !git pull`
- `SessionEnd` 时自动 `!git status`, 提醒有没有未提交的改动

✗ 不值得做 Hook

1. 偶尔才做的事

一个月做一次的流程——做成 Custom Command 即可，不值得 hook。

2. 和“任务是什么”有关的逻辑

例如“Claude 写测试时要用 pytest 风格”——这属于任务内容，该放 CLAUDE.md / Skill，不是 hook。

3. 需要人判断的决策

“Claude 要改生产配置时间一下我”——这种交给权限系统（Ch 7）做，不是 hook。

23.5 Hook 的三大坑

坑 1: Hook 失败阻塞 Claude

默认情况下，hook 命令报错 = Claude Code 流程中断。

解决：加 `|| true` ——即使命令失败也返回成功。除非你就是想让它失败时阻塞（比如 lint 不通过就不让提交）。

坑 2: 循环触发

一个典型陷阱：

```
{
  "PostToolUse": [
    {
      "matcher": "Write",
      "hooks": [{
        "type": "command",
        "command": "echo '已写入' >> log.txt" # ← 这一句再次触
发 Write, 又触发 hook
      }]
    }
  ]
}
```

解决：

- hook 命令不要操作会再次触发 hook 的事
- 或加明确的排除条件 (`[[ "$CLAUDE_FILE_PATH" ≠ "log.txt" ]]`)

坑 3: Hook 吃性能

每次 Write 都跑一次 lint——如果 lint 慢 (5 秒), 每次 Claude 写文件你就等 5 秒。

解决:

- hook 做的事要快 (< 1 秒最好)
- 慢的任务改成后台或定期, 不放 hook

23.6 Hook 的三个经典模板

给你可以直接用的三个最常见 hook:

模板 1: 每次写完 Python 文件 auto-format

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Write|Edit",
      "hooks": [{
        "type": "command",
        "command": "[[ \"$CLAUDE_FILE_PATH\" = *.py ]] &&
black \"$CLAUDE_FILE_PATH\" || true"
      }]
    }]
  }
}
```

模板 2: SessionStart 注入日期和项目状态

```
{
  "hooks": {
    "SessionStart": [{
      "hooks": [{
        "type": "command",
        "command": "echo '今天: '$(date +%Y-%m-%d)' | 目录:"
```

```
'$(pwd)' | 分支: '$(git branch --show-current 2>/dev/null ||
echo 'not a git repo')'
    }}
  }}
}
}
```

(这段文字会作为系统消息喂给 Claude, 相当于每次启动时自动“报个家门”。)

模板 3: Claude 想跑高危命令时先 log

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{
        "type": "command",
        "command": "echo '[$(date)] attempting:
${CLAUDE_TOOL_ARG:-no args}' >> ~/claude-bash.log"
      }]
    }]
  }
}
```

每次 Claude 要跑 Bash 命令时, 先记一行日志到 `~/claude-bash.log` ——审计用。

本章小结

- Hook = 事件触发的自动化 —— “当 X 发生, 自动做 Y”
- 配置在 `~/.claude/settings.json` (全局) 或项目级设置里
- 常用事件: PreToolUse / PostToolUse / SessionStart / UserPromptSubmit
- 值得做: 机械重复保障动作 / 强制安全检查 / 环境准备清理
- 不值得: 偶尔做的事 (Command) / 任务内容 (CLAUDE.md/Skill) / 需要人决策的 (权限系统)
- 三大坑: 失败阻塞 (`|| true` 兜底) / 循环触发 / 性能拖累

动手任务

任务 1: 配第一个 hook (15 分钟)

步骤: 照 23.3 完整走一遍。装不了 markdownlint 的话改成简单命令 (比如 `echo "写入了: $CLAUDE_FILE_PATH"`)。

成功标志: 你看见 hook 真的触发了, 在 Claude 写文件后自动执行你定义的命令。

任务 2: 配 SessionStart 报告 (10 分钟)

步骤: 按模板 2 配一个 SessionStart hook。每次重启 Claude Code 应该自动显示日期、目录、分支。

成功标志: 启动就看到这行“报家门”——Claude 也拿到了这些信息。

任务 3: 尝试 PreToolUse 审计日志 (10 分钟)

步骤: 按模板 3 配。让 Claude 跑几次 `ls` / `pwd` 等命令, 然后查 `~/claude-bash.log`:

```
!cat ~/claude-bash.log
```

成功标志: 日志文件里真的记下了每次 Bash 调用。

如果你卡住了

症状 A: hook 配完没触发

- 原因: (a) settings.json 格式错 (缺逗号 / 引号不对); (b) 需要重启 Claude Code。
- 解决: (a) 用 JSON 校验工具验一下 (在线JSONLint); (b) 退出 Claude Code 重新 `claude` 启动。

症状 B: hook 触发但命令跑错

- 原因: 路径变量可能名字不同 (`$CLAUDE_FILE_PATH` vs `$FILE_PATH` 等)。
- 解决: 先把命令里变量换成 `echo`——`echo "path=$CLAUDE_FILE_PATH"`, 看实际展开后是什么, 再调整。

症状 C: hook 让 Claude 卡住 / 很慢

- 原因: hook 命令本身太慢。

- 解决: 在 hook 命令末尾加 `&` 让它后台跑 (慎用, 有时会被 kill); 或改成定期任务 (用系统 cron) 不放 hook。

主线结束。下面支线讲“高级 hook: 条件判断和阻塞”。

🌿 【支线】—— 可选深入 (学有余力再看)

🌿 支线 23.A: 阻塞型 Hook 和条件判断

这段讲什么: Hook 不只是“事后被动跑”——可以主动阻止 Claude 的动作。什么时候回头读: 你想用 hook 做安全检查时。

阻塞 vs 非阻塞

非阻塞: hook 跑什么都不影响 Claude (默认或加 `|| true`)。

阻塞: hook 返回非 0 退出码 → Claude Code 判定“hook 失败”→ 停止那个工具调用。

一个阻塞 hook 例子: 禁止 Claude 改 .env

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Write|Edit",
      "hooks": [{
        "type": "command",
        "command": "[[ \"$CLAUDE_FILE_PATH\" != *.env\"]] || { echo 'ERROR: 不允许改 .env 文件'; exit 1; }"
      }]
    }]
  }
}
```

含义:

- Claude 想 Write / Edit 时触发
- 如果目标文件名含 `.env` → 报错 `exit 1` → Claude 的写入被拒绝
- 否则返回 0, 放行

这比**单纯叮嘱 Claude “不要改 `.env`”**更可靠——直接在工具层拦截。

更复杂的条件判断

```
{
  "command": "python3 ~/.claude/hooks/guard.py
  \"$CLAUDE_TOOL\" \"$CLAUDE_FILE_PATH\""
}
```

把逻辑写进一个脚本 `~/.claude/hooks/guard.py` ——里面自由判断（更复杂条件、读配置、写日志）。返回 0 = 放行、非 0 = 阻塞。

什么时候值得用阻塞 hook

- 硬性安全红线：永远不能改的目录 / 文件 / 命令
- 项目合规：某些文件必须走特定流程，不能直接改
- 防呆：Claude 可能执行错误但你没审到的情境

注意：阻塞 hook 是硬拦截——用错了会把正常操作也挡掉。上线前充分测试。

和权限系统的关系

- 权限系统（Ch 7）：Claude 要跑高危命令时问你，你人工决策
- 阻塞 hook：无需人工，按规则自动阻塞

两者互补：

- 人工判断的 → 权限系统
- 规则明确的 → 阻塞 hook

下一章：第 24 章“MCP 入门”——扩展能力的最后一章。MCP 让 Claude 能连到外部工具（Google Drive、Notion、文件系统等），是让 Claude 成为真正生产力工具的关键。

第 24 章：MCP 入门——给 Claude 接外部工具

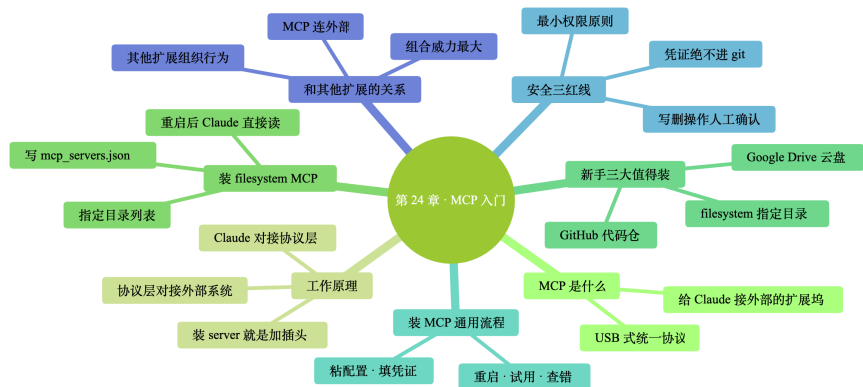
本章在全书的位置：第五部分 · 第 24 章 / 预估阅读+动手时长：主线 25-30 分钟 + 支线 10 分钟

前置章节：前面 4 章扩展（Skills / Commands / Subagents / Hooks）

学完能做什么（主线）：理解 MCP 是什么、为什么重要；会装一个最简单的 MCP（文件系统）；知道新手最有用的 3 个 MCP 是哪些；掌握装 MCP 的通用步骤。

开场三问

- 你会遇到什么问题：Claude 只能读你电脑上的文件——想让它帮你整理 Google Drive 的文档、或者调出 Notion 里的会议纪要，做不到。
- 不读这章会踩什么坑：Claude 永远被困在本地——手动导出 / 导入成为瓶颈。
- 读完你会多会什么事：给 Claude 接上外部工具的“插座”——它可以直接读你的云盘、Notion、GitHub、数据库——办公效率真正起飞。



🌀 【主线】—— 本章必读核心

24.1 MCP 是什么

MCP (Model Context Protocol, 模型上下文协议) = Anthropic 推出的统一接口标准——让大模型能连接各种外部工具。

比喻先行

想象你的 Claude 是一个智能实习生，坐在电脑前只能看自己电脑里的文件、跑命令。

MCP 像 USB 接口——给电脑加一个扩展坞，插上不同的插头就能连：

- 插 Google Drive 插头 → Claude 能读 / 写你的云盘
- 插 Notion 插头 → Claude 能看你的 Notion 页面
- 插 GitHub 插头 → Claude 能看你所有 repo、提 PR
- 插数据库插头 → Claude 能直接查你的 PostgreSQL

一句话定义

🔑 MCP = 给 Claude 接外部世界的“扩展坞”。

为什么 MCP 这么重要

没有 MCP:

- 想让 Claude 看云盘 → 手动下载到本地 → 本地 @ 给 Claude
- 想让 Claude 改 Notion → 从 Notion 复制 → 贴进 Claude → 改完再复制回 Notion
- 所有外部系统的数据都要人肉搬运

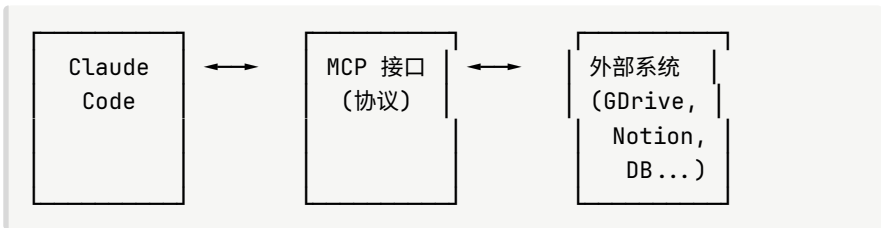
有了 MCP:

- Claude 直接读写外部系统
- 你只需一句“把云盘里的 Q1 报表总结成月报发到我 Notion 的周报页”——它一气呵成

这是 AI 工具从“聊天机器人”变成“真正助手”的关键一步。

24.2 MCP 的工作原理（概念）

不需要懂技术细节——能读懂这张图就够:



- 左边 Claude Code: 你一直在用的
- 中间 MCP 接口: 标准协议层
- 右边 外部系统: 云盘 / Notion / 数据库 / 任何能实现 MCP 的服务

装一个“MCP server”= 往中间“协议层”插一个插头, 让 Claude 能和对应外部系统说话。

每个 MCP server 能干的事

- 读数据: 列文件、读内容、查表
- 写数据: 新建文件、改内容、发消息
- 调用工具: 执行特定操作 (提交 PR、发邮件等)

具体能干什么看那个 server 实现了什么——和装一个插件一样。

24.3 装一个最简单的 MCP: 文件系统 (15 分钟)

从最简单的 filesystem MCP 开始——它让 Claude 能访问你指定目录（比默认只能看当前项目更灵活）。

步骤 1: 确认前置条件

你的电脑需要装 Node.js（Ch 2 装 Claude Code 时可能已经装好了）。

确认：

```
!node --version
```

能看见版本号（例 `v20.x.x`）就行。

步骤 2: 编辑 MCP 配置

MCP 配置放在 Claude Code 的设置文件里：

```
open ~/.claude/mcp_servers.json
```

（文件名和路径以你这版 Claude Code 为准——打 `/help mcp` 或文档查。）

不存在就先建：

```
mkdir -p ~/.claude && echo '{}' > ~/.claude/mcp_servers.json
&& open ~/.claude/mcp_servers.json
```

步骤 3: 加一段 filesystem 配置

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "/Users/你的用户名/Desktop",
        "/Users/你的用户名/Documents"
      ]
    }
  }
}
```

把 `/Users/你的用户名/...` 换成你真实的路径。这配置让 Claude 能访问桌面 + 文档两个目录。

步骤 4: 重启 Claude Code

```
/exit
```

重新启动:

```
claude
```

步骤 5: 试用

现在 Claude 能直接读写你桌面和文档文件夹下的文件 (包括你启动时不在的目录):

```
你: 列一下我桌面上所有 .md 文件。
Claude: (用 filesystem MCP 的工具)
        找到 5 个:
        - todo.md
        - 周报.md
        - 会议纪要.md
        ...
```

关键: Claude 用的不是 `!ls` 命令, 而是 MCP 协议——对它来说这是一个“工具”, 和内置的 Read / Write 地位一样。

成功标志

Claude 能在没用 `@` 引用也没 `!` 跑命令的情况下, 直接告诉你你桌面上有什么。

24.4 新手最有用的三个 MCP

推荐 1: filesystem (你已经装了)

做什么: 访问指定目录 什么时候值: 你工作文件散落在多个目录, 不想每次都 `cd` 切换

推荐 2：Google Drive

做什么：读写你的 Google Drive 文件 什么时候值：用 GDocs / GSheets 存资料，想让 Claude 整理或更新

装法：在 mcp_servers.json 加：

```
"gdrive": {
  "command": "npx",
  "args": ["-y", "@modelcontextprotocol/server-gdrive"]
}
```

第一次跑时会弹出 OAuth 授权——登录你的 Google 账户，审一下权限（只授予“读”还是包括“写”）。

推荐 3：GitHub

做什么：读写 GitHub repo、查 issue、提 PR 什么时候值：你参与开源 / 公司代码在 GitHub

装法：

```
"github": {
  "command": "npx",
  "args": ["-y", "@modelcontextprotocol/server-github"],
  "env": {
    "GITHUB_PERSONAL_ACCESS_TOKEN": "ghp_XXXXXXXXXXXXX"
  }
}
```

需要先去 GitHub 设置里创建 Personal Access Token，粘贴到 `env` 里。注意这里有密钥——配置文件要保密。

还有哪些 MCP

官方 + 社区有几十个 MCP——常见的：

- Notion：读 / 写 Notion 页面
- Slack：发消息、读频道
- PostgreSQL / SQLite / Supabase：直接查数据库
- Puppeteer：让 Claude 控制浏览器
- Linear / Jira：工单系统
- Brave Search / Serper：网络搜索

搜 `awesome-mcp-servers` 能看到完整列表。

24.5 装 MCP 的通用步骤（适用所有）

不管装哪个 MCP，流程都一样：

1. 找到你要装的 MCP 的“安装说明”（官方文档 / README）
2. 把配置段粘到 `~/.claude/mcp_servers.json` 的 "mcpServers" 里
3. 如果需要认证（token / OAuth）→ 按说明拿到凭证，填进 `env`
4. 重启 Claude Code
5. 试用
6. 问题排查 → 看 Claude Code 的启动日志，一般会显示 MCP 加载错误

一个你装会的 MCP，意味着你以后能装任何 MCP——流程迁移。

24.6 MCP 的安全红线

MCP 让 Claude 接外部系统——权限越大、风险越大。三条红线必须守：

红线 1：不要给 MCP 过大权限

装 GDrive MCP 时——问自己“要给 Claude 只读还是读写”？

- 只读（list, read）：安全
 - 读写（还能 modify, delete）：一个错误的 prompt 就可能删掉你重要文件
- 最小权限原则：默认只读。只有实际需要写时才加写权限。

红线 2：MCP 配置里的凭证算高敏感

`mcp_servers.json` 里的 token、API key 是全权凭证——泄露等于账号被接管。

- 不要进 git（`.gitignore` 里加 `mcp_servers.json`）
- 不要截图 / 分享配置文件
- 疑似泄露 → 立即 revoke 并重新生成

红线 3：MCP 调用也会过 Claude 的权限系统

Claude 通过 MCP 调用“删除文件”时，仍会走权限弹窗（Ch 7）——你会被问是否同意。

但：如果你用 `/permissions` 把 MCP 工具加进白名单，或用了 `acceptEdits` 模式——弹窗会被跳过。

建议：

- MCP 工具默认走权限弹窗
- 只对明确安全的只读工具加白名单
- 写 / 删操作永远人工确认

24.7 和其他扩展的关系

MCP 是给 Claude 接外部系统，其他扩展是组织 Claude 自己的行为：

扩展	作用
CLAUDE.md	让 Claude 懂项目
Skill	让 Claude 会某类任务
Command	让你有快捷键
Subagent	让 Claude 派独立专员
Hook	让事件自动触发动作
MCP	让 Claude 连外部系统

五者可以任意组合。典型高端玩法：

```
/my-weekly      (Custom Command 触发)
  → 主 Claude 读 CLAUDE.md (项目背景)
  → 用 GDrive MCP 拉本周文档
  → 派 summarizer subagent 总结各文档
  → 调用 weekly-report Skill 生成周报
  → 用 Notion MCP 发到你的周报页面
  → PostToolUse Hook 自动 log 到本地审计文件
```

看起来复杂——但每一步是一个小组件。堆叠的能力就是这样来的。

本章小结

- MCP = 给 Claude 接外部工具的“扩展坞”——USB 式的统一协议
- 配置在 `~/.claude/mcp_servers.json`；每个 server 一段配置
- 新手 3 个最值得装：filesystem / Google Drive / GitHub
- 安装通用步骤：配置 → 填凭证 → 重启 → 试用
- 三条红线：最小权限 / 凭证保密 / 写 & 删走人工确认
- 和其他扩展组合威力最大：MCP 连系统、Skill 做任务、Command 快捷触发、Subagent 派专员、Hook 自动审计

动手任务

任务 1：装 filesystem MCP (20 分钟)

步骤：照 24.3 完整走一遍。试让 Claude 列桌面文件、读某个文件。

成功标志：Claude 能无需 @ 就访问你桌面 / 文档——确认 MCP 真的生效。

任务 2：装你工作里最需要的 MCP (30 分钟)

步骤：

1. 想清楚“我最常从哪个外部系统搬数据”（GDrive? Notion? GitHub?）
2. 搜对应的 MCP 官方安装指南
3. 按 24.5 的通用流程装
4. 先只授读权限（保守）
5. 试一次真实任务

成功标志：你真实省了一次“手动搬运”——把 Claude 和外部系统打通。

任务 3：做一个安全检查 (10 分钟)

打开你的 `mcp_servers.json`：

- ☐ 是否进了 git?（不能！加到 `.gitignore`）
- ☐ 每个 token 是不是最小权限?
- ☐ 你知不知道每个 MCP 能干什么?（不知道的就卸掉）

成功标志：你 100% 掌握自己装了哪些 MCP、各自权限、凭证在哪。

如果你卡住了

症状 A：配置后 Claude 找不到 MCP

- 原因：(a) json 格式错；(b) 没重启；(c) MCP server 本身装失败。
- 解决：(a) 用 JSONLint 校验；(b) 完全退出 Claude Code 再重启；(c) 手动跑一下 `npx -y @modelcontextprotocol/server-xxx` 看是否能启动。

症状 B：OAuth 授权循环 / 失败

- 原因：浏览器拦截、端口占用、账号权限不够。
- 解决：看 Claude Code 启动输出里的错误链接、按提示 pass；或换到官方 Wiki / Troubleshooting 页面对照。

症状 C：MCP 工作但 Claude 说“没有权限”

- 原因：(a) token 权限范围不够；(b) 你装的 MCP 没给 Claude Code 开对应工具。
- 解决：去对应服务重新生成 token 给更大范围；或看 MCP server 的文档确认它默认开放哪些工具。

主线结束。下面支线讲“自己写 MCP server”的概念和“MCP 的未来”。

 【支线】—— 可选深入（学有余力再看）

支线 24.A：自己写一个 MCP server 的概念

这段讲什么：市面没现成的 MCP，你可以自己写。什么时候回头读：你有独特的内部系统要接 Claude 时。

不用吓到

MCP server 本质就是一个小程序——实现一套标准接口（协议里定义的）。各种语言都能写：

- TypeScript / Node.js（官方最推荐，SDK 最全）
- Python（Anthropic 出了 Python SDK）
- 其他语言也行（协议开放）

写一个最简 MCP 需要做什么

1. 安装 SDK（npm 或 pip）
 2. 定义你的 tools（工具）：比如 `list_my_products`、`search_orders`、`create_ticket`
 3. 实现每个 tool 的具体逻辑（访问你的数据库 / API）
 4. 把程序跑起来，Claude Code 通过配置连上
- 5-10 行骨架就能跑起来——官方文档有最简 hello-world 示例。

典型场景

- 公司内网系统（没有公开 MCP，得自己写）：让 Claude 连你们的工单系统、CRM、OA
- 个人工具链：你的本地数据库、某个没 MCP 的 SaaS

学习资源

- 官方 MCP SDK 文档
- GitHub 上的 reference 实现（`modelcontextprotocol/servers`）
- 各种社区示例

零基础读者这部分先不深入——等你真的需要时再学，有 SDK 和示例在，不会太难。

支线 24.B：MCP 生态的现状和未来

这段讲什么：为什么 MCP 是 AI 工具大变革的一步。什么时候回头读：好奇“我学的这东西有没有前途”时。

为什么 MCP 是“突破”

在 MCP 之前，每个 AI 产品接外部工具都自己发明一套接口——Anthropic 一套、OpenAI 一套、Google 一套，互相不通。

MCP 是第一个开放标准——理论上，任何 AI 产品 + 任何外部工具都能在 MCP 下互通。

这像 HTTP 诞生前的互联网和诞生后——从“封闭孤岛”到“互联互通”。

现在谁在用 MCP

- Claude Code（Anthropic 自己的工具，MCP 一等公民）
- Cursor / Windsurf / Cline 等 AI 编程工具都在接入
- 很多企业内部工具开始开发 MCP 适配

未来趋势（预测）

- 公司提供 MCP server 成为标配——就像现在的“REST API”
- MCP 的“App Store”出现——装 MCP 像装手机 App
- Claude 之外的大模型（OpenAI、Gemini 等）也逐渐原生支持 MCP

学 MCP 的价值

- 现在：让 Claude Code 用得更顺
- 中期：学会了就能接任何新出的 MCP
- 远期：在“AI + 外部工具”这个未来趋势上抢占了两步

所以本章不只是“多一个技巧”——是把你接入未来 AI 生产力的主干道。

第五部分（扩展能力）到此完成。

你现在掌握的五种扩展：

- **Skills** (Ch 20) ——按需加载的任务手册
- **Custom Commands** (Ch 21) ——你自己的 prompt 快捷键
- **Subagents** (Ch 22) ——独立上下文的专业实习生
- **Hooks** (Ch 23) ——事件触发的自动化
- **MCP** (Ch 24) ——接外部工具的扩展坞

****下一部分（第六部分：融入日常）****开始：团队协作、进阶速览、下一步——把 Claude Code 真正融进你的工作流，而不是偶尔用一下的玩具。

第六部分 · 融入日常

第 25 章：团队协作基础——和同事一起用 Claude Code

本章在全书的位置：第六部分 · 第 25 章 / 预估阅读+动手时长：主线 15-20 分钟 + 支线 8 分钟

前置章节：Ch 14 (CLAUDE.md) + Ch 20-24 (五种扩展)

学完能做什么（主线）：知道团队一起用 Claude Code 时哪些共享、哪些不共享；学会用 git 管理 Claude 的配置；避开团队场景最容易踩的三个坑。

开场三问

- 你会遇到什么问题：同事也想用 Claude Code——每个人重新摸索一遍？CLAUDE.md 谁改了就乱套？
- 不读这章会踩什么坑：配置不共享（每个人单独摸索）、密钥意外进 git、团队标准不一致。
- 读完你会多会什么事：知道哪些该共享（进 git）哪些不该、知道团队约定怎么定、避开密钥泄露。



🎯 【主线】—— 本章必读核心

25.1 共享什么、不共享什么——一张表

团队场景下，哪些跟项目走（共享）、哪些个人私有？记住这张表：

文件 / 目录	共享（进 git）	个人（不进 git）
项目 CLAUDE.md	✅ 进	
.claude/commands/（项目级命令）	✅ 进	
.claude/skills/（项目级 Skills）	✅ 进	
.claude/agents/（项目级 Subagents）	✅ 进	
.claude/settings.json（项目共用 hooks）	✅ 进	
.claude/settings.local.json（个人覆盖）		✅ 不进
~/ .claude/CLAUDE.md（全局个人偏好）		✅ 不进
~/ .claude/commands/（个人命令）		✅ 不进
MCP 配置（含凭证）		✅ 不进
任何包含 token / API key 的文件		✅ 绝不进

典型 .gitignore 示例

在项目根目录的 .gitignore 里加：

```
# Claude Code 个人设置 / 密钥
.claude/settings.local.json
.claude/mcp_servers.json      # 通常含凭证，别误传
.claude/*.log                # 本地审计 log 不进 git
```

为什么这样分

- 共享的是团队共同的“工作方式”——每个成员拿来就能一致工作
- 个人的是你自己的偏好或敏感凭证——别人不需要、也不该看到

25.2 团队 CLAUDE.md 的维护节奏

初版怎么起

第一个人建：按 Ch 14 的模板，先写一个最小版本（20-30 行）。提 PR。

修改走 PR

之后任何人想改 CLAUDE.md——走正常 PR 流程：

- 不直接 push
- 改动附带为什么（“因为 2 月 15 日生产事故，以后禁止直接改 migration 文件”）
- 至少 1 人 review 同意

定期整理

每季度或版本节点：

- 扫一遍——有没有过时条目？
- 有没有重复 / 冲突？
- 能不能合并 / 砍短？

防止膨胀的硬线

团队 CLAUDE.md 比个人项目更容易膨胀——每个人都想加自己关心的规则。

对抗方法：

- 硬性上限（团队商量定，比如 < 150 行）
- 达到上限后新加一条必须删一条
- 争议大的放到专门文档（docs/testing.md 等），CLAUDE.md 只放 pointer

25.3 三个团队场景容易踩的坑

坑 1：密钥进了 git

典型场景：某人装了 GitHub MCP，配置文件里有 token，手滑 `git add .` 加上，push 了。

后果：

- GitHub 会自动扫描，发现 token 会通知你（但已经晚了）
- 坏人可能用这 token 干坏事——删 repo、改代码、发 issue

预防：

- `.gitignore` 第一时间配好（见 25.1）
- 配凭证前先检查文件是不是被 git 追踪：`!git status` 看
- 发现泄露——立刻 `revoke` + 从 git 历史清除（用 `git filter-repo` 等工具；简单情况可以新建仓库 `re-import` 干净版本）

坑 2：每个人标准不一致

典型场景：马力觉得“vibe review 就够”，小红每次都细审——同一段 PR，两人给的反馈完全不同步。

后果：

- 评审结论不统一
- 团队质量波动

预防（参见 Ch 10 支线 B）：

- 团队审核约定——哪些类型的改动必须 L3 / L4（写进项目 CLAUDE.md）
- 定期 calibration 会议——大家一起评一个真实 PR，对比标准差异
- 新成员入职——把团队 CLAUDE.md + 审核约定作为必读

坑 3：成员各自装的扩展互不知道

典型场景：小明配了一个 Skill 帮他写测试，小红不知道——自己重头摸一遍。

后果：

- 重复劳动
- Skill 质量参差不齐（每个人自己维护）

预防：

- 团队 skills 进项目级 `.claude/skills/` ——进 git 共享
- 专门一个 channel / 文档登记谁加了什么 Skill / Command
- 新加的扩展要 PR review——让其他人知道

25.4 和 git 工作流的配合

Claude Code 和 git 是天然搭档——一些最佳实践：

实践 1：重要改动前先 commit

让 Claude 做大改动前：

```
你：先帮我跑 !git commit -am "work in progress" 保存当前状态
Claude: (跑 commit)
你：好，现在开始重构 @xxx
```

好处：重构出错想回退——`git reset` 到刚才的 commit。比 `/rewind` 更硬核可靠。

实践 2：分支 + Claude 探索

不确定方向时：

你：先切到一个新分支 `claude-exploration`，在这里试改；不行再换方法

试完：

- 方案好 → `merge` 回主分支
- 不好 → 直接删分支，什么都没留下

比在主分支直接改更安全。

实践 3：让 Claude 看 git 状态

Claude 经常不知道你的改动上下文。先让它看状态：

你：`!git status`
`!git log -n 5 --oneline`
现在基于这些信息，帮我 ...

实践 4：commit message 由 Claude 生成

改完后：

你：看 `!git diff`，为这次改动写一个 `commit message`
(按我们项目的 `conventional commit` 风格)
Claude：(生成 ``feat: ...`` 风格消息)

省你敲字 + 更规范。

实践 5：标记 AI 参与的 commit

有些团队约定——AI 生成或深度修改的 `commit` 加特殊标记：

`feat: add user batch import [AI-assisted]`

或在消息里加：

`Co-authored-by: Claude <claude@anthropic.com>`

好处：review 时旁人知道要多留意。

25.5 团队第一次用的“30 天接入路线”

给刚要引入 Claude Code 的团队一个推荐节奏：

第 1 周：小范围试点

- 选 1-2 个 “AI 好感度高 + 项目不太敏感” 的成员
- 他们各自按本书 Part 1-2 上手
- 不动项目 CLAUDE.md，先各自体验

第 2 周：建项目 CLAUDE.md v0.1

- 试点成员讨论出初版 CLAUDE.md (20-30 行)
- 提 PR，全员 review
- 不建议一次性把团队所有规矩都塞进去——先最硬的几条

第 3 周：建团队审核约定

- 讨论哪些目录 / 文件 必须 L3 / L4 审核
- 哪些绝不能让 AI 动
- 写进 CLAUDE.md 或单独文档

第 4 周：扩展 + 共享

- 试点成员把已经好用的 Custom Commands / Skills 放到 `.claude/commands/`
`.claude/skills/`（项目级）
- 全员 pull 下来就能用
- 收集反馈、微调

30 天后的常态

- 所有成员都用上
- CLAUDE.md 每月维护一次
- 新 Skill / Command 走 PR
- 月度回顾：“哪些用得好、哪些没人用、CLAUDE.md 要不要砍”

本章小结

- 共享 vs 私有：项目级目录下的所有东西共享、个人设置 + 凭证不共享
- `.gitignore` 第一件 事 配 好 —— `.claude/settings.local.json` / `.claude/mcp_servers.json`

- CLAUDE.md 走 PR，每季度整理，硬性上限防膨胀
- 三个典型坑：密钥进 git / 标准不一致 / 扩展不共享——各有预防方法
- git 和 Claude 搭档：重改前 commit / 分支试 / 先看状态 / 让 Claude 写 commit message / 标记 AI 参与
- 30 天接入路线：第 1 周试点、第 2 周初版 CLAUDE.md、第 3 周审核约定、第 4 周扩展共享

动手任务

任务 1：给你的项目配 .gitignore (5 分钟)

步骤：

1. 打开（或新建）项目根目录的 .gitignore
2. 加入 25.1 的推荐行
3. `git status` 检查是否还有敏感文件被追踪；有的话用

```
git rm --cached <文件> 移除（文件保留本地）
```

成功标志：`git status` 里看不到任何含凭证或个人配置的文件。

任务 2：扫一下你现在的 git 历史 (10 分钟)

步骤：

1. 在 `项` `目` `根`
跑：`!git log --all --full-history -- .claude/ 2>/dev/null | head -50`
2. 看有没有历史 commit 涉及敏感配置

3. 如果有 → 警觉：去对应服务 revoke 所有 token，重建

成功标志：确认你的 git 历史里没有未处理的敏感泄露。

任务 3：写一份团队 30 天接入计划 (15 分钟，如你要给团队引入)

步骤：照 25.5 的路线图，具体到人、具体到日期写出你团队的计划：

```
# 团队 Claude Code 接入计划
```

```
## 第 1 周 (4/22-4/28) 试点
```

```
- 试点成员：马力、李四
```

- 他们各自读本书 Part 1-2

第 2 周 (4/29-5/5) CLAUDE.md v0.1

- 5/2 前出初版 PR (马力负责起草)
- 全员 5/4 前 review + 合并

...

成功标志：你有了一份能给老板看的接入路线，不是空喊“我们要用 AI”。

如果你卡住了

症状 A：同事不愿意用，觉得 AI 靠不住

- 原因：合理——AI 有出错案例、别人可能听说过。
- 解决：低风险任务起步——让他们先用在“写周报草稿”这种最安全的任务，建立信任后再扩大。不要强推。

症状 B：团队 CLAUDE.md 改动总是引起争吵

- 原因：规则背后的价值观分歧。
- 解决：争议大的条目先不进——放到专门文档（docs/style.md）讨论，达成共识再移回 CLAUDE.md。

症状 C：密钥已经 push 了怎么办

- 原因：手滑或 .gitignore 晚了。
- 解决：两步走——① 立刻 revoke 泄露的 token / key；② 从 git 历史清除（用 git filter-repo 或 BFG Repo-Cleaner；力量不够就新建一个干净仓库重新 push）。历史不清干净，泄露的凭证永远在。

主线结束。下面支线讲“Claude Code 和 CI/CD 的配合”。

🌱 【支线】—— 可选深入（学有余力再看）

支线 25.A：Claude Code 和 CI/CD 的边界

这段讲什么：有人问“能不能把 Claude 接到 GitHub Action 里自动审 PR”——能，但有边界。什么时候回头读：你的团队想把 AI 深度集成进自动化流水线时。

能做什么

- PR 自动审核：每次开 PR 触发一次 Claude 审查，结果评论到 PR
- 自动生成 CHANGELOG：每次打 tag 让 Claude 生成发版说明
- 自动回复 issue：常见问题 Claude 先答，人工 review

具体怎么做：用 Anthropic API + GitHub Action，或用 Claude Code 的非交互模式（`claude -p "..."`）。

不该做什么

- Claude 自动 merge PR：永远要人审——merge 是不可逆的
- Claude 直接部署：部署事故成本高，人工批准 > 自动
- Claude 改生产配置：永远不要给自动化流程改生产权限

一个踏实的入口

从低风险 + 易观察的开始：

- Claude 自动给 PR 打 summary 评论（只说看到了什么，不给评判）
- 观察几周，调整 prompt
- 质量稳定后再加审查评判
- 永远不自动化不可逆的事

和本书定位的关系

本书是零基础入门——CI/CD 集成是进阶话题。先把交互式的 Claude Code 用熟、团队协作建好，再考虑 CI。

下一章：第 26 章“进阶能力速览”——指路牌性质。告诉你“还有这些更深的东西存在”，但不教——你读完本书已经达到 80% 场景需求，剩下 20% 给你指路。

第 26 章：进阶能力速览——指路牌

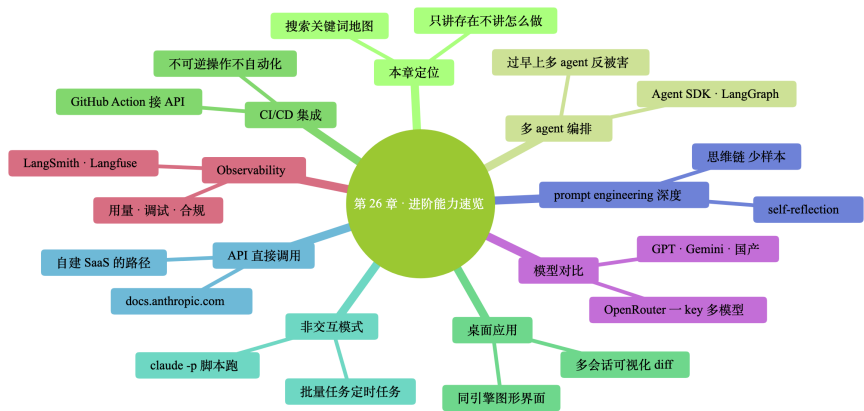
本章在全书的位置：第六部分 · 第 26 章 / 预估阅读+动手时长：主线 15-20 分钟 + 支线 5 分钟

前置章节：本书所有前面章节的概貌

学完能做什么（主线）：知道除了本书教的之外还有哪些进阶能力存在——够你在需要时有搜索关键词和大致路径，不会“听说过但不知道叫什么”。

开场三问

- 你会遇到什么问题：读完全书你已能覆盖 80% 日常——但剩下 20% 高阶场景没碰过，连叫什么都不知道。
- 不读这章会踩什么坑：某天需要做某种进阶的事——“感觉 Claude 可以做但不知道从哪查”，白走弯路。
- 读完你会多会什么事：有一张进阶能力地图——遇到新需求能定位到对应的进阶区域，搜对关键词、找对文档。



【主线】—— 本章必读核心

26.1 本章不教你“怎么做”——只告诉你“有这些”

每一小节只讲 3 件事：

1. 是什么（1-2 句）
2. 什么场景会想到它
3. 想学去哪里

别指望本章看完你就会——这是目录 / 索引 / 搜索关键词库。

26.2 进阶方向 1：多 Agent 编排

是什么

让多个 AI agent 协作完成复杂任务——比 Subagent（Ch 22）更复杂的协作模式。

什么场景

- 长链任务：需求分析 → 架构设计 → 代码生成 → 测试编写 → 文档生成——每步一个专门 agent
- 对抗式 review：两个 agent 互相挑对方的毛病
- 模拟会议：不同“人设”的 agent 讨论一个方案

关键词 / 资源

- Claude Agent SDK（Anthropic 官方）
- LangGraph（多 agent 工作流框架）
- CrewAI、AutoGen、MetaGPT（各种多 agent 框架）

本书立场

只有当 subagent 不够用时才考虑多 agent 编排。很多团队过早上多 agent，结果是一堆 agent 互相干扰反而比单个 Claude 做得差。

26.3 进阶方向 2：CI/CD 集成

是什么

把 Claude 接到 GitHub Actions / GitLab CI / Jenkins 等自动化流水线——让每次 PR / 每次部署自动触发 Claude 做事。

什么场景

- 每次 PR 自动审查、生成摘要
- 自动生成 CHANGELOG、release notes
- 自动回复常见 issue
- 自动补充测试用例

关键词 / 资源

- Anthropic API（不是 Claude Code，是底层 API）
- Claude Code non-interactive mode (`claude -p "..."`)
- GitHub Action 模板：搜“claude-code-action”

本书立场

见 Ch 25.A——可以做，但永远不要让 AI 执行不可逆的自动化。

26.4 进阶方向 3：Claude Code 桌面应用（2026-04 大改版）

是什么

和本书教的“终端版”Claude Code 同一引擎，但图形界面——多会话侧栏、拖拽面板、可视化 diff、内嵌终端、git worktree 隔离。

什么场景

- 你本质上想要 Claude Code，但受不了终端
- 需要同时管理多个 Claude 会话（比如在三个分支上并行探索）
- 需要可视化的 diff 审核（大型改动时）

关键词 / 资源

- 下载：claude.com/download

- 本书附录 H 有简明导览

本书立场

你读完本书学到的所有概念（权限、上下文、CLAUDE.md、三层记忆、五种扩展）在桌面应用里一样适用——只是界面不同。无缝切换。

26.5 进阶方向 4：非交互模式 / 脚本化

是什么

Claude Code 可以不开对话、直接跑一个任务：

```
claude -p "用 meeting-notes Skill 整理 @~/meetings/2026-04-19.txt"
```

跑完就退出，输出结果——适合嵌到脚本、自动化、批量处理。

什么场景

- 批量处理：循环跑 100 个文件
- 定时任务：每天早上 7 点自动整理昨日日报
- shell 脚本里的一环

关键词 / 资源

- `claude --help` 查看所有命令行参数
- `claude -p` (print mode)
- 写 bash 脚本把多个 Claude 调用串起来

本书立场

99% 零基础读者用不到——但知道这种模式存在，以后想自动化时不至于懵。

26.6 进阶方向 5：Anthropic API 直接调用

是什么

Claude Code 是 Anthropic 基于自家 API 做的一个工具。你也可以直接用 API 写自己的程序——把 Claude 集成进任何你自己的软件。

什么场景

- 想做一个自己的 AI 应用（SaaS 产品、内部工具）
- 想让非终端用户也能用 Claude 能力
- 想深度定制交互方式

关键词 / 资源

- docs.anthropic.com
- SDK: Python、TypeScript、Go、Java
- Tool use / Agent SDK

本书立场

这就是程序员的工作——非程序员用不到。但如果你未来学了编程，这是一条连续路径：Claude Code 上手 → 理解 AI → 自己调 API 做东西。

26.7 进阶方向 6: prompt engineering 深度

是什么

怎么写 prompt 本身是一门学问。本书讲了基本技巧（WHAT-WHERE-HOW-VERIFY、角色设定、明确约束等），但还有更深的：

- Chain-of-Thought（思维链）：让模型 step-by-step 展开思考
- Few-shot learning（少样本）：给几个例子让它模仿
- Self-reflection：让它自己审自己的答案
- Constitutional AI 原则：建立“宪法式”行为约束

什么场景

- Claude 在某类任务上反复表现不够好——调 prompt 可能比换模型更有效
- 做高要求的内容（专业写作、精确推理）
- 做教育 / 培训产品

关键词 / 资源

- Anthropic 官方 [prompt engineering 文档](#)
- 社区: learnprompting.org
- 书: 《The Prompt Report》等

本书立场

本书 + 官方 prompt engineering 文档 $\approx$ 覆盖 95% 需要。剩下 5% 是研究性工作。

26.8 进阶方向 7：模型对比 + 模型切换

是什么

Claude 不是唯一的。其他大模型：GPT (OpenAI)、Gemini (Google)、Grok (xAI)、Qwen / DeepSeek / Kimi (国产)。

各有擅长——不同任务用不同模型可能效果更好。

什么场景

- Claude 在某领域稍弱，听说 GPT 更好——想对比
- 国内网络不稳定 → 用国产模型做备份
- 成本敏感 → 不同模型性价比不一样

关键词 / 资源

- LMSYS Chatbot Arena：模型对比排行榜
- OpenRouter：一个 key 调多家模型
- Cursor / Cline 等 AI 编程工具：多模型支持

本书立场

本书不多评价其他模型——但知道“有得选”，就知道 Claude 不是唯一答案。真正的专业用户经常同时用多个模型。

26.9 进阶方向 8：Observability / 日志追踪

是什么

把 Claude 的每一次调用都记下来——看花了多少 token、响应多久、有没有出错。

什么场景

- 团队用量审计
- 调试：某次 Claude 回答奇怪，回溯当时的完整 prompt + 回复

- 合规：证明“这件事是人工 review 后做的”

关键词 / 资源

- Claude Code 的 log 目录 (`~/claude/logs/`)
- 社区工具：LangSmith、Langfuse、Phoenix——AI observability 平台
- 自建：Hook + 数据库

本书立场

个人用户基本不需要。企业场景合规可能要。

26.10 你的进阶路线图

根据你最关心什么，推荐下一步：

你最想	先看哪个
把 Claude 用得更顺	重读 Ch 14 (CLAUDE.md) + Ch 20-24 (五扩展)
在团队里推广	Ch 25 + 附录 E FAQ
做自动化 / CI	本章 26.3 (CI/CD) + 26.5 (脚本化)
做产品 / 自建应用	本章 26.6 (API) + 官方文档
学得更深、做研究	本章 26.7 (prompt) + 26.8 (多模型)
就是想继续进阶用	下一章 Ch 27 下一步——推荐资源 + 挑战清单

本章小结

- 本章是指路牌，不是教学
- 8 个进阶方向：多 agent 编排 / CI/CD / 桌面应用 / 非交互模式 / API 直调 / prompt engineering / 模型对比 / Observability
- 每个方向给了关键词 + 资源 + 本书立场
- 按你最关心什么挑一个方向深入

动手任务

任务 1：选你的下一个“进阶方向”（10 分钟）

步骤：看 26.10 的表，问自己：

- 当下我最想解决的问题是什么？
- 对应哪个方向？
- 我决定接下来花多久探索它？（1 周 / 1 月 / 3 月）

成功标志：你有了一个具体的下一步，不是“读完没事做”。

任务 2：收藏关键资源（5 分钟）

建一个书签文件夹“Claude Code 进阶”，放入：

- docs.anthropic.com
- claude.com/download（桌面应用）
- 你对应进阶方向的主要资源（见本章每节）

成功标志：需要时不用重新搜——直接点开。

如果你卡住了

症状 A：所有方向都想学

- 原因：正常——全是新世界。
- 解决：一次只学一个。选最能解决你当前问题的那个。其他先放一边。

症状 B：选不出哪个最重要

- 原因：本书读完后还没真用起来。
- 解决：先不选进阶——回去把本书教的真正用 1-2 个月，遇到瓶颈再来看进阶。

症状 C：觉得所有进阶都太技术、自己学不会

- 原因：零基础焦虑再次发作。
 - 解决：不是所有进阶都需要编程——CLAUDE.md 维护、团队协作、扩展使用都不要求写代码。先深化你已经会的，别勉强自己学“看起来牛”但用不到的东西。
-

主线结束。下面支线讲“AI 工具格局未来 1-2 年”——让你对趋势有个感觉。



【支线】—— 可选深入（学有余力再看）

支线 26.A：AI 工具未来 1-2 年的几个趋势

这段讲什么：不是预测未来——是告诉你有哪些变化正在发生，好让你不被突袭。什么时候回头读：你想“这东西会不会很快过时”时。

趋势 1：Agent 化——从“chat bot”到“能干活的助手”

AI 从“你问一句它答一句”变成“你说大方向它自己一步步做完”。Claude Code 本身就是这个趋势的代表。

对你的影响：

- 不要只学 prompt 技巧——要学“怎么和 agent 协作”
- 信任校准、权限管理、验证这些能力越来越重要

趋势 2：多模态——文本 → 图片 → 声音 → 视频

模型能看图（已经能）、能看视频、能听声音、能生成声音。

对你的影响：

- 截图给 AI 已经成常态——本书 Ch 19 讲了
- 未来：直接对着屏幕说话让 AI 操作
- 新场景：让 AI 看一段视频总结、听一段录音整理

趋势 3：本地模型 + 云端模型混合

本地开源模型（Llama、Qwen、DeepSeek）越来越强。但云端模型（Claude / GPT-5 等）仍然最聪明。

未来格局：

- 日常简单事 → 本地跑（隐私、免费、离线）
- 复杂深度事 → 云端跑（最强能力）
- 自动判断用哪个 = 下一代工具的亮点

趋势 4：MCP 生态爆发

第 24 章讲的 MCP——这是跨 AI 工具的统一协议。一旦生态起来：

- 你装一次 MCP，在 Claude / Cursor / 任何支持 MCP 的工具里都能用
- 公司提供 MCP server 成标配——就像以前提供 API
- 本书教你的 MCP 知识跨工具迁移

趋势 5：AI 变基础设施

像电和互联网——早期是稀奇玩意，最后成了水电煤。

- 现在：用 AI 是“本事”
- 两年后：不用 AI 才是奇怪的

对你的影响：

- 现在学 AI 工具 = 抢先
- 但别把 AI 工具本身当目的——目的始终是你要完成的工作
- 学怎么用好比追最新工具重要

趋势 6：价格下降 + 能力提升

AI 模型价格每年跌一个数量级，但能力每年涨 1-2 倍。

这意味着：

- 今年你觉得贵的任务，明年可能随便跑
- 今年你觉得 AI 做不好的任务，明年可能它专门擅长

对你的影响：

- 不要因为“现在某任务贵 / 某任务做不好”就放弃——明年再试
- 持续关注最新能力 $\approx$ 每 3-6 个月看一次 changelog

稳住心态的建议

- 不要为了追新而追新——先把现在已有的工具用熟
- 基本功重于花样——本书教的 prompt、权限、上下文、记忆，未来依然有效
- 选 1-2 个核心工具深入，不要每个都浅尝

下一章：第 27 章“下一步路线”——本书最后一章。给你一份持续成长的实操路线：推荐资源 + 学习节奏 + 30 天挑战清单，让你读完书不是结束而是起点。

第 27 章：下一步路线——读完书只是起点

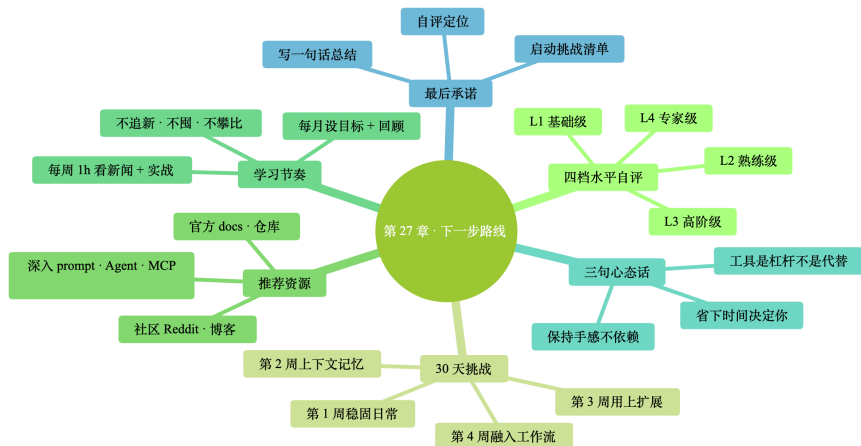
本章在本书的位置：第六部分 · 第 27 章 / 预估阅读+动手时长：主线 15-20 分钟 + 支线 5 分钟

前置章节：读完整本书

学完能做什么（主线）：拿到一份可落地的持续成长路线图——30 天挑战清单 + 推荐资源 + 学习节奏——让你读完不会“不知道怎么继续”。

开场三问

- 你会遇到什么问题：书读完了、技巧都会了，但真正习惯没养成——用一阵又回到旧工作方式。
- 不读这章会踩什么坑：所学知识没机会“生根”——3 个月后忘光。
- 读完你会多会什么事：30 天让 Claude Code 变成习惯——从“每次要想起来用”到“不用就不自在”。



🎯 【主线】—— 本章必读核心

27.1 先认识自己现在的水平

把本书知识转化成“能力档”，简单自评：

L1：基础级（读完 Part 1–2 + 少量动手任务）

- 能独立启动 Claude Code
- 会 @ 引用文件、审 diff、基本权限决策
- 会 `/clear`、`/compact`

L2：熟练级（完成大部分动手任务 + 日常使用 2 周以上）

- 能主动用 `/plan` `/rewind`
- 有自己的信任档案 + 止损规则
- 写过至少一版可用的 CLAUDE.md
- 有模型切换意识 (Sonnet / Opus)

L3：高级级（读完全书 + 扩展能力实际用起来）

- 至少用过 1 个 Skill / Command / Subagent 之一

- 装过至少一个 MCP
- 已经为团队或个人工作流程节省了时间

L4: 专家级 (用 Claude Code 改变了工作方式)

- Claude Code 嵌到每日节奏里
- 已经帮别人入门或主导团队接入
- 对扩展 / 定制有深度实践

你现在大概在哪一档？这决定了下面挑战清单怎么用。

27.2 30 天挑战清单

目标：30 天后达到 L3（高级别）。

第 1 周：稳固日常 (L1 → L2 过渡)

每天 20-40 分钟，连续 7 天：

- ☐ Day 1: 早上用 `/daily` 命令列今日任务
- ☐ Day 2: 让 Claude 整理至少 5 个文件（重命名 / 归档）
- ☐ Day 3: 让 Claude 写一份会议纪要
- ☐ Day 4: 改一个真实文档（日报 / 邮件 / 博客）
- ☐ Day 5: 用 `/plan` 做一个稍大的任务
- ☐ Day 6: 遇到错误时用 `/rewind` 而不是手动修
- ☐ Day 7: 回顾一周——哪件事最省时？哪件事最卡？

第 2 周：管理好上下文和记忆

- ☐ Day 8: 写你的第一版 CLAUDE.md (< 40 行)
- ☐ Day 9: 用了 `@ Tab` 自动补全至少 5 次
- ☐ Day 10: 主动 `/compact` 了至少 2 次
- ☐ Day 11: 每次完成任务就 `/clear`，养成习惯
- ☐ Day 12: 截图给 Claude 解读一次
- ☐ Day 13: 查一次 `/status` 和用量页，看自己花多少钱
- ☐ Day 14: 回顾——CLAUDE.md 有没有改进空间？

第 3 周：用上扩展

- ☐ Day 15: 装一个现成 Skill（本书 20.3 的 meeting-notes 或你找的别的）
- ☐ Day 16: 写一个你自己的 Custom Command

- ☐ Day 17: 写你的第二个 Command（已经找到规律了）
- ☐ Day 18: 建一个最简 Subagent（按 Ch 22.3）
- ☐ Day 19: 配一个 hook（按 Ch 23.6 模板）
- ☐ Day 20: 装一个 MCP（filesystem / GDrive / GitHub 择一）
- ☐ Day 21: 回顾——这周哪个扩展最值？哪个没用上？

第 4 周：融入工作流

- ☐ Day 22: 把本周至少 3 个重复任务做成模板（prompt 或 Command）
- ☐ Day 23: 用 Claude Code 做一件“平常不会用 AI 做”的事
- ☐ Day 24: 教一个朋友 / 同事启动 Claude Code
- ☐ Day 25: 在团队群或朋友圈分享一次你的使用经验
- ☐ Day 26: 写一篇自己的“Claude Code 使用心得”（200 字也行）
- ☐ Day 27: 整理你的 CLAUDE.md、Skills、Commands——砍掉不用的
- ☐ Day 28: 大回顾——30 天用了多少次？节省了多少时间？
- ☐ Day 29: 设定下 30 天的一个主目标（进一步精通某领域 / 推广给团队 / 做特定工具）
- ☐ Day 30: 休息 + 准备下一阶段

回顾的三个关键问题

1. 这 30 天，Claude Code 帮我做成了哪件之前做不好 / 做不了的事？
2. 还有哪类任务我没让 AI 碰？为什么？（判断是合理 vs 是惯性）
3. 下 30 天我要在哪个方向进一步？

27.3 推荐资源

官方

1. Anthropic 官方文档: docs.anthropic.com
 - 最权威、更新最及时
 - 英文为主，浏览器自带翻译能看懂
2. Claude Code 官方仓库 (GitHub):
 - 搜 “claude-code”
 - Issue + Discussion 区有大量真实问题解答
3. Anthropic 官方 Blog / Cookbook:
 - Blog: 产品重大更新

- Cookbook：具体用法示例

社区资源

1. 英文：

- Reddit r/ClaudeAI：用户社区、经验分享
- HumanLayer 的 CLAUDE.md 文章（本书 Ch 14 很多原则来自这里）
- awesome-claude-code 类 GitHub 仓库：工具 / Skill / Command 合集

2. 中文：

- 国内开发者博客里的 Claude Code 实践
- B 站 / 微信公众号的入门教程（注意时效——Claude Code 迭代快）
- 群 / Discord：各大 AI 社区都有 Claude Code 相关讨论

深入学习

1. prompt engineering：

- Anthropic 官方 prompt engineering 指南
- learnprompting.org

2. Agent SDK（若你想自己构建）：

- Anthropic Agent SDK 文档
- Tool use 示例

3. MCP：

- MCP 官方文档 (modelcontextprotocol.io)
- awesome-mcp-servers

本书未涵盖但值得知道

- Claude Cowork（如果你是非编程工作者，反而更适合）
- Claude 桌面应用（本书附录 H 有简介）
- Anthropic 商业产品线（企业版 / Enterprise）

27.4 学习节奏建议

每周节奏

- 1 小时：刷最近一周 Anthropic 产品更新 + 1-2 个社区帖子
- 3-5 小时：真实项目里用 Claude Code
- 偶尔：试一个新扩展 / 新 MCP

每月节奏

- 月初：设一个“本月想提升哪方面”的目标
- 月末：回顾——做到了吗？用量如何？
- 季度：整理 CLAUDE.md、清理不用的扩展

不要做的

- **✗** 每天刷最新功能——工具迭代太快，天天追会焦虑
- **✗** 囤工具不用——装了 10 个 MCP 平时只用 1 个
- **✗** 追“最 pro”设定——你用得顺比配置好看更重要
- **✗** 和别人比——不同角色、不同需求，不可比

27.5 关于 AI 时代的工作心态

最后给你三句话——超越 Claude Code 本身：

1. 工具是杠杆，不是代替

Claude Code 让你做得更快更多——但做什么、要达到什么目的、质量标准仍然是你的事。

AI 不会替你思考人生目标。别指望工具回答“我该做什么人”。

2. 保持手感

太依赖 AI 会让基础能力生锈：

- 写作能力
- 思考能力
- 动手的耐心

建议：每周至少一项工作完全不用 AI。保持“没工具也能做”的手感。

3. AI 帮你省时间——但省下的时间你怎么用决定你成为什么样的人

省下 2 小时你拿去刷短视频 → 你是“刷短视频更多的人”
省下 2 小时你拿去学习 / 锻炼 / 陪家人 → 你是“更丰富的人”

工具放大一切——放大你的好习惯和坏习惯。

本章小结

- 4 档水平：L1 基础 / L2 熟练 / L3 高阶 / L4 专家——自评你现在在哪
- 30 天挑战清单：第 1 周稳固日常 → 第 2 周管理上下文记忆 → 第 3 周用扩展 → 第 4 周融入工作流
- 推荐资源：官方优先 + 社区补充；中文资源注意时效
- 学习节奏：每周 1 小时看新闻 + 3-5 小时实战；每月目标 + 回顾；不要追最新 / 囤工具 / 和人比
- 工具是杠杆不是代替；保持手感；省下的时间怎么用决定你成为什么样的人

动手任务（最后一个!）

任务 1：自评 + 定位（5 分钟）

照 27.1 四档判断你现在在哪：_____

任务 2：启动 30 天挑战（5 分钟）

步骤：

1. 在日历上标 30 天后的日期
2. 把 27.2 的 30 个 Day 任务贴到备忘录 / Notion / 任何你每天都看的地方
3. 从明天开始，每完成一个打勾
4. 30 天后回来回顾

成功标志：你有了一份可打勾的挑战清单——不是空喊。

任务 3：写一句话总结（10 分钟）

步骤：写下一句话——

“我读完这本书后，打算用 Claude Code 改变的一件事是：_____”

可以写得很小（“每天用它写日报”）也可以写得远（“帮团队全面接入 AI”）——只要是具体的。

成功标志：你有了一句清晰的承诺。每次打开 Claude Code 前默念一次。

如果你卡住了

症状 A：30 天清单做不完，觉得挫败

- 原因：你的节奏比清单慢——正常。
- 解决：延长到 60 天甚至 90 天——重要的是每天进步一点，不是赶进度。

症状 B：读完书但没感觉“真的学到了”

- 原因：读和做之间有鸿沟，你可能没真的做动手任务。
- 解决：回头补动手任务——哪怕只做每章的第一个。读 10 章不如做 3 章。

症状 C：工作中没机会用

- 原因：工作内容没什么“AI 能帮上的”。
 - 解决：从个人生活开始用——整理照片、写旅行日记、规划周末、改私人博客。熟了再用到工作上。
-

写在最后

你从不知道终端是什么读到能用扩展 + MCP 打造自己的 AI 工作流——这一路不容易。

- 你建立了一整套对新工具的判断力：怎么信、怎么验证、怎么止损
- 你学会了和 AI 协作的基本礼仪：给上下文、审改动、管权限、防泄密
- 你有了一份活的 CLAUDE.md、一个信任档案、一个止损规则——这些是你的财富

这本书的使命到这里结束了，你的使命正式开始。

去做你想做的事吧——现在你有了一个永不疲倦、愿意学习任何任务、每天变更聪明的伙伴。

好好用它。

 【支线】—— 可选深入（学有余力再看）

支线 27.A：书没写的那些

这段讲什么：本书有意没写的东西，以及为什么。什么时候回头读：你好奇“本书是不是避开了某些话题”时。

1. 没深入 Linux

覆盖 Mac + Windows，但 Linux 用户必须自行类推。原因：

- 零基础读者基本没人只用 Linux
- Linux 用户通常有较强自学能力
- 省篇幅和认知负担

2. 没详细讲编程

本书假设读者不会编程。所以：

- 示例偏日常（整理文件、写文档）
- 涉及代码的章节（Ch 22 代码审查 subagent）只做演示
- 不教 Python / JavaScript 任何语言

想学编程 → 另找教程（本书不替代）。

3. 没写 Linux / Android / iOS 平台

只覆盖主力桌面系统。移动端 Claude 有独立产品线（Claude.ai App、Cowork），不在本书范围。

4. 没深入“AI 伦理”

没展开讨论：

- AI 会抢工作吗
- AI 的偏见和公平性
- AGI / 超级智能

这些重要，但和“入门 Claude Code”关系不大。想深入这些话题，去看专门的书（如《AI 超级大陆》《超越人类》等）。

5. 没给“完整作业”

没有“用 Claude Code 做一个 Todo App”这种端到端作业。原因：

- 零基础读者目标多样，一个示例作业满足不了所有人

- 本书重心智模型 + 判断力，而不是“照抄一个项目”
- 真正的作业是你自己的生活 / 工作任务

6. 没推荐具体付费订阅

模糊说了“Pro / Max / API”——没给“你应该买 X”。原因：

- 价格 / 套餐会变
- 每个人需求不同
- 本书不接受广告，不做购买引导

附录开始——下一部分是 9 个附录，查阅型内容：Mac/Windows 速查、slash 命令全表、决策流程图、FAQ、术语表、报错自救、桌面应用导览、延伸阅读。用到的时候翻。

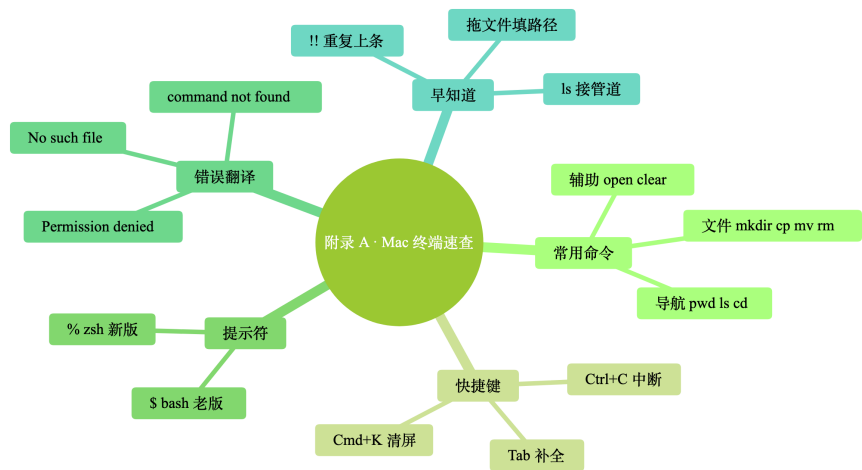
祝你用得愉快。

——马力 于 2026 年春 li.ma.aria@gmail.com

附录

附录 A：Mac 终端速查

用法：打开终端.app（Spotlight 搜“终端”），按速查表敲命令。



最常用 15 个命令

命令	干什么	例
<code>pwd</code>	看我现在在哪个文件夹	<code>pwd</code>
<code>ls</code>	列当前文件夹里的东西	<code>ls</code>
<code>ls -la</code>	连隐藏文件和详细信息都列	<code>ls -la</code>
<code>cd 文件夹名</code>	进入某个文件夹	<code>cd Documents</code>
<code>cd ..</code>	上一层文件夹	<code>cd ..</code>
<code>cd ~</code>	回自己的用户文件夹（家）	<code>cd ~</code>
<code>cd -</code>	回上次那个文件夹	<code>cd -</code>
<code>mkdir 名字</code>	建一个新文件夹	<code>mkdir notes</code>
<code>touch 名字</code>	建一个空文件	<code>touch todo.md</code>
<code>cp 源 目标</code>	复制	<code>cp a.txt b.txt</code>
<code>mv 源 目标</code>	移动 / 重命名	<code>mv a.txt b.txt</code>
<code>rm 文件</code>	删除（小心，不进回收站）	<code>rm old.txt</code>
<code>rm -rf 文件夹</code>	删文件夹及里面所有内容（最危险，再三确认）	<code>rm -rf temp/</code>
<code>open .</code>	在 Finder 打开当前文件夹	<code>open .</code>
<code>clear</code>	清屏	<code>clear</code>

最常用快捷键

快捷键	作用
<code>Tab</code>	自动补全文件名 / 命令
<code>↑ / ↓</code>	翻命令历史
<code>Ctrl+A</code>	光标跳到行首
<code>Ctrl+E</code>	光标跳到行尾
<code>Ctrl+C</code>	中断当前运行的命令
<code>Ctrl+D</code>	退出当前 shell
<code>Ctrl+L</code>	清屏（等同 <code>clear</code> ）
<code>Cmd+T</code>	新标签页
<code>Cmd+N</code>	新窗口
<code>Cmd+K</code>	清屏并清历史（彻底清）
<code>Cmd+加号 / 减号</code>	字号变大 / 变小

提示符号速读

- `%`（zsh，新版 Mac 默认）：等着你打字
- `$`（bash，老 Mac）：一样意思
- 看到 `~` = 你在自己的用户文件夹

常见错误翻译

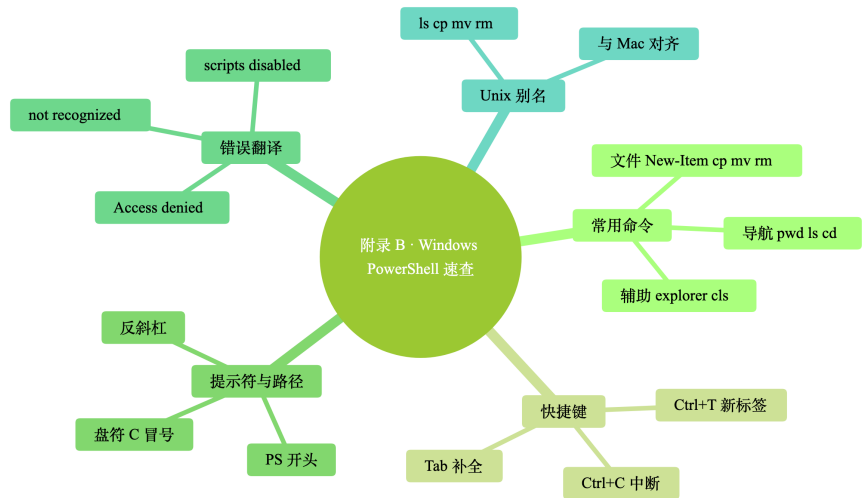
看到	意思	怎么办
<code>command not found: xxx</code>	没装 xxx 这个程序	先 装 （ 比 如 <code>brew install xxx</code> ）
<code>No such file or directory</code>	找不到这个文件 / 文件夹	<code>pwd</code> 看你在哪、 <code>ls</code> 看有没有
<code>Permission denied</code>	权限不够	加 <code>sudo</code> （慎用）；或 <code>chmod +x 文件</code>
<code>zsh: parse error</code>	命令打错了（缺引号 / 括号）	仔细看命令，重新敲
<code>Operation not permitted</code>	Mac 的 SIP 保护拦的	该目录是系统目录，别动

一些“我希望早点知道”

- 拖文件进终端 → 自动填入完整路径（省去手打）
- `ls | less`（带竖线）→ 列表太长时分页看，按 `q` 退出
- `!!` → 重复上一条命令（装完 `sudo` 忘了加，敲 `sudo !!` 直接补）
- `cd` 单独一个 → 等同 `cd ~`
- `open 文件.pdf` → 用默认程序打开这个文件

附录 B：Windows PowerShell 速查

用法：按 Win + X 选“终端”（Windows 11）或“PowerShell”。本书不推荐 WSL / CMD——统一用 PowerShell。



最常用 15 个命令

命令	干什么	例
<code>pwd</code>	看我在哪个文件夹	<code>pwd</code>
<code>ls</code>	列当前文件夹	<code>ls</code>
<code>ls -Force</code>	连隐藏文件都列	<code>ls -Force</code>
<code>cd 文件夹名</code>	进去	<code>cd Documents</code>
<code>cd ..</code>	上一层	<code>cd ..</code>
<code>cd ~</code>	回用户文件夹	<code>cd ~</code>
<code>mkdir 名字</code>	建文件夹	<code>mkdir notes</code>
<code>New-Item 名字</code>	建空文件	<code>New-Item todo.md</code>
<code>cp 源 目标</code>	复制	<code>cp a.txt b.txt</code>
<code>mv 源 目标</code>	移动 / 重命名	<code>mv a.txt b.txt</code>
<code>rm 文件</code>	删除 (小心, 不进回收站)	<code>rm old.txt</code>
<code>rm -r 文件夹</code>	删文件夹及内容	<code>rm -r temp</code>
<code>explorer .</code>	在资源管理器打开当前	<code>explorer .</code>
<code>cls</code>	清屏	<code>cls</code>
<code>notepad 文件</code>	用记事本打开	<code>notepad todo.md</code>

最常用快捷键

快捷键	作用
<code>Tab</code>	自动补全
<code>↑ / ↓</code>	翻历史
<code>Home</code>	光标跳行首
<code>End</code>	光标跳行尾
<code>Ctrl+C</code>	中断
<code>Ctrl+L</code> 或 <code>cls</code>	清屏
<code>Ctrl+T</code>	新标签页 (Windows Terminal)
<code>Ctrl+Shift+N</code>	新窗口
<code>Ctrl+加号 / 减号</code>	字号

提示符号速读

- `>` (如 `PS C:\Users\你>`): PowerShell 等你打字
- `PS` = PowerShell
- 路径用反斜杠 `\` (和 Mac 的 `/` 不一样)

常见错误翻译

看到	意思	怎么办
The term 'xxx' is not recognized	没装或没在 PATH 里	先装; 重开终端
Cannot find path 'xxx' because it does not exist	路径错	<code>pwd</code> 和 <code>ls</code> 核查
Access to the path 'xxx' is denied	权限不够	以管理员身份运行 PowerShell
... is currently disabled on this system	脚本执行政策限制	管理员 PowerShell 跑 <code>Set-ExecutionPolicy RemoteSigned</code> (理解后再做)
... cannot be loaded because running scripts is disabled	同上	同上

Windows 特别要注意的路径差异

- 反斜杠 `\` (不是 `/`)
- 盘符 `C:\` `D:\` ——Mac/Linux 没有
- 空格路径用引号: `cd "Program Files"`
- 用户文件夹: `C:\Users\你的名字` (不是 `/Users/...`)

PowerShell 的“别名”

很多 PowerShell 原本的命令有 Unix 风格别名, 方便跨平台用户:

PowerShell 原生	别名 (Mac / Unix 风格)
Get-ChildItem	ls (也支持 dir)
Set-Location	cd
Copy-Item	cp (也支持 copy)
Move-Item	mv (也支持 move)
Remove-Item	rm (也支持 del)

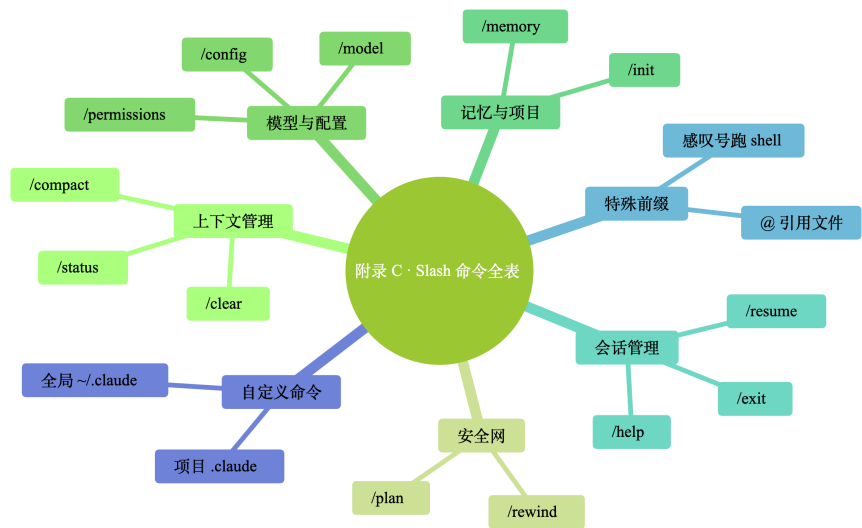
本书统一用 Unix 风格别名——和 Mac 版速查表对应一致。

一些“我希望早点知道”

- 拖文件进终端 → 自动填路径（和 Mac 一样）
- 以管理员身份运行：右键 PowerShell 图标 → “以管理员身份运行”——某些系统命令需要
- `dir` 也能用（老 CMD 风格，PowerShell 兼容）
- `clip` 命令 → `echo "hello" | clip` 把输出复制到剪贴板

附录 C：Claude Code Slash 命令全表

用法：Claude Code 输入框里输 `/`，选命令。这里按用途分组。



上下文管理

命令	作用	本书哪讲
<code>/clear</code>	清空当前会话（Ctx 归零）	Ch 12
<code>/compact</code>	压缩对话为摘要（大幅降低 Ctx）	Ch 12
<code>/status</code>	看当前模型、Ctx%、用量	Ch 15

安全网

命令	作用	本书哪讲
<code>/plan</code>	进计划模式（只出方案不动手）	Ch 9
<code>/rewind</code>	精确回退到某个状态	Ch 9

模型与配置

命令	作用	本书哪讲
<code>/model</code>	切换模型（Sonnet / Opus / Haiku / Opus 1M）	Ch 15
<code>/config</code>	看 / 改 Claude Code 配置	Ch 18
<code>/permissions</code>	管理权限白名单	Ch 7

记忆与项目

命令	作用	本书哪讲
<code>/memory</code>	打开 MEMORY.md 管理界面	Ch 13
<code>/init</code>	自动生成 CLAUDE.md（不推荐）	Ch 14

会话管理

命令	作用	本书哪讲
<code>/help</code>	所有命令说明	Ch 18
<code>/exit</code>	退出 Claude Code	Ch 3
<code>/resume</code>	恢复上次会话	Ch 3
<code>/login / /logout</code>	登录 / 登出	Ch 2

特殊前缀（非 slash 但属于“特殊输入”）

前缀	作用	例	本书哪讲
<code>!</code>	直接跑 shell 命令	<code>!git status</code>	Ch 19
<code>@</code>	引用文件	<code>@CLAUDE.md</code>	Ch 5

常用组合套路

套路	步骤
Plan → Execute	<code>/plan</code> → 审方案 → 退出 plan → 执行
Opus 规划 + Sonnet 执行	<code>/model</code> Opus → 出方案 → <code>/model</code> Sonnet → 执行
中段压缩	Ctx 50% → <code>/compact</code> → 继续
Rewind 而非修补	Claude 出错 → <code>/rewind</code> → 重新描述

自定义命令

自己的 `/xxx` 命令放在:

- 全局: `~/.claude/commands/xxx.md`
- 项目: `<项目>/.claude/commands/xxx.md`

详见 Ch 21。

注意

- 具体命令名和行为可能因 Claude Code 版本而异
- 出现新命令 / 某命令消失 → 跑 `/help` 看当前版本全表
- 自定义命令会和内置一起在 `/help` 输出里出现

附录 D：决策流程图

本附录把书里反复出现的几个“该怎么选”汇总成流程图。图全部用 Mermaid 手绘风格绘制，GitHub 上直接渲染，可以直接右键保存图片或截图打印。

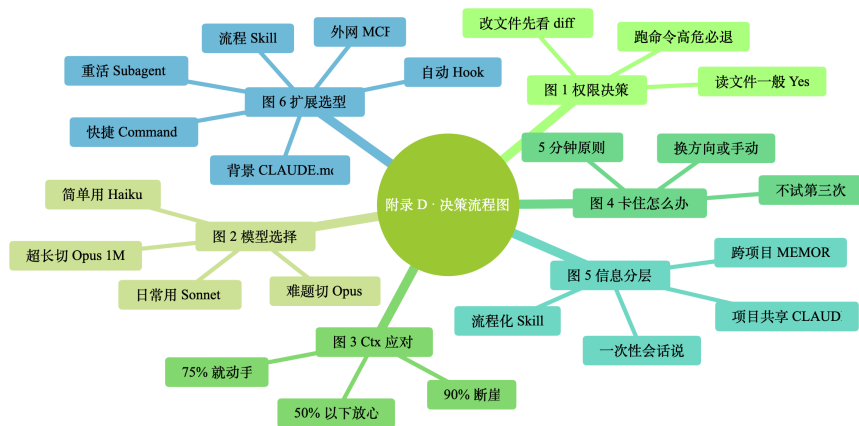


图 1：权限决策流程（遇到弹窗怎么选）

```
%%{init: {"look": "handDrawn", "theme": "neutral",
"themeVariables": {"fontFamily": "'Chalkboard SE','Comic Sans MS','Segoe Print','Kaiti SC','STKaiti','KaiTi','Bradley Hand',cursive", "fontSize": "15px"}}}%
flowchart TD
    A["Claude 弹出权限窗口<br/>它要干什么?"] --> B{"操作类型"}
    B -->| "读文件" | C["通常 Yes<br/>除非是 .env、密钥文件"]
    B -->| "改文件" | D{"先看 diff<br/>改的都是我想改的地方?"}
    B -->| "跑命令" | E{"是高危命令吗?<br/>rm / force push / 删表..."}
    D -->| "是" | F["Yes 接受"]
```

```

D →|"否"| G["No 退回让它改"]
E →|"否"| H["Yes 接受"]
E →|"是"| I["详细审一次再决定"]

```

口诀：

- 读文件 → 一般 Yes（除非是 .env、密钥文件）
- 改文件 → 永远看 diff，多改了就 No
- 跑命令 → 高危必退（详见附录 G）

图 2：模型选择流程（任务来了用哪个）

```

%%{init: {"look": "handDrawn", "theme": "neutral",
"themeVariables": {"fontFamily": "'Chalkboard SE','Comic Sans
MS','Segoe Print','Kaiti SC','STKaiti','KaiTi','Bradley
Hand',cursive", "fontSize": "15px"}}}%
flowchart TD
    A["新任务来了"] → B{"需要深度推理吗?"}
    B →|"否": 简单查询 / 翻译 / 重命名"| C["Haiku"]
    B →|"是"| D{"上下文会超 200K 吗?"}
    D →|"是"| E["Opus 1M"]
    D →|"否"| F{"日常活还是硬核活?"}
    F →|"日常 (80% 场景)"| G["Sonnet (默认)"]
    F →|"复杂推理 / 卡住了"| H["Opus"]

```

新手一句话：默认 Sonnet，卡住切 Opus，简单事用 Haiku。

图 3：Ctx% 使用率应对流程

```

%%{init: {"look": "handDrawn", "theme": "neutral",
"themeVariables": {"fontFamily": "'Chalkboard SE','Comic Sans
MS','Segoe Print','Kaiti SC','STKaiti','KaiTi','Bradley
Hand',cursive", "fontSize": "15px"}}}%
flowchart TD
    A["瞥一眼状态栏<br/>看 Ctx 百分比"] → B{"落在哪个区间?"}
    B →|"0 - 50%"| C["继续写，无需操心"]
    B →|"50 - 75%"| D["准备整理<br/>快完就收尾<br/>还长就 /
compact"]

```

```
B →|"75 - 90%"| E["立刻 /compact 或 /clear"]
B →|"超过 90%"| F["断崖区<br/>马上 /clear 或退出重启"]
```

口诀：看到 75% 就动手，别等 90%。

图 4：卡住了怎么办（5 分钟原则）

```
%%{init: {"look": "handDrawn", "theme": "neutral",
"themeVariables": {"fontFamily": "'Chalkboard SE','Comic Sans
MS','Segoe Print','Kaiti SC','STKaiti','KaiTi','Bradley
Hand',cursive", "fontSize": "15px"}}}%
flowchart TD
    A["第 1 次尝试失败"] → B["调整 prompt 再试一次"]
    B → C{"第 2 次还是不行?"}
    C →|"是"| D["触发 5 分钟原则<br/>先估算手动要多久"]
    D → E{"手动完成大概需要?"}
    E →|"5 分钟以内"| F["关掉 Claude<br/>自己动手做完"]
    E →|"20 分钟以上"| G["继续用 AI，换方向:<br/>/rewind 重开<br/>/compact 清包袱<br/>换模型 / 拆小任务"]
    G →|"5 - 20 分钟"| H["任选：自己做 或 换方向"]
```

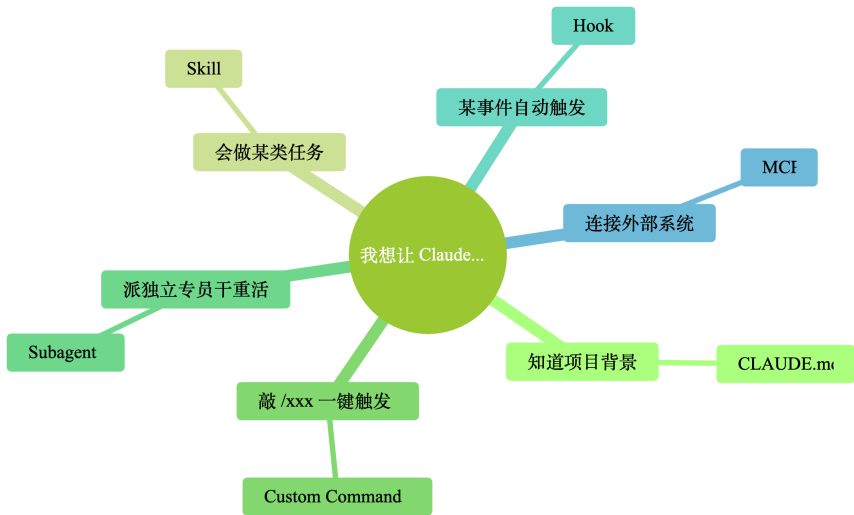
口诀：同一个 prompt 不要试第三次。

图 5：信息分层决策（该放在哪一层）

```
%%{init: {"look": "handDrawn", "theme": "neutral",
"themeVariables": {"fontFamily": "'Chalkboard SE','Comic Sans
MS','Segoe Print','Kaiti SC','STKaiti','KaiTi','Bradley
Hand',cursive", "fontSize": "15px"}}}%
flowchart TD
    A["这条信息..."] → B{"是什么性质?"}
    B →|"一次性用的"| C["会话里说就行<br/>不用存"]
    B →|"跨项目 + 关于我的"| D["MEMORY.md"]
    B →|"项目规则 + 团队共享"| E["CLAUDE.md"]
    B →|"结构化任务流程（偶尔用）"| F["Skill"]
    B →|"外部已有长文档"| G["在 CLAUDE.md 写 Pointer<br/>（详见 XXX）"]
```

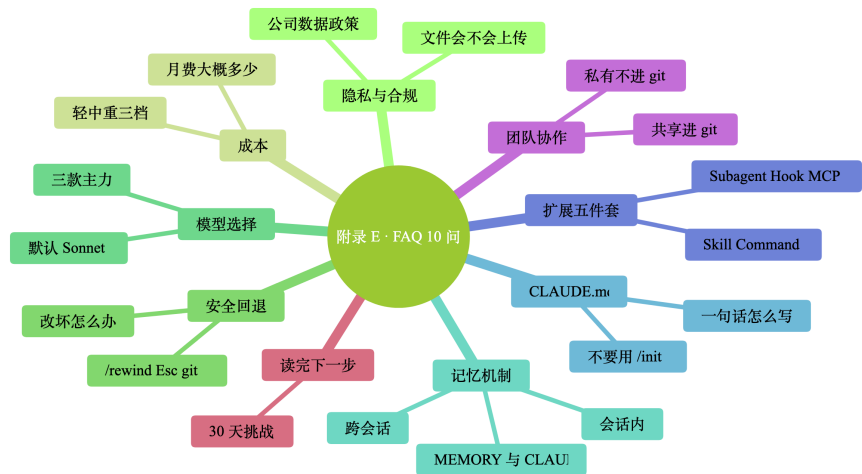
口诀：越私人越往 MEMORY，越团队越往 CLAUDE，越流程化越往 Skill。

图 6：扩展机制选型



口诀：CLAUDE.md 讲背景，Skill 讲流程，Command 给快捷键，Subagent 派替身，Hook 搞自动，MCP 接外网。

附录 E: FAQ (常见问题 10 问)



1. Claude 会偷看我的文件吗？

不会“偷”，但你 @ 引用或让它 !cat 的文件内容会上传到 Anthropic 服务器 (HTTPS 加密传输)，最多暂存 30 天后删除，不用于训练。

Claude 不会主动读你没让它读的文件——它只有工具权限，权限外动不了。详见 Ch 11。

2. 用 Claude Code 大概多少钱一个月？

按使用强度粗略：

- 轻度（每天 10-30 分钟）：\$10-20 / 月
- 中度（每天 1-2 小时）：\$50-150 / 月
- 重度（每天 3+ 小时）：\$200+ / 月

推荐订阅 Claude Pro 起步。详见 Ch 15。

3. 它改坏我的文件了怎么办？

三种挽回：

1. `/rewind`：回退到改之前
2. `Esc Esc`：快速撤销最近一步
3. `git`：`git checkout <文件>` 恢复到上次 commit

最可靠是重要任务前先 `!git commit -am "wip"` ——有回退锚点。

详见 Ch 9。

4. 我在公司用会不会违反数据政策？

先查公司 AI 政策。没有明文政策就发邮件问 IT / 合规。

永远别发：

- 密钥、token、密码
- 未脱敏的客户 / 员工信息
- NDA 覆盖的文档

合规场景 → 争取企业版（Zero Retention、私有部署）。详见 Ch 11。

5. 有哪些模型？怎么选？

三款主力：

- Haiku 4.5：快 + 便宜，简单查询
- Sonnet 4.6：中坚，日常默认
- Opus 4.7：最强 + 最贵，复杂推理

新手建议：默认 Sonnet、卡了切 Opus、简单事用 Haiku。 `/model` 随时换。详见 Ch 15。

6. Claude 会记住我上次说的话吗？

三层记忆：

- 会话内：记（到 `/clear` 或退出为止）
- 跨会话：看情况——放在 `MEMORY.md`（Claude 自动记你偏好）或 `CLAUDE.md`（你手写的规则）的内容会记
- 完全无记忆：每次新会话，你没明确存过的就不记

详见 Ch 13。

7. 一句话: CLAUDE.md 怎么写?

不要用 `/init`。手写, < 60 行:

- WHAT: 项目是什么 (1-2 句)
- WHY: 关键决策为什么这么选
- HOW: 3-5 条硬规则
- Pointers: 指向详细文档的链接

发现 Claude 重复犯错时再补条目。详见 Ch 14。

8. Skill / Command / Subagent / Hook / MCP 怎么分?

- Skill: Claude 按需加载的任务手册
- Command: 你主动敲 `/xxx` 触发
- Subagent: 派独立小 Claude 干重活
- Hook: 事件自动触发
- MCP: 接外部工具 (GDrive / Notion / DB)

五者可组合使用。详见 Part 5 (Ch 20-24)。

9. 我可以和同事一起用吗?

可以。原则:

- 共享: 项目 `CLAUDE.md`、`.claude/commands/`、`.claude/skills/`、`.claude/agents/` → 进 git
- 私有: `settings.local.json`、个人凭证、MCP 配置 → 不进 git
- 走 PR: CLAUDE.md 改动都走 PR review

详见 Ch 25。

10. 读完书了, 接下来干嘛?

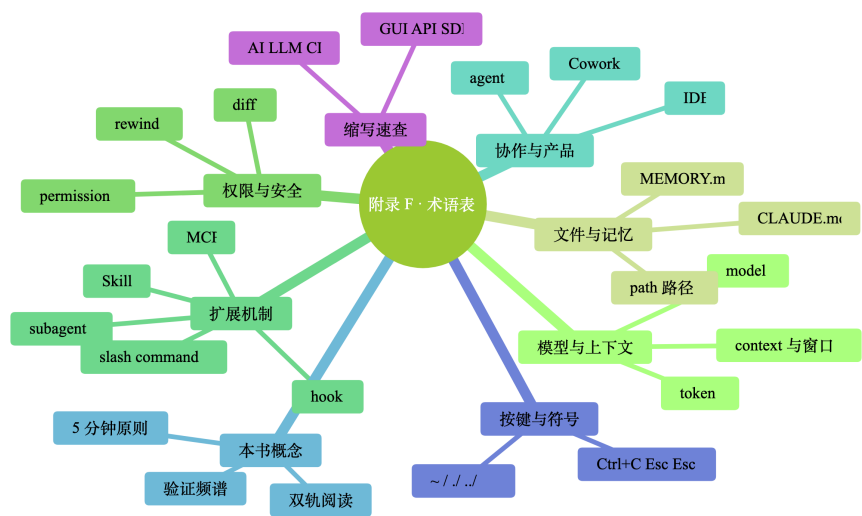
30 天挑战清单:

- 第 1 周: 稳固日常用法
- 第 2 周: 管理上下文和 CLAUDE.md
- 第 3 周: 用上扩展 (至少一个 Skill / Command / MCP)
- 第 4 周: 融入工作流 + 清理

核心原则：选 1-2 个方向深入，不要追最新 / 囤工具 / 和人比。详见 Ch 27。

附录 F：术语表

本书出现的所有 Claude Code / AI / 终端相关术语。跨章节查阅用。



A-Z 英文术语

术语	生活化翻译	本书出处
agent	有特定岗位的 Claude 实习生	Ch 4, 22
agentic coding	AI 主动执行任务的编程方式	Ch 4 支线
API	应用程序接口——程序之间说话的约定	Ch 24, 26
CLI	命令行工具——只能打字（不能点）的工具	Ch 1
CLAUDE.md	项目说明书——Claude 每次启动先读	Ch 13, 14
command	命令——给电脑或 Claude 下的一条指令	Ch 18, 21
context	短期工作记忆——这次对话能记住的总量	Ch 12
context window	上下文窗口——context 的容量上限	Ch 12
Cowork	Anthropic 面向非编程用户的 GUI 产品	Ch 4
diff	修改对比图——文件改前改后两栏	Ch 6
directory / folder	文件夹（本书统一用“文件夹”）	Ch 1
hook	自动触发规则——事件→动作	Ch 23
IDE	代码编辑器	Ch 4
install	安装	Ch 2
JSON	一种结构化文本格式	Ch 23, 24
MCP	外接工具插座——连外部系统的协议	Ch 24
MEMORY.md	自动记忆文件——Claude 自己记的你的偏好	Ch 13
model	大模型——Claude 背后的大脑	Ch 15
OAuth	一种第三方登录授权协议	Ch 24
pane	分屏区域	Ch 2
path	路径——文件在电脑里的地址	Ch 1
permission	权限	Ch 7

本书内置概念

概念	定义	本书出处
双轨阅读	🎯 主线（必读）+ 🌿 支线（选读）	Ch 0
WHAT-WHERE-HOW-VERIFY	提示词结构模板	Ch 5
验证频谱 L1-L4	按风险分档的审核深度	Ch 10
信任档案	你个人的“Claude 靠谱度”分级记录	Ch 10
止损规则	预设的“什么时候该停”判断	Ch 17
5 分钟原则	手动 < 5 分钟就别磨 AI	Ch 17
渐进披露	CLAUDE.md 慢慢加、不一次堆	Ch 14
Pointers > Copies	CLAUDE.md 里指路优于复制内容	Ch 14
最小权限原则	扩展 / Subagent / MCP 只给够用的权限	Ch 22, 24
三件套（章末）	小结 / 动手任务 / 卡住了	Ch 0

按键与符号

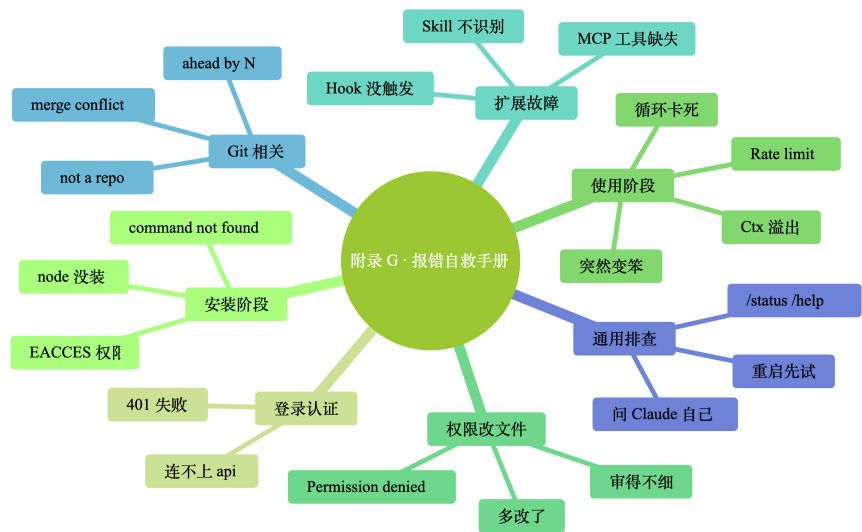
符号	读法	说明
~	波浪号	用户文件夹
~/	波浪号斜杠	从用户文件夹开始的路径
./	点斜杠	当前文件夹
../	双点斜杠	上层文件夹
/	正斜杠	Mac/Linux 路径分隔
\	反斜杠	Windows 路径分隔
\$ %	美元 / 百分号	Mac 终端提示符
>	大于号	Windows PowerShell 提示符
`	`	竖线 / 管道符
Ctrl+C	—	中断运行
Cmd+C	—	Mac 复制 (= Windows Ctrl+C)
Esc Esc	—	Claude Code 快速撤销

缩写速查

缩写	全称	翻译
AI	Artificial Intelligence	人工智能
API	Application Programming Interface	应用程序接口
CI/CD	Continuous Integration / Continuous Deployment	持续集成 / 持续部署
CLI	Command Line Interface	命令行界面
Ctx	Context	上下文 (Claude Code 状态栏缩写)
DAU	Daily Active Users	日活用户
GUI	Graphical User Interface	图形界面
IDE	Integrated Development Environment	集成开发环境
LLM	Large Language Model	大语言模型
MCP	Model Context Protocol	模型上下文协议
NDA	Non-Disclosure Agreement	保密协议
OS	Operating System	操作系统
PR	Pull Request	合并请求
SaaS	Software as a Service	软件即服务
SDK	Software Development Kit	软件开发工具包
SSO	Single Sign-On	单点登录
VPC	Virtual Private Cloud	虚拟私有云

附录 G：常见报错与自救手册

按关键词搜。每条格式：看到什么 → 最可能原因 → 怎么办。



安装阶段

command not found: claude

原因：Claude Code 没装；或装了但 PATH 没更新。怎么办：

- 重装： `npm install -g @anthropic-ai/claude-code`
- 关掉终端重开（让 PATH 生效）
- 确认 `which claude`（Mac） / `where claude`（Windows）能找到

EACCES: permission denied（安装时）

原因：npm 全局安装时权限不够。怎么办：

- Mac: 用 `sudo npm install -g ...` (输入 Mac 密码)
- 更好: 改 npm 前缀到用户目录 (避免 sudo)

node: command not found

原因: Node.js 没装。怎么办: 去 nodejs.org 下载 LTS 版本装。

npm WARN deprecated (装的时候一堆警告)

原因: 依赖库版本提示, 通常不影响功能。怎么办: 忽略, 装完能跑就行。

登录 / 认证

Authentication failed / 401

原因: token 过期 / 输错 / 没权限。怎么办:

- `/logout` 然后 `/login` 重新登录
- 确认订阅仍在有效期

Unable to connect to api.anthropic.com

原因: 网络问题 (防火墙 / 代理 / 梯子)。怎么办:

- 确认你能在浏览器访问 <https://anthropic.com>
- 公司网络 → 问 IT 是否需要配代理
- 国内用户可能需要特殊网络

使用阶段

Context window exceeded / 上下文溢出

原因: Ctx 爆了。怎么办:

- `/compact` 压缩; 或 `/clear` 重启
- 下次避免一次 @ 太多文件

Rate limit exceeded

原因: 短时间请求过多。怎么办:

- 等 1-5 分钟
- 或切到 Haiku (额度独立)

- 重度用户 → 升 Max / 换 API key

Claude 回答突然变笨 / 答非所问

可能原因 A: Ctx > 80%，模型开始糊涂 办法： `/compact` 或 `/clear`

可能原因 B: 模型换错了（你上次切到 Haiku 忘了切回） 办法： `/status` 看当前模型， `/model` 切回合适的

可能原因 C: prompt 太模糊 办法：按 Ch 5 的 WHAT-WHERE-HOW-VERIFY 重构提示

Claude 一直执行错误命令 / 循环中

原因：陷入局部最优。怎么办：

- 立刻 Ctrl+C 中断
 - `/rewind` 或 `/clear`
 - 用更具体的提示重来
-

权限 / 改文件

Permission denied (Claude 想动某文件时)

原因：系统级权限不够。怎么办：

- 文件 owner 是 root? → `chmod` / `chown` 改权限（小心）
- 系统保护目录? → 不要改

diff 看着对，但改完发现错了

原因：你 vibe review 不够细。怎么办：

- `/rewind` 回到改之前
- 重新走，这次细审或 `/plan` 先

改完文件后发现多改了很多

原因：Claude “顺手”多动 怎么办：

- `/rewind` 回到改前
 - 重讲：“只改 A，不动 B、C”
-

Hooks / 扩展

Hook 没触发

原因 A: `settings.json` 格式错 办法: 用 JSONLint 校验

原因 B: 需要重启 办法: 完全退出 Claude Code 重开

原因 C: `matcher` 匹配不上 办法: 先用 `"matcher": "*"` 匹配全部试, 再缩小

Hook 让 Claude Code 卡住

原因: `hook` 命令慢 / 卡住。怎么办:

- `hook` 里的命令加 `timeout 5` 限时 (Mac/Linux)
- 暂时注释掉 `hook` 看是不是它的问题

Skill / Command 没被识别

原因 A: 文件路径错 办法: 确认在 `~/.claude/skills/<名>/SKILL.md` 或 `.claude/commands/<名>.md`

原因 B: `frontmatter` 格式错 办法: 确认有三条横线包围的 `---` 块

原因 C: 需要重启 办法: 退出 Claude Code 重开

MCP 装了但 Claude 说没有这个工具

原因 A: `mcp_servers.json` 格式错 / 路径错 办法: JSON 校验 + 看启动输出有没有报错

原因 B: MCP server 本身没跑起来 办法: 手动跑一次
`npm run start @modelcontextprotocol/server-xxx`

Git 相关

`fatal: not a git repository`

原因: 当前目录不是 git 项目。怎么办: `git init` 初始化; 或 `cd` 到正确的项目目录。

`Your branch is ahead of 'origin/main' by N commits`

原因: 本地比远程新。怎么办: `git push` 推到远程 (注意确认要不要)。

Merge conflict

原因：你和别人改了同文件。怎么办：

- 不要乱删——打开冲突文件看 <<<<<< ===== >>>>>> 标记
 - 手动选择保留哪边
 - 或让 Claude 帮你：“@<冲突文件> 解决这里的 merge conflict，优先保留我们本地的业务逻辑”
-

通用排查顺序

遇到任何问题按这个顺序试：

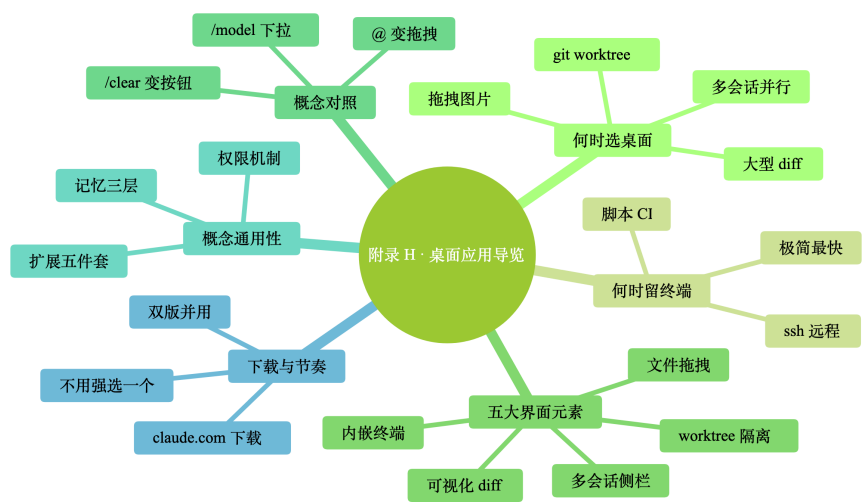
1. 重启 Claude Code（解决 50% 的问题）
 2. `/status` 看状态（模型对吗？Ctx 高吗？）
 3. `/help` 看命令说明
 4. 看启动时的日志输出（有错误提示吗？）
 5. 搜关键词（GitHub issues / Reddit）
 6. 问 Claude Code 自己：“我遇到 XXX 错误，怎么办？”（Claude 知道自己的错误！）
-

还是解决不了？

- 官方 GitHub 开 issue（附完整错误信息 + 你的版本 + OS）
- Reddit r/ClaudeAI 发帖
- 发 `/version` 看你的版本——升级到最新版可能就好了

附录 H：Claude Code 桌面应用简明导览

说明：本书主讲终端版 Claude Code。如果你嫌终端麻烦——桌面应用（2026-04 大改版）是同一引擎的图形版本，核心能力完全通用。这份导览让你快速对号入座。



什么时候切桌面应用值

- 看多会话不方便（终端只能一个窗口一个会话）
- 大型 diff 审核（图形化的 diff 比终端强多了）
- 需要 git worktree 并行工作（桌面应用原生支持）
- 截图 / 图片输入频繁（图形界面拖拽更顺）
- 对纯终端有抵触 但想要 Claude Code 能力

什么时候终端版更好

- 追求极简（终端最快）
- 远程服务器 / ssh 场景（桌面应用只在本机跑）
- 深度定制 hooks / pipe
- 脚本化 / CI 集成（非交互模式）

桌面应用的五大界面元素

1. 多会话侧栏

左侧列表——每个会话独立 Ctx。

好处：

- 给不同项目 / 任务开多个会话，互不干扰
- 切换只需点一下，不用 `/exit + cd + claude`
- 每个会话显示模型 / Ctx% / 名字

2. 可视化 diff

Claude 要改文件时——左右两栏图形化 diff，不是终端的 ASCII 版本。

好处：

- 颜色清晰（删除红底、新增绿底）
- 行号对齐
- 接受 / 拒绝按钮直接点（不用 y/n）

3. 内嵌终端

应用里内建一个终端，随时切。

好处：

- 不用开第二个 app
- 终端命令输出也在同一窗口，方便对照

4. 拖拽面板 / 文件浏览器

左侧有 workspace 文件树，可以拖文件到 prompt——自动 @ 引用。

5. git worktree 隔离

高阶功能——让 Claude 在独立 git 分支的副本上工作，不污染你当前分支。

工作流程:

- 1. 创建 worktree (独立副本)
 - 2. Claude 在这个副本里改东西
 - 3. 满意 → merge 回主分支
 - 4. 不满意 → 扔掉副本, 主分支毫发无损
- 比普通分支更隔离——Claude 甚至不知道主分支存在。

概念对照表 (终端 ↔ 桌面)

终端版	桌面应用对应
claude 启动	点图标启动
/exit	关闭会话 tab
新会话	点 “+ New Session”
@文件	拖文件 / 点 @ 按钮
看 Ctx%	会话条显示
/model	会话顶部模型下拉
/clear	点 Clear 按钮
/rewind	点 Rewind 按钮 / 点消息左边的小箭头
/plan	模式切换按钮

本书所有概念在桌面应用里适用吗?

完全适用:

-  三层记忆 (CLAUDE.md / MEMORY.md / 会话)
-  权限机制 (弹窗可能是图形对话框, 逻辑一样)
-  信任校准 L1-L4
-  模型与成本
-  Skills / Commands / Subagents / Hooks / MCP

界面不同但概念一样:

- @ 引用文件 → 拖拽或点按钮
- / 斜杠命令 → 大部分可用, 或改用顶部按钮

哪里下载

官方: claude.com/download

支持 Mac 和 Windows (桌面应用)。

一个建议的迁移节奏

如果你已经用熟终端版:

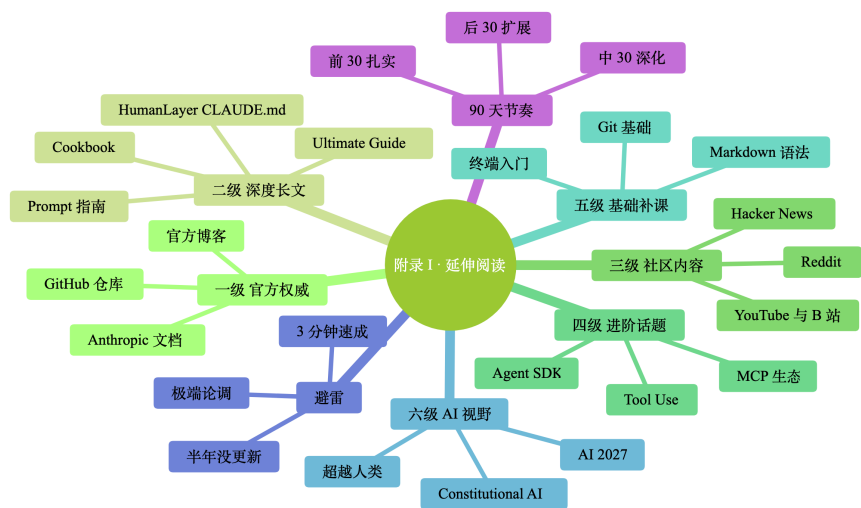
1. 装桌面版
2. 打开同一个项目
3. 做几件和终端版一样的事 (读文件、改文件、审 diff)
4. 对比哪边你更顺
5. 不用强迫选一个——两个可以并用

如果你还没用过终端版:

- 终端版和桌面版任选其一开始都行
- 本书教的所有概念适用
- 实际上很多人桌面版起步、终端版收尾 (熟了之后发现终端更快)

附录 I：延伸阅读

读完本书后，值得进一步看的资料。按优先级排列——先看前面的。



一级：官方权威（最该看）

1. Anthropic 官方文档

- 地址：docs.anthropic.com
- 内容：Claude Code、API、prompt engineering 全套
- 最权威 + 更新最及时
- 英文为主，浏览器翻译能看懂

2. Claude Code GitHub 官方仓库

- 搜 GitHub “claude-code” 找 Anthropic 官方
- Issues / Discussions 区有大量真实问题

- Changelog 跟踪最新特性
- 装完就加收藏

3. Anthropic Blog

- 地址: anthropic.com/news
 - 产品大更新、研究进展、最佳实践
 - 每月 1 次浏览就好
-

二级：高质量长文（核心思想）

4. HumanLayer 的 CLAUDE.md 文章

- 搜 “HumanLayer CLAUDE.md”
- 本书 Ch 14（CLAUDE.md 最佳实践）很多原则来自这里
- 深度讨论为什么 CLAUDE.md 应该短、为什么不用 `/init`

5. FlorianBruniaux 的 Claude Code Ultimate Guide

- GitHub 上的 `claude-code-ultimate-guide`（本书的灵感来源之一）
- 英文、面向有经验的开发者
- 信息密度极高，不适合零基础，但深度超过本书

6. Anthropic Prompt Engineering Guide

- docs.anthropic.com 的 “Prompt engineering” 章节
- 官方最全的 prompt 技巧合集
- 学完本书后精进 prompt 必读

7. Anthropic Cookbook

- GitHub `anthropics/anthropic-cookbook`
 - 真实代码示例合集
 - 覆盖 API 调用、tool use、agent 等
-

三级：社区内容（获取手感）

8. Reddit r/ClaudeAI

- 用户社区
- 真实使用心得 / 技巧 / 踩坑
- 不要照单全收——有时 karma 最高的不一定对

9. Hacker News

- 搜“Claude”关键词
- 资深用户讨论
- 观点独到、也最冷峻

10. YouTube / B 站

- 关键词：“Claude Code”
 - 视频教程时效性强——注意发布时间（Claude Code 迭代快，半年前的视频可能过时）
 - 中英文都有
-

四级：进阶话题（有需求再看）

11. Agent SDK / Multi-Agent

- Anthropic Agent SDK 文档
- LangChain / LangGraph
- CrewAI、AutoGen、MetaGPT

先问自己：我真的需要多 agent 吗？大多数场景 subagent 够用。

12. MCP 生态

- MCP 官网：modelcontextprotocol.io
- GitHub `modelcontextprotocol/servers`（官方 MCP 集合）
- `awesome-mcp-servers`（社区集合）

13. Tool Use / Function Calling

- Anthropic 文档的“Tool use”章节
- 如何让 AI 调用外部工具

- 想自己写 Agent 必读
-

五级：基础知识补课（零基础补充）

14. 终端 / 命令行入门

- Mac: Apple 官方 “Terminal User Guide”
- Windows: Microsoft Learn “PowerShell 快速入门”
- 本书附录 A、B 也有速查

15. Git 基础（如果你想更深入用 Claude Code 处理代码）

- [pro git](#) 中文版在线免费
- 只需前 3 章（安装、基础、分支）

16. Markdown 语法

- CLAUDE.md / Skill / Command 都用 markdown
 - [Markdown 语法速查](#)（15 分钟学完）
-

六级：AI 全景视野（大方向思考）

17. 《AI 2027》（非虚构报告）

- 关于 AI 未来 1-2 年的详细推演
- 帮你理解“为什么 Claude Code 这种工具会出现”

18. 《超越人类：AI 浪潮下我们如何生存》

- 非技术视角讨论 AI 对工作 / 生活的影响
- 适合读完本书觉得“我好像需要更大视野”时读

19. Anthropic 的 Constitutional AI 论文

- 关于 Claude 为什么比较“守规矩”的技术原理
 - 学术向，但理解 AI 背后设计哲学有帮助
-

不推荐（避雷）

- 所有标题“Claude Code 终极速成”的 3 分钟视频 —— 太浅
 - 超过 6 个月没更新的 GitHub 仓库 / 教程 —— Claude Code 变化快
 - 和你工作无关的炫技项目 —— 学了没用
 - 鼓吹“Claude 能代替程序员”或“AI 只是玩具”的极端内容 —— 都不对
-

一份推荐的 90 天学习节奏

Day 1–30: 基础扎实

- 本书（优先）+ 官方文档关键章节
- 30 天挑战清单（Ch 27）
- 每天真实场景用

Day 31–60: 深化一个方向

- 选你最关心的一个（CLAUDE.md / Skills / MCP / prompt engineering / ...）
- 读对应的深度资源（见上面二级、三级）
- 动手做 3-5 个实际项目

Day 61–90: 横向扩展 + 分享

- 了解其他 AI 工具（Cursor、Windsurf、GPT、Gemini 等）
- 在团队 / 社区分享你的经验
- 开始尝试更复杂场景（多 agent、MCP 定制、CI 集成等）

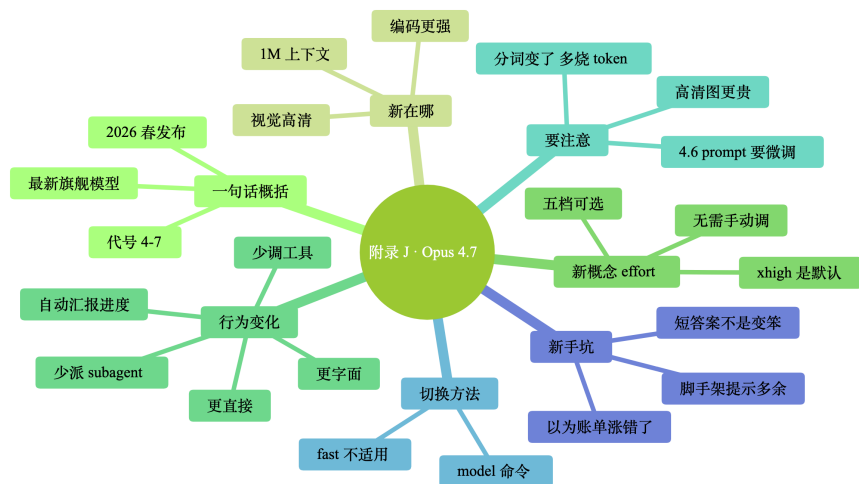
90 天后：你会是身边人里最懂 Claude Code 的。

祝阅读愉快，用得顺手。

本书作者：马力 · @mali9527 · li.ma.aria@gmail.com 版权所有 © 2026 马力 (Ma Li)，以 CC BY-NC-SA 4.0 协议开源。转载 / 翻译 / 引用请保留本署名。

附录 J: Claude Opus 4.7 新手指南

2026-04-16 Anthropic 发布 Claude Opus 4.7，是当前 Claude Code 默认推荐使用的模型。本书默认以 Opus 4.7 为基线写作。这一篇把 4.7 相对 Opus 4.6 的变化、行为差异、新概念、新手坑集中讲一遍——你只需要读这一附录，就能知道“Opus 4.7 到底带来了什么”。



J.1 一句话概括 Opus 4.7

Anthropic 于 2026 年 4 月 16 日 发布的旗舰模型，API 代号 `claude-opus-4-7`——当前 Claude Code 在付费用户那里使用的默认 Opus 版本。

它的定位：

- 当下最强的通用可用模型（官方原话：“our most capable generally available model”）
- 专门为复杂编码 + 长期自主工作 + 视觉理解 场景优化

- 价格和 Opus 4.6 相同——升级不加价
- 原生支持 100 万 token 上下文 (1M)，且无长上下文溢价

相比 Opus 4.6，它的核心变化可以归成三句话：

1. 更聪明——尤其是编程、视觉、长任务自主执行
2. 更直接——回复更短、更像工程师少客套、emoji 少
3. 更字面——你说一，它就做一，不自作主张推广到二

J.2 你会直接感受到的差异

这一节给从 Opus 4.6 切过来的人。如果你是第一次用 Claude Code，可以跳到 J.3。

差异 1：回答变短了

4.7 根据任务复杂度动态调整回复长度——简单问题给简短答案，不再“I'd be happy to help you...”开场。

❌ 别误以为：AI 是不是变笨了 / 是不是罢工了 ✅ 正确理解：这就是新版本的默认风格

差异 2：更“照字面办事”

例子：你说“把 `user.email` 字段改成 `user.contact_email`”。

- 4.6 行为：可能顺手把邻近的 `user.phone` 也重命名成 `user.contact_phone`（它觉得“你大概想统一”）
- 4.7 行为：只改你说的那一个字段。如果要连带别的，你得明说

差异 3：默认调更少工具、派更少 subagent

4.7 会更多用推理，更少用工具调用。如果你之前习惯让它“读 10 个文件再判断”，现在可能只读 3 个它就开始下结论。

应对：需要它一次并行读多文件时明确说——

“请同时读取 `src/auth/*.py` 里所有文件后再判断”

差异 4: 长任务中会主动汇报进度

以前 (4.6) 你得问“现在到哪一步了”，4.7 会自己在合适的节点告诉你。如果你之前在 CLAUDE.md / prompt 里加过“每完成一步告诉我”，可以试着拿掉看看还需不需要。

差异 5: 同样一段话，可能多烧 35% token

4.7 用了新分词器——同样的中文或代码内容，可能用 1.0~1.35 倍的 token (最多 35%)。

这不是 bug，是设计上的权衡（新分词器对模型能力有帮助）。但月账单可能因此上涨——见 J.6 坑 3。

J.3 新概念：effort 档位

Opus 4.7 引入了 5 档 effort (思考投入档位)，在“聪明 vs 成本/速度”之间切换：

档位	速度	智力	适合
low	最快	一般	成本敏感、简单批量
medium	快	中等	日常轻量问题
high	中	强	并发多会话、想省一点
xhigh	中慢	很强	Claude Code 官方默认
max	慢	最强	极难问题（可能过度思考，建议克制使用）

对零基础最重要的一句话

你不用手动调。Claude Code 已经把 Opus 4.7 的默认档位设成 `xhigh` ——这是官方推荐的最佳值。

💡 只有在你用 Anthropic API 直接写代码（见 Ch 26）时，effort 才是你要关心的 API 参数。Claude Code 用户保持默认即可。

J.4 在 Claude Code 里用 Opus 4.7

切到 Opus 4.7

和 Ch 15 讲的 `/model` 命令完全一致：

```
/model
```

在弹出的菜单里选 Claude Opus 4.7（或者你看到的版本号可能显示成“Opus”简写）。

现在默认就是 4.7 了吗？

- 2026-04-23 起，Enterprise 和 API 用户的默认 Claude Code 模型切到 Opus 4.7
 - Pro / Max 订阅用户在 `/model` 菜单里点“Opus”时，指向的就是 4.7
- 你可以随时 `/model` 看当前是哪个版本——如果显示 Opus 4.7，就是它。

`/fast` 在 4.7 下失效了

Claude Code 的 `/fast`（快速模式）是 Opus 4.6 专用的加速通路——Anthropic 通过更快的服务器路径响应请求。

切到 Opus 4.7 后：

- `/fast` 不起作用或不显示（取决于具体 Claude Code 版本）
- 这不是 bug，是正常行为
- 如果你非常在意响应速度，可以回切 Opus 4.6 + `/fast`

1M 上下文——该用吗？

Opus 4.7 原生支持 100 万 token 上下文（5 倍于传统 200K）。好消息：按标准 API 价格，没有长上下文溢价。

但——上下文越长，每轮输入越烧 token。除非你真的需要（比如让它读整个大型代码库），日常用默认上下文就够（见 Ch 12 讲上下文原理）。

J.5 prompt 写法的几个微调

如果你之前用过 4.6（或抄过 4.6 的教程），4.7 下可以试试这几个调整：

调整 1: 第一轮就说清楚

官方推荐“把 Claude 当作受委托的工程师”——意图、约束、验收标准、文件路径一次性给全。这和 Ch 19（输入技巧）讲的 WHAT / WHERE / HOW / VERIFY 模板完全一致。

调整 2: 要更深思熟虑时显式要求

```
Think carefully and step-by-step before responding; this problem is harder than it looks.
```

（中文等效：“请仔细、一步一步地思考再回答；这个问题比看起来更难”）

调整 3: 要快速响应时也显式要求

```
Prioritize responding quickly rather than thinking deeply.
```

（中文等效：“优先快速回复，不必深度思考”）

调整 4: 需要并行子任务时明确说

4.7 默认会派更少 subagent。如果你明确需要并行——

“请在同一轮内启动多个 subagent 分别处理这些文件”

调整 5: 把 4.6 的“脚手架提醒”拿掉

- “记得检查一下幻灯片布局”
- “记得复审一下代码”
- “输出前请确认格式正确”

这类加在 prompt 末尾的提醒，给 4.7 可能多余——它默认就会做这些。试着去掉，看质量是否还稳。如果没降，以后就不用写了。

J.6 新手最容易踩的 5 个坑

坑 1: 以为回答短就是 AI 变笨

事实：4.7 的简短回答是主动设计——任务简单就不写长文。

别做：反复“你能详细说说吗 / 多举几个例子吗”——除非你真的需要。

坑 2: 沿用 4.6 的 prompt 感觉“变味了”

你熟悉的某个 4.6 prompt 在 4.7 下结果不对劲？可能是因为：

- 4.7 更字面，不会自作主张泛化
- 4.7 默认少调工具/少派 subagent
- 以前“隐含”的指令，现在要显式写

解法：把指令拆得更明确 + 把验收标准写出来。

坑 3: 账单突然涨了——两个原因叠加

- 分词器换了：同文本可能多 1.0~1.35 倍 token
- 默认档位 `xhigh`：比 4.6 的默认更高档

如果成本敏感，两种办法：

1. 切回 Sonnet 4.6 做日常任务（见 Ch 15 的选型建议）
2. API 模式下可手动把 effort 调成 `high`（Claude Code 用户一般不需要）

坑 4: 粘高清图截图一次烧几千 token

Opus 4.7 最大支持 2576px ($\approx 3.75\text{MP}$) 的图像（旧版只到 1568px）。清晰度大升级，但单张高清图消耗成倍增长。

建议：

- 截图先压缩到 1000~1500px（一般够用）
- 只有在需要 OCR、看清设计稿细节、验证像素级内容时用高清图

坑 5: `/fast` 找不到了，以为功能被删了

如前所述，`/fast` 是 Opus 4.6 的快速通路。切到 4.7 后它不起作用是正常的。

如果你特别想要 `/fast`：回切 Opus 4.6（`/model` → Opus 4.6）。

J.7 什么时候该用 Opus 4.7?

这一节是 Ch 15（模型与成本）“选型原则”的 4.7 补丁版：

场景	推荐
复杂重构、架构设计、长任务 (30 min+) 自主执行	Opus 4.7 (默认 xhigh)
看截图 / 分析设计稿 / 看清图表细节 / OCR	Opus 4.7 (视觉升级最大)
需要跨很多文件的代码审查	Opus 4.7
日常文件读写、写文档草稿、普通修改	Sonnet 4.6 够用 (便宜 + 快)
简单批量任务 (改格式、翻译、速查)	Haiku 4.5 (最便宜)

90% 用户的经验法则:

- 默认 Sonnet 4.6——成本和速度的甜区
- 复杂任务切 Opus 4.7——深度推理一次值回票价
- 超简单的切 Haiku——不要浪费

J.8 官方权威资源

以下是学完本附录后可以深入读的官方原文 (优先级从高到低):

1. 发布公告 (浏览 5 分钟): anthropic.com/news/claude-opus-4-7
2. Opus 4.7 Claude Code 最佳实践 (核心 + 实用, 浏览 15 分钟): claude.com/blog/best-practices-for-using-claude-opus-4-7-with-claude-code
3. 新特性完整清单: platform.claude.com/docs/en/about-claude/models/whats-new-claude-4-7
4. 4.6 → 4.7 迁移指南: platform.claude.com/docs/en/about-claude/models/migration-guide
5. effort 档位官方建议: platform.claude.com/docs/en/build-with-claude/effort

本节小结

- Opus 4.7 是 2026-04-16 发布的旗舰模型, Claude Code 默认推荐
- 相比 4.6: 更聪明、更直接、更字面, 但分词器换了, 会多烧 10~35% token
- 新增 5 档 effort (low / medium / high / xhigh (默认) / max), Claude Code 用户不用手动调
- `/model` 切换、`/fast` 只对 4.6 有效
- 4.6 的 prompt 在 4.7 下可能需要写更明确 + 去掉脚手架提醒
- 视觉升级最大 (支持 2576px), 适合看截图 / 设计稿——但烧 token

- 成本敏感回切 Sonnet 4.6 或 Haiku 4.5 没有任何问题

本附录随 Anthropic 新版本发布持续更新。任何时候想知道当前用的哪个版本：在 Claude Code 里敲 `/model` 。

如果未来出现 Opus 4.8 / 5.x，本附录会第一时间补写差异。